

ARM PrimeCell

DMA Controller (PL081)

Design Manual

ARM PrimeCell DMA Controller (PL081) Design Manual

© Copyright ARM Limited 2001. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com/>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.1.1	Current status and anticipated changes	1
1.1.2	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
1.4	Conventions used in this document	2
1.4.1	Signals	2
1.4.2	Bytes, Half-words and Words	2
1.4.3	Bits, bytes, K and M	3
2	SCOPE	3
3	INTRODUCTION	3
4	DMAC DESIGN OVERVIEW	3
4.1	Signal Description	4
4.2	Programmer's Model	7
4.3	AHB Slave Interface and Register Block	11
4.4	AHB Master Interface	14
4.4.1	DMAC Internal Arbiter	15
4.4.2	AHB-Lite Master Interface	16
4.4.3	AHB-Lite-to-AHB wrapper	19
4.5	Channel Logic	19
4.5.1	DMAC Channel Register Block	19
4.5.2	DMAC Channel Source Transfer Block	22
4.5.3	DMAC Channel Destination Transfer Block	23
4.5.4	DMAC Channel LLI Load Block	25
4.5.5	DMAC FIFO Packing-Unpacking Block	26
4.5.6	DMAC Channel Register File	28
4.5.7	DMAC Channel Request Processing Block	29
4.5.8	DMAC Channel Request Masking Block	31
4.6	DMAC Response Routing Block	32
4.7	DMAC Request synchronising logic	34
4.8	Revision Designator Block	35
5	DMAC IMPLEMENTATION DETAILS	35

5.1	Design Hierarchy	35
5.2	DMAC Top Level Module (Dmac)	36
5.3	AHB Slave Interface Module (DmacAhbSlavelf)	36
5.3.1	Register Write Logic	39
5.3.2	Register Read Logic	39
5.4	AHB Master Interface(DmacAhbMasterIf)	40
5.4.1	Internal Arbiter (DmacArbiter)	40
5.4.2	AHB-Lite Master State machine (DmacLiteMaster)	41
5.4.3	AHB Wrapper (DmacMasterWrap)	44
5.5	Channel Control Logic (DmacChannel)	44
5.5.1	DMAC Channel Register Block (DmacChRegBlock)	44
5.5.2	DMAC Channel Source-Transfer Block (DmacChSrcXfer)	46
5.5.3	DMAC Channel Destination Transfer Block (DmacChDstXfer)	50
5.5.4	DMAC Channel LLI Load Block (DmacChLLILoad)	53
5.5.5	DMAC FIFO Packing Unpacking Block (DmacChPckUnpck)	55
5.5.6	DMAC Channel Register File (DmacChRegFile)	56
5.5.7	DMAC Channel Request Processing Block (DmacChReqProc)	56
5.5.8	DMAC Channel Request Masking Block (DmacChReqMask)	57
5.6	DMAC Response Routing Block (DmacRspRoute)	58
5.7	DMAC Request Synchronising Block (DmacRqstSync)	58
5.8	Revision Designator block (DmacRevAnd)	58
5.9	Software/System Assumptions	59
6	DMAC VERIFICATION DETAILS	60
6.1	DMAC VHDL Verification Testbench	60
6.1.1	Unit Under Test	62
6.1.2	Bus Master Protocol Checker	63
6.1.3	AHB Master/Arbiter Model	63
6.1.4	Trickbox	63
6.1.5	Protocol Checker	64
6.1.6	Test Vectors	64
6.1.7	Generics	64
6.1.8	Running Simulations	65
6.2	DMAC VERILOG VERIFICATION TESTBENCH	65
6.2.1	Unit Under Test	68
6.2.2	Bus Master Protocol Checker	69
6.2.3	AHB Master/Arbiter Model	69
6.2.4	Trickbox	69
6.2.5	Protocol Checker	70
6.2.6	Test Vectors	70
6.2.7	Parameters	70
6.2.8	Running Simulations	71
6.3	Trickbox Description	72
6.3.1	Trickbox register summary	73

6.3.2 Behavioural Model of DMAC	73
6.3.3 Grant Generation Block (DmacTrGntGen)	84
6.3.4 Checker Block	85
6.4 Changes which needs to be done for above Trickbox for DMAC variant	102
6.4.1 Module level changes to be done	102
6.4.2 Changes which needs to be done in Tbench for Behaviour Trickbox	102
6.5 Functional Tests	103
6.5.1 Register Tests	104
6.5.2 Enabled Channel Test	108
6.5.3 Interrupt Tests	109
6.5.4 Channel Test	113
6.5.5 DMAC DISABLE Test	258
6.5.6 DMAC HALT Test	259
6.5.7 LLI Tests	259
6.5.8 MultiChannel Tests	260
6.5.9 AHB Master Test Cases	291
6.5.10 Corner cases	294
7 DMAC INTEGRATION TEST DETAILS	295
7.1 DMAC VHDL Integration testbench	295
7.1.1 Unit Under Test	297
7.1.2 Test Vectors	298
7.1.3 Running Simulations	298
7.2 DMAC Verilog Integration testbench	299
7.2.1 Unit Under Test	300
7.2.2 Test Vectors	301
7.3 Integration Trickbox	302
7.4 Integration Tests	302
7.4.1 Integration Testing of non-AMBA intra-chip outputs	302
7.4.2 Integration Testing of non-AMBA intra-chip inputs	302
7.4.3 Integration Testing of AMBA ports	303
7.5 Integration Test Vector Generation	303

1 ABOUT THIS DOCUMENT

1.1 Change history

1.1.1 Current status and anticipated changes

First release of the document.

1.1.2 Change history

Issue	Date	Change
A01	29 th April, 2001	First Draft
A	17 th May, 2001	First Release

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM IHI0011	AMBA Specification Rev2.0
2	ARM DDI 0218 A	ARM PrimeCell DMA Controller (PL081) Technical Reference Manual
3	PL081 INTM 0000 A	ARM PrimeCell DMA Controller (PL081) Integration Manual
4	ARM DUI 0006	AMBA Compliance Test Suite User Guide
5	ARM DDI 0138	Example AMBA System Technical Reference Manual
6	ARM DUI 0092	Example AMBA System User Guide
7	SCB00 QPRO 0008 A05	ARM PrimeCell Integration Vector Strategy
8	SC056 TRM 0000 A01	AHB Example Bus Master Technical Reference Manual

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	An AMBA bus designed for high-performance peripherals.
AHB master	A device that can send out requests over an AHB bus.
AHB slave	A device that fulfils requests provided by an AHB master.
AMBA	Advanced Microcontroller Bus Architecture.
DMA	Direct Memory Access.
DMA channel	The hardware which monitors and controls the progress of a DMA transaction.

DMA request	A request usually provided by a peripheral, to indicate to the DMA controller that data is ready to be transferred.
DMA stream	A DMA stream is the set up needed to be able to transfer data from a source to a destination. This usually comprises of a DMA channel, DMA requests and configuration information.
DMAC	DMA Controller.
DMAFC	DMAC Flow Control
DRRB	DMA Request and Router Block
DstFC	Destination Flow Control
FIFO	First In First Out Buffer
Flow controller	The peripheral which has knowledge, and controls of the length of the packet. The flow controller is usually the DMA Controller where the packet length is programmed by software before the DMA channel is enabled. If the packet length is unknown, when the DMA channel is enabled, either the source or destination peripheral is used as the flow controller.
LLI	Linked List Item
M2M	Memory to Memory
Mux	Multiplexer
Packet	A "packet" of data is the total amount of data to be transferred under the control of a single LLI. A packet is usually made up of several DMA bursts.
SrcFC	Source Flow Control
TC	Terminal Count

1.4 Conventions used in this document

1.4.1 Signals

- Signal names are shown in bold font type.
- Active low signals are prefixed by a lower case 'n'. In the case of AHB signals by 'Hn', or postfixed by 'n'.
- AHB signals are prefixed by an upper case 'H'.
- When a signal is described as being 'Asserted', the actual level depends on whether the signal is active HIGH or active LOW. 'Asserted' means HIGH for active high signals and LOW for active low signals.
- (L)(B/S)REQ indicates LBREQ, LSREQ, BREQ and SREQ.

1.4.2 Bytes, Half-words and Words

- A byte is 8 bits.
- A half-word is two bytes i.e. 16 bits.
- A word is 4 bytes i.e. 32 bits.

1.4.3 Bits, bytes, K and M

- The suffix 'B' (upper case) is used to indicate BYTES.
- The suffix 'b' (lower case) is used to indicate BITS.
- When used to indicate an amount of memory, the suffix 'k' means 1024.
- When used to indicate a frequency, the suffix 'k' means 1000.
- When used to indicate an amount of memory, the suffix 'M' means $1024^2 = 1048576$.
- When used to indicate a frequency, the suffix 'M' means 1,000,000.

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell PL081 DMAC (DMA Controller). The design description is split between two sections – Section 4 and Section 5. The testbenches and the test programs for functional verification and integration testing are described in Section 6 and Section 7.

Section 4 presents an overview of the DMAC design, describing the functional partitioning of the DMAC and the signal interconnections. Section 5 presents detailed descriptions on the DMAC design. Section 6 describes the testbench and the test programs used to generate test vectors for functional verification of the DMAC. Section 7 describes the testbenches and the test programs used to generate test vectors for Integration Testing of the DMAC.

3 INTRODUCTION

The DMAC is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM DMA Controller (PL081) Technical Reference Manual [Ref.2]. The ARM PrimeCell DMA Controller (PL081) Integration Manual [Ref.3] describes how the block may be integrated into a typical industry standard ASIC design flow.

The AMBA Compliance Test Suite User Guide [Ref.4] is a source for further information about AMBA bus compliance testbenches.

The Example AMBA System Technical Reference Manual [Ref.5] and User Guide [Ref.6] describe an example microcontroller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of TICTalk test vectors for production test.

4 DMAC DESIGN OVERVIEW

The DMA Controller (DMAC) is an Advanced Microcontroller Bus Architecture (AMBA) block that connects to the Advanced High-performance Bus (AHB). There is an AHB slave interface for programming the DMAC and an AHB master for data transfer. The DMAC has two channels, which can buffer up to 4 words each. Each channel can transfer data through the AHB Master interface with the programmed data width and endianness. The DMAC PL081 supports 16 DMA requestors, and generates individually maskable interrupts for Terminal count and Transfer error for each channel.

4.1 Signal Description

The following tables describe the DMA controller interface signals. Table 4.1-1 lists the AHB slave interface signals, while Table 4.1-2 and 4.1-3 lists the master interface signals. Table 4.1-4 and 4.1-5 list the Interrupts, the DMA requests and the DMA response signals for the DMAC.

For information on the AHB, see AMBA Specification Rev2.0 [Ref. 1].

Signal Name	Type	Source / Destination	Description
HCLK Bus clock	Input	Clock Source	This clock times all bus transfers. All signal timings are related to the rising edge of HCLK.
HRESETn Reset	Input	Reset controller	The Bus reset signal is active LOW and is used to reset the system and the bus. This is the only active LOW signal.
HADDR[11:2] Address bus	Input	AHB Master	The system address bus.
HWRITE Transfer direction	Input	AHB Master	When HIGH this signal indicates a write transfer and when LOW, a read transfer.
HSIZE[2:0] Transfer size	Input	AHB Master	Indicates the size of the transfer. The DMAC supports only 32 bit WORD accesses to all its registers. So all transfers to and from the DMA controller must be 32-bit (HSIZE[2:0] = '010').
HTRANS Transfer type	Input	AHB Master	Indicates whether a data-transaction is to be done for the current transfer. HTRANS[1] on the AHB bus must be connected to the single-bit-wide HTRANS input of the DMAC. '1' represents NSEQ or SEQ and thus a valid data transfer. '0' represents BUSY or IDLE.
HWDATA[31:0] Write data bus	Input	AHB Master	The write data bus is used to transfer data from the master to the DMAC Slave Interface during write operations. The DMAC AHB Slave interface supports only 32-bit accesses.
HSELDMAC Slave select	Input	AHB Decoder	The DMAC AHB slave has its own slave select signal. This signal indicates that the current transfer is intended for the DMAC. This signal is a combinatorial decode of the address bus.
HRDATA[31:0] Read data bus	Output	AHB Master	The read data bus is used to read data from the DMAC AHB Slave Interface. The DMAC AHB Slave interface supports only 32-bit accesses.
HREADYIN Transfer done	Input	AHB Slave	When HIGH the HREADYIN signal indicates that a transfer has finished on the bus.
HREADYOUT	Output	AHB Master	When HIGH the HREADYOUT signal indicates that a transfer targeting the DMAC AHB Slave Interface has finished on the bus.

Transfer done			The DMAC AHB Slave interface inserts one wait state for read accesses to Channel registers. All other read/write accesses to registers are completed with zero wait states.
HRESP[1:0] Transfer response	Output	AHB Slave	The transfer response provides additional information on the status of a transfer. Defined responses from AHB slave are, OKAY, ERROR, RETRY and SPLIT. The DMA Controller AHB Slave Interface only drives OKAY or ERROR response.

Table 4.1-1 AHB slave signals

Signal Name	Type	Source / Destination	Description
HADDRM[31:0] Address bus	Output	AHB Slave	The 32-bit system address bus.
HTRANSM[1:0] Transfer type	Output	AHB Slave	Indicates the type of the current transfer, which can be NONSEQ, SEQ, IDLE or BUSY.
HWRITEM Transfer direction	Output	AHB Slave	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZEM[2:0] Transfer size	Output	AHB Slave	Indicates the size of the transfer. The DMA controller allows 8, 16 and 32-bit transfer sizes. 8-bit: HSIZEM[2:0] = '000' 16-bit: HSIZEM[2:0] = '001' 32-bit: HSIZEM[2:0] = '010'
HPROTM[3:0] Protection control	Output	AHB Slave	Provides information about a bus access.
HLOCKDMAC Lock information for locked transfers	Output	AHB Arbiter	This signal is used to indicate to the arbiter that the master is going to perform a number of indivisible transfers.
HBURSTM[2:0]	Output	AHB Slave	Indicates the size of the burst for burst transfer. The types of bursts supported by the DMAC AHB Master are INCR, INCR4, INCR8 and INCR16.
HWDATAM[31:0] Write data bus	Output	AHB Slave	The write data bus is used to transfer data from the master to the bus slaves during write operations.
HRDATAM[31:0] Read data bus	Input	AHB Slave	The read data bus is used to transfer data from bus slaves to the bus master during read operations.

HREADYINM Transfer done	Input	AHB Slave	When HIGH the HREADYINM signal indicates that a transfer has finished on the bus. This signal may be driven LOW to extend a transfer, by the AHB slave.
HRESPM[1:0] Transfer response	Input	AHB Slave	The transfer response provides additional information on the status of a transfer. Four different responses are supported: OKAY, ERROR, RETRY and SPLIT. To indicate that the transfer has completed successfully, the slave uses the OKAY response. To indicate that an error has occurred, the slave uses the ERROR response. The DMA controller will then assert the Error Interrupt request. To indicate that the data is not ready, the slave uses the RETRY and SPLIT responses. The DMA controller should retry the transfer at a later date.

Table 4.1-2 AHB Master signals

Signal Name	Type	Source / Destination	Description
HBUSREQDMAC	Output	AHB arbiter	Bus request signal used by the DMA controller to request access to the AHB bus.
HGRANTDMAC	Input	AHB arbiter	Grant signal generated by the arbiter. This signal indicates that the DMAC is currently the highest priority master requesting the bus. The DMAC gains bus ownership when HGRANTDMAC and HREADYINM are high on the rising edge of HCLK.

Table 4.1-3 AHB Master bus request signals

Signal Name	Type	Source / Destination	Description
DMACINTERR	Output	Interrupt Controller	DMAC Error interrupt request to interrupt controller. Note that this interrupt signal is active HIGH.
DMACINTTC	Output	Interrupt controller	DMAC Terminal Count interrupt request to interrupt controller. Note that this interrupt signal is active HIGH.
DMACINTR	Output	Interrupt controller	DMAC combined interrupt request to interrupt controller. This signal is the combination of DMACINTERR and DMACINTTC. Note that this interrupt signal is active HIGH.

Table 4.1-4 DMAC Interrupt signals

Signal Name	Type	Source / Destination	Description
DMACBREQ[15:0]	Input	Peripheral	Burst DMA transfer request.
DMACSREQ[15:0]	Input	Peripheral	Single DMA transfer request.
DMACLBREQ[15:0]	Input	Peripheral	Last burst DMA transfer request.
DMACLSREQ[15:0]	Input	Peripheral	Last single DMA transfer request.
DMACCLR[15:0]	Output	Peripheral	DMA Request Acknowledge/Clear. It is asserted one clock after the end of the data-phase of the last beat of the transfer.
DMACTC[15:0]	Output	Peripheral	DMAC Terminal Count. Indicates that the transaction has completed and the packet of data has been transferred.

Table 4.1-5 DMAC request and response signals

Signal Name	Type	Source / Destination	Description
SCANENABLE	Input	Test Controller	Scan Enable signal. Indicates that the circuit is in scan-test mode.
SCANINHCLK	Input	Test Controller	Scan data input for clocking in the test pattern in scan-test mode.
SCANOUTHCLK	Output	Test Controller	Scan data output for observing the flip-flop contents in scan test mode.

Table 4.1-6 Scan Test related signals

4.2 Programmer's Model

In this section, information about all the registers is given.

Address	Type	Width	Reset Value	Register Name	Description
DMAC Base + 0x000	R	1-0	0x0	DMACIntStat	This register provides the interrupt status of the DMA controller. A high bit indicates that a specific DMAC channel interrupt is active.
DMAC Base + 0x004	R	1-0	0x0	DMACIntTCStat	This register is used to determine whether an interrupt was generated due to the transaction completing (Terminal Count). A high bit indicates that the transaction completed.

Address	Type	Width	Reset Value	Register Name	Description
DMAC Base + 0x008	W	1-0	-	DMACIntTCClr	When writing to this register, each data bit that is HIGH that causes the corresponding bit in the DMACIntTCStat register to be cleared. Data bits those are LOW does not have any effect on the corresponding bit in the register.
DMAC Base + 0x00C	R	1-0	0x0	DMACIntErrStat	This register is used to determine whether an interrupt was generated due to an error being generated. A high bit indicates that an error was generated.
DMAC Base + 0x010	W	1-0	-	DMACIntErrClr	When writing to this register, each data bit that is HIGH that causes the corresponding bit in the DMACIntErrStat register to be cleared. Data bits those are LOW do not have any effect on the corresponding bit in the register.
DMAC Base + 0x014	R	1-0	0x0	DMACRawIntTC	This register provides the raw status of DMAC Terminal Count interrupts prior to masking. A high bit indicates that the interrupt request is active prior to masking.
DMAC Base + 0x018	R	1-0	0x0	DMACRawIntErr	This register provides the raw status of DMAC error interrupts prior to masking. A high bit indicates that the interrupt request is active prior to masking.
DMAC Base + 0x01C	R	1-0	0x0	DMACEnbldChns	This register shows which DMAC channels are enabled. A high bit indicates that a DMAC channel is enabled.
DMAC Base + 0x020	R/W	15-0	0x0	DMACSoftBReq	This register allows DMAC Burst Requests to be generated by software. Reading the register indicates which sources are requesting DMA transfers. A request can be generated from either a peripheral or the software request register.
DMAC Base + 0x024	R/W	15-0	0x0	DMACSoftSReq	This register allows DMAC Single Requests to be generated by software. Reading the register indicates which sources are requesting DMA transfers. A request can be generated from either a peripheral or the software request register.
DMAC Base + 0x028	R/W	15-0	0x0	DMACSoftLBReq	This register allows DMAC last Burst Requests to be generated by software. Reading the register indicates which sources

Address	Type	Width	Reset Value	Register Name	Description
					are requesting DMA transfers. A request can be generated from either a peripheral or the software request register.
DMAC Base + 0x02C	R/W	15-0	0x0	DMACSoftLSReq	This register allows DMAC last Single Requests to be generated by software. Reading the register indicates which sources are requesting DMA transfers. A request can be generated from either a peripheral or the software request register.
DMAC Base + 0x030	R/W	1-0	0x0	DMACConfig	This register is used to configure the DMA controller.
DMAC Base + 0x034	R/W	15-0	0x0	DMACSync	This register is used to give the information regarding the synchronisation of the DMA requests.
DMAC Base + 0x100	R/W	31-0	0x0	DMACC0SrcAddr	DMAC channel 0-source address.
DMAC Base + 0x104	R/W	31-0	0x0	DMACC0DestAddr	DMAC channel 0-destination address.
DMAC Base + 0x108	R/W	31-2	0x0	DMACC0LLIReg	DMAC channel 0 linked list address..
DMAC Base + 0x10C	R/W	31-0	0x0	DMACC0Control	DMAC channel 0 control.
DMAC Base + 0x110	R/W	17-0	0x0	DMACC0Config	DMAC channel 0-configuration register.
DMAC Base + 0x120	R/W	31-0	0x0	DMACC1SrcAddr	DMAC channel 1 source address.
DMAC Base + 0x124	R/W	31-0	0x0	DMACC1DestAddr	DMAC channel 1 destination address.
DMAC Base + 0x128	R/W	31-2	0x0	DMACC1LLIReg	DMAC channel 1 linked list address..
DMAC Base + 0x12C	R/W	31-0	0x0	DMACC1Control	DMAC channel 1 control.
DMAC Base + 0x130	R/W	17-0	0x0	DMACC1Config	DMAC channel 1 configuration register.
DMAC Base + 0x500	R/W	0 (one-bit register)	0x0	DMACITCR	This register is used to select the Integration test mode. This register is cleared upon reset.
DMAC Base + 0x504	R/W	15-0	0x0	DMACITOP1	Control and read the DMACCLR[15:0] output lines.

Address	Type	Width	Reset Value	Register Name	Description
DMAC Base + 0x508	R/W	15-0	0x0	DMACITOP2	Control and read the DMACTC[15:0] output lines.
DMAC Base + 0x50C	R/W	1-0	0x0	DMACITOP3	Control and read the DMACINTERR, DMACINTTC output lines.
DMAC Base + 0xFE0	R	7-0	0x81	DMACPeriphId0	Peripheral ID register Bits 7:0.
DMAC Base + 0xFE4	R	7-0	0x10	DMACPeriphId1	Peripheral ID register Bits 15:8.
DMAC Base + 0xFE8	R	7-0	0x04	DMACPeriphId2	Peripheral ID register. Bits 23:16.
DMAC Base + 0xFEC	R	7-0	0x00	DMACPeriphId3	Peripheral ID register. Bits 31:24.
DMAC Base + 0xFF0	R	7-0	0x0D	DMACPCellId0	PrimeCell ID register. Bits 7:0.
DMAC Base + 0xFF4	R	7-0	0xF0	DMACPCellId1	PrimeCell ID register. Bits 15:8.
DMAC Base + 0xFF8	R	7-0	0x05	DMACPCellId2	PrimeCell ID register. Bits 23:16.
DMAC Base + 0xFFC	R	7-0	0xB1	DMACPCellId3	PrimeCell ID register. Bits 31:24.

Table 4.2-1 Memory map for the DMAC

NOTE :

1. Writes to a read-only register will not change the contents of any register. However, the DMAC AHB slave will return an OKAY response on the AHB for such accesses.

2. Similarly reads to a write-only register will not return any valid data to the AHB. The DMAC AHB slave will return 0x00000000 on to the HRDATA lines and will return an OKAY response on the AHB for such accesses.

At the topmost level, the DMAC consists of the following major blocks:

- AHB Slave Interface
- Channel Logic (instantiated twice)
- Internal Arbiter
- AHB Master Interface

In addition, the following blocks are also instantiated at the top-level of the Dmac:

- DMA Response Router
- DMA Request synchronisers
- Revision designator block

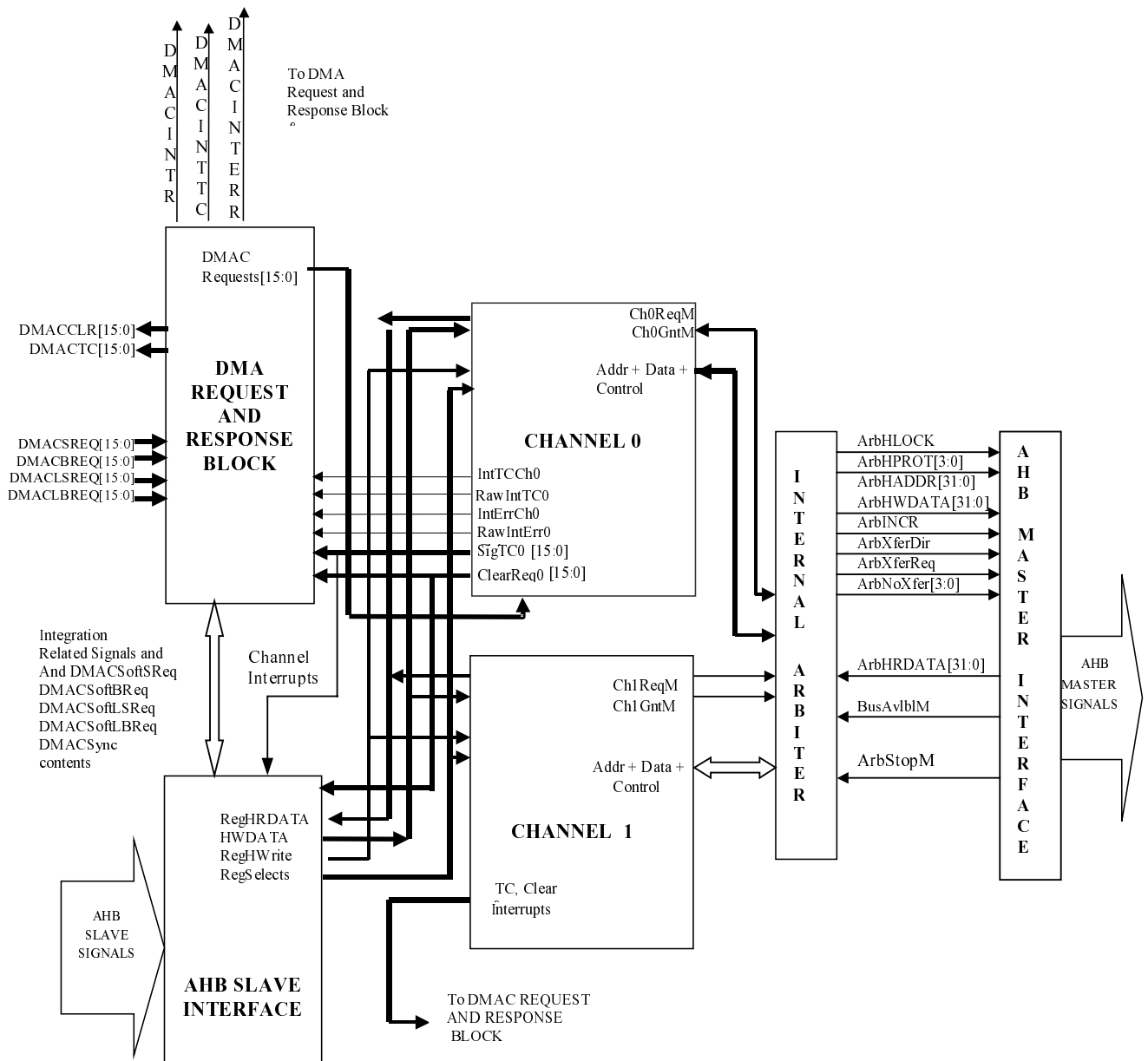
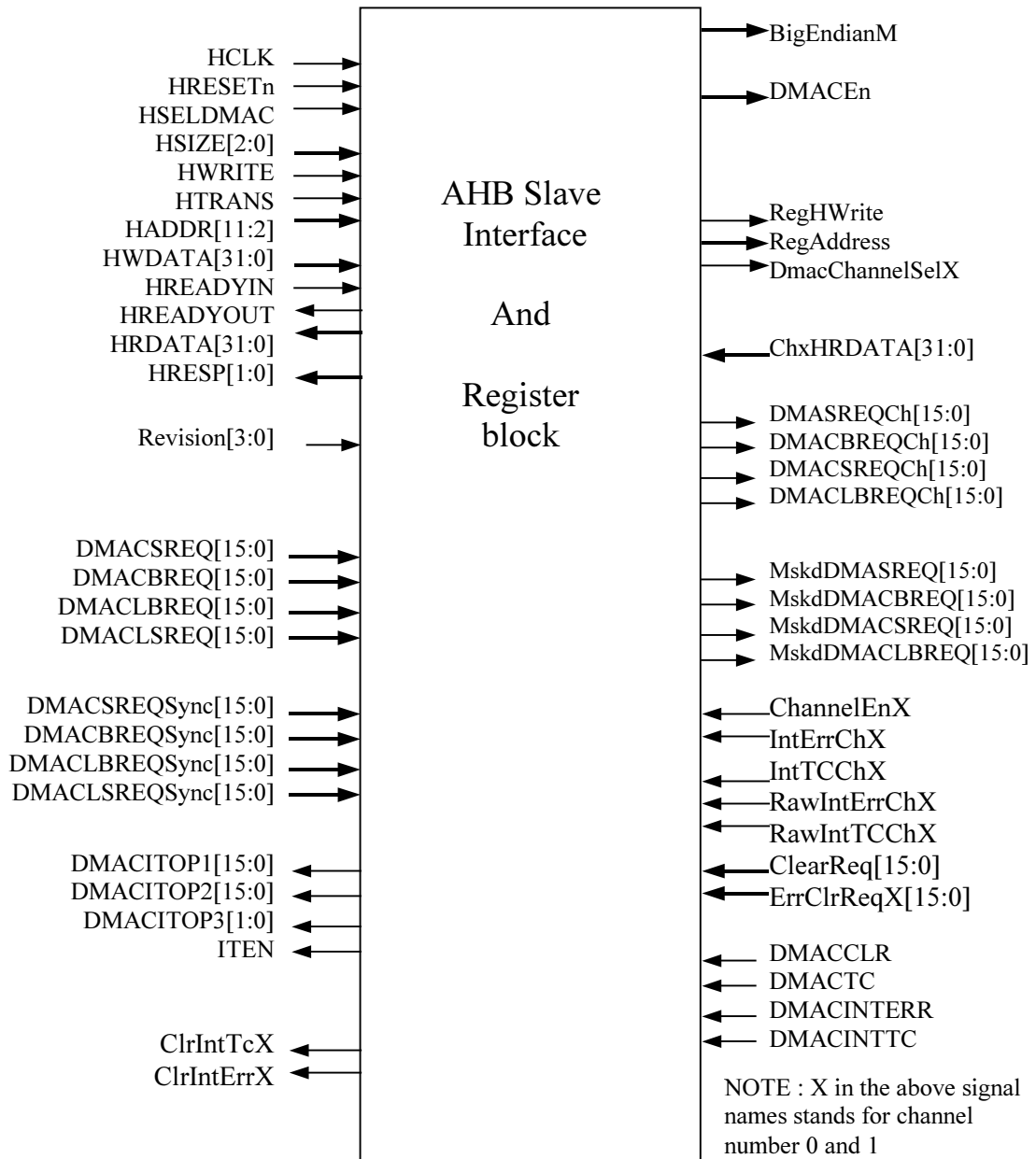


Figure 4.2-1 Top-level connectivity between the major sub blocks in the DMAC

4.3 AHB Slave Interface and Register Block

The AHB Slave interface deals with reads and writes to registers in the DMAC through the AHB. This interface is used for programming the DMAC and the DMAC channels. The port level connections of AHB slave interface module are shown in **Figure 4.3-1**.

**Figure 4.3-1 AHB Slave Interface**

This block decodes addresses and generates write enables for all the write-able registers in the module and in the channels. The write-able registers of the DMAC are as given in the register memory map in programmers model (Section 4.2). The list of registers implemented in this block is as follows.

- **Integration Test related registers:** DMACITCR, DMACITOP1, DMACITOP2, and DMACITOP3.
- **Functional registers:** DMACSoftSReq, DMACSoftBReq, DMACSoftLSReq, DMACSoftLReq, DMACConfig, DMACSync, all virtual registers such as DMACIntTCStat, DMACIntErrStat, DMACIntStat, DMACRawIntErr, DMACRawIntTC, DMACIntTCClr, DMACIntErrClr and DMACE nblDChns registers are implemented in this module.

On read, this module returns the data in the selected register on to the HRDATA bus on AHB.

The port level signal description:

Inputs

- **HCLK** : AHB Clock for the registers in the DMAC Slave module.
- **HRESETn** : AHB Reset
- **HSELDMAC** : The Slave select signal for the DMAC indicating that the DMAC Slave is selected for either a read or a write transfer.
- **HWRITE** : When high, this signal indicates that the DMAC Slave is selected for the write transfer. This signal is used to generate the individual register write signals when DMAC slave is selected.
- **HTRANS** : The signal to indicate the transfer type on the AHB. This is bit1 of HTRANS[1:0] on AHB.
- **HWDATA** : This signal vector is the data bus for write transfers from AHB to DMAC registers. Whenever the DMAC slave is selected for a write data transfer the data on this bus is written into the corresponding register in the DMAC.
- **HADDR** : This signal gives the address to which the transfer is to happen. The lower order address bits [11:2] are registered when the DMAC Slave is selected for the transfer. The registered version of the address is decoded to generate the individual select signals for the registers in the DMAC.
- **HSIZE** : This signal gives the width of the data transfer.
- **HREADYIN** : This input indicates that the previous AHB Slave on the AHB transfer has completed the transfer and the DMAC Slave can proceed if it is selected on the AHB(i.e. HSELDMAC = 1).
- **Revision** : This signal indicates the revision number of the peripheral.
- **ChXHRDATA** : This signal is the read data bus coming from the channels. All the read data buses coming from the channels are ORed with RegHRDATA to drive the final HRDATA on the AHB.
- **DMACBREQ/DMACLBREQ/DMACSREQ/DMACLSREQ** : These signals are the DMAC requests coming from the peripherals to the DMAC. These are processed before being passed onto the Channel. The DMAC requests are masked with the DMACEn bit (DMAC enable bit). Then these masked request are passed onto the double synchronisation block and the intermediate buffer. Out of these 2 sources (buffered and double synchronised) one request is selected depending on the contents of the DMACSync register. This selected request is ORed with the Soft Request register to generate the final request to the channel.
- **DMACSync/DMACLSync/DMACLSReqSync/DMACLBReqSync** : These are the double synchronised version of the DMA requests.
- **DMACCLR/DMACINTERR/DMACINTTC** : These signals are the DMACCLR/ DMACINTERR/DMACINTTC response ports going out of the DMAC. These ports are brought into this slave module for integration test read purposes.
- **ChannelEnX** : This is the channel enable status signal coming from each of the channels. This is used to return the status of the channels when DmacChEnbldChns register is read from the AHB through the AHB Slave interface.
- **IntErrChX/IntTCChX/RawIntErrChX/RawIntTCChX** : These are the Interrupt status signals from each of the channels. These are brought in the slave interface module to return the interrupt status of the channels when the corresponding interrupt registers are read from the DMAC AHB Slave interface.
- **ClearReq/ErrClrReqX** : : These are the requests coming from the channels to clear the Soft request registers in the slave interface. The ClearReq signal has higher priority as compared to a write from AHB to the Soft request registers in the Slave Interface. When 1 is detected on the corresponding bit of the ClearReq signal, the SoftReq register bit in all 4 Soft request register gets cleared.

Outputs

- **HREADYOUT** : This signal goes to the AHB from the DMAC Slave Interface, indicating the ready status of the slave interface. This signal is driven HIGH in all write transfers to the DMAC Slave Interface (thereby providing zero-wait state responses). This signal is driven LOW value for one clock cycle (to add one wait state) for all read transfers to the channel registers.

- **HRESP** : This is the response given to the master on the AHB by the DMAC Slave Interface. The DMAC Slave drives Okay response when not selected. It drives ERROR responses only when the DMAC Slave is accessed with non-word-aligned bus accesses from the AHB. For all other accesses to the DMAC, the AHB Slave interface returns Okay responses.
- **HRDATA** : This is the read data bus on the AHB. Whenever the DMAC AHB Slave is selected, The HRDATA bus is driven with data from the selected register in the DMAC, depending on the address.
- **RegHWrite** : This is the registered version of the HWRITE signal. This signal is generated by clocking HWRITE when the DMAC Slave is selected. This signal controls read/write to the channel registers through the DMAC AHB Slave Interface.
- **RegAddress** : This is the registered version of HADDR. This signal is generated by clocking HADDR when the DMAC Slave gets selected for a valid transfer. This signal is used for the selection of registers within a channel when that channel is selected (i.e. DmacChannelSel for that channel is asserted).
- **DmacChannelSelX** : This signal indicates that the address space reserved for channel X is being accessed through the AHB Slave interface.
- **MskdDMACSREQ/MskdDMACBREQ/MskdDMACLSREQ/MskdDMACLBREQ** : These are the masked versions of the raw DMAC requests coming from the external peripherals to the DMAC. The raw DMAC request are ANDed with the DMACEn bit (for power saving when the DMAC is disabled) to generate these masked requests.
- **ClrIntTCX/ClrIntErrX** : Interrupt clear signals are generated when writes occur to the Interrupt Clear registers (DmacIntTcClr/DmacIntErrClr) from the DMAC Slave Interface and the corresponding bit in the HWDATA bus is HIGH. These signals are derived combinationally (ANDing of write enable of the corresponding register and HWDATA bits) and are single clock wide signals.
- **DMACBREQch/DMACLBREQch/DMACSREQch/DMACLSREQch** : These are the DMAC requests routed to the Channels after ORing the raw DMA requests and Soft DMA requests.
- **ITEN** : Bit 0 of the DMACITCR register used to enable/disable the integration test mode.
- **DMACITOP1/DMACITOP2/DMACITOP3** : These are integration test related registers implemented in the AHB Slave interface. The contents of these registers are passed on to the DmacRspRoute module for integration test multiplexing.
- **DMACEn** : The DMAC enable/disable control bit. This is bit 0 of the DMACConfig register in the AHB Slave Interface.
- **BigEndianM** : This is the Endianess information from the DMACConfig register going to the AHB master interface.

4.4 AHB Master Interface

The AHB Master Interface instantiates 3 modules – the AHB-Lite Master Interface, the AHB-Lite-to-AHB wrapper and the Internal Arbiter. This module acts as a full AHB Master for the DMAC with support for handling all types of AHB slave responses.

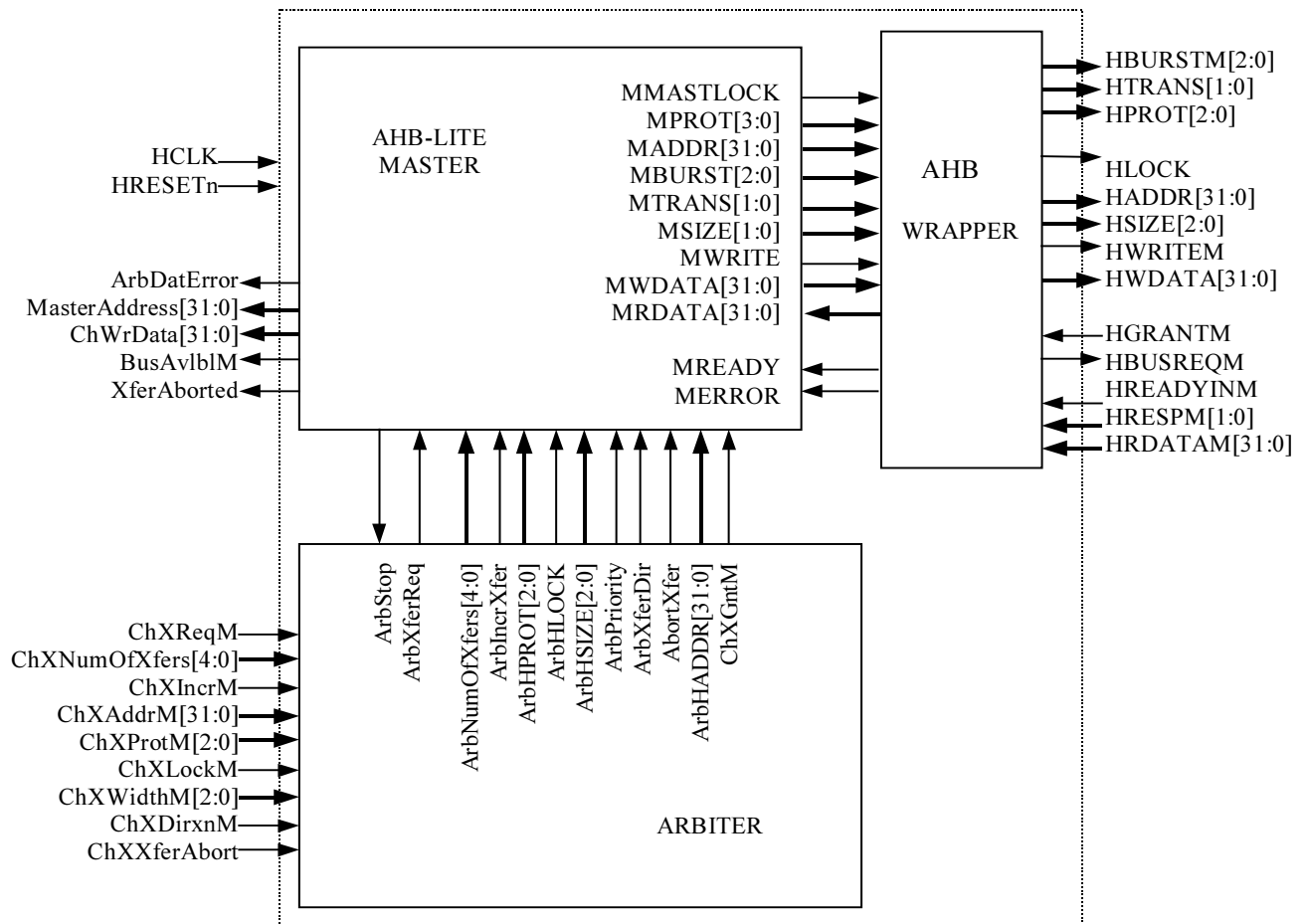


Figure 0-1 AHB Master Interface

4.4.1 DMAC Internal Arbiter

The DmacArbiter performs the following functions:

- Routing the transfer requests from the channels to the AHB-Lite Master Interface.
- Routing the abort-transfer-requests from the channels to the AHB-Lite Master Interface.
- Granting AHB access to one of the channels, if more than one channel has raised a transfer request.
- Passing the address and control information of the granted channel to the Master Interface.

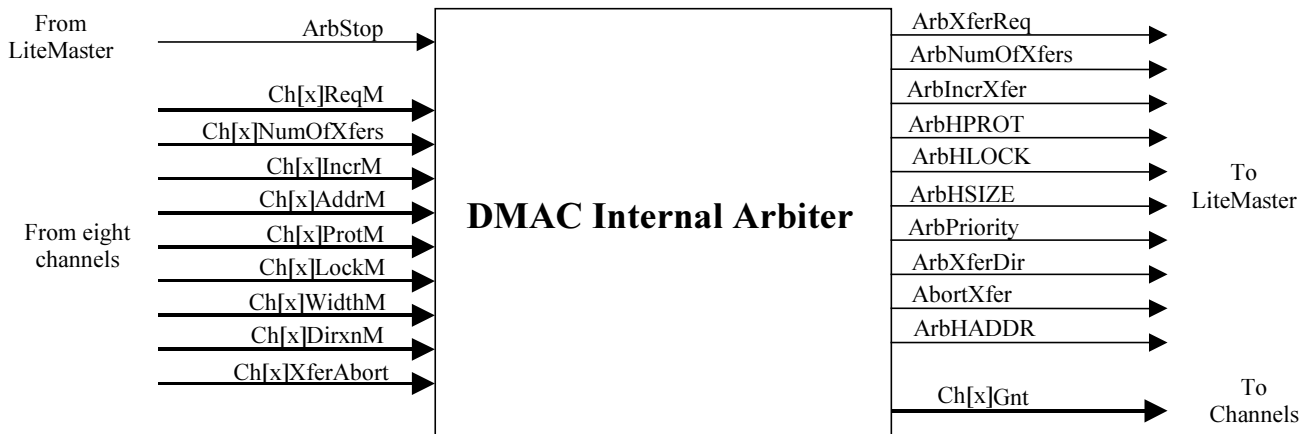


Figure 4.4-2 Block Inputs and Outputs for DmacRevAnd

(Note : Here x stands for the number of Channels)

A brief description of the interface signals of the internal-arbiter block is as follows:

Inputs

- **From LiteMaster:** **ArbStop**, when low, allows the internal arbiter to re-arbitrate and change its grant. When **ArbStop** is high, the arbiter de-grants all the channels.
- **From DMAC channels:** **Ch[x]ReqM** are the request lines, one from each of the two channels, to the arbiter. When request from a channel is high, the internal-arbiter grants the channel based on its priority over other requesting channels. Through **Ch[x]NumOfXfers**, the channels indicate the number of AHB transactions they wish to perform. **Ch[x]IncrM**, **Ch[x]AddrM**, **Ch[x]ProtM**, **Ch[x]LockM**, **Ch[x]WidthM** and **Ch[x]DirxnM** are the address and control signals from the channels, one of which is routed to the LiteMaster on the basis of internal-arbiter grant. **Ch[x]XferAbort** are transfer-abort requests from the channels.

Outputs

- **To LiteMaster:** Except **ArbPriority**, all the Arb-prefixed signals are information from one of the two channels routed to the LiteMaster on the basis of internal-arbiter grant. If channel-1(the lower priority channel in the two channel configuration) requests is granted, then the **ArbPriority** line is pulled low indicating that it is a 'low' priority channel (Low priority channel is not allowed to saturate the AHB). If high priority channel is granted, then **ArbPriority** is pulled high.
- **To DMAC channels:** **Ch[x]Gnt** represent 2 grant lines to the channels. If a channel request is high and according to the fixed priority scheme, the channel of highest priority gets the grant.

4.4.2 AHB-Lite Master Interface

This module implements the Master Interface of the DMA controller and performs all the DMA AHB accesses for the channels. The port level connections of the AHB-Lite Master Interface are shown in **Figure 4.4-3**. The ports **ArbxHLOCK** and **ArbxHPROT** are inputs from the arbiter and are used by the master, to drive the **HLOCK** and **HPROT** information on to the AHB. **HWDATA** is combinationally driven by ORing the data buses from the channels (**ChxHWDATA**). The AHB-Lite Master Interface does not support **SPLIT** and **RETRY** responses from the AHB Slave. To make it a full AHB Master, the AHB-Lite-to-AHB wrapper is used.

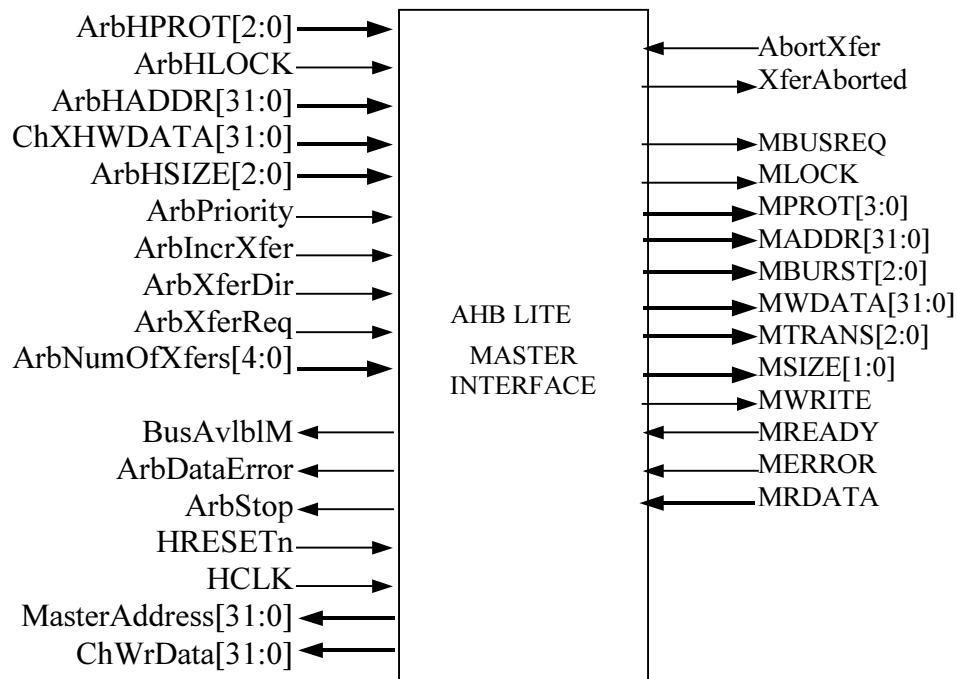


Figure 4.4-3 Block Inputs and Outputs for DmacLiteMaster

Other interface signals between the master and internal channel arbiter and their use in the master/arbiters is as follows:

Inputs

- **HCLK/HRESETn** : AHB clock and reset signal
- **MREADY** : This signal indicates to the AHB-Lite master interface that the AHB bus is available for use and it can drive the control/data signals onto the AHB. All the state transitions occur on detecting MREADY high in the AHB-Lite master interface.
- **MERROR** : This signal indicates that the error response is received from the AHB slave. On detecting this signal HIGH AHB-Lite aborts the current burst of transfer, gives signal for re-arbitration to the internal arbiter. Then on next clock it re-starts the new burst of transfer.
- **MRDATA** : This is the HRDATA bus routed through the wrapper.
- **BigEndianM** : This information is used for endianizing the data coming out from the channel to the HWDATA bus through MWDATA (wrapper). A HIGH on this bit indicates that the data bus is BigEndian.
- **ChXHWDATA** : The write data buses from both the channels. All the data from the channels for this AHB port (to which this master corresponds) are ORed to generate the final MWDATA (in effect, HWDATA).
- **ArbHLOCK** : This information is used for initiating locked transfers on the AHB. The value on this signal is passed onto the MLOCK (in effect HLOCK on the AHB) for all AHB transfers for the current request from the channels.
- **ArbHPROT** : This information is driven onto the MPROT (in effect HPROT) lines.
- **ArbHSIZE** : This indicates the width of the data transfer. The width of the transfer can be WORD, HALFWORD or BYTE. This information is also used for incrementing the address on the MADDR lines for subsequent transfers on the AHB.
- **ArbHADDR** : This is the address with which the AHB master starts transfers on the AHB, when the internal arbiter requests the data bus.

- **ArbIncrXfer** : Used to indicate whether the transfer is incrementing or non-incrementing. Depending on the value of this bit, NSEQ or SEQ is put on the AHB.
- **ArbPriority** : This signal is used for giving information regarding the priority of the channel requesting the DMA access.
- **ArbXferDir** : Used to indicate whether the access is read or write
- **ArbXferReq** : This is the actual transfer request, which will be acted upon by the master. This is the ORing of the internal grants given by the Internal Arbiter to the channels. This signal is driven by the arbiter on seeing the request from the channels.
- **ArbNumOfXfers** : This indicates the number of transfers to be done by the master for the particular ArbXferReq
- **AbortXfer** : This signal is driven by the internal arbiter to the master to stop the current burst of transfer which was requested by the internal arbiter. The Master Interface will complete the current committed AHB burst on the AHB, and assert the XferAborted signal to the Internal Arbiter.

Outputs

- **MWDATA** : The write data buses from both the channels (ChXHWDATA) are ORed together and then passed on as MWDATA. One level of internal buffering for MWDATA is also present to take care of AHB transfers with wait states and to avoid affecting the channel flow.
- **MLOCK** : The value on ArbHLOCK is latched onto the MLOCK line for all requests from the internal arbiter.
- **MPROT** : The value on ArbHPROT is latched onto the MPROT lines for all requests from the internal arbiter.
- **MBURST** : This signal indicates the type of burst (number of transfers) for the current burst. The DMAC supports only INCR, INCR4, INCR8 and INCR16 types of burst.
- **MTRANS** : The type of transfer (NSEQ or SEQ) depends on the mode of the current transfer (First-in-the-burst / not-the-first-in-a-burst or Incrementing / NonIncrementing)
- **MADDR** : This is address to which the Master intends to do the next transfer. This is driven as HADDR from the wrapper.
- **MSIZE** : This signal indicates the size of the next transfer.
- **MWRITE** : For write transfers this signal is driven HIGH, and for read it is driven LOW.
- **BusAvlBlM** : This signal is driven by the AHB Master Interface to the Internal Arbiter to indicate the completion of the current transfer. In case of destination transfers, this information is used by the arbiter for giving out the next data on the ArbHWDATA lines. In case of source transfers, this information is used by the internal arbiter/channel to register the in-coming data from ArbHRDATA.
- **XferAborted** : This signal is used for hand-shaking between the master interface and the internal arbiter when the channel is disabled (through the AHB Slave interface) even as its transfer is in progress on the AHB Master port. When the master interface detects the AbortXfer signal from the internal arbiter, it completes the current burst which was committed on to the AHB and then asserts the XferAborted signal.
- **ArbStop** : This signal is given to the internal arbiter to stop the arbitration. When this signal is LOW, internal arbiter re-arbitrates between any pending channel requests for AHB accesses. The AHB-Lite master interface pulls this signal low when the last transfer from the previous request, is in progress.
- **ArbDataError** : If the AHB Master Interface gets an error response from an AHB slave, it asserts the ArbxDatError signal to the Internal Arbiter.
- **MasterAddress** : This signal is used for updating the source and destination address registers in the DMAC channels. This signal has the same timings as MADDR with the difference being in its value at the start of a new AHB access, when it holds the incremented value of the previous access address. The channel uses the information on this bus to update its registers when the BusAvlBl signal is high.
- **ChWrData** : This is the write data bus to the channel. The HRDATA is first buffered into the register, then it is converted into the little endian format. This endianised data is ChWrData. Thus channel uniformly assumes the little endian format independent of the Endianness of the Master(i.e. AHB on which it is sitting).

4.4.3 AHB-Lite-to-AHB wrapper

This block is used to convert the AHB-Lite master into a full AHB Master by adding support for all AHB Slave responses (including SPLIT and RETRY). The port level connections are as given in the following diagram.

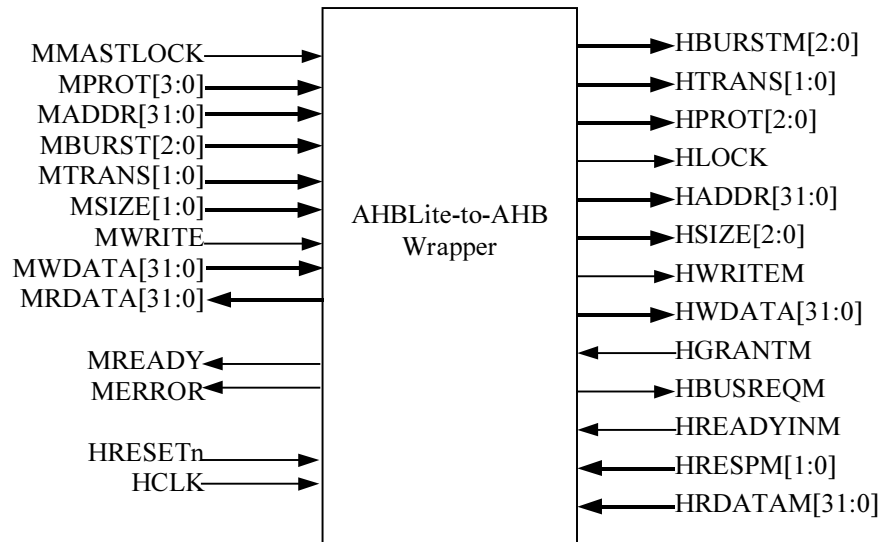


Figure 4.4-4 Block Inputs and Outputs for DmacMasterWrap

The Wrapper function is described in AHB Example Bus Master Technical Reference Manual [Ref. 8].

4.5 Channel Logic

This block is the top level of the DMAC Channel. This block instantiates the following functional sub-blocks:

- DmacChRegBlock (mainly contains the logic for inferring of channel registers and bits)
- DmacChSrcXfer (handles the AHB transfers during source-to-DMAC transfer operation)
- DmacChDstXfer (handles the AHB transfers during DMAC-to-destination transfer operation)
- DmacChLLILoad (handles the AHB transfers during LLI loading operation)
- DmacChPckUnpck (contains FIFO data packing-unpacking logic)
- DmacChReqProc (DMAC Channel request processor)
- DmacChReqMask (DMAC Channel request mask)

In the following subsections, each of the functional sub blocks are described briefly along with a pin level module diagram and description of key-signals.

4.5.1 DMAC Channel Register Block

This block contains the internal channel registers namely, source-address-register, destination-address register, control register, LLI-load-register and channel-configuration register. Apart from these registers, the DMAC channel register block contains counters like TrfSizeDst (transfer-size in terms of destination width) and other internal flag bits. This block also splits the bit-fields of DMACCxControl register, DMACCxLLIReg register and DMACCxConfig register [x = 0 and 1 (channel number)] and generates the write-enable signals for DMAC channel registers.

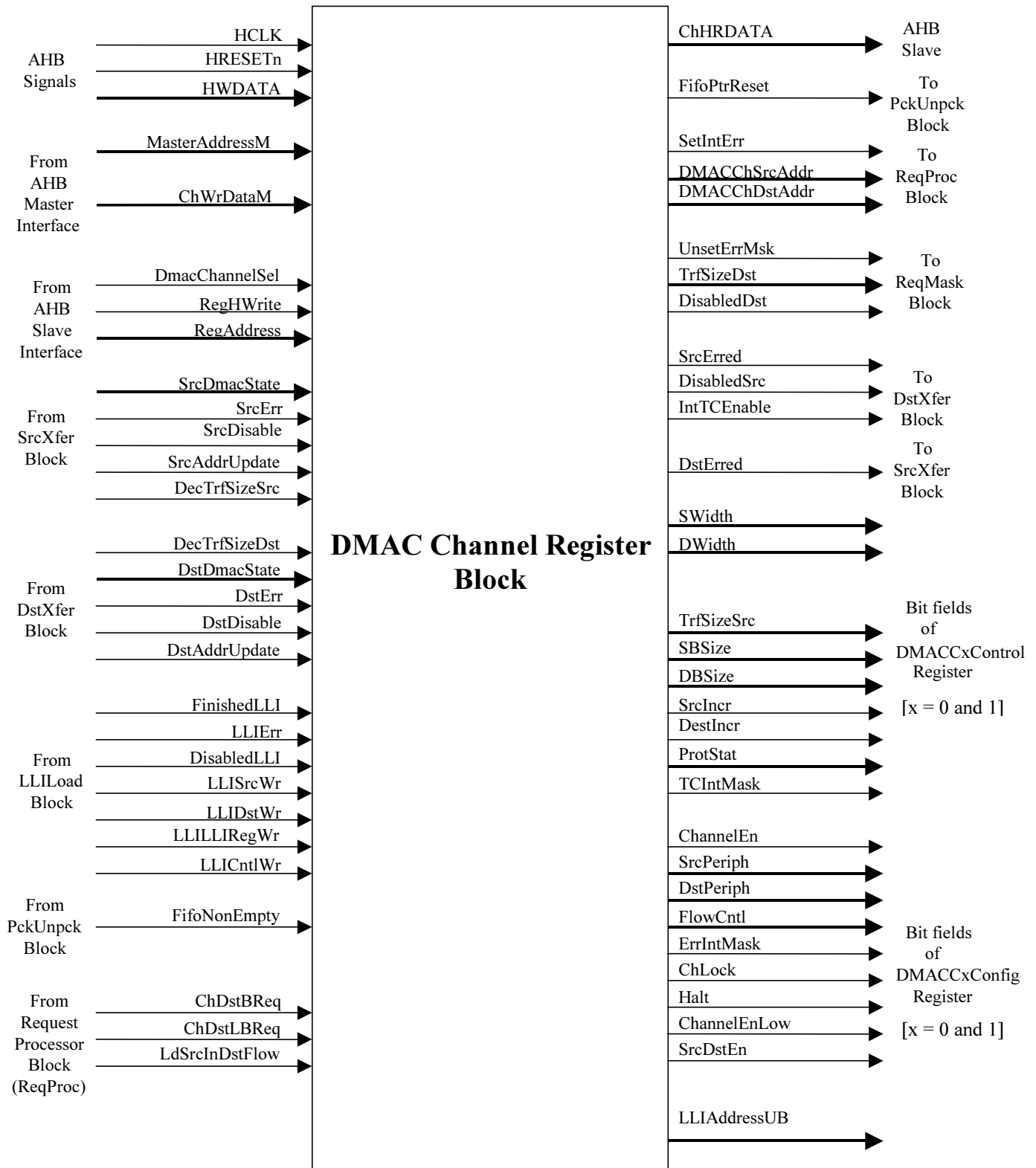


Figure 4.5-1 Block Inputs and Outputs for DmacChRegBlock

Inputs

- **AHB signals:** Apart of HCLK and HRESETn, HWDATA (of the slave side AHB) is an input to the module. HWDATA is used for updating the channel register contents during programming of DMAC.
- **From AHB Master Interface (DmacAhbMaster) :** MasterAddressM is the input from the AHB master port of DMAC. This signal is copy of HADDRM which is fed back internally. MasterAddressM is used by the source/destination address registers to update themselves with the latest address transacted on Master AHB. ChWrDataM is the Master AHB data port, which transfers the new channel-register-values during LLI load operation.
- **From AHB Slave Interface (DmacAhbSlavelf) :** The signals from the slave interface are used for writing into the channel registers during DMAC programming.
- **From DMAC SrcXfer Block (DmacChSrcXfer) :** SrcAddrUpdate is used to update the DMACChSrcAddr register with DMAC AHB master addresses (fed back into the DMAC as MasterAddrM). DesTrfSizeSrc is used by the TrfSizeSrc counter to downcount. SrcDmacState, SrcErr and SrcDisable signals are used (along with other signals) to make ChannelEn bit zero.
- **From DMAC DstXfer Block (DmacChDstXfer) :** DstAddrUpdate is used to update the DMACChDstAddr register with DMAC AHB master address (fed back into the DMAC as MasterAddrM). DecTrfSizeDst is used by the TrfSizeDst counter to downcount (TrfSizeDst stores the number-of-transfers-remaining, in terms of destination width). DstDmacState, DstErr and DstDisable signals are used (along with other signals) to make ChannelEn bit zero. ChannelEn is a control-bit which also doubles up as a status bit. A write-of-zero in ChannelEn while programming, is taken as a trigger to disable the channel. The ChannelEn is actually pulled low, only after channel is inactive. (Src/Dst)DmacState, (Src/Dst)Err and (Src/Dst)Disable are used for ensuring that shutting-down of channel is complete.
- **From LLI Load Block (DmacChLLILoad) :** LLISrcWr, LLIDstWr, LLILLIRegWr and LLICntlWr are used to update the respective channel registers during LLI loading operation. DisabledLLI and LLIErr are used (along with other signals) to make ChannelEn bit zero. FinishedLLI indicates end of LLI loading operation and is used by the register-block to reset the FIFO pointers.
- **From Packing-Unpacking block (DmacChPckUnpck) :** FifoNonEmpty is a status bit-field in DMACChConfig register. This is required when DMACChConfig register is read back from the AHB slave interface.
- **From Request Processor Block (DmacChReqProc) :** LdSrcInDstFlow and ChDst(L)BREQ are used to load the TrfSizeSrc counter in Destination-Flow-Control mode. Please note that the TrfSizeSrc counter is only reused in Destination-Flow-Control mode. Otherwise a value programmed into TrfSize has got no meaning in this flow-control-mode.

Outputs

- **To AHB Slave (DmacAhbSlavelf) :** ChHRDATA returns the channel-register contents, when read back by the AHB slave interface.
- **To Packing Unpacking block (DmacChPckUnpck) :** FifoPtrReset is asserted whenever the channel-enable goes low or LLI loading finishes before the start of new packet.
- **To request processor block (DmacChReqProc) :** SetIntErr, is asserted when the channel gets an error response. DMACChSrcAddr and DMACChDstAddr are the channel-source/destination-address-register contents.
- **To request mask block (DmacChReqMask) :** UnsetErrMsk is asserted when the channel is disabled after an error response
- **To DstXfer Block (DmacChDstXfer) :** SrcErred is asserted whenever SrcErr is high and is pulled low when the source state machine goes to idle state. Similarly DisabledSrc is asserted when SrcDisable is high and is pulled low when the source-machine goes to idle state. Note that SrcErr and SrcDisable are input to DMAC channel register block.
- **Bit fields of channel-registers :** The bit-fields of the channel-registers are split in this block and sent out.

4.5.2 DMAC Channel Source Transfer Block

This block mainly describes the Source-DMAC-data-transfer state machine. It also generates SrcFactor, which is an optimisation factor in pipelined mode of operation.

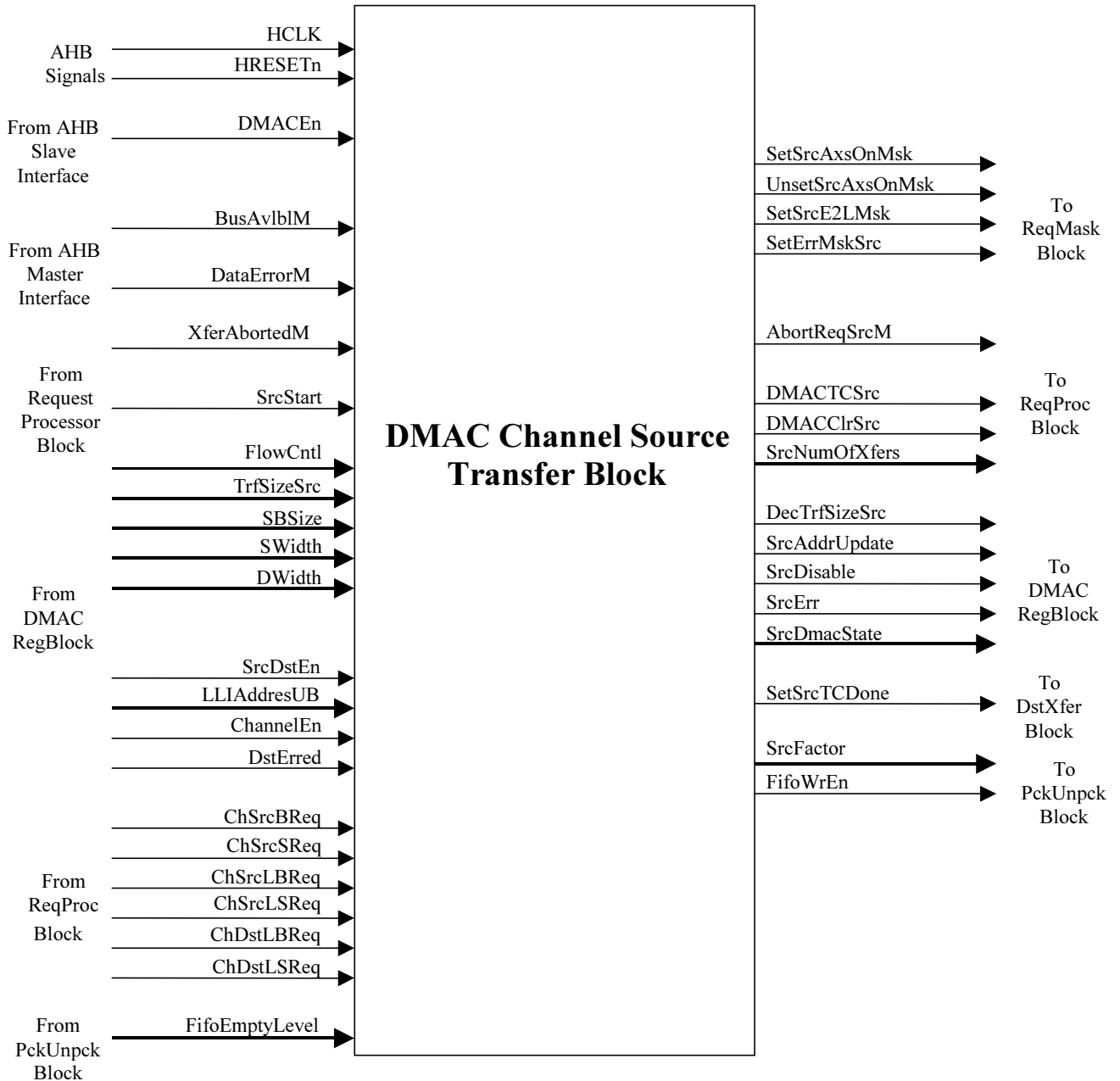


Figure 4.5-2 Block Inputs and Outputs for DmacChSrcXfer

Inputs

- **From AHB Slave Interface:** A low on DMACEn is a global-disable for the DMAC. When DMACEn is zero, the source-DMAC-state-machine does not start-off, even if all the triggers-to-start are present.
- **From AHB master interface:** BusAvlblM indicates that the valid data had appeared on the bus. XferAbortedM is an acknowledge-signal in response to abort-request from the channel. When a channel is midway through its transfer and the software disables the channel, the disabling may not take effect

immediately. Instead, the channel raises an abort-request to the AHB-master-interfaces; the request is acknowledged by XferAbortedM.

- **From DMAC channel register block:** The source-state-machine in this block handles the transfers for all types of programmed variables like transfer-sizes, all types of flow-control configuration etc. Depending on the variables, different actions are taken on the state transitions. This requires all the signals that are shown coming from DMAC register block.
- **From request processor block:** SrcStart indicates that the source-DMAC-state machine can start provided DMACEn and ChannelEn are high. Depending on whether ChSrcBReq or ChSrcSReq is asserted, a burst of data or a single data is fetched from the source. If last requests are high (ChSrcLBReq or ChSrcLSReq) in a suitable flow control mode, the packet is terminated. ChDstLSReq or ChDstLBReq are required by the source-state-machine in Destination-Flow-Control mode.
- **From packing unpacking block:** FifoEmptyLevel tells the source-state-machine about the limit of source-accesses that can be done. If there is not a single empty space in FIFO, in spite of requests, the source-machine cannot start off.

Outputs

- **To request mask block:** SetSrcAxsOnMsk is a one-clock-wide signal, asserted at the start of any source access. On the other hand, UnsetSrcAxsOnMsk is asserted whenever a source access terminates, be it during a normal termination or abnormal abort operation. SetSrcE2LMsk is asserted at the end of source transfers for a packet only if there are more LLI to load. SetErrMskSrc is asserted when an error is encountered on source AHB.
- **To request processor block:** AbortReqSrcM is asserted if software attempts to disable a channel, when an AHB source access is in progress. DMAC(TC/Clr)Src are asserted at the end of an AHB access qualified with other conditions (like, if a DMA burst is completed DMACClrSrc is asserted and if a packet finishes, DMACTCSrc is also asserted). DMAC(TC/Clr)Src are de-asserted after the target peripheral pulls down the DMA request. SrcNumOfXfers is the minimum of two values - data-that-FIFO-can-accommodate and data-that-is-required-from-source.
- **To DMAC register block:** DecTrfSizeSrc and SrcAddrUpdate are asserted at the completion of each source-data-transfer for one clock period duration. SrcErr is asserted with the second cycle of AHB error response, if the error occurs for the channel source transfers. SrcDisable indicates that the packet has been terminated (may be normally or abnormally).
- **To DstXfer block:** SetSrcTCDone is asserted only in source-flow-control mode when the TC has been given by the DMAC to the source peripheral.
- **To packing unpacking block:** FifoWrEn is asserted after every successful intake of source data. SrcFactor is an optimisation factor generated at the end of source-accesses so that the destination access starting in pipeline immediately afterwards can start with a burst of higher value.

4.5.3 DMAC Channel Destination Transfer Block

This block mainly describes the Destination-DMAC-data-transfer state machine. It also generates DstFactor which is an optimisation factor for pipelined mode of operation.

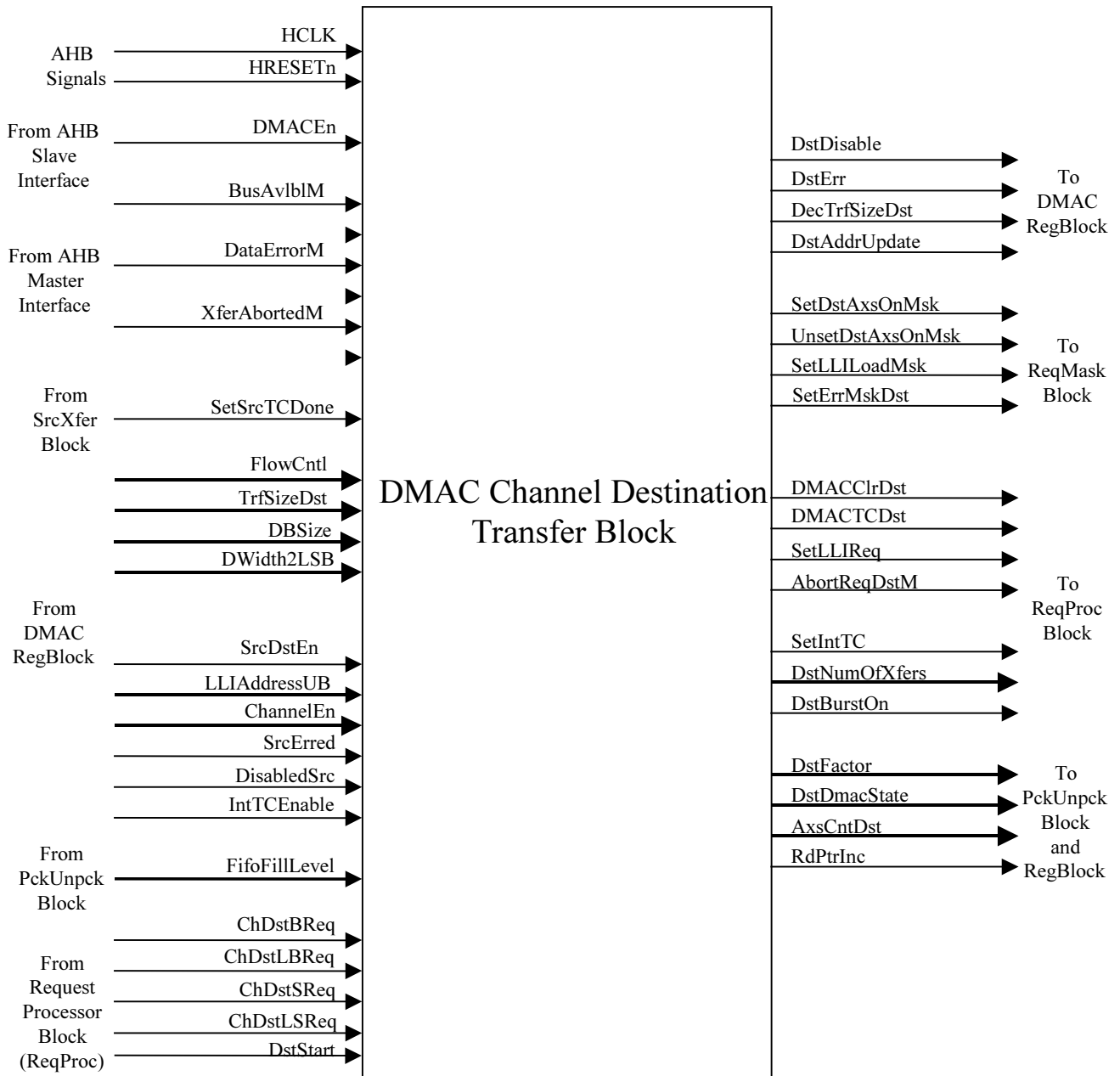


Figure 4.5-3 Block Inputs and Outputs for DmacChDstXfer

Inputs

- **From AHB slave interface:** A low on DMACEn is a global-disable for the DMAC. When DMACEn is zero, the destination-DMAC-state-machine does not start-off, even if all the triggers-to-start are present.
- **From AHB master interface:** BusAvlBlM indicates that the data driven by the DMAC has been 'consumed'. XferAbortedM is an acknowledge-signal in response to abort-request from the channel. When a channel is midway through its transfer and the software disables the channel, the disabling does not take effect immediately. Instead, the channel raises an abort-request to the AHB-master-interfaces; the request is acknowledged by XferAbortedM.
- **From SrcXfer block:** In source flow control mode, the destination raises TC only when it 'knows' that the source TC has been given (In case of DMAC-Flow-Control, there is a transfer-size value to indicate that the

packet is completed. In case of Destination-Flow-Control, the LBReq or LSReq indicates the end of packet). This information of 'source-TC-been-given' is conveyed in by SetSrcTCDone.

- **From DMAC channel register block:** The destination-state-machine in this block handles the transfers for all types of programmed variables like transfer-sizes, all types of flow-control configuration etc. Depending on the programmed variables, different actions are taken on state transitions. This requires all the signals, that are shown coming from DMAC register block.
- **From packing unpacking block:** FifoFillLevel tells the destination-state machine the limit of destination-accesses that can be done. If there is not a single data in FIFO, in spite of requests, the destination-machine cannot start off.
- **From request processor block:** DstStart indicates that the destination-DMAC-state machine can start provided DMACEn and ChannelEn are high. Depending on whether ChDstBReq or ChDstSReq is asserted, a burst of data or a single data is given to the destination. If last requests are high (ChDstLBReq or ChDstLSReq) in suitable flow control mode, the packet is terminated.

Outputs

- **To DMAC register block:** DecTrfSizeDst and DstAddrUpdate are asserted for one clock period duration at the completion of each destination-data-transfer. DstErr is asserted with the second cycle of AHB error response, if the error occurs for the channel destination transfers. DstDisable indicates that the packet has been terminated (may be normally or abnormally).
- **To request mask block:** SetDstAxsOnMsk is a one-clock-wide signal, asserted at the start of any destination access. On the other hand, UnsetDstAxsOnMsk is asserted whenever a destination access terminates, be it during a normal termination or abnormal abort operation. SetLLILoadMsk is asserted at the end of packet if there is a new LLI to load. AbortReqDstM is asserted if the software attempts to disable a channel, when an AHB destination access is in progress. SetErrMskDst is asserted when an error is encountered source AHB.
- **To request processor block:** . DMAC(TC/Clr)Dst are asserted at the end of an AHB access qualified with other conditions (like, if a DMA burst is completed DMACClrDst is asserted and if some packet finishes, DMACTCDst is also asserted). DMAC(TC/Clr)Dst are de-asserted after the target peripheral pulls down the DMA request. SetLLIReq is asserted after the completion of packet transmission only if the DMACChLLIReg contains a non null value. SetIntTC is asserted at the end of a packet transmission only if IntTCEnable (TC interrupt enable) is high. DstNumOfXfers is the minimum of two values - data-that-FIFO-contains and data-that-is-required-by-destination. DstBurstOn indicates that the destination burst (time between raising of DMA request and clearing of the request with DMAClr/TC) is continuing.
- **To packing-unpacking block:** DstFactor is an optimisation factor generated at the end of destination-accesses so that the source access starting in pipeline immediately afterwards can start with a burst of higher value. RdPtrInc is asserted with every successful consumption of data driven by DMAC. AxsCntDst is a 5 bit vector indicating the number of destination-accesses remaining to be done on destination AHB.

4.5.4 DMAC Channel LLI Load Block

The main functionality of this block is to handle the AHB transfers during LLI loading operation. It also clocks out LLIFinish and LLIDisable.

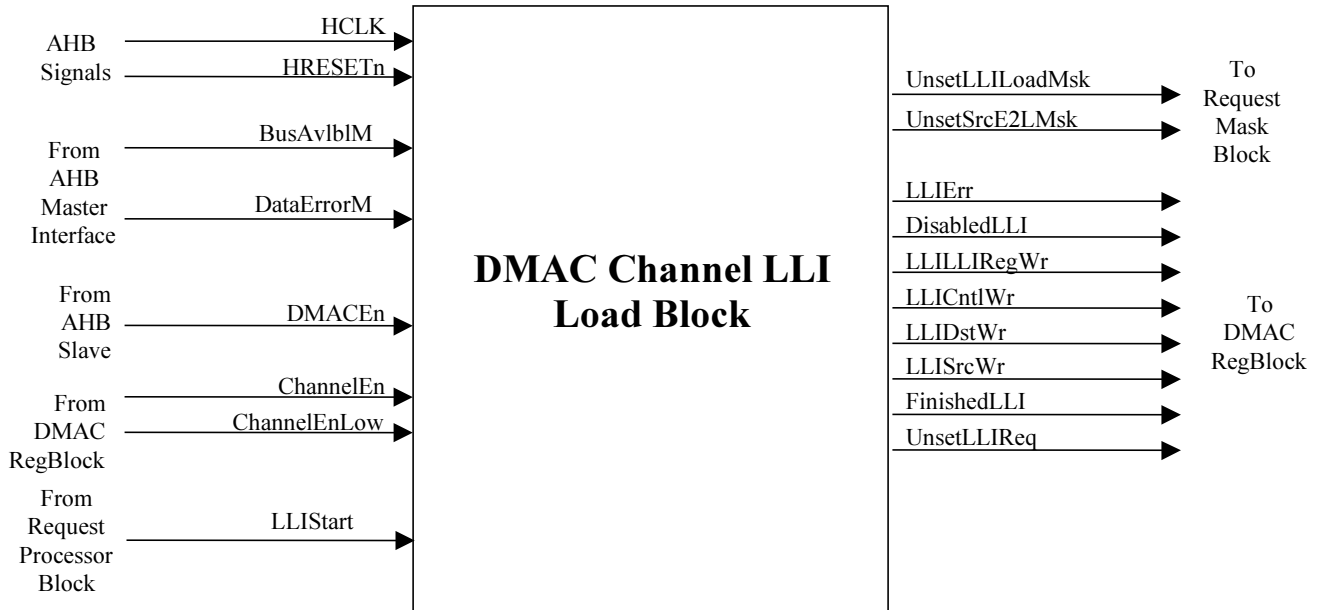


Figure 4.5-4 Block Inputs and Outputs for DmacChLLILoad

Inputs

- **From AHB master interface:** BusAvlBlM indicates that valid data has appeared on the bus. DataErrorM is asserted during the second cycle of error response on the LLI-load-AHB.
- **From AHB slave:** A low on DMACEn is a global-disable for the DMAC. When DMACEn is zero, the LLI-load-state-machine does not start-off, even if all the triggers-to-start are present.
- **From DMAC channel register block:** ChannelEn is one of the bit fields of DMACChConfig register. ChannelEnLow is asserted for a clock whenever the ChannelEn bit is being written with zero from AHB slave side. ChannelEnLow is used to disable the LLI loading if the state-machine is yet to move out of its IDLE state.

Outputs

- **To request mask block:** UnsetLLILoadMsk is asserted for a clock when the LLI loading process finishes or terminates abnormally due to an error response. UnsetSrcE2LMsk is asserted for a clock when the LLI-load-machine moves out of its IDLE state.
- **To DMAC channel register block:** LLIErr is asserted when the LLI loading is aborted due to an error response on LLI-loading-AHB (indicated by DataErrorLLIM being high). LLIDisable is asserted high for one clock if software disables the channel and the LLI-load-machine is in its IDLE state. DisabledLLI is a clocked version of LLIDisable. LLISrcWr, LLIDstWr, LLICntlWr and LLILLIRegWr are asserted as and when the data for source-address-register, destination-address-register, channel-control-register and channel-LLI-register arrives. FinishedLLI is a clocked version of LLIFinish. LLIFinish is asserted when the LLI loading operation terminates normally after fetching all the four data-words for the channel-registers. UnsetLLIReq is asserted when the LLI loading operation starts on the AHB master.

4.5.5 DMAC FIFO Packing-Unpacking Block

This block contains packing logic for channel-FIFO-write-data and unpacking logic for channel-FIFO-read-data. The FIFO is a 32-bit-wide FIFO. If there are narrow source data accesses, the data needs to be 'packed' into the FIFO for optimal usage. Similarly if narrow destination data accesses are going on, then only a portion of a FIFO word is to be extracted – this is called unpacking. Apart from packing-unpacking, this block also calculates the fill-level and empty level of the FIFO.

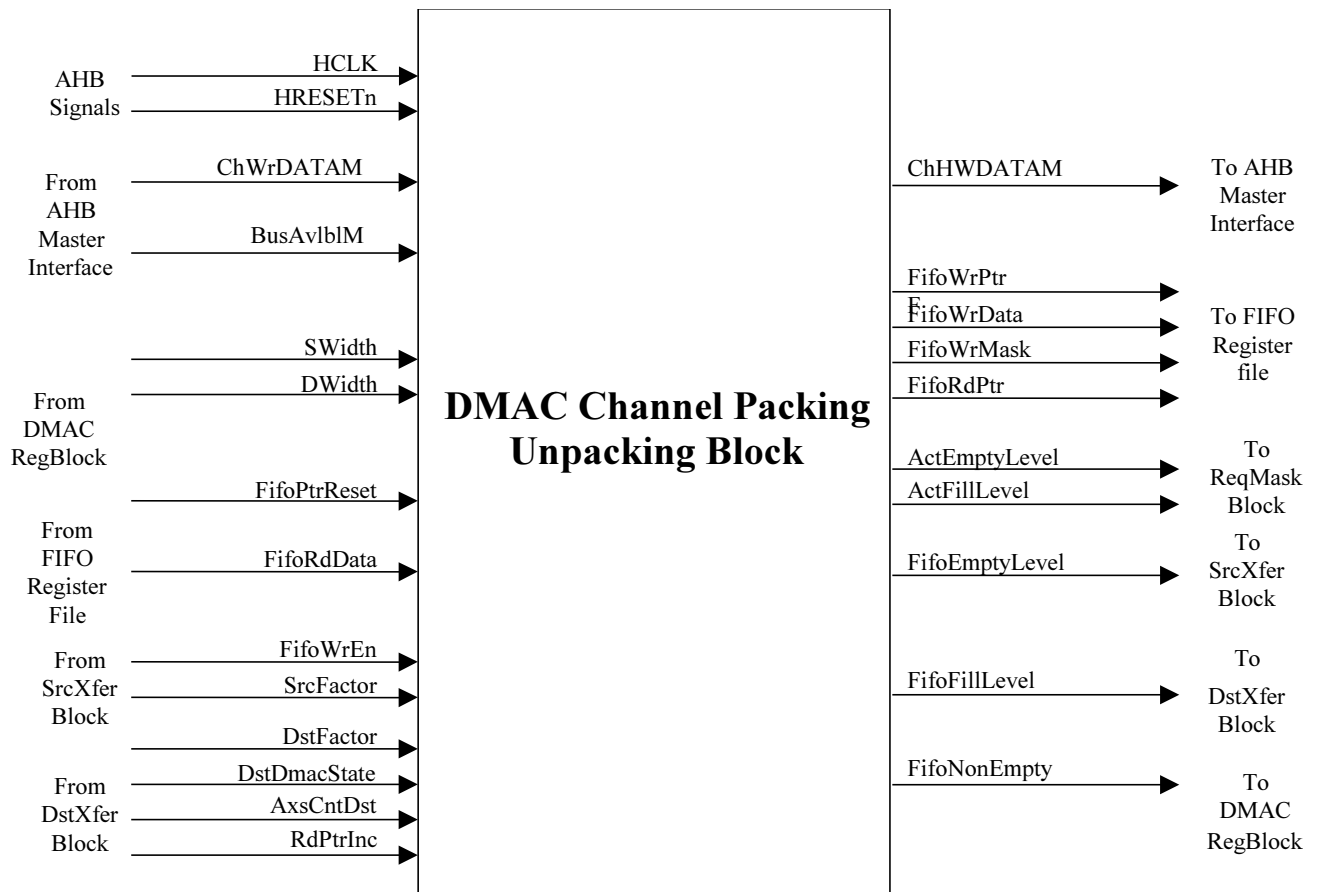


Figure 4.5-5 Block Inputs and Outputs for DmacChPckUnpck

Inputs

- **From AHB master interface:** ChWrDataM lines are the data input to the channel-FIFO. During source-data-fetch operations, the ChWrDataM contents are written into the channel-FIFO. BusAvlblM indicates that valid data has appeared on the bus.
- **From DMAC channel register block:** SWidth and DWidth values are used for packing and unpacking. These are also used to calculate the (empty/fill)level in terms of source and destination width. When FifoPtrReset is asserted, the FIFO pointers (read and write pointers and sub-pointers) are reset to "00". FIFO-pointer-reset is typically done when channel is disabled. This is helpful because next time when the channel is programmed, the data-width to be transferred might not be the same as previous case. It is then difficult to increment the FIFO pointers, left unaligned by the previous channel-enable session.
- **From channel SrcXfer block:** Assertion of FifoWrEn indicates to the packing-unpacking logic to move a data into the channel-FIFO. SrcFactor is a prediction factor generated by the SrcXfer block only if source and destination are on the same AHB. This factor is added to the actual FIFO fill level to show a higher fill-level to destination-transfer-block. This enables the destination-transfer-block to initiate a transfer of greater-burst-length. This does not lead to data corruption, as the pipeline nature of the bus ensures that by the time the destination data needs to be driven out on the bus, the FIFO would have been filled to the requisite level by source.
- **From channel DstXfer block:** DstFactor is a prediction factor generated by the DstXfer block only if source and destination are on the same AHB. This factor is added to the actual FIFO level to show a higher empty-level to source-transfer-block. This enables the source-transfer-block to initiate a transfer of greater-burst-length. This does not lead to data corruption, as the pipelined nature of the bus ensures that by the time the source data arrives, the FIFO will be drained out. AxsCntDst indicates the number of destination accesses

remaining to be done on AHB. The maximum value of AxsCntDst can be 16 (when FIFO is full and the destination transfer size is byte wide).

Outputs

- **To AHB master interface:** ChHWDATAM contains the data which is extracted from the FIFO and which is to be written into the destination. If the destination width is narrower than 32 bits, then ChHWDATAM contains the data in the lower byte lane(s). When no data transfers are being done to the destination, ChHWDATAM is driven to all-zeroes.
- **To channel FIFO register file:** FifoWrPtr and FifoRdPtr are the two-bit write and read pointers for the four-deep FIFO. FifoWrData is a 32-bit wide signal. If the 'data' to be written into the FIFO is narrower than 32 bits, then the narrow data is placed on the byte-lane indicated by write sub-pointer value. FifoWrMask is a signal, generated along with FifoWrData. FifoWrMask is also a 32 bit wide signal in which the bits corresponding to byte-lane(s) carrying data (in case of narrow destination width) are marked by '1's.
- **To request mask block:** ActFillLevel and ActEmptyLevel are the conventional fill/empty level of the channel FIFO. The optimisation factors – SrcFactor and DstFactor, are not added to these fill levels.
- **To channel SrcXfer block:** FifoEmptyLevel is ActEmptyLevel added with DstFactor.
- **To channel DstXfer block:** FifoFillLevel is ActFillLevel added with SrcFactor.
- **To DMAC channel register block:** If the FIFO is not completely empty, FifoNonEmpty signal is high. Otherwise, the signal is zero.

4.5.6 DMAC Channel Register File

This block contains the bank of flip-flops which constitute the register-file of the FIFO.



Figure 4.5-6 Block Inputs and Outputs for DmacChRegFile

Inputs

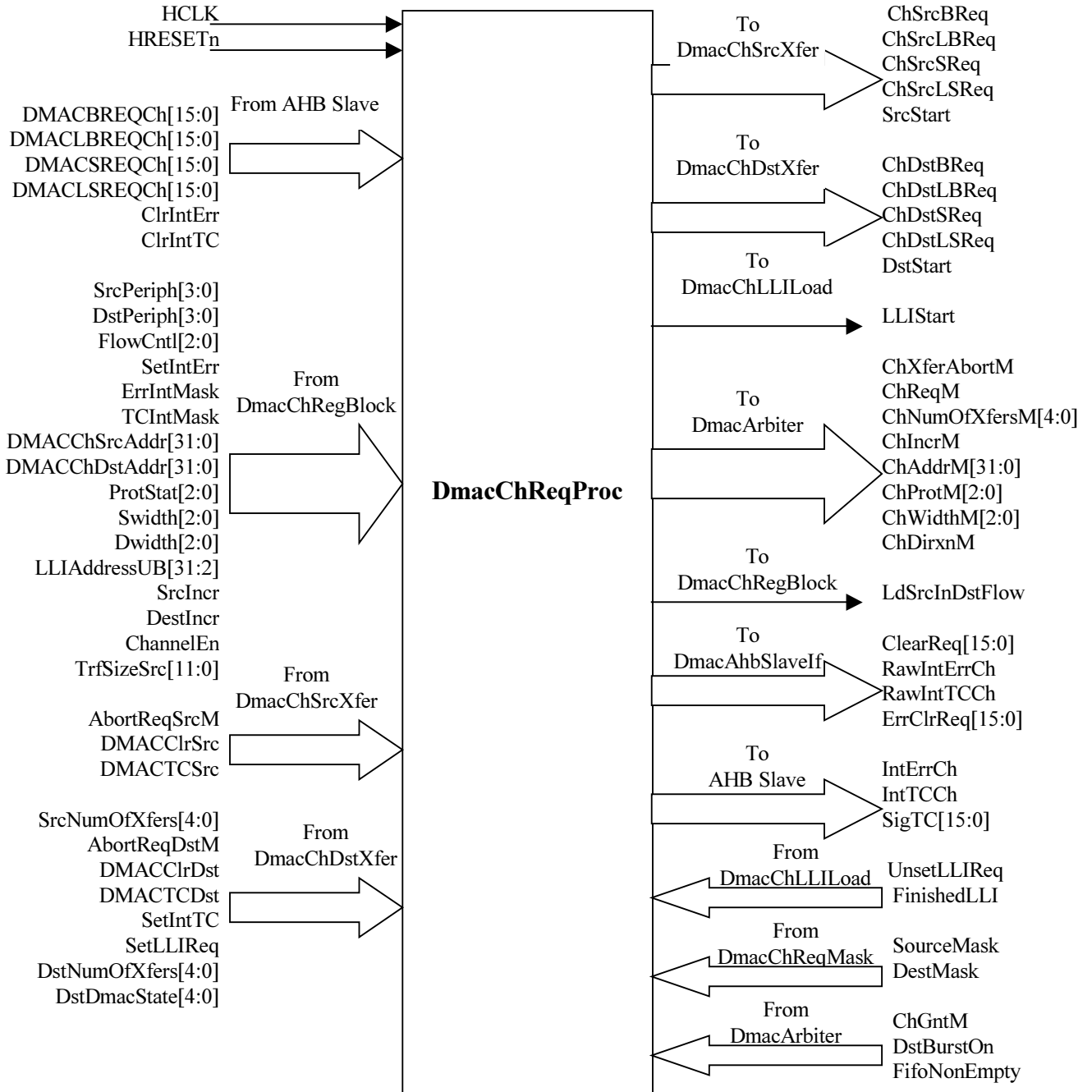
- **From packing unpacking block:** FifoWrEn indicates that the data present on FifoWrData can be written into the FIFO word as pointed by FifoWrPtr. As the data being written into the FIFO might be narrower than a word, it is sometimes required that data is written into specific byte-lane(s), without destroying the contents on remaining byte-lanes. This is achieved by FifoWrMask.

Outputs

- **To packing unpacking block:** FifoRdData is the entire content of the 32-bit word pointed by FifoRdPtr.

4.5.7 DMAC Channel Request Processing Block

This block maps the DMAC-request-lines to the channel, routes the channel-information to the AHB Master port and resolves the internal-arbiter grant to start source/destination/LLI transfers. Description of the salient group of input and output signals is given below.



Inputs

- **From AHB slave interface:** DMAC(L)(B/S)REQCh are four signals and all are 16-bit wide, indicating 16 requestors. Any bit high indicates the type of transfer the peripheral wants to do. ClrIntErr and ClrIntTC are the interrupt-clear signals.

- **From AHB master interface:** ChGntM indicate that the channel has the permission to access the AHB. The ChGntM is resolved to determine whether the source or destination or the LLI should start on the AHB.
- **From DMAC channel register block:** The bit-fields of DMACChControl, DMACChConfig and DMACChLLIReg registers are input to the request-processing-block. SrcPeriph and DstPeriph signals are used to resolve the requests specific to the channel, out of 16 requests. FlowCntl is used to determine whether certain requests are applicable to the channel in specified flow-control-mode. For example, in Memory-to-Memory configuration for DMA transfers, all the request lines are ignored (even when these are high). ErrIntMask is required to mask out the raw-error-interrupt. DMACChSrcAddr and DMACChDstAddr are routed to the AHB master port.
- Note: All the inputs from DMAC channel register block to request-processing-block have not been explained in order to enhance clarity.
- **From DMAC channel SrcXfer block:** AbortReqSrcM is logically ORed with the corresponding AbortReqDst signal (coming from DMAC channel DstXfer block) and sent to the master port. DMAC(Clr/TC)Src is a single bit signal which is de-multiplexed onto one out of 16 DMACCLR/DMACTC lines (one for each requestor). SrcNumOfXfers is given to the AHB master interface.
- **From DMAC channel DstXfer block:** AbortReqDstM is logically ORed with the corresponding AbortReqSrc signal (coming from DMAC channel SrcXfer block) and sent to the master port. DMAC(Clr/TC)Dst is a single-bit signal which is de-multiplexed onto one out of 16 DMACCLR/DMACTC lines (one for each requestor). DstNumOfXfers is assigned to the AHB master interface. SetLLIReq is used to set the LLIReq bit, which is then assigned to the AHB master port.
- **From DMAC channel LLI load block:** UnsetLLIReq is used to reset the LLI request. FinishedLLI is used in Destination-Flow-Control mode to load the destination-transfer-size counter after LLI loading.
- **From packing unpacking block:** FifoNonEmpty is used to load destination-transfer-size counter in Destination-Flow-Control mode.
- **From request masking block:** SourceMask and DestMask are ANDed with source and destination requests to generate the final requests to the master interfaces.

Outputs

- **To DMAC AHB master interfaces:** All the address and control signals are to be routed to the master interface. A set of address and control signals (for the master port) are output of the request-processing block.
- **To DMAC channel SrcXfer block:** ChSrc(L)(B/S)Req are the requests which are mapped to the particular channel using the SrcPeriph bit-field. The SrcPeriph muxes out a single bit ChSrc(L)(B/S)Req from the 16-bit DMAC(L)(B/S)REQCh. SrcStart is asserted, if the channel is granted and channel-source request is high. However if during the grant, channel destination request for the same bus is also high, the SrcStart is not asserted. Instead, DstStart is asserted.
- **To DMAC channel DstXfer block:** ChDst(L)(B/S)Req are the requests which are mapped to the particular channel using the DstPeriph bit-fields. The DstPeriph muxes out a single bit ChDst(L)(B/S)Req from the 16-bit DMAC(L)(B/S)REQCh. DstStart is asserted, if the channel is granted and channel-destination request is high.
- **To DMAC channel LLI loading block:** LLISart is the signal which is asserted when the channel is granted and the LLI request is asserted by the channel.
- **To DMAC channel register block:** In Destination-Flow-Control case, the source transfers are to be started when any destination request is asserted. LdSrcInDstFlow is generated in this request-processing-block by detecting an edge on destination-requests or by detecting pending destination transfers.
- **To AHB slave interface:** ClearReq is a 16 bit signal, which is generated by de-muxing the single bit DMACClrSrc and DMACClrDst signals (input to request-processing block from SrcXfer and DstXfer block respectively). RawIntErrCh is a bit which is set with the clocked version of SetIntErr (input to this block) and reset with ClrIntErr (input to this block). Similarly RawIntTCCh is set with SetIntTC and reset with ClrIntTC. ErrClrReq is a 16-bit signal in which the bits corresponding to source and destination peripherals are asserted for one clock when the channel gets an error response.

- **To DmacResponseRoute:** SigTC is a 16-bit signal, which is generated by de-muxing the single-bit DMACTCSrc and DMACTCDst signals (input to request-processing block from SrcXfer and DstXfer block respectively). IntErrCh and IntTCCh are generated by logically ANDing RawIntErrch and RawIntTCCh with the interrupt masks.

4.5.8 DMAC Channel Request Masking Block

This block describes the mask to be put on DMAC transfer requests corresponding to the channel. Description of the masks that have been used is given below.

- **SrcE2LMask :** Source-Transfer-End to LLI load mask. This is used in source-flow control mode. After the Source transfers for a packet have finished, no more source requests should be serviced unless the destination transfer for the packet finishes.
- **SrcAxsOnMask:** When the AHB transfers corresponding to a source-request have started, no more source requests should be serviced until the transfer finishes. This is taken care of by SrcAxsOnMask.
- **LLILoadMask:** When the LLI loading is proceeding, no source or destination transfers should be started off with. This is taken care of by the LLILoadMask.
- **DstAxsOnMask:** When the AHB transfers corresponding to a destination-request have started, no more destination requests should be serviced until the transfer finishes. This is taken care of by DstAxsOnMask, which prevents the same destination request from triggering off one more access.
- **SrcTrfSizeMask:** Ensures that in DMAC flow control mode, no transfer starts off when the Transfer Size programmed is zero.
- **FifoSrcMask:** When there are no empty spaces in the FIFO to accommodate source data, then in spite of requests, AHB transfers should not start off. This is taken care of by FifoSrcMask.
- **DestNonReqMask:** In Destination-Flow-Control case, unless a destination request is raised, no source transfers should start off. This is taken care of by the DestNonReqMask.
- **FifoDstMask:** When there are no data in the FIFO to start destination transfers, then in spite of requests, AHB transfers should not start off. This is taken care of by FifoDstMask.
- **DstTrfSizeMask:** This is for masking destination accesses on AHB when destination transfer size counter is zero (in Dmac Flow control mode).

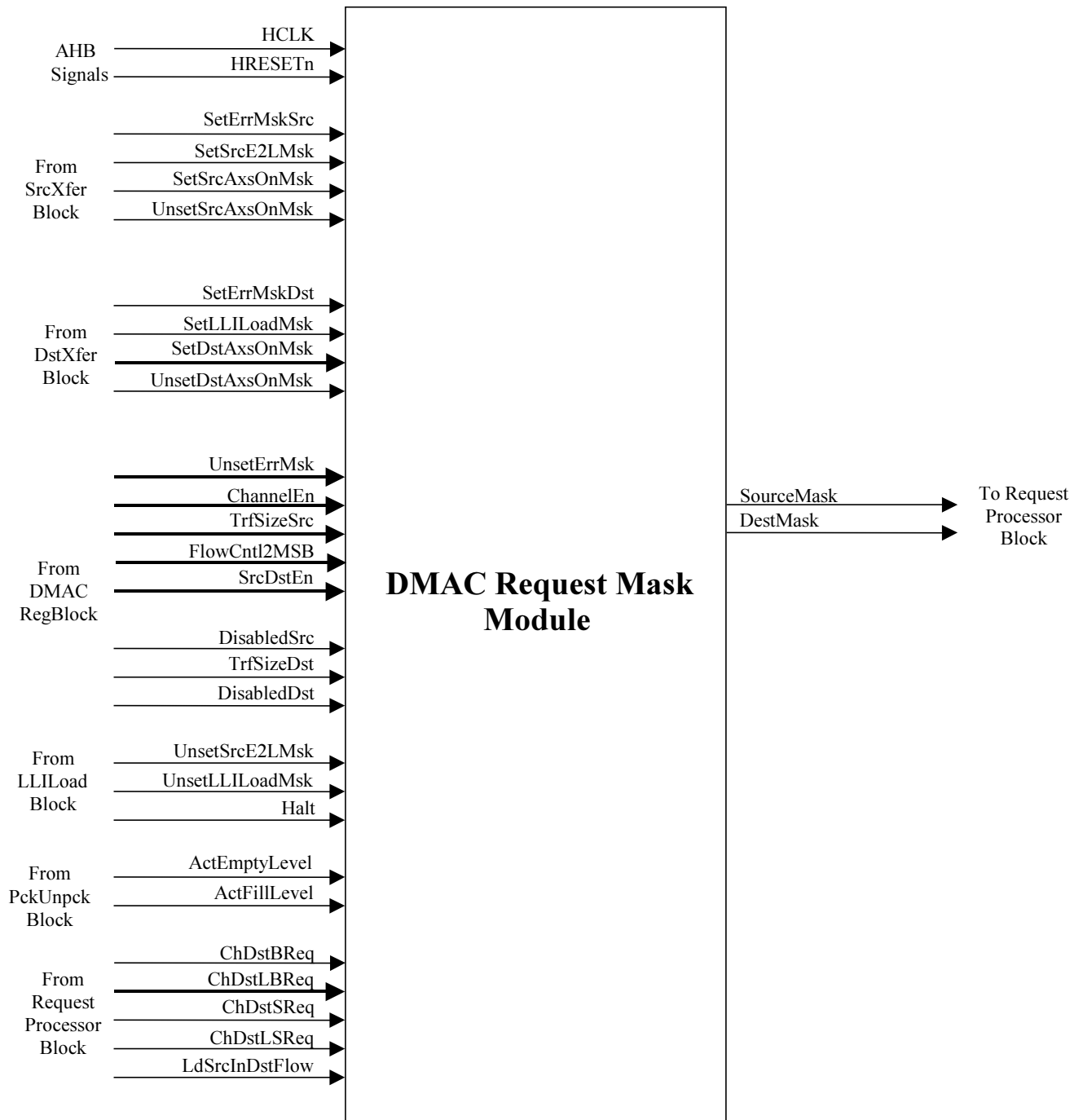


Figure 4.5-7 Block Inputs and Outputs for DmacChReqMask

4.6 DMAC Response Routing Block

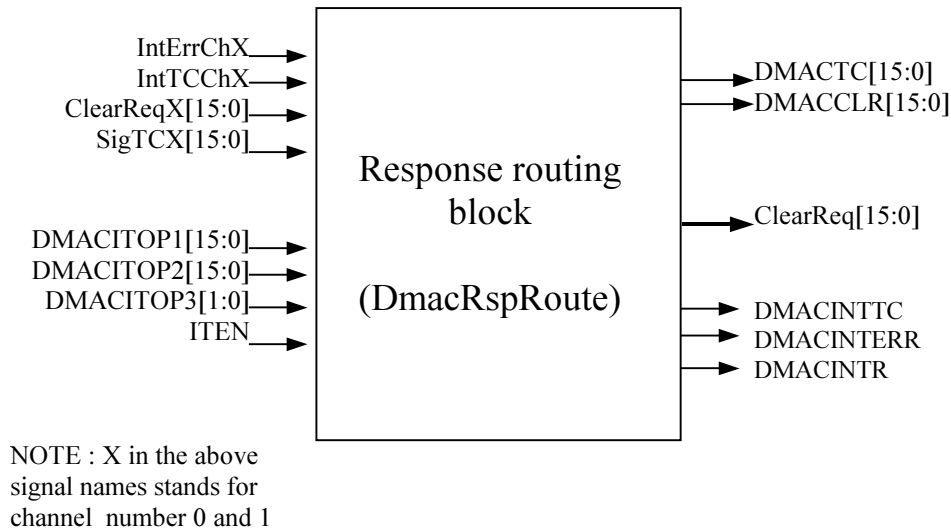


Figure 4.6-1 Block Inputs and Outputs for the Response Routing logic

The suffix 'X' has been used with many signals like ClearReq, SigTC, IntTCCh etc. The suffix 'x' represents the channel number (0 and 1).

Each channel generates a Terminal Count interrupt and an Error Interrupt when the relevant conditions occur. These interrupt lines are active high. The interrupts are combined into the DMACINTTC interrupt and the DMACINTERR interrupt by ORing the respective channel interrupt lines. The interrupt status for each channel is readable through the DMACIntErrStat and the DMACIntTCStat registers.

Each channel generates a 16-bit vector ClearReqX[15:0] [x = 0 and 1 (channel number)] to indicate DMA request clear and a 16-bit vector SigTCX[15:0] [x = 0 and 1 (channel number)] to indicate Terminal Count. Each bit in these vectors corresponds to a DMA requestor. The related bits in these vectors from the channels are ORed to generate the requestor-specific DMACCLR[15:0] and DMACTC[15:0] outputs.

This module contains the ORing logic for the above-mentioned functions. In addition, this module contains Integration test muxes for DMACCLR[15:0], DMACTC[15:0], DMACINTTC, DMACINTERR and DMACINTR.

Inputs

- **ClearReqX[15:0] and DMACCLR[15:0]:** The clear (DMACCLR) signal is specific to each requestor and not each channel. Thus in place of a single-bit from a channel (indicating completion of request-servicing), a 16 bit signal, ClearReq[15:0], comes out from each channel, with the appropriate bit set to high. This block does a bit-wise logical OR of all the ClearReq signals and passes it to the output DMACCLR[15:0].
- **SigTCX[15:0] and DMACTC[15:0]:** The TC (terminal count) signal is similar to clear-signal, in the sense that it is also requestor-specific. Thus a 16-bit signal SigTCX[15:0] bit vector comes out from each channel, with the appropriate bit set high. This block does a bit-wise logical OR of all the SigTCX signals and passes it to the output DMACTC[15:0].
- **DMACITOP1, 2, 3 and ITEN :** These are integration test register inputs to the module. These are used for integration test multiplexing.
- **IntTCChX, IntErrChX [X= 0 and 1 (channel number)] :** These are the Terminal Count interrupt and Error Interrupt lines from the Channels.

Outputs

- **DMACINTTC, DMACINTERR and DMACINTR:** These three signals are DMAC outputs. All the IntTCChx lines from the channel are ORed to generate DMACINTTC, which is a DMAC output. The DMACINTERR is obtained from the IntErrChx lines in a similar manner. The DMACINTR is a logical OR of DMACINTTC and DMACINTERR.

- **ClearReq[15:0]** : These output lines are the Ored version of the ClearReq from all the channels.

4.7 DMAC Request synchronising logic

DMA request signals are masked with the DMACEn bit in the AHB slave interface and are passed onto the Request Synchronising block. In this module, these signals are double synchronised to the HCLK domain.

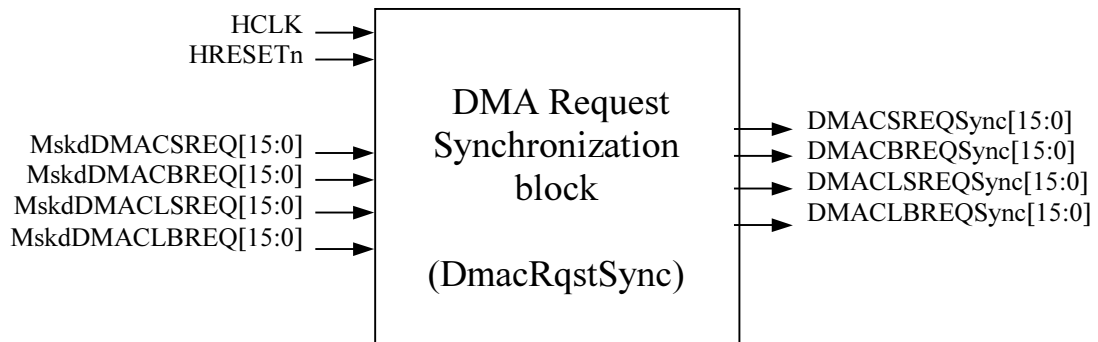


Figure 4.7-1 Block Inputs and Outputs for Request Synchronisation logic

The output of this module are fed back to the AHB slave interface before being passed onto the channels. In the AHB Slave interface, a selection is done between the double synchronised version of the DMA requests and the buffered version of the DMA requests, depending on the corresponding control bit in the DmacSync register. The requests are then passed on to the channel. This is illustrated in **Figure 4.7-2** below. All logic shown outside the DmacRqstSync block in this figure, is implemented in the DmacAhbSlavelf block.

Inputs

- **Masked DMARequests:** There 4 DMA request vectors, all 16 bit wide - DMACSREQ[15:0], DMACLSREQ[15:0], DMACBREQ[15:0] and DMACLBREQ[15:0]. All the 4 vectors are gated with DMACEn (in the DmacAhbSlavelf block) so as to reduce the power consumption by avoiding the toggling of the internal nodes in the DMAC. The masked version of the requests (MskdDMAC{L}{S/B}REQ[15:0]) are fed to the DmacRqstSync block for double synchronisation.

Outputs

- **DMAC{L}{S/B}REQSync[15:0]:** These signals are the double synchronised versions of the corresponding inputs.

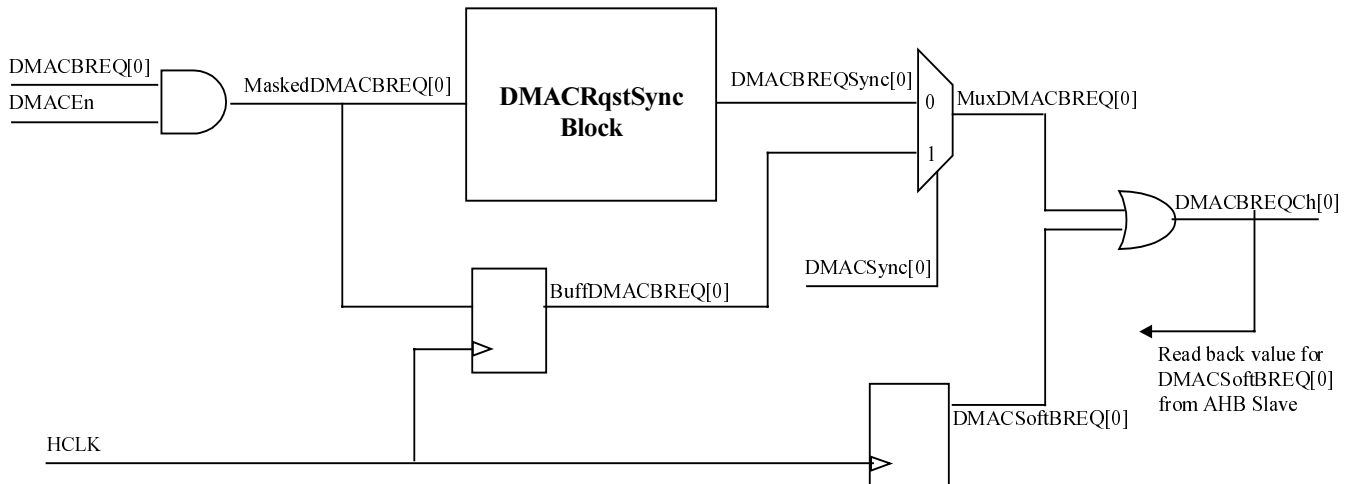


Figure 4.7-2 Selection of synchronised or unsynchronised requests (DMACBREQ[0] used as an example)

4.8 Revision Designator Block

This module contains a single AND gate to be used as a place-holder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed. The port level connections for the module are as below.

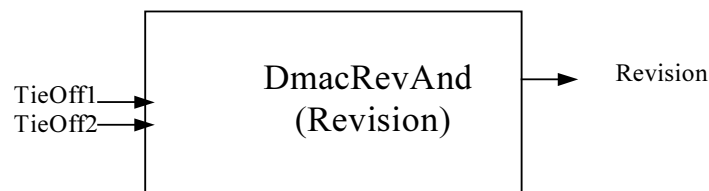


Figure 4.8-1 Block Inputs and Outputs for DmacRevAnd

5 DMAC IMPLEMENTATION DETAILS

In this section, the details of the blocks listed in section 4 are provided.

5.1 Design Hierarchy

The hierarchy in the DMAC is shown in **Figure 5.1-1**. A short description about the functionality of each of the sub-modules is also given within brackets.

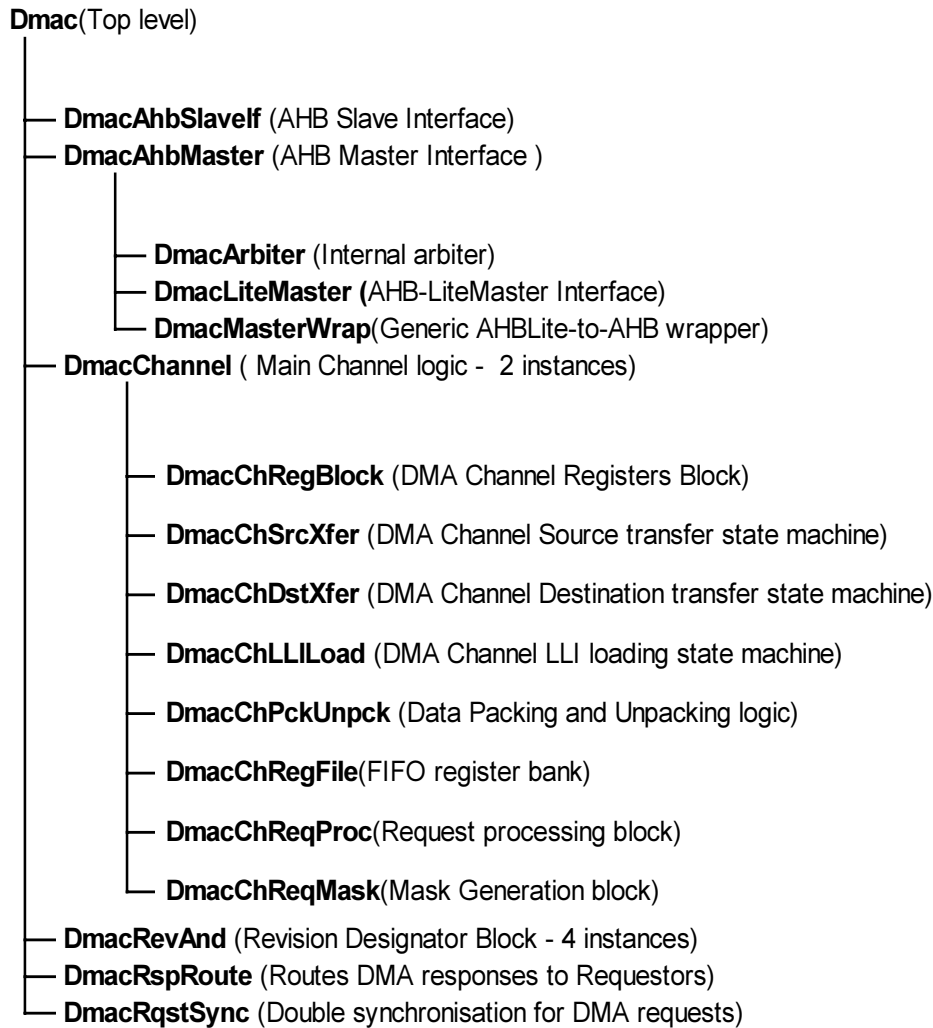


Figure 5.1-1 DMAC Design Hierarchy

5.2 DMAC Top Level Module (Dmac)

The top-level module is a structural block which instantiates and interconnects the main functional blocks in the DMAC.

5.3 AHB Slave Interface Module (DmacAhbSlaveIf)

The AHB Slave interface and Register block consists of logic to respond to AHB accesses to the DMAC AHB Slave interface, by returning AHB responses and by registering the address and control information from the AHB.

AHB response generation logic:

The responses driven on the AHB by the AHB Slave are described below.

The DMAC AHB Slave drives OKAY responses on the HRESP lines under following conditions

- When the DMAC Slave is not selected. (i.e. HREADYIN = 0 or HSELDMAC = 0)

- When the DMAC Slave is selected (i.e. HREADYIN =1 and HSELDMAC =1) but the HTRANS is IDLE or BUSY.
- When the DMAC Slave is selected (i.e. HREADYIN =1 and HSELDMAC =1) but the HTRANS is SEQ or NSEQ and the HSIZE is WORD.

The DMAC AHB Slave drives ERROR response on to the HRESP lines under following condition.

- When the DMAC Slave is selected (i.e. HREADYIN =1 and HSELDMAC =1) but the HTRANS is SEQ or NSEQ and the HSIZE is **other than** WORD.

Registering the Address and control information from the AHB

Under the following condition, the DMAC slave generates internal select signals for register read/write and registers the HADDR and HWRITE information:

- When the DMAC Slave is selected (i.e. HREADYIN =1 and HSELDMAC =1) but the HTRANS is SEQ or NSEQ and the HSIZE is WORD.

Generation of individual write signals for registers in the module

When the internal select signal of the DMAC AHB Slave is high and HWRITE is also high, the registered address (AddrBuff) is combinationally decoded to generate the individual write signals for the registers in the module.

Driving HRDATA on the AHB

According to the address on the AddrBuff lines the Slave Interface returns the content of the corresponding register on to the HRDATA bus.

Channel-specific registers reside in the respective channel block. The write logic for these registers is also present in the channel blocks. The channel returns the data on the ChxHRDATA line when any of the registers in it, is selected. The channel drives out the registered version (ChHRDATA) of the internal read data, which is derived by multiplexing the registered AHB address (AddrBuff). Hence, one WAIT state is added for any of the channel register reads from the AHB. This registering of the read data is done to meet synthesis timing requirements on the HRDATA output.

The AHB Slave Interface OR's the ChxHRDATA from both the channels with the RegHRDATA from non-channel specific registers within the AHB Slave interface to drive out the HRDATA on to the AHB bus.

Writing into DMAC registers from the AHB

The write enable for each register is generated by decoding the registered version of the AHB address bus (AddrBuff) when the RegHWrite (registered version of HWRITE) signal is high. For channel-specific register this logic is located inside the channel register block. When the write enable for a register is sampled high on the rising edge of the HCLK, the data on the HWDATA lines from AHB is registered into the respective register.

Registers implemented in the AHB Slave Interface module

- Registers used for testing. (DMACITCR, DMACITOP1, DMACITOP2, DMACITOP3)
- Interrupt status registers. (DMACIntStat, DMACIntTCStat, DMACIntERRStat, DMACRawIntTC, DMACRawIntErr). These registers are virtual registers in the sense that, when the register address is decoded (i.e. the register is selected), the AHB Slave interface module just returns the status of the respective signals on the HRDATA lines. Hence, these registers are not implemented with flip-flops.
- Channel enabled status register (DMACEnbldChns). This is also a virtual register used to read the enable status of both the channels.
- Registers used for clearing the interrupts (DMACIntTCClr, DMACIntErrClr). These registers are virtual registers, in the sense that these registers are not implemented with flip-flops. The interrupt clear signal is generated combinationaly when a write is detected to the corresponding register and the bits in the HWDATA line are high.
- Software DMA request generation registers (DMACSoftSReq, DMACSoftBReq, DMACSoftLSReq and DMACSoftLBReq) and Synchronisation control register DMACSync.

Registers implemented in other modules

- All the channel-specific registers are implemented in the channels. (DMACCxConfig, DMACCxControl, DMACCxSrcAddr, DMACCxDestAddr and DMACCxLLIReg).
- The 'Revision' field in the DMACPeriphId2 register is implemented using the DmacRevAnd module.

Passing register bit values to other modules in the DMAC

- The ITEN signal (corresponding to the ITEN bit of the DMACITCR register) is driven to the DmacRspRoute module for implementing the integration test muxes.
- When a write is detected to the DMACIntTCClr or the DMACIntErrClr register, clear signals are given to the channels depending on which bit in the HWDATA lines is high, for clearing the channel interrupts.
- The DMACEn and BigEndian information are given to other modules from DMACConfig register in the AHB Slave Interface module.
- The contents of the DMACSoftSReq, DMACSoftBReq, DMACSoftLSReq and DMACSoftLBReq registers are ORed with the hardware DMA requests and passed onto the Channels as DMA requests.

Passing status information from the Channels to the AHB Slave module

- Enable status information is given from all the channels to the AHB Slave Interface. The AHB Slave Interface returns it on to the HRDATA bus when the DMACEnbldChns register is read.
- The clear requests (either ClearReq[15:0] or ErrClrReqX[15:0] where x = 0 and 1 (channel number)) from the Channels to the AHB Slave module are used to clear the DMA Soft request bits in the DMACSoftSReq, DMACSoftBReq, DMACSoftLSReq and DMACSoftLBReq registers. When a DMA transfer, initiated by setting a Soft Request bit, completes normally, the ClearReq signal is asserted by the respective Channel. When a DMA transfer, initiated by setting a Soft Request bit, receives an error response from the AHB slave, the respective Channel asserts the ErrClearReq signal to clear the Soft request registers.
- When a channel-specific register is selected by the AHB Slave module for read, the channel returns the register contents on to the ChxHRDTA [x = 0 and 1 (channel number)] lines.
- The status of the masked and unmasked interrupts is also given by the channel to the AHB Slave module.

Write Control logic for the SoftRequest registers

The write control logic for the DMACSoftSReq, DMACSoftBReq, DMACSoftLSReq and DMACSoftLBReq registers is as below:

- When a bit in the ClearReq[15:0] input is detected high, the corresponding Soft request register bit is cleared independent of the status of the write from the AHB slave side.
- If '0' is written through the AHB slave interface into any bit in the SoftReq registers, it is ignored in the normal functional mode, but is allowed in the integration test mode. This is because, the clearing of the SoftReq bits in the normal mode is done by the Channel control logic on completion of the programmed DMA transfer. In the Integration test mode, the focus is on checking connectivity. Hence, an explicit clearing of the SoftReq bits through AHB slave write is allowed in the Integration test mode.
- In the normal mode, all the bits in the Soft request registers are cleared when the DMAC is disabled. In the integration test mode this clearing does not happen even if the DMAC is disabled. This allows Integration testing to proceed even as the Channels remain idle.

Read logic for SoftRequest registers

Whenever a read is done from the address corresponding to the Soft Request registers, the actual requests going to the channels (i.e. ORing of the HARD and SOFT requests) is driven on to the HRDATA lines. This allows for Integration testing of the DMA request input lines.

5.3.1 Register Write Logic

The register write logic for the DMAC slave is as in **Figure 5.3-1**. A DMAC register is written with the data on HWDATA when the write enable for that register is sampled high on the rising edge of HCLK. The register to be written is selected by decoding the sampled address (HAddrBuff). The write enable signals are generated by combining the address decode with the sampled HWRITE signal (RegHWrite).

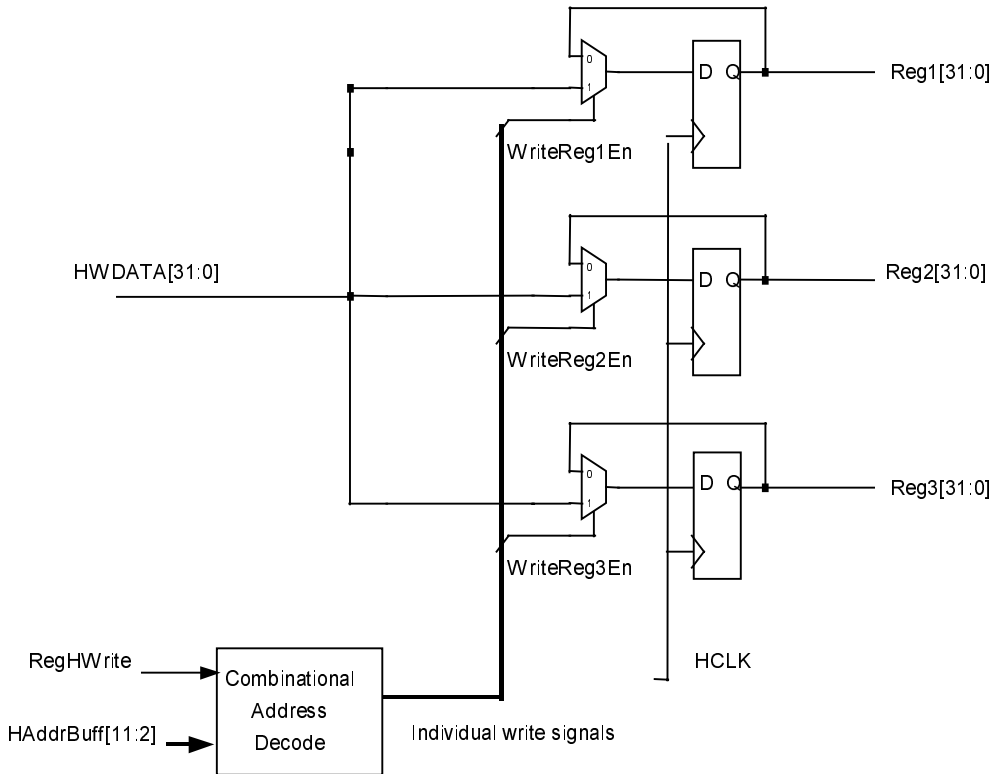


Figure 5.3-1 Register Write Logic

5.3.2 Register Read Logic

The Read logic for the registers in the DMAC is as given in **Figure 5.3-2**.

For reads, the registers in the AHB Slave interface are muxed (based on HAddrBuff) to generate an intermediate RegHRDATA bus. Similarly, the registers in each of the channels are muxed and clocked to generate intermediate registered ChHRDATA buses for each channel. These buses are then OR'd to generate the final HRDATA output. Thus the logic for driving out the HRDATA uses MUX-OR implementation to ensure optimum timings. Also, for reads to both the channel registers, one wait state is added. This is done to meet synthesis timing requirements.

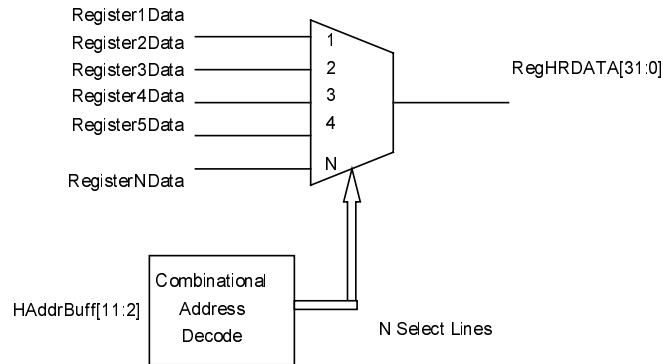


Figure 5.3-2 Register Read Logic

5.4 AHB Master Interface(DmacAhbMasterIf)

In this section, the sub-blocks within the channel AHB-master-interface logic are explained in more details.

5.4.1 Internal Arbiter (DmacArbiter)

The function of the internal-arbiter is to arbitrate between requests from channels and to route selected requests to the AHB Master block. The functions are described in detail below.

Raising channel-transfer-requests to the master-interface

The transfer request to the master-interface (ArbXferReq) is a logical OR of all the channel requests. The channel that is granted, gives out the address and control information to the master-interface.

Raising abort-requests to the master-interface

The abort request to the master-interface (AbortXfer) is a logical OR of the two abort-requests from the two channels. The channels can assert abort requests only in ST_XXX_DATAXFER (where XXX stands for SRC or DST) state. At a time, on the same AHB port, more than one channel (and within a channel, more than one state machine – among source and destination machines) cannot be active. Thus AbortXfer is generated by logically ORing all the channel-abort-requests.

Granting of channels

The priority scheme that has been used for giving grant is 'fixed-priority'. Thus the logic is a priority encoder with the highest priority given to channel 0 and the lowest to channel 1. Moreover, when channel 1 is granted, the ArbPriority line is pulled low, as this channel (1) is defined as "lower priority" channel in DMAC specification.

The internal grant to the channels is given, only when ArbStop is LOW. When ArbStop is high, none of the channels are granted. All the internal channel- grant-lines (ChxGntM) are pulled low. The internal-arbiter architecture is such that it arbitrates combinatorially according to the fixed-priority scheme, unless ArbStop is high. Once a channel has been granted, then the next phase of arbitration should not start till the end of the ongoing transfer. To stop the arbiter from arbitrating during the current channel's transfer phase, the signal ArbStop is used.

Routing granted channel's address and control information to master-interface

Using the channel-grant lines (Ch[0:1]GntM), the address and control signals are routed to the master-interface. If the channel is granted, (corresponding Ch[0:1]GntM line is high) then the control signal for that channel is assigned to an intermediate signal. If not granted then the intermediate-signal is assigned zero. Finally at the top level, all the intermediate-controls-signals from all the channels are bit-wise ORed to generate the final control-signal which is passed to the master-interface.

5.4.2 AHB-Lite Master State machine (DmacLiteMaster)

The AHB-Lite Master State machine transits through different states depending on the requested number of transfers and the type of transfer requested by the internal arbiter. The state machine has the following states:

ST_MASTER_INITIAL

ST_MASTER_ADDRPHASE

ST_MASTER_DATAPHASE

ST_MASTER_LOWPRIO_IDLE

On Reset the State machine enters the ST_MASTER_INITIAL state.

ST_MASTER_INITIAL

In this state it waits for the transfer request from the internal arbiter (ArbXferReq) to go high. If MREADY is sampled high at the instant when the ArbXferReq is high, it goes to the ST_MASTER_ADDRPHASE state. When it sees MREADY high, it drives NSEQ on MTRANS and drives MADDR, MSIZE and MPROT with values registered from the internal arbiter. MBURST is driven with a value derived from ArbNumOfXfers, ArbHSize and ArbHADDR. A burst is started with the correct address to ensure that the 1KB boundary is not crossed. In case of non-incrementing accesses, the HBURST information is always INCR (undefined increment).

ST_MASTER_ADDRPHASE

This state marks the address phase of the first AHB transfer on the bus.

When MREADY is sampled high, the state machine moves to the ST_MASTER_DATAPHASE state with the following actions:

If the current transfer is last in the number of transfers requested by the internal arbiter (i.e. RemNumOfXfers = 1), the state machine checks whether ArbXferReq is high. If ArbXferReq is high, the AHB-Lite Master puts the new address from the internal arbiter on to MADDR, and NSEQ on to the MTRANS lines. All other control signals are driven from the arbiter, derived in a manner similar to that in the case of the state transition from the ST_MASTER_INITIAL state to the ST_MASTER_ADDRPHASE state. The new burst transfer is started back to back without any IDLE cycles. If, the current transfer is last in the number of transfers requested by the internal arbiter, and ArbXferReq is low then the state machine puts IDLE on MTRANS lines and clears the RemNumOfXfers counter. The state machine then moves to ST_MASTER_DATAPHASE. If MREADY is sampled LOW then it remains in the ST_MASTER_ADDRPHASE state without changing any control information.

If the current transfer is the last in the burst (RmnFrmBurst = 1), but the total number of transfers are not yet over, a new burst is started for the remaining number of transfers. MTRANS is driven as NSEQ. The RemNumOfXfers counter is decremented by one. If the requested AHB accesses for the ongoing transfer were from a low priority channel then the state machine puts out an IDLE cycle on the AHB before the next burst of transfers and moves to the ST_MASTER_LOWPRIO_IDLE state.

If the current transfer is not the last in the burst (RmnFrmBurst != 1), the incremented MADDR is put on the bus (In case of non-incrementing DMA accesses, the address incrementing is not done and the same address is put out) and the RemNumOfXfers and RmnFrmBurst counters are decremented by one. A SEQ is put on the HTRANS lines (NSEQ in case of non-incrementing DMA accesses).

ST_MASTER_DATAPHASE

When MREADY is sampled high:

If the current transfer is the last in the number of transfers requested by the internal arbiter (RemNumOfXfers = 1), the state machine checks whether ArbXferReq is high. If ArbXferReq is high then the FSM puts the new address on to the bus and NSEQ on MTRANS for starting the new transfer requested by the internal arbiter. If ArbXferReq is not high and if the RemNumOfXfers counter has counted down to 0, it moves on to the ST_MASTER_INITIAL state; else the state machine remains in the ST_MASTER_DATAPHASE state.

If the current transfer is the last in the burst/single transfer initiated due to peripheral boundary restrictions, and the total number of transfers requested by the internal arbiter are not over (i.e. $\text{RemNumOfXfers} \neq 1$ & $\text{RmnFrmBurst} = 1$) and the AbortXfer signal is low, then the state machine drives the incremented address on the MADDR lines according to the MSIZE information (In the case of non-incrementing DMA accesses, MADDR remains unchanged) and NSEQ on the MTRANS lines. The RemNumOfXfers counter is decremented by 1. The MBURST information is now calculated from RemNumOfXfers and $(\text{MADDR} + 1)$ (Increment is done according to HSIZE). If AbortXfer is sampled HIGH then the state machine remains in the same state but de-asserts ArbStop for re-arbitration and sets an internal signal (RstDueToAbort) to indicate the aborting condition on the next clock cycle.

If the requested transfers for the previous transfer are over and if these were requested from a low priority channel then the state machine puts out an IDLE cycle at the end of the transfer and moves to $\text{ST_MASTER_LOWPRIO_IDLE}$ state. The state machine also pulls MBUSREQ low.

If the current transfer is not the last (i.e. $\text{RmnFrmBurst} \neq 1$) in the burst then the incremented MADDR is put on the bus (same Address in case of non-incrementing DMA access) and the RemNumOfXfers and RemFrmBurst counters are decremented by one. SEQ is put on the MTRANS lines (NSEQ in case of non-incrementing bus access).

When an ERROR is sampled on MERROR , the state machine moves to the ST_MASTER_INITIAL state. Also, it resets all the counters (i.e. RemNumOfXfers , RmnFrmBurst) and drives IDLE on to the AHB.

ST_MASTER_LOWPRIO_IDLE

This state indicates that an IDLE cycle is put on the bus due to low priority channels initiating transfers on to the bus. This IDLE cycle is inserted at the end of the AHB access requested by the LOW priority channels. On sampling MREADY high, on the next clock edge, the state machine moves to the $\text{ST_MASTER_ADDRPHASE}$ state.

Figure 5.4-1 shows the state diagram of the AHB-Lite Master Interface.

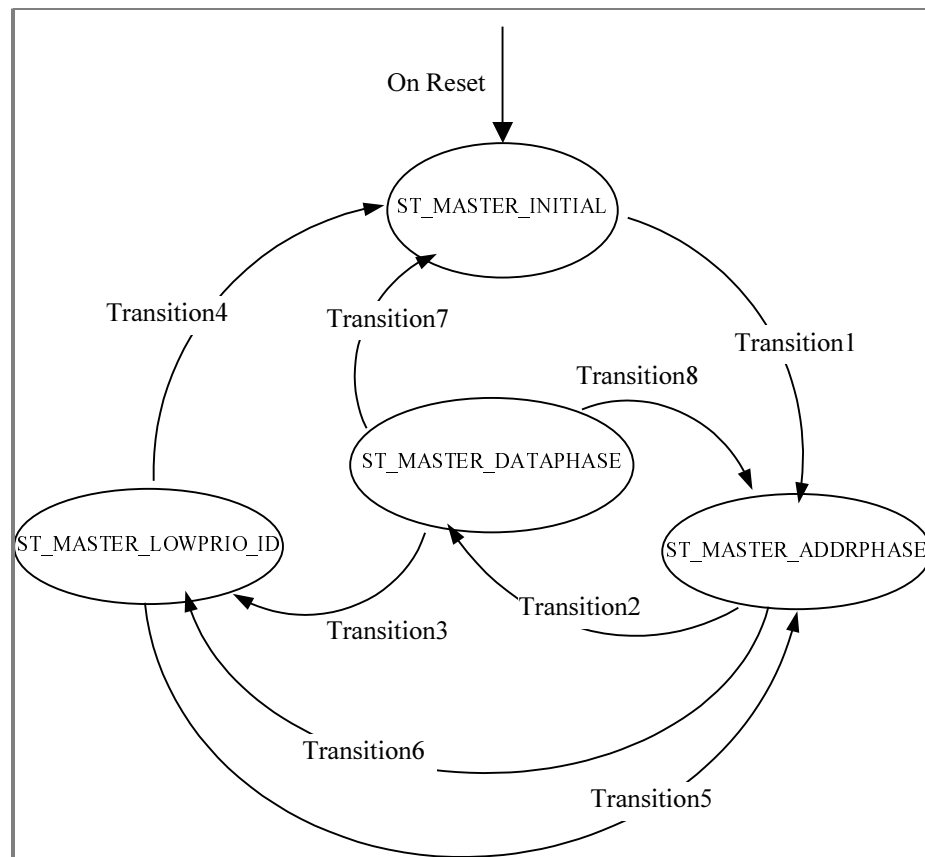


Figure 5.4-1 AHB-Lite Master Interface of DMA Controller

Transition table for the AHB-Lite Master Interface state machine is given in **Table 5.4-1**. In this table, wherever multiple transition conditions are shown, the transition occurs when one or more of the conditions is satisfied.

Transition Number	Trigger Expression
Transition1	1. MREADY && (WaitOneClk = '0') && (StartFrmBuff = '0') && (ArbXferReq = '1') 2. MREADY && (WaitOneClk = '0') && (StartFrmBuff = '1')
Transition2	MREADY && (!(RemNumOfXfers = "00001" && RstDueToAbort = '0') && (BuffPriority = '0'))
Transition3	MREADY && (RemNumOfXfers = "00001" && RstDueToAbort = '0') && (BuffPriority = '0')
Transition4	1. MREADY && (StartFrmBuff = '0') && (ArbXferReq = '0') 2. MERROR && !MREADY
Transition5	MREADY && (!(StartFrmBuff = '0') && (ArbXferReq = '0'))
Transition6	MREADY && (RemNumOfXfers = "00001" && RstDueToAbort = '0') && (BuffPriority = '0')
Transition7	MREADY && (RemNumOfXfers = "00000") && (StartFrmBuff = '0') && (ArbXferReq = '0')
Transition8	1. MREADY && (RemNumOfXfers = "00000") && (StartFrmBuff = '0') && (ArbXferReq = '1') 2. MREADY && (RemNumOfXfers = "00000") && (StartFrmBuff = '1')

Table 5.4-1 AHB-Lite Master Interface State Machine transition table

Some of the critical signals are described below:

ArbDataError: This signal is the clocked version of the MERROR response signal in the data phase of the transfer. When asserted, this signal indicates that the master interface has got an ERROR response from the targeted AHB slave.

BusAvlblM : This signal is an indication that the channel is active on the AHB. It is a clocked version of MREADY. This signal is used for pipelining data from the channels to the AHB master interface.

The HRDATA sampled on the AHB port can be of any Endianisation depending on the BigEndian bit in the DmacConfig register. The Channel assumes the data to be Little Endian. Hence, Endianization is done in the AHB Master interface to make the DmacChannel independent of the BigEndian bit in the DmacConfig register. The data sampled on HRDATA is fed to the Endianization logic. This logic routes the byte lanes of the input HRDATA onto the ChWrData lines accordingly to make ChWrData Little Endian.

On the HWDATA path, the Channels drive data onto the lower order BYTE lanes depending on the width of the transfer (i.e. HSIZE). The AHB Master Interface duplicates these BYTE lanes onto the other BYTE lanes of HWDATA.

Since data from the Channel is driven depending on the BusAvlblM signal, and since the BusAvlblM signal is a registered version of MREADY, there is a holding buffer for the HWDATA coming out from the Channel in order to support waited transfers (MREADY pulled LOW). In normal cases when MREADY is HIGH, the Channel data is

routed to HWDATA directly. But in case of waited transfers, write data from the internal buffer in the Master Interface is routed onto HWDATA on the AHB.

The combinational process (Next State logic) of the main state machine is split into 3 separate processes.

- First process for generating intermediate signals.
- Second for generating the signals after qualifying with the MERROR signal
- Third for generating D inputs for the flipflops in the state machine after qualifying with MREADY.

This splitting is done for better synthesis timing on late-arriving signals. In this case, MREADY and MERROR are late-arriving signals, as these come from AHB ports.

Functions used

In the RTL, several functions have been defined in this block. These are described below.

AddrIncr: This function is used for incrementing the address on the HADDR lines depending on HSIZE information. The function implements a 32 bit incrementer with look-ahead carry generation logic. The inputs for this function are the 32-bit address and the HSIZE information. The incremented version of the address is the output of the function. The address is incremented by 1 in case of BYTE-wide transfers, by 2 in case of Half-Word-wide transfers and by 4 for WORD-wide transfers.

CalcPossible: This function takes the current address and HSIZE as inputs and gives out the number of transfers possible on the AHB without crossing the 1KB boundary. This function checks the higher order bits of the address to see if the address is near the 1KB boundary. If the address is within 16 transfers of the 1KB boundary, the 4 lower order address bits are subtracted from 16 to find the number of transfers that can be done in the burst.

StartNewBurst: This function takes two inputs:

- The number of transfers requested by the internal arbiter.
- The number of transfers possible in a single burst without crossing the 1KB boundary (These are calculated from the CalcPossible function).

From these two inputs, the function calculates the optimum valid AHB burst that can be initiated on the AHB.

5.4.3 AHB Wrapper (DmacMasterWrap)

The Wrapper function is described in AHB Example Bus Master Technical Reference Manual [Ref. 8].

5.5 Channel Control Logic (DmacChannel)

In this section, the sub-blocks within the channel core logic are explained in detail.

5.5.1 DMAC Channel Register Block (DmacChRegBlock)

Details of DMAC-channel-register-block functionality are described in this sub-section.

Generation of control signals for AHB slave read/write to channel registers

The AHB slave interface generates DmacChannelSel to indicate that a particular channel is selected among two DMAC channels. The slave-interface also passes RegAddress which is a slice of the actual address (HADDR[4 : 2]), which can resolve a particular register within the channel. With the help of RegAddress and DmacChannelSel, the individual register-select namely DmacSrcRegSel, DmacDstRegSel, DmacLLIRegSel, DmacCntlRegSel and DmacChCnfgSel are generated. Based on these 'selects' the channel multiplexes the register contents as the AHB-read-data to the slave interface. If RegHWrite is high (an input to channel-register-block from DMAC AHB slave interface) along with individual-register-select, then single-clock-wide register-write-enables are generated. The signal names for the channel registers' write-enables are SrcAddrWrEn, DstAddrWrEn, CntlWrEn and LLIWrEn.

Clocking LLISelect at packet-beginning

LLI loading takes place through the AHB-master-port. If the LLI loading is done directly, the loading of DMACCxControl (which happens after LLI-register-loading) takes place through the master-port.

Source address register

This register can be updated from three sources, namely, AHB Slave port, channel logic and LLI-load-block. There cannot be a simultaneous write from more than one source (By design, mutual exclusivity is ensured between Source address register accesses from Channel logic and LLI-load-block. It is assumed that Channel registers will not be written when the DMAC is active, hence the AHB Slave port accesses to this register are mutually exclusive with the channel logic and LLI-load-block).

- The write from AHB-slave-port takes place during programming of the channel, during which SrcAddrWrEn goes high.
- When source-transactions for the channels are performed, the AHB addresses corresponding to the transactions are fed back into the channel and clocked into source-address-register with each successful transaction. This is termed as a write by channel-logic and is indicated by SrcAddrUpdate.
- At the end of the packet, if the DMAC self-programs its channel through LLI loading, then the arrival of source-address on the DMAC-AHB-master-data-bus is indicated by LLISrcWr. When LLISrcWr is high, ChWrDataSrc (which is assigned the 32 bit data from the AHB master port) is clocked into source-address-register.

Destination address register

This register can be updated from three sources, namely, AHB Slave port, channel logic and LLI-load-block. There cannot be a simultaneous write from more than one source, as in the case of the Source address register.

- The write from AHB-slave-port takes place during programming of the channel, during which DstAddrWrEn goes high.
- When destination-transactions for the channels are performed, the AHB addresses corresponding to the transactions are fed back into the channel and clocked into destination-address-register with each successful transaction. This is termed as a write by channel-logic and is indicated by DstAddrUpdate.
- At the end of the packet, if the DMAC self-programs its channel through LLI loading, then the arrival of destination-address on the DMAC-AHB-master-data-bus is indicated by LLIDstWr. When LLIDstWr is high, ChWrDataDst (which is assigned the 32 bit data from the AHB master port) is clocked into destination-address-register.

Channel control register (higher 20 bits)

This register's upper-20-bits are treated separately as the upper bits are updated from two sources – AHB-slave-port and LLI-loader-block. There cannot be a simultaneous write from more than one source.

- The write from AHB-slave-port takes place during programming of the channel, during which CntlWrEn goes high and upper 20 bits of AHB slave data-bus are clocked into the DMACCxControl (x = 0 and 1) register.
- At the end of the packet, if the DMAC self-programs its channel through LLI loading, then the arrival of control-register-contents on the DMAC-AHB-master-data-bus is indicated by LLICntlWr. When LLICntlWr is high, ChWrDataCntl (which is assigned the upper-20-bits of 32 bit data from the AHB Master port) are clocked into the upper-20-bits of the DMACCxControl (x = 0 and 1) register.

Transfer-Size-Source counter (lower 12 bits of channel control register)

This register-field is updated from four sources, namely, AHB slave port, LLI-load-block, channel-logic in Destination-Flow-Control-mode and channel-logic in DMAC-Flow-Control-mode. There cannot be a simultaneous write from more than one source.

- The write from AHB-slave-port takes place during programming of the channel, during which CntlWrEn goes high.

- At the end of the packet, if the DMAC self-programs its channel through LLI loading, then the arrival of control register contents (and hence TrfSizeSrc too) on the DMAC-AHB-master-data-bus is indicated by LLICntlWr. When LLICntlWr is high, ChWrDataTrf (which is assigned the 12-bit data from AHB Master port) is clocked into TrfSizeSrc counter.
- In Destination-Flow-Control-mode, the TrfSizeSrc counter is used for tracking Source transfers. (It is to be noted that TrfSizeSrc has got no conceptual relevance in Destination-Flow-Control-mode). Whenever the LdSrcInDstFlow (an input from DMACChReqProc block) is asserted, the TrfSizeSrc counter is loaded with a value, which will fetch the data required by destination.
- If the transfers are progressing in DMAC-Flow-Control-mode, the TrfSizeSrc counter is decremented with each transfer.

Transfer-Size-Destination counter (TrfSizeDst)

TrfSizeDst counter is only applicable in DMAC-Flow-Control-mode. This counter tracks the destination transfers in DMAC-Flow-Control-mode. A different counter from TrfSizeSrc is needed, as the source-request and the destination requests are given separate Terminal-Count-signals (TC). TrfSizeDst counter is loaded with TrfSizeDstCo whenever the channel is enabled, or loading of an LLI is finished. TrfSizeDstCo is "TrfSizeSrc (Transfer Size in accordance to source width) multiplied by ratio of DWidth to SWidth". Numerically,

$$\text{TrfSizeDst} = \text{TrfSizeSrc} * (\text{DWidth}/\text{SWidth}).$$

When the destination transfers progress, TrfSizeDst decrements with each successful destination transfer.

Channel LLI register

This register is updated from two sources, namely, AHB Slave port and LLI-load-block. There cannot be a simultaneous write from more than one source.

- The write from AHB-slave-port takes place during programming of the channel, during which LLIWrEn goes high.
- At the end of the packet, if the DMAC self-programs its channel through LLI loading, then the arrival of LLI-register-contents on the DMAC-AHB-master-data-bus is indicated by LLILLIRegWr. When LLILLIRegWr is high, ChWrDataLLIReg (which is assigned the 32-bit data from AHB Master port) is clocked into channel LLI register.

Channel configuration register (Bit 18 and 16-downto-1)

Bit 17 (also called 'active-bit') is skipped because it is a status bit (read-only) which gives a '1' on read-back if FIFO is non-empty. If active bit is zero, it indicates that the FIFO is empty. Bit-0 (also called ChannelEn bit) is skipped because it is physically separate from rest of DMACChConfig register. The rest of the configuration-register is updated from only one source i.e. AHB-slave-side, while programming the DMAC.

Channel enable bit (ChannelEn) (DmacCxConfig register-bit 0)

ChannelEn is a control-bit when it is written-into from AHB slave side. A write of '1' into channel-enable bit makes the channel ready for transactions. ChannelEn bit may be read back through the AHB slave interface to check the 'active-status' of the channel. So ChannelEn doubles up as a status bit during read. Writing a zero into ChannelEn bit is not immediately reflected with a read-back of zero. Instead, writing a zero in ChannelEn is considered as a disable request from software. Only after the source and destination state machines have come to the respective IDLE states after finishing the ongoing set of AHB transfers, ChannelEn is actually taken to zero. ChannelEn is also de-asserted if the channel received an error response on the AHB during source, destination or LLI transactions.

5.5.2 DMAC Channel Source-Transfer Block (DmacChSrcXfer)

Details of the DMAC-channel-source-transfer-block functionality are described in this sub-section. Before description, three keywords namely transfer-size, burst-size and access-size are described. These three keywords have been repeatedly used in the document.

- **Transfer Size:** Its value is stored in TrfSizeSrc counter. In case of DMAC-Flow-Control mode, the end of packet is signalled by the expiry of TrfSizeSrc counter.
- **Burst-Size:** Its value is stored in BrstCntSrc counter. When BrstCntSrc expires, the DMACLR signal is given to the peripheral (In case of source/destination being a memory, the clear line is not toggled). BrstCntSrc is always less than or equal to TrfSizeSrc.
- **Access-Size:** Its value is stored in AxsCntSrc counter. It indicates the number of transfers “delegated” by the channel to the master-interface, to be performed on bus. The master-interface may break this into smaller AHB bursts at the memory-page boundaries. AxsCntSrc is always less than or equal to BrstCntSrc. (AxsCntSrc may be less than the DMA peripheral burst size in the case where the Channel FIFO does not contain enough empty locations to store data from one full peripheral burst).

Generation of BrstCntSrcCo

BrstCntSrcCo is used by the source-DMAC state machine to load BrstCntSrc (burst-counter, on the expiry of which, DMACLR is asserted for peripherals) and AxsCntSrc (the number of AHB transactions that the channel delegates to AHB-master-interface to perform), when moving from ST_SRC_IDLE to ST_SRC_SELECTED. In Source-Flow-Control mode BrstCntSrcCo contains a value according to the type-of-request raised. If single request is raised, then BrstCntSrcCo has a value of 1. If a burst-request is asserted, BrstCntSrcCo reflects the value of SBSIZE. In case of source being a memory (where no requests are raised), BrstCntSrcCo always reflects the value corresponding to SBSIZE.

In DMAC-Flow-Control mode, the lesser of the following two values is stored in BrstCntSrcCo - transfer-size remaining (TrfSizeSrc counter) and SBSIZE-indicated-burst-length. If a single-request is asserted, BrstCntSrcCo reflects 1. If source is a memory-device, then SBSIZE is used in the computation, since all DMA accesses to memory are bursts.

In Destination-Flow-Control mode, the lesser of the following two values is stored in BrstCntSrcCo - minimum of transfers-required-by-destination (this value is also stored in TrfSizeSrc counter) and SBSIZE-indicated-burst-length. If a single-request is asserted, BrstCntSrcCo reflects 1. If source is a memory-device, then SBSIZE is used in the computation, since all DMA accesses to memory are bursts.

Source DMAC transfer state machine

In this sub-section the state-machine handling the Source to DMAC transfer is explained. In **Figure 5.5-1**, the state diagram is shown. The details of the state-transition-triggers are described in **Table 5.5-1**. The tasks that are done on each transition are described after the table.

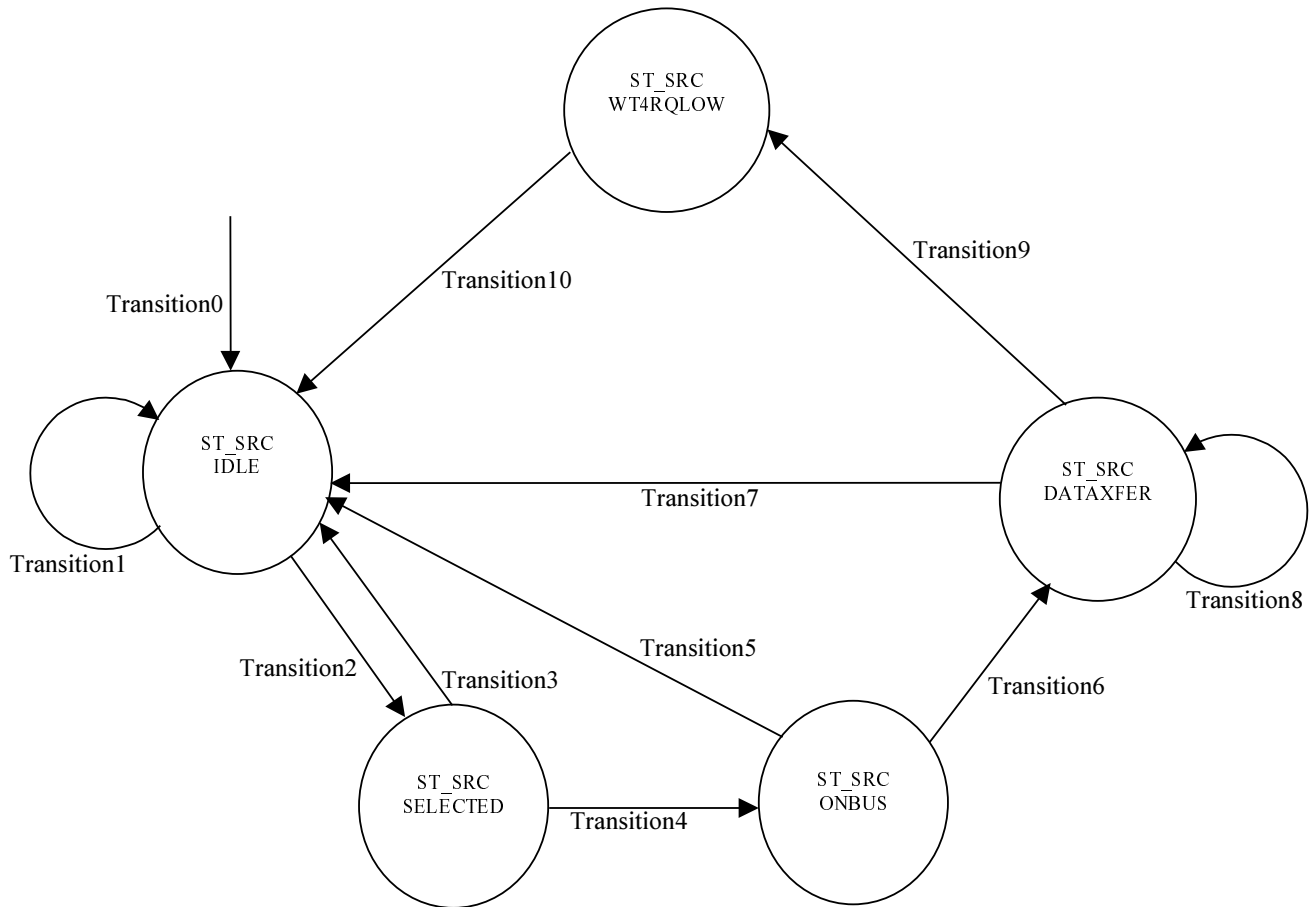


Figure 5.5-1 State machine for DMAC source-transfer block

Transition Number	Trigger Expression
Transition 0	When HRESETn is asserted, transition-0 takes place.
Transition 1	When channel is disabled (indicated by SrcDstEn = 0) before source-transactions start-off (SrcStart still not asserted), or the destination of the channel receives an error response, transition-1 takes place.
Transition 2	When source-machine gets the grant (SrcStart = 1) and no error responses are being returned (DataErrorSrc != 1), transition-2 is made.
Transition 3	When source-AHB receives an error response to the data transfer (DataErrorSrc = 1) transition-3 is made.
Transition 4	When BusAvlbl is asserted, indicating OKAY response on source AHB transition-4 is made.
Transition 5	When source-AHB receives an error response to the data transfer (DataErrorSrc = 1) transition-5 is made.
Transition 6	BusAvlbl is asserted, indicating OKAY response on source AHB, transition-6 is made.
Transition 7	When AHB returns an error response (DataErrorSrc = 1)

	OR Master-interface acknowledges an abort-request.(XferAbortedSrc = 1) OR the current access on the source AHB has terminated, although the source-burst is still ON OR the current source access as well as source-burst has completed (i.e. AxsCntSrc = 0 and BrstCntSrc = 1), but the source being a memory, the state-machine returns to IDLE instead of doing the request-clear handshake, transition-7 is made.
Transition 8	When AHB access has yet not finished (AxsCntSrc not equal to 0) and valid data has arrived (BusAvlBlSrc is high) transition-8 is made.
Transition 9	When current-source-access as well as source-burst has completed (i.e. AxsCntSrc = 0 and BrstCntSrc = 1) and the source is a peripheral (i.e. it is supposed to go through a request-clear handshake) transition-9 is made.
Transition 10	When peripheral de-asserts the request, transition 10 is made.

Table 5.5-1: Transition description for DMAC Source Transfer State Machine**Tasks done on the transition edges****Transition 0:**

- The state-machine is initialised to ST_SRC_IDLE.

Transition 1:

- The source-machine is disabled – indicated by assertion of SrcDisable (which sets a flag called DisabledSrc in DmacChRegBlock). SrcBurstOn flag is reset and BrstCntSrc counter is reset to zero on transition 1.

Transition 2:

- This transition indicates that the access has started and hence no more requests (corresponding to which the access has started) are to be entertained, until the burst completes. So SrcAxsOnMask (in DmacChReqMask) is set which masks the source-request. If the access is the first access for the burst (indicated by SrcBurstOn flag being zero), then SrcBurstOn flag is set and BrstCntSrc counter is loaded with BrstCntSrcCo. If it is not the first-access of the burst, the BrstCntSrc counter is left unchanged.

Transition 3:

- SrcAxsOnMask is reset while making transition 3. AxsCntSrc is also reset along with access-history-information (Pr1AxsCntSrc and Pr2AxsCntSrc).

Transition 4:-

- Only the state changes. No other tasks or signal changes are done.

Transition 5:

- SrcAxsOnMask is reset while making transition 3. AxsCntSrc is also reset along with access-history-information (Pr1AxsCntSrc and Pr2AxsCntSrc).

Transition 6:

- AxsCntSrc counter decrements by one and the previous value of AxsCntSrc is loaded in Pr1AxsCntSrc.

Transition 7:

- SrcBurstOn, SrcAxsOnMask and BrstCntSrc counters are reset. AxsCntSrc and its delayed versions are also reset. Moreover, SrcErr is asserted which sets a flag SrcErred in DmacChRegBlock. There is a latency between the source-machine receiving an error response and the channel getting disabled.

Transition 8:

- Transfer-size, burst-size and access-size counters decrement by one. The history of AxsCntSrc is recorded in Pr1AxsCntSrc and Pr2AxsCntSrc. FifoWrEn is asserted which clocks the incoming data into the FIFO.

Transition 9:

- This transition marks the end of transfers due to a request. Transfer-size and burst-size counters decrements by one. AxsCntSrc and its delayed versions are cleared. FifoWrEn is asserted, which clocks the incoming data into the FIFO. Since it is the end of a burst, request-clear signal is asserted. If the corresponding condition of TC for each flow-control mode is present, then TC signal is asserted.

Transition 10:

- Request-clear and TC signals are de-asserted. SrcAxsOnMask is reset.

5.5.3 DMAC Channel Destination Transfer Block (DmacChDstXfer)

Details of the DMAC-channel-destination-transfer-block functionality are described in this sub-section. As in the description of the previous section, three key-words have been extensively used – transfer-size, burst-size and access-size. In destination-transfer-block, the counter depicting the above-mentioned three variables are TrfSizeDst, BrstCntDst and AxsCntDst respectively. However, the interpretation of these three counters is the same as their counterparts in the source-transfer block. TrfSizeDst is loaded in DMAC-Flow-Control mode with a value of TrfSizeSrc multiplied by (DWidth/SWidth).

Generation of BrstCntDstCo

BrstCntDstCo is used by the source-DMAC state machine to load BrstCntDst (burst-counter, on the expiry of which, DMACLR is asserted for peripherals) and AxsCntDst (the number of AHB transactions that the channel delegates to AHB-master-interface to perform), when moving from ST_DST_IDLE to ST_DST_SELECTED.

In DMAC-Flow-Control mode, BrstCntDstCo reflects minimum of transfer-size remaining (TrfSizeDst counter) and DBSize-indicated-burst-length. If a single-request is asserted, BrstCntDstCo reflects 1. In case of destination being a memory (where no requests are raised), BrstCntDstCo always reflects the value corresponding to DBSize.

In Source-Flow-Control mode, the DMAC asserts request-clear after every access. DMAC does not wait for the burst number of transfers to get over, before giving a DMACLR. Thus BrstCntDstCo reflects the minimum of two values - DBSize-indicated-burst-length and FifoFillLevel (in term of destination-peripheral/memory-width).

In Destination-Flow-Control mode BrstCntDstCo reflects the number according to the type-of-request raised. If single request is raised, then BrstCntDstCo has a value of 1. If a burst-request is asserted, BrstCntDstCo reflects the value of DBSize. In case of destination being a memory (where no requests are raised), BrstCntDstCo always reflects the value corresponding to DBSize.

DMAC-Destination-transfer-state-machine

In this sub-section the state-machine handling the DMAC-to-destination transfer is explained. In **Figure 5.5-2**, the state diagram is shown. The details of the state-transition-triggers are described in **Table 5.5-2**. The tasks that are done on each transition are described after the table.

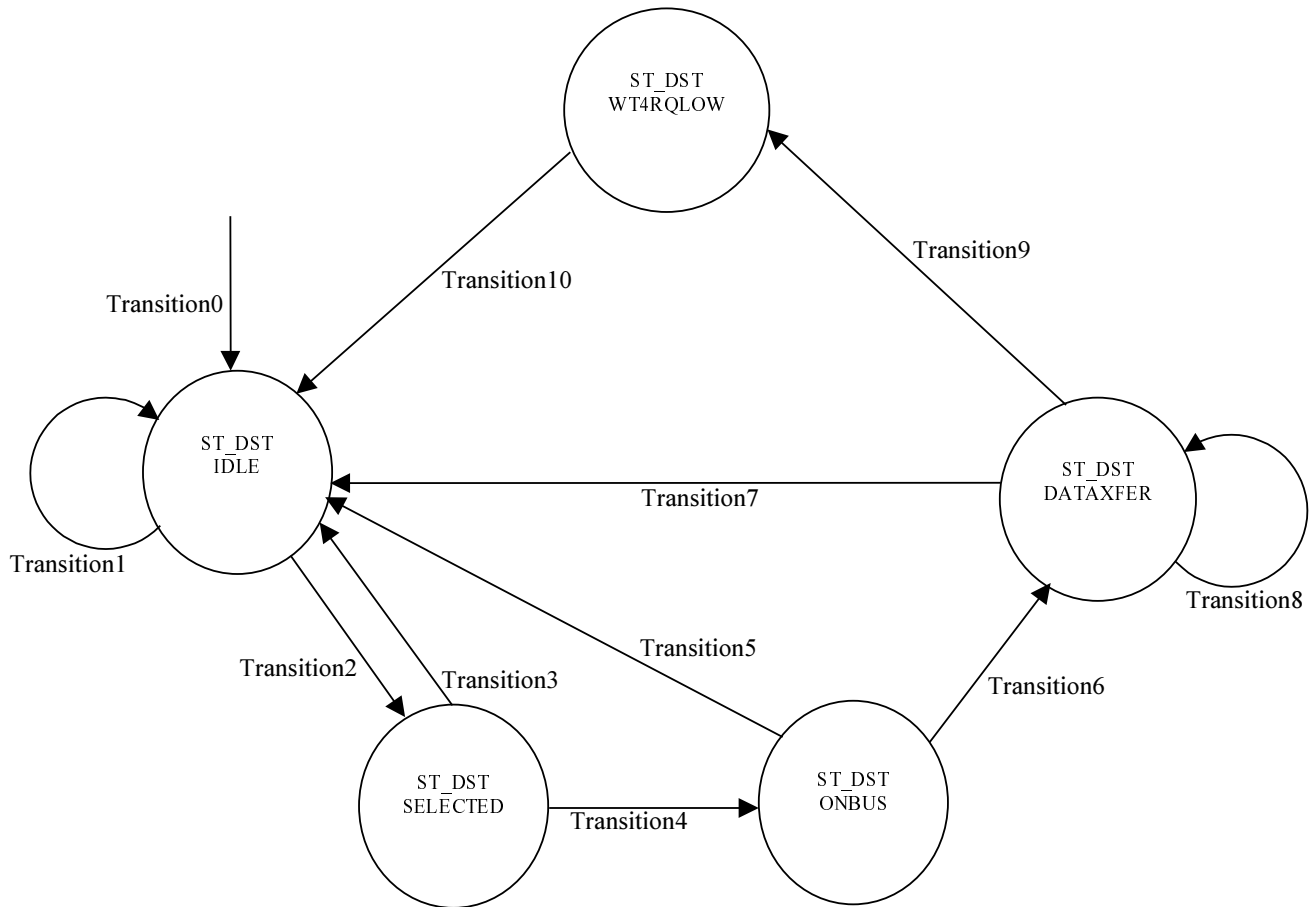


Figure 5.5-2 State machine for DMAC-destination transfer state machine

Transition Number	Trigger Expression
Transition 0	When HRESETn is asserted, transition-0 takes place.
Transition 1	When channel is disabled (indicated by SrcDstEn = 0) before destination-transactions start-off (DstStart still not asserted), or the destination of the channel receives an error response, transition-1 takes place.
Transition 2	When destination-machine gets the grant (DstStart = 1) and no error responses are returned (DataErrorDst ≠ 1), transition-2 takes place.
Transition 3	When destination-AHB returns an error to the data transfer (DataErrorDst = 1) transition-3 is made.
Transition 4	When BusAvbl is asserted, indicating OKAY response on the destination AHB, transition-4 is made.
Transition 5	When destination-AHB returns an error to the data transfer (DataErrorDst = 1) transition-5 is made.
Transition 6	BusAvbl is asserted, indicating OKAY response on destination AHB transition-6 is made.
Transition 7	When AHB returns an error response (DataErrorDst = 1)

	OR Master-interface acknowledges an abort-request (XferAbortedDst = 1) OR the current access on the destination AHB has terminated, although the destination-burst is still ON OR the current destination access as well as destination-burst has completed (i.e. AxsCntDst = 0 and BrstCntDst = 1), but the destination being a memory, the state-machine returns to IDLE instead of doing the request-clear handshake, transition-7 takes place.
Transition 8	When AHB access has yet not finished (AxsCntDst not equal to 0) and a "proper-data" has arrived (BusAvlBlDst is high) transition-8 takes place.
Transition 9	When current-destination-access as well as destination-burst has completed (i.e. AxsCntDst = 0 and BrstCntDst = 1) and the destination is a peripheral (i.e. it is supposed to go through a request-clear handshake) transition-9 takes place.
Transition 10	When peripheral de-asserts the request, transition 10 takes place.

Table 5.5-2: Transition description for Destination DMAC Transfer State Machine**Tasks done on the transition edges****Transition 0:**

- The state-machine is initialised to ST_DST_IDLE.

Transition 1:

- The destination-machine is disabled – indicated by assertion of DstDisable (which sets a flag called DisabledDst in DmacChRegBlock). DstBurstOn flag is reset and BrstCntDst counter is reset to zero on transition 1.

Transition 2:

- This transition indicates that the access has started and hence no more requests (corresponding to which the access has started) are to be entertained, until the burst completes. So a DstAxsOnMask (in DmacChReqMask) is set which masks the destination-request. If the access is the first access for the burst (indicated by DstBurstOn flag being zero), then DstBurstOn flag is set and BrstCntDst counter is loaded with BrstCntDstCo. If it is not the first-access of the burst, the BrstCntDst counter is left unchanged.

Transition 3:

- DstAxsOnMask is reset while making transition 3. AxsCntDst is also reset.

Transition 4:-

- FIFO read pointer is increased by one to fetch the next data. For this, the RdPtrInc is asserted for a clock.

Transition 5:

- DstAxsOnMask is reset while making transition 3. AxsCntDst is also reset.

Transition 6:

- AxsCntDst counter decrements by one and FIFO-read-pointer is increased by one (RdPtrInc is asserted)

Transition 7:

- DstBurstOn, DstAxsOnMask and BrstCntDst counters are reset. AxsCntDst and its delayed versions are also reset. Moreover, DstErr is asserted which sets a flag DstErred in DmacChRegBlock. There might be a latency between the destination-machine receiving an error response and the channel getting disabled.

Transition 8:

- Transfer-size, burst-size and access-size counters decrement by one. RdPtrInc is asserted for a clock, to fetch the next data.

Transition 9:

- This transition marks the end of transfers due to a request. Transfer-size and burst-size counters decrement by one. Since it is the end of a burst, request-clear signal is asserted. If the corresponding condition of TC for each flow-control mode is present, then TC signal is asserted. If there is one more LLI to load after completion of this packet, then LLIREq is set high by asserting SetLLIREq for a clock.

Transition 10:

- Request-clear and TC signals are de-asserted. DstAxsOnMask is reset.

5.5.4 DMAC Channel LLI Load Block (DmacChLLILoad)

In this section, the LLI-load state-machine is described with a state diagram in **Figure 5.5-3**. The details of the state-transition-triggers are described in **Table 5.5-3**. The tasks that are done on each transition are described after the table.

The flow of operation is similar to Source-DMAC or DMAC-destination transfer state machine. But unlike source and destination bursts, the burst length for LLI-loading is always four and the AHB access width (HSIZE) is word. The starting address of the LLI-burst is taken from the LLI field in DMACCxLLIReg register. The data corresponding to the starting address is written into DMACCxSrcAddr (x = 0 to 7) register. The next three data are written into DMACCxDestAddr, DMACCxLLIReg and DMACCxControl (x = 0 to 7) registers respectively.

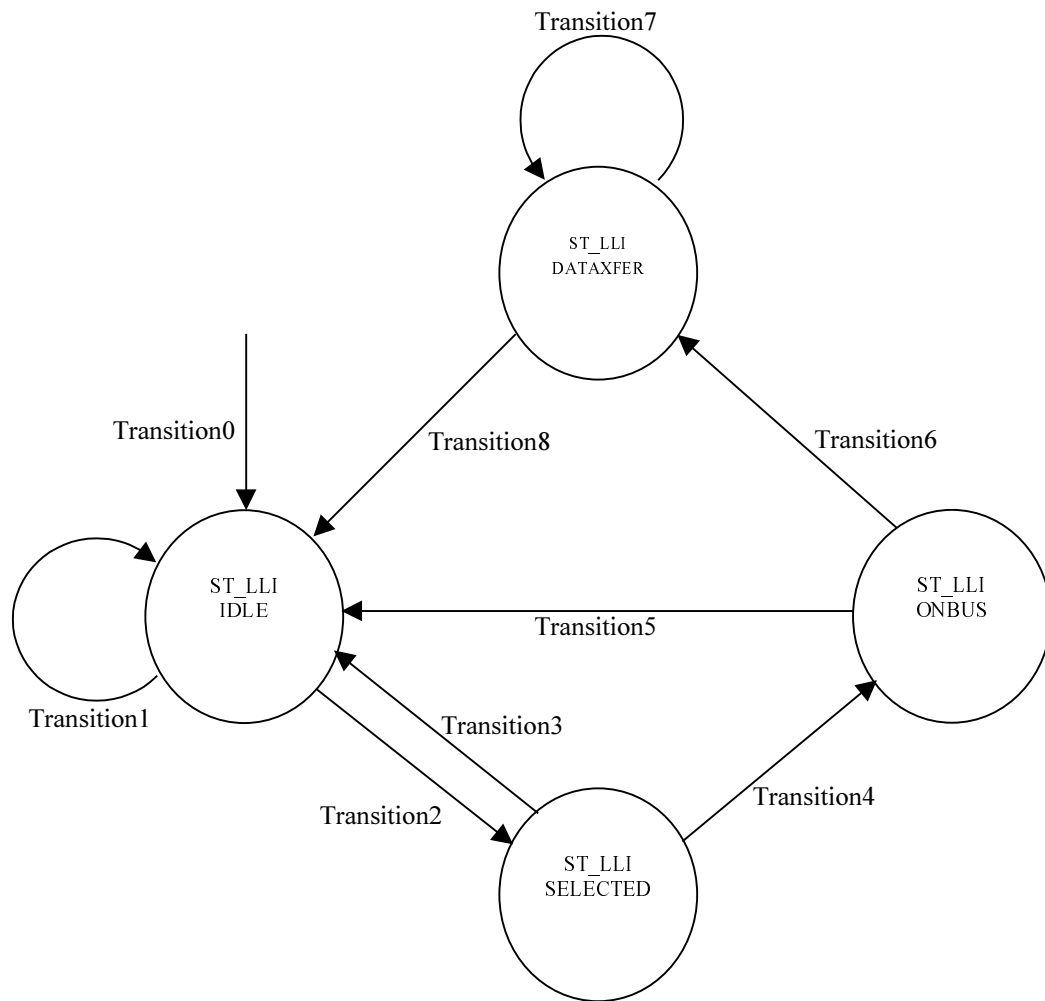


Figure 5.5-3 DMAC LLI load state machine

Transition Number	Trigger Expression
Transition 0	When HRESETn is asserted, transition-0 takes place.
Transition 1	When channel is disabled (indicated by ChannelEnLow = 1) before LLI-transactions start-off (LLIStart still not asserted), transition-1 takes place.
Transition 2	When LLI-machine gets the grant (LLIStart = 1) and no error responses are being returned (DataErrorLLI != 1), transition-2 takes place.
Transition 3	When LLI-loading-AHB receives an error response to the data transfer (DataErrorLLI = 1) transition-3 takes place.
Transition 4	When BusAvbl is asserted, indicating OKAY response, AHB transition-4 takes place.
Transition 5	When destination-AHB returns an error to the data transfer (DataErrorLLI = 1) transition-5 takes place.
Transition 6	When BusAvbl is asserted, indicating OKAY response on destination AHB, transition-6 takes place.

Transition 7	When BusAvbl is asserted, indicating OKAY response and access-count is yet to reach zero (indicating that the 4 LLI-transactions are yet to be completed), transition 7 takes place.
Transition 8	When BusAvbl is asserted, indicating OKAY response and access-count has reached zero (indicating that the 4 LLI-transactions are completed) OR AHB returns an error response while loading LLI, transition 8 takes place

Table 5.5-3 Transition conditions for LLI load state machine**Tasks done on the transition edges****Transition 0:**

- The state-machine is initialised to ST_LLI_IDLE.

Transition 1:

- LLI loading is disabled by asserting LLIDisable. This sets a flag called DisabledLLI. This flag is used by the DmacChRegBlock to shut down the channel (pulling down ChannelEn bit to zero).

Transition 2:

- Access count for LLI loading sequence (AxsCntLLI) is loaded with 4.

Transition 3:

- Access-count (AxsCntLLI) is reset to zero.

Transition 4:

- Only the state changes. No other tasks or signal changes are done.

Transition 5:

- Access-count (AxsCntLLI) is reset to zero

Transition 6:

- Access count is decreased by one and the LLI-transfer-request to AHB master-interface is pulled low (by asserting UnsetLLIReq).

Transition 7:

- Transition 7 indicates that the LLI transfers are being performed and the AHB-read-data is loaded into the respective channel registers. The value of AxsCntLLI counter determines which particular channel-register will be loaded with the incoming read data. When AxsCntLLI is 3, the first item i.e. DMACCxSrcAddr (x = 0 and 1) is loaded. Subsequently, DMACCxDstAddr, DMACCxLLIReg and DMACCxControl (x = 0 and 1) registers are loaded.

Transition 8:

- If the transition is made due to an error-response, LLIErr is asserted which is used for disabling the channel.

5.5.5 DMAC FIFO Packing Unpacking Block (DmacChPckUnpck)

This block contains packing-logic for the Channel FIFO Write data and unpacking logic for the Channel FIFO Read data. The DmacChPckUnpck block performs the following:

The AHB read data from the AHB master is driven into the channel .

The FIFO is a word-wide FIFO. If source data accesses are going on, with widths less than 32 bits, then the bytes/halfwords need to be stored into FIFO after 'packing' so that the FIFO capacity is used optimally. The packing-logic needs to write into only a portion of the FIFO-word during byte and halfword accesses. So FifoWrData and FifoWrMask both are generated from the process. The DmacChRegFile uses the FifoWrMask and FifoWrData to register the new value into the FIFO.

If the destination is narrower than a word, then only a portion of one FIFO word has to be given to the destination, at a time. This process is called unpacking and is performed in this block. This block controls the main-read-pointer (FifoRdPtr) and the sub-pointer (RdSubPtr). The data corresponding to the read sub-pointer location is combinationally driven as FIFO read data. The read-pointers are incremented according to the destination peripheral/memory's width.

The data retrieved from the FIFO is routed onto the AHB data bus.

The DmacChPckUnpck block also generates the fill-level of the channel FIFO which is required by the source and destination state machines for loading their transfer-counters.

5.5.6 DMAC Channel Register File (DmacChRegFile)

This block is the data storage area for the channel FIFO. It contains four 32-bit registers. There are read-pointers and write-pointers for writing into the register-file. The read and write pointers are implemented to have a FIFO data structure. The signal name for read and write pointers are FifoRdPtr and FifoWrPtr respectively and are input to the module from the packing-unpacking block. The functionality described in this block is write-logic and read-logic. The details are as follows.

FIFO write logic: Whenever the FifoWrEn signal goes high, data present in FifoWrData is written into the FIFO. However, the 'write' is done using a mask. It might happen that 'data – narrower than 32 bits' is to be written into the FIFO. The mask ensures that the narrow data is written only into specified byte-lane(s) and data in other byte-lanes remain unaffected.

FIFO read logic: FIFO read logic is implemented using a mux. The contents of the location pointed-to by the current value of FifoRdPtr is driven into the read databus.

5.5.7 DMAC Channel Request Processing Block (DmacChReqProc)

This block maps the DMAC-request-lines to the channel, routes the channel-information to the AHB Master port and resolves the internal-arbiter grant to start source/destination/LLI transfers. A brief listing of its functions are listed as follows.

- The transfer-abort-request for the master-AHB port (asserted when software disables the channel) from the source and destination block are ORed and sent to the Master Interface as a single ChXferAbortM.
- Resolution of the request mapped to the channel is done. This is done on the basis of SrcPeriph and DstPeriph bit-fields in DMACCxConfig (x = 0 and 1) register.
- The DMA transfer requests from the peripherals are qualified to filter out illegal requests. For example if an SREQ for destination is raised in DMAC flow control mode, it should be ignored.
- Source Request is formed by masking the 'qualified-request' with the consolidated source-mask from DmacChReqMask block.
- Destination Request is formed by masking the request with the consolidated destination-mask from DmacChReqMask block.
- LdSrcInDstFlow signal is generated, which loads the TrfSizeSrc in destination-flow-control mode. In destination-flow-control mode, the TrfSizeSrc (source-transfer-size counter) is reused for performing source transfers. (It is to be noted that the TrfSize has no functional relevance in Destination Flow Control mode). The TrfSizeSrc counter value is loaded whenever the destination request goes high and when channel is enabled.

- The clear and TC de-multiplexer for source is implemented. The source-transfer-block asserts the clear and TC. These signals are to be de-muxed to one out of 16 requestors.
- The clear and TC de-multiplexer for destination is implemented. The destination-transfer-block raises the clear and TC. These signals are to be de-muxed to one out of 16 requestors.
- The Internal Grant decipher block is implemented. When the channel is granted by the internal arbiter, it needs to be resolved to one of source/destination/LLI block grant.
- The control signals, depending on which block has been given the internal grant (i.e. source, destination or LLI block), are routed to the master interface.

5.5.8 DMAC Channel Request Masking Block (DmacChReqMask)

In DMAC, there are a number of 'masks' which when 'set', prevents the channel from requesting for transfers to AHB-master-interface. For example, once the 'servicing' (AHB transfers corresponding to a request) of a request starts, the request should be 'masked' to avoid triggering of further AHB transfers. The set of masks can be classified in three categories.

- Exclusive Masks for source-transaction related requests.
- Common masks, which 'mask' both source-transaction and destination-transaction related requests.
- Exclusive masks for destination-transaction related requests.

The salient masks, which come under the above-mentioned categories are described below.

Exclusive Masks for source-transaction related requests.:

- **SrcE2LMask (Source-Transfer-End to LLI load mask):** This is used in source-flow control mode. After the source-transfers for a packet have finished, no more source requests are serviced until the destination transfer for the packet finishes.
- **SrcAxsOnMask (Source access on mask):** When the AHB transfers corresponding to a source-request have started, no more source requests are serviced until the transfer finishes. This is taken care of by SrcAxsOnMask.
- **Halt:** This bit in DMACChConfig register indicates a halt to filling up of channel-FIFO. That is, this bit is used to stop the source-requests from requesting AHB-master-interface for transactions. Thus Halt is a mask for source-requests.
- **DisabledSrc:** If a source-packet-transfer has been finished, or it has been aborted due to channel-disable, then the source-requests are to be ignored till the channel is reprogrammed again. DisabledSrc masks the source-requests in this situation.
- **SrcTrfSizeMask (source-transfer-size mask):** This mask ensures that in DMAC-Flow-Control mode, no transfer starts off when transfer-size has been programmed as zero.
- **FifoSrcMask (FIFO source mask):** When there are no empty spaces in the FIFO to accommodate source data, then, in spite of requests, AHB transfers should not start off. This is taken care of by FifoSrcMask.
- **DestNonReqMask (destination non-requirement mask):** In destination flow control case, unless a destination request is raised, no source transfers should start off. This is taken-care-of by the DestNonReqMask.

Common masks

- **LLILoadMask (LLI loading mask):** When the LLI loading is proceeding, no source or destination transfers should be started. This is taken care of by the LLILoadMask.

Exclusive masks for destination related requests:

- **DstAxsOnMask (destination access on mask):** When the AHB transfers corresponding to a destination-request have started, no more destination requests are serviced until the transfer finishes. This is taken care of by DstAxsOnMask.

- **DisabledDst:** If a packet-transfer has been finished, or has been aborted due to channel-disable, then the destination-requests are to be ignored till the channel is reprogrammed again. In this situation the destination-requests are masked by DisabledDst.
- **DstTrfSizeMask (destination-transfer-size mask):** This mask ensures that, no transfer starts off when transfer-size has been programmed is zero.
- **FifoDstMask (FIFO destination mask):** When there is no data in the FIFO to start destination transfers, then in spite of requests, AHB transfers should not start. This is taken care of by FifoDstMask.

5.6 DMAC Response Routing Block (DmacRspRoute)

This block is used to combine the output signals from the channels (DMA responses to the DMA requestors and Interrupts to the processor) and route these to the output of the DMAC.

This module contains OR-logic to combine the ClearReqX[15:0] [X = 0 and 1 (channel number)] signals from both the channels to generate the ClearReq[15:0] internal signal. Each bit in ClearReq[15:0] corresponds to a DMA requestor. Similarly, this module contains OR-logic to combine the SigTCX[15:0] [X = 0 and 1 (channel number)] signals from both the channels to generate the SigTC[15:0] internal signal. ClearReq[15:0] and SigTC[15:0] are then passed through Integration test muxes. ClearReq[15:0] is muxed with DMACITOP1[15:0] to generate the DMACCLR[15:0] output. Similarly, SigTC[15:0] is muxed with DMACITOP2[15:0] to generate the DMACTC[15:0] output.

This module contains OR-logic to combine the IntTCChX [X = 0 and 1 (channel number)] signals from both the channels to generate the InINTTC internal signal. Similarly, this module contains OR-logic to combine the IntErrChX [X = 0 and 1 (channel number)] signals from both the channels to generate the InINTERR internal signal. Also, InINTTC and InINTERR are OR-ed to generate the internal version InINTR of the combined DMAC interrupt. InINTR, InINTTC and InINTERR are then passed through Integration test muxes to generate the DMACINTR, DMACINTTC and DMACINTERR interrupts. Since there is no separate Integration test register bit to directly control DMACINTR, the OR of bits 0 and 1 of DMACITOP3 is used. This is because the combined DMAC Interrupt is meant to be an OR of the individual DMAC interrupts.

5.7 DMAC Request Synchronising Block (DmacRqstSync)

This block consists of double synchronisers for the DMA requests. The masked version of the DMA requests (DMA requests ANDed with the DMACEn bit) are passed as inputs to this module. These inputs are then passed through two-level registering in HCLK domain for double synchronization.

5.8 Revision Designator block (DmacRevAnd)

This module contains a single AND gate to be used as a place-holder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

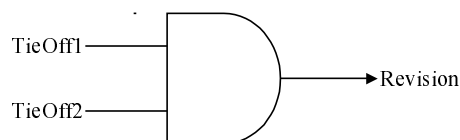


Figure 5.8-1 RevisionAnd component

This AND gate is instantiated four times at the top level, since the revision number is a four bit quantity. In the present version of the DMAC, all the inputs are tied to zero at the top level. Outputs of all AND gates are concatenated and connected to the output read multiplexer of the AHB slave interface logic. This helps software to identify the revision number through an AHB read.

5.9 Software/System Assumptions

The following software assumptions have been made regarding programming of the DMA Controller:

- There should not be any write-operation to Channel registers in an active channel, after the Channel Enable is made high. If any DMAC channel parameters are required to be reprogrammed, the reprogramming should be done after disabling the DMAC channel cleanly.
- If the Source Width is lesser than that of Destination, then the TransferSize value multiplied by the Source Width should be an integral multiple of the Destination-Width.
- When the Source peripheral is the Flow controller, and the Source Width is less than the destination width, the number of transfers performed by the Source peripheral (before asserting an LSReq/LBReq) should be such that the number of transfers multiplied by the Source Width should be an integral multiple of the Destination-Width. If the above is violated, data could get stuck and lost in the FIFO and the results are UNPREDICTABLE. The transfer can be aborted by disabling the relevant DMAC Channel.
- The SrcPeripheral and DestPeripheral bit-fields in DMACCxConfig [x = 0 and 1 (channel number)] register should not be programmed with any value greater than 15.
- The SWidth and DWidth bit-fields in DMACCxControl [x = 0 and 1 (channel number)] register should not indicate more than 32-bit wide peripheral.
- After the software disables a channel by clearing the Channel Enable bit in DMACCxConfig [x = 0 and 1 (channel number)] register, it should re-enable the bit only after it has polled a '0' in the corresponding DMACEnbldChns (Enabled Channels register) bit. This is because the actual disabling (indicated by DMACEnbldChns) does not immediately happen with the clearing of ChannelEnable bit. The latency of the ongoing AHB burst needs to be accommodated.
- The LLI field in DMACCxLLIReg [x = 0 and 1 (channel number)] should not indicate an address greater than 0xFFFFFFFF0. Otherwise, the four-word LLI burst will wrap over at 0x00000000 and the LLI-data-structure will not be in contiguous memory location.
- When the transfer size programmed in the DMAC is greater than the depth of the FIFO in a Source/Destination peripheral, the DMAC should only be programmed for non-incrementing address generation.
- On receiving the DMACCLR[x] [x = 0 to 15 (requestor number)], a peripheral is expected to de-assert all its DMACxREQ [x = S, B, LS or LB] signals, irrespective of which request the DMACCLR was asserted in response to. This is because DMACCLR is not specific to a single-request (SREQ) or burst-request (BREQ) signal. Internal to the DMAC, the handshaking of DMACCLR is done with a logical OR of all the DMA requests.
- If transfer-size field in DMACCxControl [x = 0 and 1 (channel number)] register is programmed as zero and the DMAC is the flow controller (in other flow-control modes, transfer-size field has got no meaning), then the channel does not initiate any transfers. It is the responsibility of the programmer to disable the channel by writing into the ChannelEn bit of DMACCxConfig [x = 0 and 1 (channel number)] register and re-program the channel again.
- The normal read-write tests should not be run on DMACCxControl [x = 0 and 1 (channel number)] register, as the TrfSize field is not a typical "write and read-back" register field. While writing, TrfSizeSrc bit-field is like a control-register (in the sense that it determines how many transfers the DMAC should perform). However, during read-back, TrfSizeSrc is expected to behave like a "status-register". It is supposed to return the number of transfers remaining (in terms of source width). So when TrfSizeSrc is read back, the no. of "destination-transfer-completed" (stored in a separate counter called TrfSizeDst) multiplied by a factor is returned. Hence the same physical-register is not being written-into and read-from. Thus normal write-and-read-back tests are not applicable.
- In the Destination-Flow-Control mode (with peripheral-to-peripheral transfer), if sufficient data is present in the Channel FIFO to service a DMACLSREQ or DMACLBREQ request raised by a destination peripheral (without requiring data to be fetched from the Source peripheral), the Source peripheral will not be issued a DMACTC.

- For destination flow controlled case (peripheral-to-peripheral transfer) with $DWidth < SWidth$, the number of data bytes requested by the destination peripheral must be an integral multiple of $SWidth$ (expressed in bytes). If this is not ensured, the DMAC may fetch more data from the Source peripheral than is absolutely required, resulting in data loss.
- At the end of accesses corresponding to low-priority channel, an IDLE cycle is inserted on the AHB bus to allow other masters access to the bus. This ensures that a low-priority channel does not hog the bus. It does, however, mean that the bus could be occupied by transactions corresponding to a low priority for upto 16 beats in the worst case. This applies to all transfer configurations, including M2M transfers.

6 DMAC VERIFICATION DETAILS

The DMAC Functional Verification testbench is an AHB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the **verification/vhdl** and **verification/verilog** directories respectively. The test vectors that are applied to the BusTalk testbenches are in the **verification/bustest** directory.

6.1 DMAC VHDL Verification Testbench

The ARM PrimeCell DMAC VHDL test world uses a testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide [Ref. 4].

The testbenches are designed to be used with most common VHDL simulators, and they are suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

The DMAC has one AHB slave and one AHB master port. The AHB slave port is used to program the control registers of the DMAC and its channels. The AHB master port perform the DMA transfer. Two separate AHB buses are created in the testbench, one for the AHB slave port and one for the AHB master port, so that the programming of the DMAC and both its channels is independent of the activity on the AHB master port. Moreover one AHB bus for the AHB master port also helps in exercising maximum portion of the logic at a time.

The testbench is dealing with the AHB slave port of the DMAC. The test code to the testbench consists of the transfer descriptions required to program the DMAC for the required DMA transfer, through the required channel. The testbench acts as an AHB master on this bus 1.

The DMAC is the AHB master on the AHB bus 2 and hence it initiates transactions on this bus to perform the programmed DMA transfer. The trickbox acts as a slave in these buses to respond to the transactions initiated by the DMAC. The decoder, the default slave and the master buswatcher models are added to make the AHB buses complete. These models are same as those in the given AMBA compliance test environment.

Figure 6.1-1 illustrates the Bustalk VHDL testbench for running test vectors.

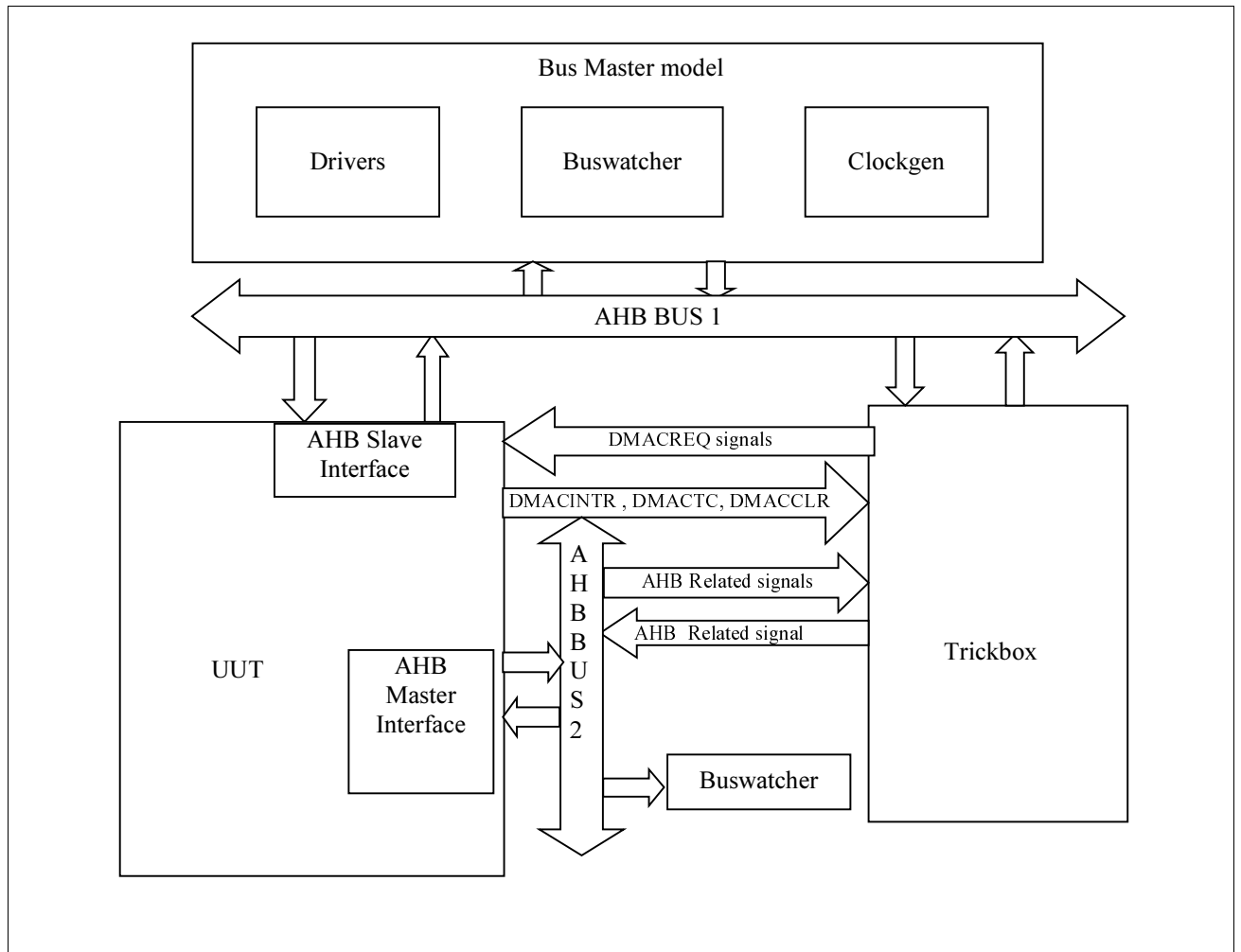


Figure 0-1 VHDL Functional Verification Testbench

Figure 6.1-2 illustrates the hierarchy for the VHDL testbench.

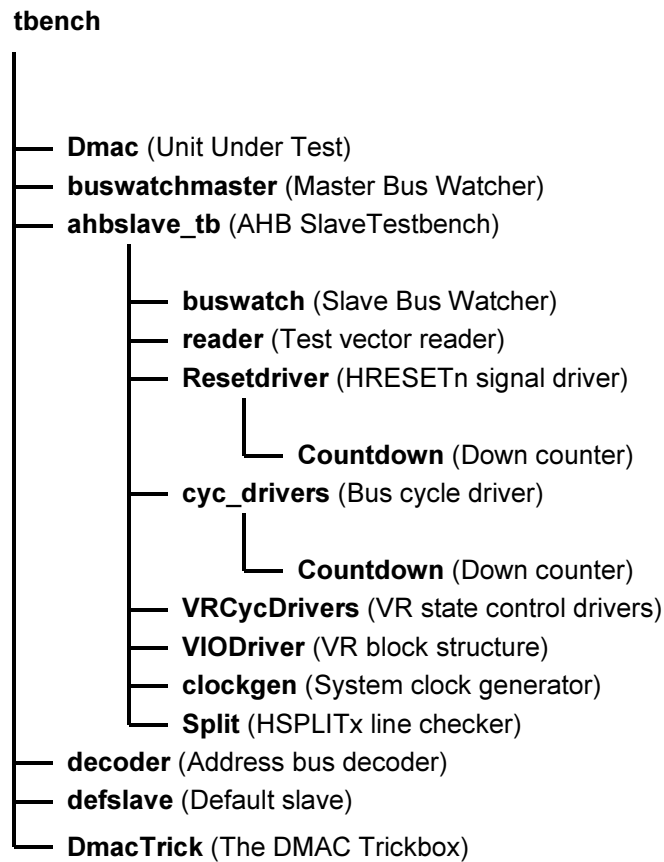


Figure 0-2 VHDL Functional Verification Testbench hierarchy

The testbench, **tbench.vhd** is used for functional testing of the DMA controller (DMAC). This block instantiates the DMAC (Unit Under Test) and the behavioural model of the trickbox on both the AHB buses. The behavioural model of the AHB Master/Arbiter connected only the AHB bus 1 to which the AHB slave ports of the UUT and the trickbox are connected. The behavioural models of the decoder, the default slave and the master bus watcher are replicated for both the AHB buses.

All the Scan input pins are tied low to keep them from interfering with the test process.

The default test frequency of HCLK is set to 100MHz. This is the same frequency to which the DMAC is constrained during synthesis. The HCLK frequency and AHB signal timing budgets are defined in the timing.vhd file.

6.1.1 Unit Under Test

The Unit Under Test may be one of the following

- **DMAC VHDL RTL**
- **DMAC scan-inserted VHDL netlist from VHDL Source**

One out of these two versions may be referenced by creating a link [named **uut**] in the **verification/vhdl** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

6.1.2 Bus Master Protocol Checker

The testbench instantiates the master bus watcher, `buswatchmaster.vhd` which checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

6.1.3 AHB Master/Arbiter Model

The UUT is stimulated primarily by AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issues commands on the AHB bus under program control.

The AHB Master/Arbiter model, `ahbslave_tb.vhd` instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the .bif file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the HRESETn signal with the correct timings. The `cyc_drivers` drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The `VRcycdrivers` selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The Split module verifies that the HMASTER number is asserted on the HSPLITx lines on the expected clock. The clockgen module generates the system clock, HCLK.

The reader reads test vectors from the `infile.bif` residing in the ***verification/bustest/invec*** directory.

Decoder

This module decodes the addresses and generates the HSELx signals for the slaves. It can generate the select signals for a maximum of 16 slaves.

The UUT (DMAC) is connected to HSEL[0] and the Trickbox is connected to HSEL[1].

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT via the HSEL[0] select line.

Accesses to the address range

0xA0000000 to 0xAFFFFFFF

will access the Trickbox via the HSEL[1] select line.

Accesses to any address outside the above ranges will select the Default Slave.

Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its HREADY output high, when other slaves are not selected. It drives a zero wait state OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

6.1.4 Trickbox

The functionality of the DMAC is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the input terminals of the DMAC and sample DMAC's outputs. The Trickbox implements a mirrored model of the DMAC within it and continuously compares the locally generated outputs with the actual outputs generated by the DMAC, on every rising edge of HCLK.

6.1.5 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the timing.vhd file in the **verification/vhdl/tbench** directory. The buswatch module residing in the **verification/vhdl/buswatcher** directory checks the Read/Write databus timings and reports any timing violations.

6.1.6 Test Vectors

The test vectors for the verification of the DMAC are coded into separate C files, in the **verification/bustest** directory. There are separate test files created for each category tests described in the verification plan. Make options have been provided to use all the test vectors together or to use them individually.

6.1.7 Generics

The DMAC testbench provides some configurability options using generics. All these generics except **SuppressOnReset** are configurable through the tbench.vhd file. **SuppressOnReset** is configurable through the ahbslave_tb.vhd file in the **verification/vhdl/tbench** directory.

XonSig

XonSig is used in the cyc_drivers.vhd file that resides in **verification/vhdl/bid**.

This generic is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If **XonSig** is set to **0** in the top level of the testbench, these signals go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the reset_driver.vhd and the cyc_drivers.vhd which reside in **verification/vhdl/bid**. It is also used in the buswatch.vhd file that resides in **verification/vhdl/buswatcher** and the VIODrivers.vhd which reside in **verification/vhdl/vr**.

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read.

HaltOnMismatch

This is used in the cyc_drivers.vhd, which resides in **verification/vhdl/bid**, the buswatch.vhd which resides in **verification/vhdl/buswatcher** and the VIODriver.vhd which resides in **verification/vhdl/vr**.

This generic is used to stop the simulation if an error occurs. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged, but the simulation continues.

Databuswidth

Databuswidth is used in the cyc_drivers.vhd, which resides in **verification/vhdl/bid**.

This generic controls the width of the AHB databus. For the DMAC testbench this is set to **32**.

SuppressOnReset

SuppressOnReset is used in the buswatch.vhd, which resides in **verification/vhdl/buswatcher**.

This generic is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If **SuppressOnReset** is TRUE, the set-up and hold timing violations on these signals are not flagged when

HRESETn is asserted. If **SuppressOnReset** is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When **SuppressOnReset** is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

6.1.8 Running Simulations

The **README** file in the **verification/vhdl/tbench** directory gives step by step instructions for running simulations using the DMAC test vectors on the VHDL source code.

Run time logs for all the combinations are available in the following files in the **verification/vhdl/Dmac_rtl** and **verification/vhdl/Dmac_scaninsert_vhdl_net_max** directories.

DmacCh0_rtl_modelsim.log (vhdl) - Channel tests on Channel0

DmacCh1_rtl_modelsim.log (vhdl) - Channel tests on Channel1

DmacCorner_rtl_modelsim.log (vhdl) - Corner2 tests on all channels.

Dmac_scaninsert_vhdl_vhdl_net_max_modelsim.log - Channel tests for default channel(channel0).

6.2 DMAC VERILOG VERIFICATION TESTBENCH

The ARM PrimeCell DMAC Verilog test world uses a testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide [Ref. 4].

The test bench is designed to be used with most common Verilog simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The test bench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

The DMAC has one AHB slave and one AHB master port. The AHB slave port is used to program the control registers of the DMAC and both its channels. The AHB master port perform the DMA transfer. Two separate AHB buses are created in the testbench, one for the AHB slave port and one for the AHB master port, so that the programming of the DMAC and its channels is independent of the activity on the AHB master port. Moreover one AHB bus for the AHB master port also helps in exercising maximum portion of the logic at a time.

The testbench is dealing with the AHB slave port of the DMAC. The test code to the testbench consists of the transfer descriptions required to program the DMAC for the required DMA transfer, through the required channel. The testbench acts as an AHB master in this bus.

The DMAC is the AHB master in the other bus and hence it initiates transactions on these buses to perform the programmed DMA transfer. The trickbox acts as a slave in this bus to respond to the transactions initiated by the DMAC. The decoder, the default slave and the master buswatcher models are added to make the AHB buses complete. These models are same as those in the given AMBA compliant test environment.

Figure 6.2-1 illustrates the BusTalk Verilog testbench for simulating test vectors.

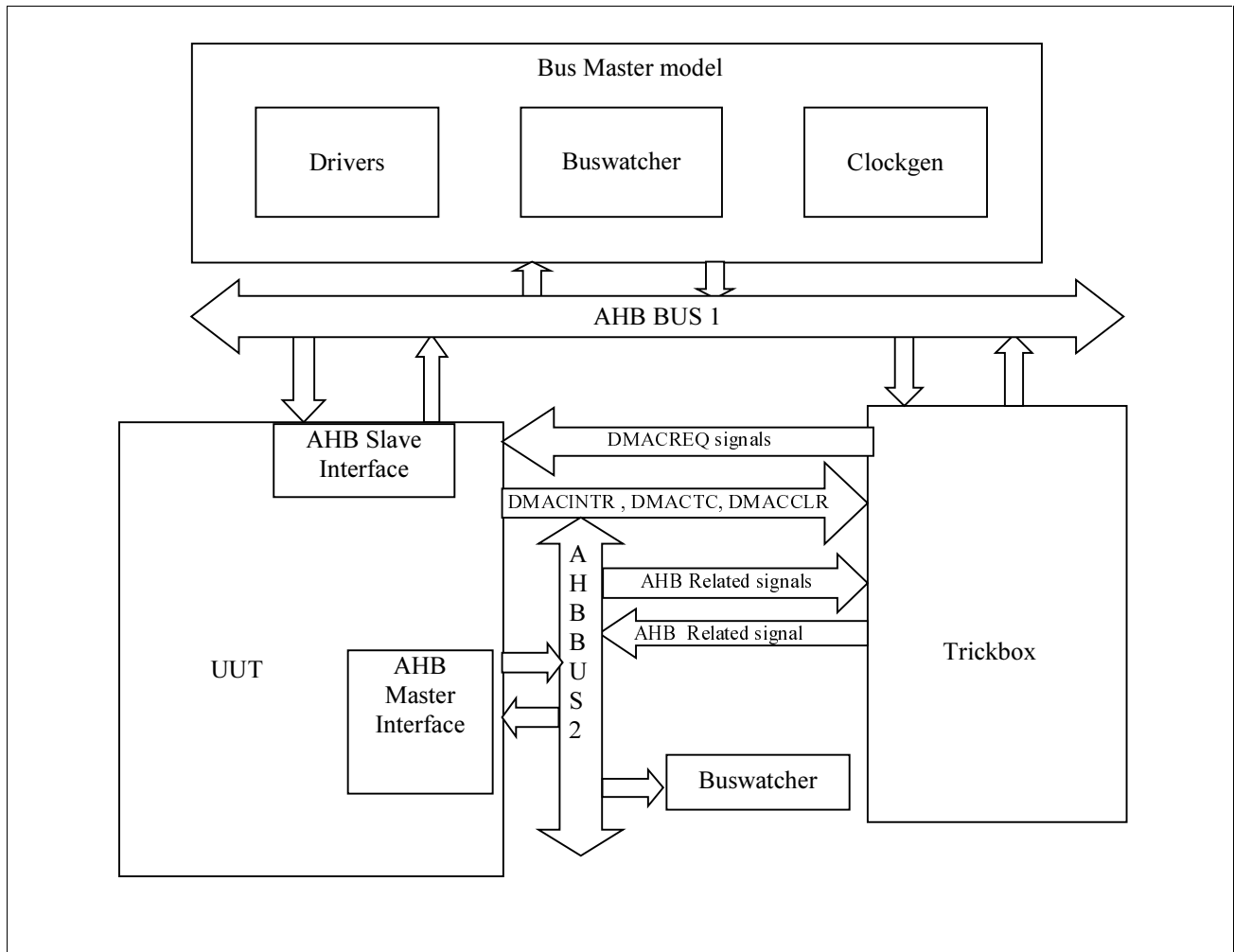


Figure 6.2-1 Verilog Functional Verification Testbench

Figure 6.2-2 illustrates the hierarchy for the Verilog testbench.

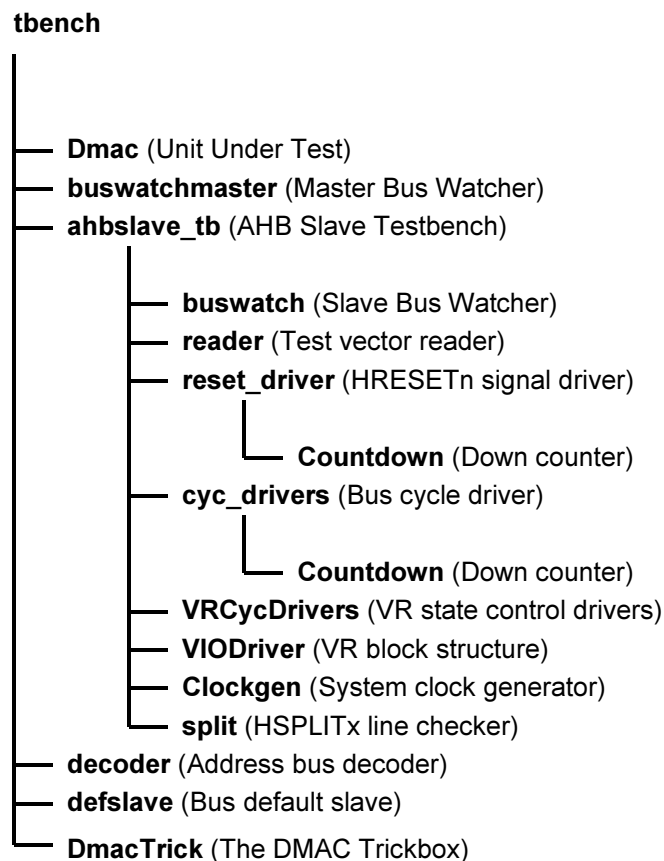


Figure 6.2-2 Verilog Functional Verification Testbench hierarchy

The testbench, **tbench.v** is used for functional testing of the Dma controller (DMAC). This block instantiates the DMAC (Unit Under Test), the behavioural model of the trickbox on all 2 AHB buses. The behavioural model of the AHB Master/Arbiter connected only the AHB bus 2 to which the AHB slave ports of the UUT and the trickbox are connected. The behavioural models of the decoder, the default slave and the master bus watcher are replicated in all AHB buses.

All the Scan input pins are tied low to keep them from interfering with the test process.

The default test frequency of HCLK is set to 100MHz. This is the same frequency to which the DMAC is constrained during synthesis. The HCLK frequency and AHB signal timing budgets are defined in the timing.v file.

6.2.1 Unit Under Test

The Unit Under Test may be one of the following

- **DMAC Verilog RTL**
- **DMAC scan-ready Verilog netlist from Verilog Source**
- **DMAC non-scan Verilog netlist from VHDL Source**

One out of these three versions may be referenced by creating a link [named **uut**] in the **verification/verilog** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

6.2.2 Bus Master Protocol Checker

The testbench instantiates the master bus watcher, `buswatchmaster.v`, which checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and flags warning and error messages if any timing or protocol violations are detected during simulations.

6.2.3 AHB Master/Arbiter Model

The UUT is stimulated primarily by AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issues commands on the AHB bus under program control.

The AHB Master/Arbiter model, `ahbslave_tb.v` instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the `.sim` file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the `HRESETn` signal with the correct timings. The `cyc_drivers` drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The `VRcycdrivers` selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The split module verifies that the `HMASTER` number is asserted on the `HSPLITx` lines on the expected clock. The Clockgen module generates the system clock, `HCLK`.

The reader reads test vectors from the `bif.sim` residing in the ***verification/bustest/invec*** directory.

Decoder

This module decodes the addresses and generates the `HSELx` signals for the slaves. It can generate the select signals for a maximum of 16 slaves.

The UUT (DMAC) is connected to `HSEL[0]` and the Trickbox is connected to `HSEL[1]`.

Any access to the address range

0x50000000 to 0x5FFFFFFF

will access the UUT via the `HSEL[0]` select line.

Accesses to the address range

0xA0000000 to 0xAFFFFFFF

will access the Trickbox via the `HSEL[1]` select line.

Accesses to any address outside the above ranges will select the Default Slave.

Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its `HREADY` output high, when other slaves are not selected. It drives a zero wait state OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

6.2.4 Trickbox

The functionality of the DMAC is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the input terminals of the DMAC and sample DMAC's outputs. The Trickbox implements a mirrored model of the DMAC within it and continuously compares the locally generated outputs with the actual outputs generated by the DMAC, on every rising edge of `HCLK`.

6.2.5 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the `timing.v` file in the **`verification/verilog/tbench`** directory. The buswatch module residing in the **`verification/verilog/buswatcher`** directory checks the Read/Write databus timings and reports any timing violations.

6.2.6 Test Vectors

The test vectors for the verification of the DMAC are coded into separate C files, in the **`verification/bustest`** directory. There are separate test files created for each category tests described in the verification plan. Make options have been provided to use all the test vectors together or to use them individually.

6.2.7 Parameters

The DMAC testbench provides some configurability options via parameters. All the parameters are configurable through the `tbench.v` file.

XonSig

XonSig is used in the `cyc_drivers.v` file that resides in **`verification/verilog/bid`**.

This parameter is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If **XonSig** is set to **0** in the top level of the testbench, these signals go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the `reset_driver.v` and the `cyc_drivers.v` which reside in **`verification/verilog/bid`**. It is also used in the `buswatch.v` that resides in **`verification/verilog/buswatcher`** and the `VIODrivers.v` which reside in **`verification/verilog/vr`**.

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read.

HaltOnMismatch

This is used in the `cyc_drivers.v`, which resides in **`verification/verilog/bid`**, the `buswatch.v` which resides in **`verification/verilog/buswatcher`** and the `VIODriver.v` which resides in **`verification/verilog/vr`**.

This parameter is used to stop the simulation if an error occurs. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the `buswatch.v`, which resides in **`verification/verilog/buswatcher`**.

This parameter is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If **SuppressOnReset** is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If **SuppressOnReset** is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When **SuppressOnReset** is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

TestMode

TestMode is used in the `cyc_drivers.v`, which resides in ***verification/verilog/bid***.

If the **TestMode** parameter is set, the testbench is configured to run in Big Endian mode. By default this parameter is cleared.

Databuswidth

Databuswidth is used in the `cyc_drivers.v`, which resides in ***verification/verilog/bid***.

This parameter controls the width of the AHB databus. For the DMAC testbench this is set to **32**.

6.2.8 Running Simulations

The **README** file in the ***verification/verilog/tbench*** directory gives step by step instructions for running simulations using tests on the Verilog source code.

Run time logs for all the combinations are available in the following files in the ***verification/verilog/Dmac_rtl***, ***verification/verilog/Dmac_noscan_vhdl_net_min*** and ***verification/verilog/Dmac_scanready_verilog_net_typ*** directories.

<i>DmacCh0_rtl_modelsim.log (verilog)</i>	- Channel tests on Channel0
<i>DmacCh1_rtl_modelsim.log (verilog)</i>	- Channel tests on Channel1
<i>Dmac_noscan_vhdl_verilog_net_min_modelsim.log</i>	- Tests on the verilog netlist with vhdl rtl source
<i>Dmac_scanready_verilog_verilog_net_typ_LDV.log</i> rtl source	- Tests on the verilog scan ready netlist with verilog

6.3 Trickbox Description

The Trickbox is a programmable AHB device designed to provide a test engineer a means to drive the non-AHB input pins of the DMAC and also to sample the non AHB output responses from it.

The Trickbox monitors the activities on the AHB Bus, Clock by Clock. The Trickbox has a behavioural DMAC (consisting of AHB Slave interface, 2 AHB Master Interfaces, Internal Arbiter, Endianization block, Channel control logic and Data Router), Memory modules, Peripheral block, HGRANT generation block and a Checker box. When the UUT is programmed the Trickbox also gets programmed, as there are mirror registers. In this way the Data Integrity checking is done and also the performance of the DMAC gets monitored.

The basic blocks of the trickbox are illustrated in **figure 4.1-1 Block diagram of the trickbox**.

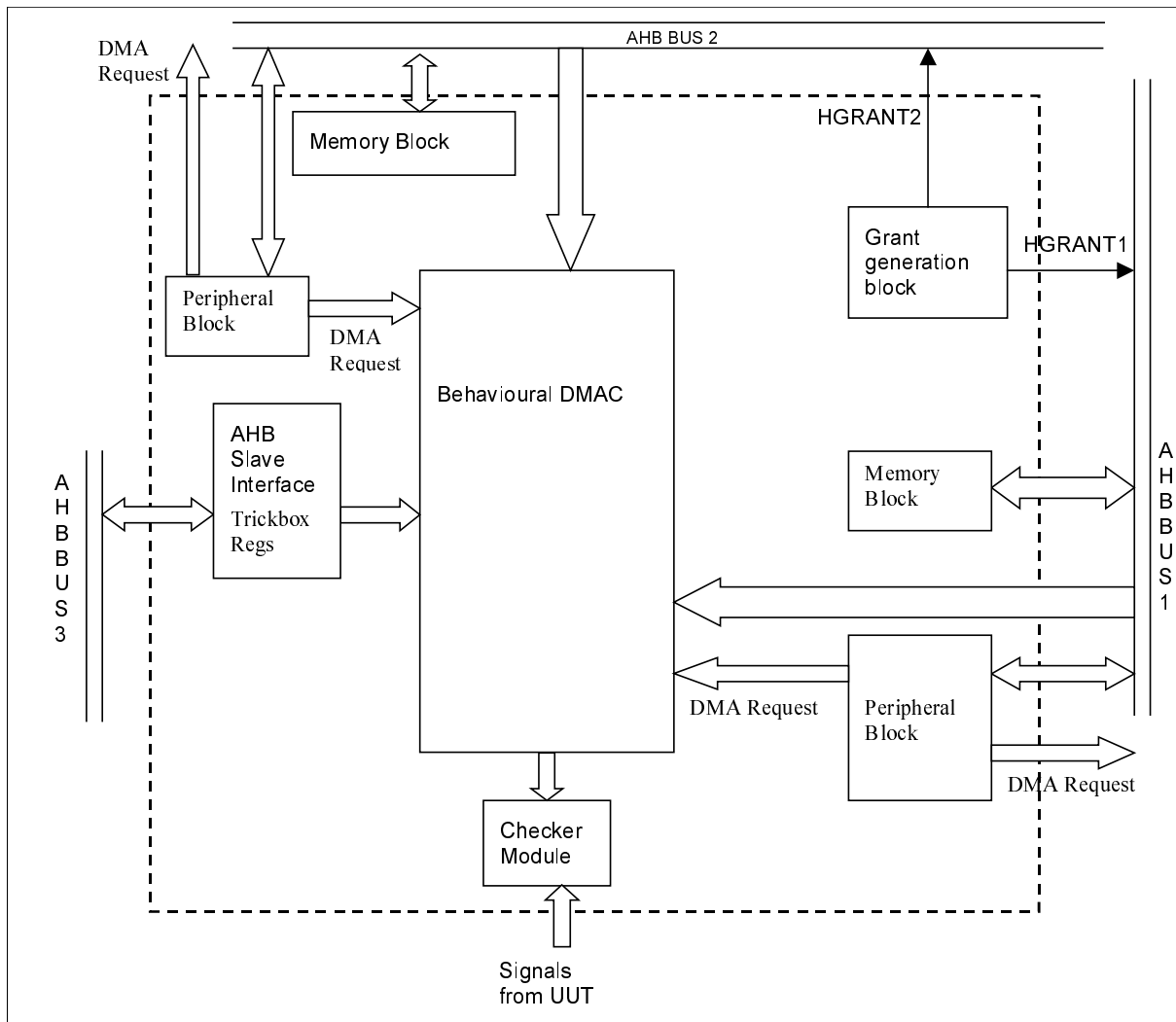


Figure 4.1-1 Block diagram of the trickbox

6.3.1 Trickbox register summary

All DMAC's channel registers are mirrored in the trickbox. So, the HSELDMAC signal of the trickbox is used to access the registers common to both Trickbox and DMAC and the HSELT signal of the Trickbox is used to access Trickbox specific registers. Hence, all the programmable registers in Trickbox, common to both DMAC and Trickbox would be selected for write when the corresponding register in DMAC is being written. AHB read access to the DMAC registers are ignored by the trickbox.

The memory and peripheral models in the trickbox have control registers, which can be accessed through HSELT of the trickbox. The details of these registers are provided with the memory and peripheral block descriptions.

6.3.2 Behavioural Model of DMAC

The trickbox has a behavioural model of DMAC to check the performance of DMAC. This behavioural DMAC monitors the activities at all the input signals of the DMAC and generate the expected values of all the output signals of the DMAC on a cycle by cycle basis. The behavioural DMAC gets enabled only in functional mode. In test mode (i.e. when Integration test is enabled in DMACITCR register) behavioural DMAC is disabled. Register bits are provided to suitably disable the protocol checkers in Checker Module.

The block diagram of the behavioural model of DMAC is provided in **figure 4.1-2**

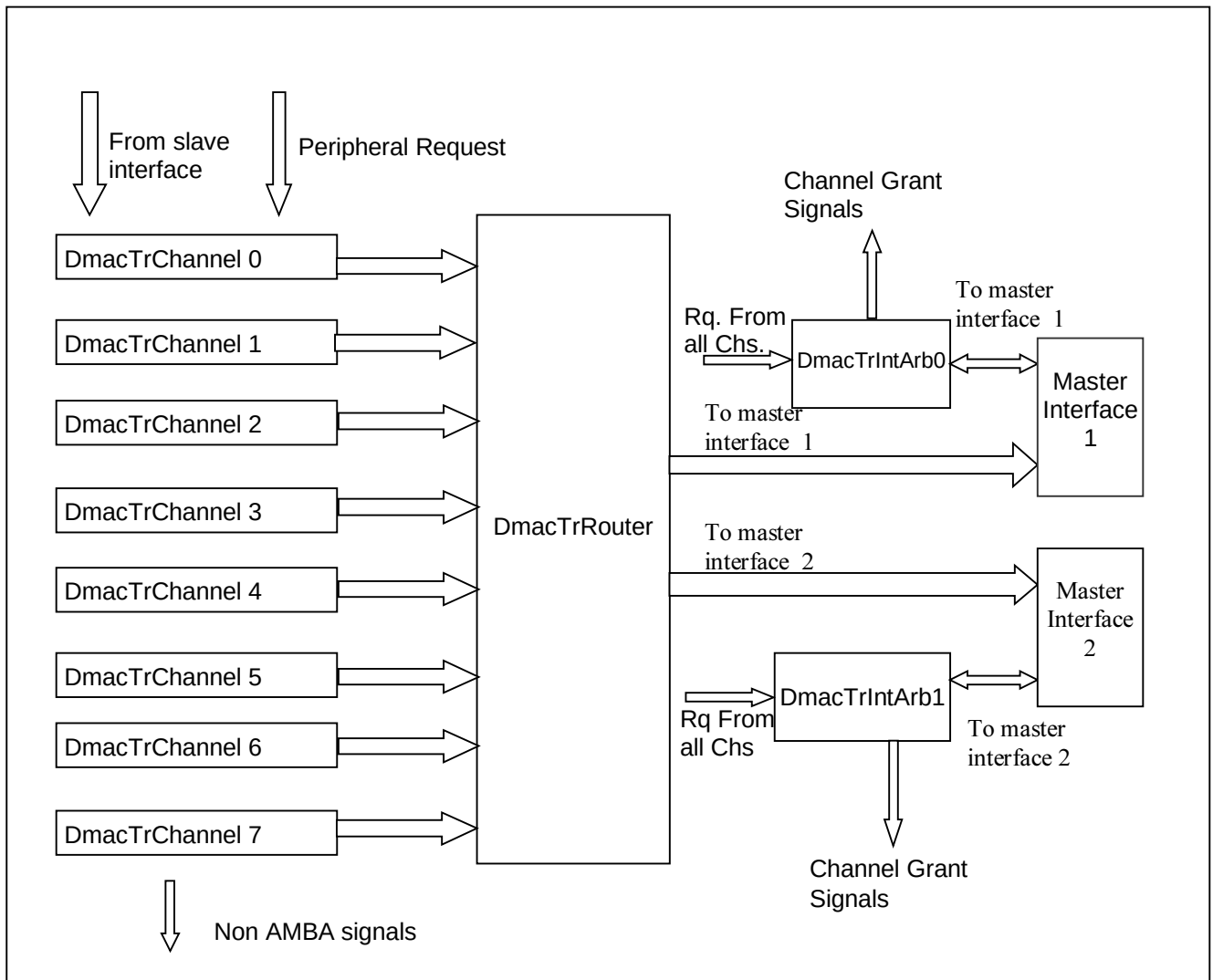


Figure 4.1-2 Block diagram of the behavioural DMAC

The following are the internal blocks of the behavioural DMAC in the trickbox.

Channel Control Logic (DmacTrChLogic)

This block is responsible for carrying out all channel related activities like, updating channel registers, generating interrupts, generating DMACCLR and DMACTC, loading the Burst and TC counters, moving the data from FIFO to peripheral and vice-versa. This module consists of a channel state machine(SM) whose state bits are used to control most of activities.

The Channel registers are updated based on Write Enables from Slave Side, Master Side (When LLI Loading is happening) and when transactions proceed for this channel. It has a FIFO, which is 4 word deep. The block diagram showing input and output signals are shown in **figure 4.1-3**

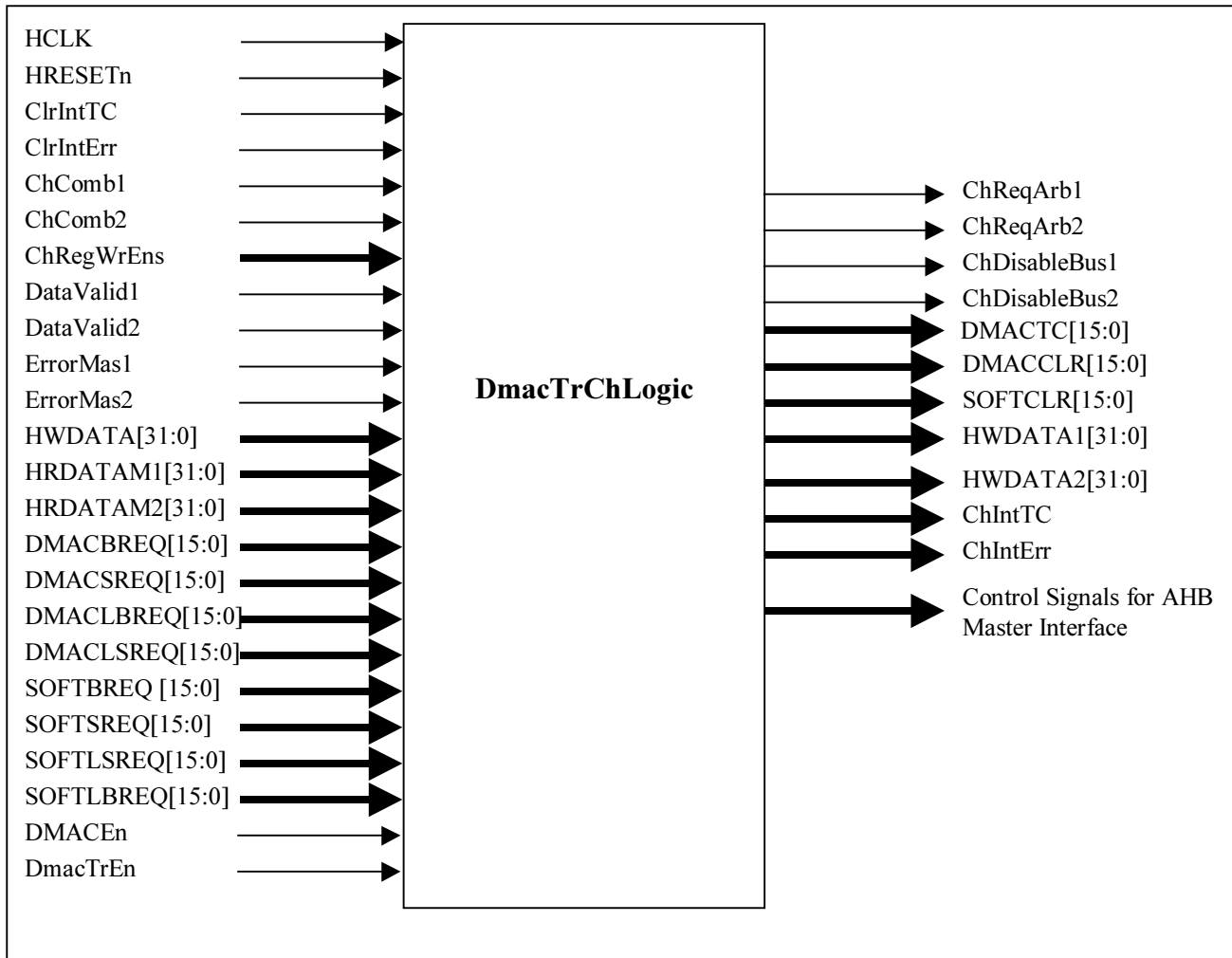


Fig 4.1-3 Block Diagram of DMAC Channel Logic

The request to the internal arbiter is generated after sampling the DMACxReq lines for that particular channel. For memory transactions and LLI loading, the requests are asserted internally and de asserted at the penultimate transfer. The request lines are masked out on following conditions.

- When SM has moved into source transfer state then SrcMask is set, when SM has moved into destination transfer state then DstMask is set.
- When DMACCLR or DMACTC is asserted for source or destination peripheral then corresponding mask's are set.
- When the Destination is the flow controller, the Source request is masked out till destination has come up with the request. This ensures that no prefetching of data is done.
- During LLI load both the source and destination request are masked out.
- When error happens then corresponding mask's are set.
- When source request is raised but there is no sufficient space to accommodate source data then SrcMask is set. When destination request is raised but there is no sufficient data to drive out for destination peripheral then DstMask is set.

Whenever data is available in the FIFO and the destination peripheral comes up with the request then the destination peripheral is given the priority.

The state machine with the conditions for transition is shown in *fig 4.1-4*.

The SM takes care of following conditions.

- When a burst or single is followed by a single, and if this channel happens to get regranted (from internal arbiter) then this information is missed out. This happens because the SM leaves a particular state only after it receives the DataValid for the last transaction committed by AHB Master interface. Whenever this condition happens then the Grant obtained from Internal Arbiter is registered. This registered grant helps in deciding the next transition for the SM.
- Wrong interpretation of the DataValid, When the state machine moves out of ST_CH_IDLE State. This happens because when the SM moves out of ST_CH_IDLE State, In this clock if the DataValid is received then it is for the previous channel's transfer. If the Last transfer of the previous channels dataphase is stretched then also an indication that the DataValid received is for previous channel's extended transfer is indicated by NotValidData signal. This signal gets set when the Channel is granted and remains set till the Address phase for the first transaction (this channel's) is over.

The logic has following states:

ST_CH_IDLE: In this state the SM (State Machine) waits for the channel to be selected. If the peripherals are on the same bus then depending on whether SrcSelComb or DstSelComb is asserted, the SM advances to ST_CH_SRC_DMA_XFER or ST_CH_DMA_DST_XFER State respectively. When advancing to these states the channel resources are outputted for AHB Master interface. If the peripheral are on different bus then SM advances to ST_CH_PERI_DUAL_BUS. The channel resources are outputted for both the Master Interfaces.

ST_CH_SRC_DMA_XFER: The SM stays in this state till the DataValid for last beat of the BeatCount is received. Since by the time the DataValid is seen for the last beat of the BeatCount, the Channel could have been regranted or other channels might have been granted. For the case where the same channel gets regranted, we will be missing the grant signal generated from internal Arbiter. Hence when the SM is in this state, and ChComb signal is seen again, then this signal gets registered. This signal would go to zero at the time of transition to other state.

This state bit information is used to pack the data before writing into the FIFO based on the endianness, and the width of the Source peripheral.

ST_CH_DMA_DST_XFER: The SM can enter this state from ST_CH_IDLE or ST_CH_SRC_DMA_XFER state. This state is similar to ST_CH_SRC_DMA_XFER State. Here also registering of Channel Grant is done as explained earlier. When the DataValid for the last beat is received the SM leaves this state to ST_CH_LLI_LOAD State if the next LLI pointer is not null. Otherwise it would move to ST_CH_SRC_DMA_XFER State if source is selected. If source is not selected it moves to ST_CH_IDLE State. The DMAEndian module uses this state information to remove the data from FIFO after suitably unpacking.

ST_CH_PERI_DUAL_BUS: The SM can enter this state when the peripherals are on different bus. In this state also care has been taken so that the grant is not missed and the DataValid's are interpreted properly. The transition from this state happens only when the DataValid for the Destination's last beat is received and LLI loading is required. Otherwise it will move to ST_CH_IDLE State. Since both source transfers and destination transfers are happening at the same time an extra signal to indicate which transactions are going on in this state are indicated by DstTxrOn and SrcTxrOn signals. These signals get set when SM enters this state and source or destination transfers are happening. This signal gets cleared when the DataValid for the Last Beat is received and the SM moves out of this state. The SM can move out of this state to load the LLI or else to restart from ST_CH_IDLE State. This state information is used to pack and unpack the data in FIFO.

ST_CH_LLI_LOAD: The SM can enter this state after finishing destination transfers. In this state a fixed beat of 4 words is done. After the DataValid for the Last Beat is received the SM will move to ST_CH_IDLE State.

The DMACCLR and DMACTC counters are loaded depending on the Flow Control. Depending on the flow control appropriate DMACXREQ lines are sampled and the burst, transfer size is loaded at the beginning of the transfer. The DMACCLR and DMACTC signal are asserted at the end of the dataphase of the last beat of the burst.

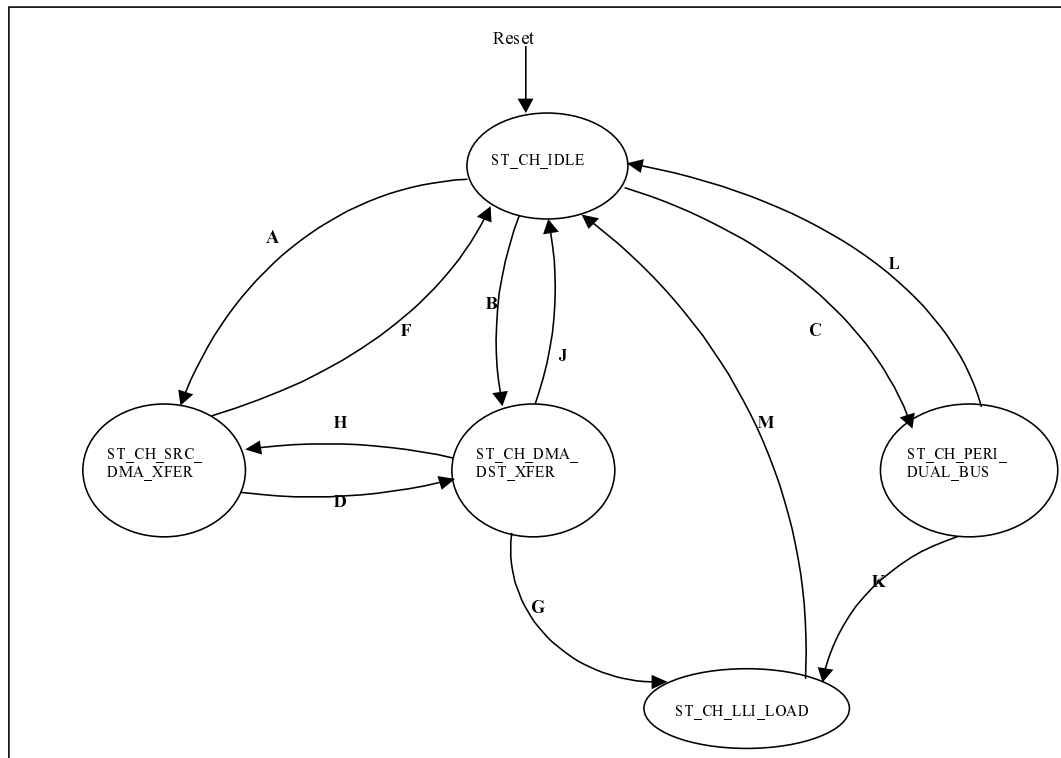


Fig 4.1.4 State Machine for Channel Logic

CONDITIONS:

A: (ChComb = 1) and (SrcSelComb = 1) and (SameBus = 1)

B: (ChComb = 1) and (DstSelComb = 1) and (SameBus = 1)

C: (ChComb = 1) and (DiffBus = 1)

D: (SrcBeat = 1) and (DataValid = 1) and (NotValidDataSrc = 0) and ((DstSelComb || DstSelReg) = 1)

F: (SrcBeat = 1) and (DataValid = 1) and (NotValidDataSrc = 0) and (DstSelComb || DstSelReg = 0)

G: (DstTC = 1) and (LLILoad not eq 0) and (DataValid = 1) and (NotValidDataDst = 0)

H: (DstBeat = 1) and (DataValid = 1) and (NotValidDataDst = 0) and ((SrcSelReg || SrcSelComb) = 1)

J: (DstBeat = 1) and (DataValid = 1) and (NotValidDataDst = 0) and ((SrcSelComb || SrcSelReg) = 1)

K: (DstTxrOn = 1) and (DstTC = 1) and (DataValid = 1) and (NotValidDataDst = 0) and (LLILoad not eq 0) and (SrcTxrOn = 0)

L: ((DstTC = 1) and (SrcTxrOn = 0) and (DataValid = 1) and (NotValidDataDst = 0) and (LLILoad = 0)) or (DstErrOccrd = 1)

M: ((LLIBeat = 1) and (DataValid = 1) and (NotValidDataLLI = 0)) or (LLIErrOccrd = 1)

Internal Arbiter (DmacTrIntArb)

This block uses fixed priority scheme to decide which channel needs to get serviced. The priority decreases from Channel0 to Channel7. I.e., Channel 0 has the highest priority and Channel 7 has the lowest priority. The block diagram showing the input and output signals is shown in **fig 4.1.5**.

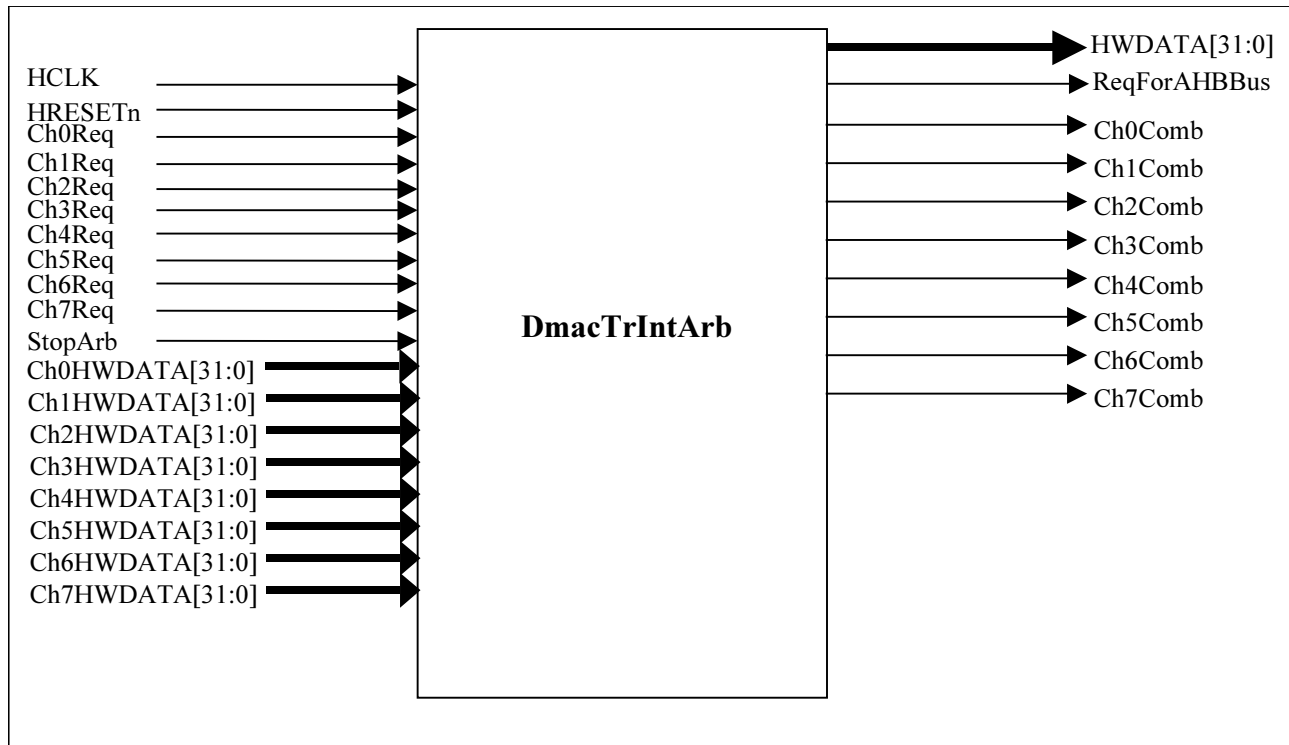


Fig 4.1.5 Block Diagram of Internal Arbiter

The inputs to this block come from channel requests and a signal StopArb (from AhbMaster interface) which indicates as to when the internal arbiter needs to start arbitration. This module is implemented such that if a Channel gets selected, then till the transactions for this channel decided by the ChBeatCount is put on to the AHB, other channel's request are masked out.

Here fixed priority is implemented. The signal ChArbMaskOn indicates whether any channel got selected. It is this signal which prevents any further re-arbitration when a channel is selected, even though StopArb goes low. The ChxComb signal is registered to generate ChArbMaskOn.

ReqForAhbBus is generated by ORing all channel select signals. The master interface samples this signal to drive out HBUSREQx signal.

Address and Data Router (DmacTrRouter)

This block is responsible for routing the channel resources like Address, HLOCK, HPROT and Burst information to the appropriate Master depending on which channel is granted. Since the AHB Master interface expects the Channel information when ChComb signal from Internal Arbiter is high, this module gates this signal with each channel's resources. The block diagram is shown in **fig 4.1.6**.

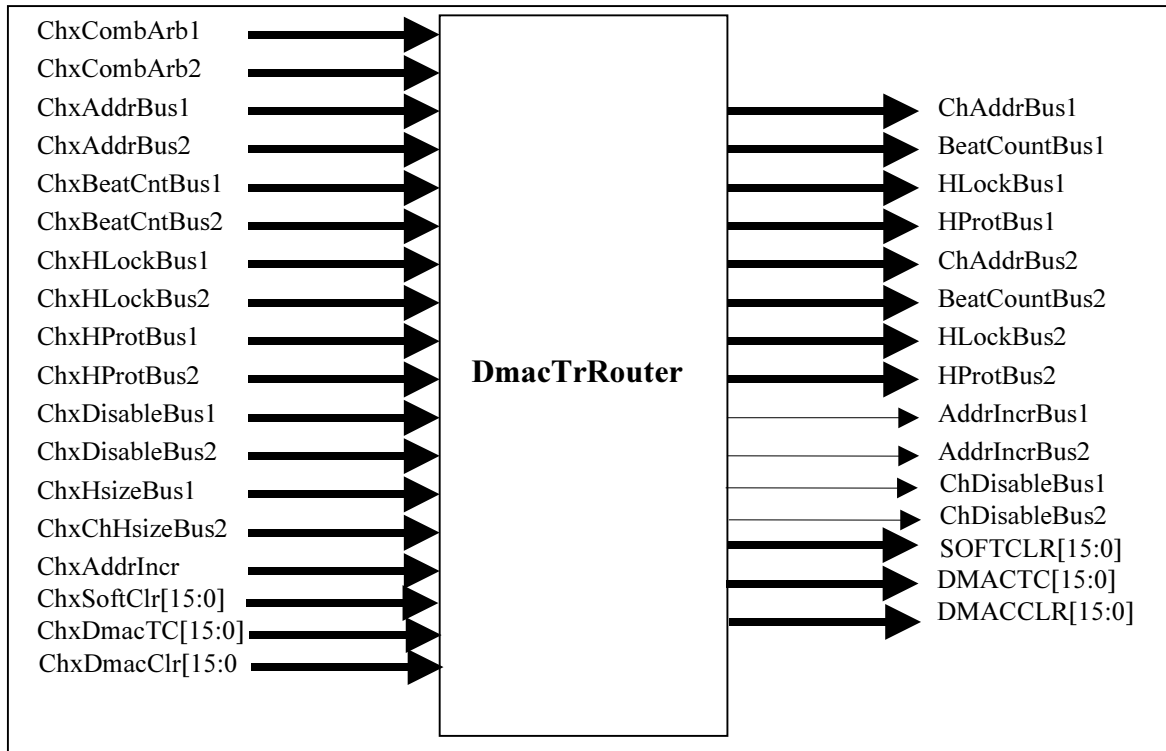


Fig. 4.1.6 Block Diagram for Data Router

AHBSlave Interface (DmacTrAhbSlaveIf)

This Interface block is provided for programming the Trickbox registers. If a Write or a Read happens to the Channel registers then the corresponding Write Enables and Read Enables are generated for Channel Logic block. All writeable registers of the DMAC are mirrored in the Trickbox. So, the HSELDMAC signal of the UUT is used to access the registers common to both Trickbox and DMAC. Hence, all the programmable registers in Trickbox, common to both DMAC and Trickbox would be selected for write when the corresponding register in DMAC is being written. AHB read accesses to the DMAC are ignored by the trickbox. The Mirrored registers are shown below.

Register	Address (Base + N)	W	Size	Description
DmacIntTCClrTr	0x008	W	7-0	When writing to this register, each data bit that is HIGH that causes the corresponding bit in the DmacIntTCStat register to be cleared. Data bit those are high does not have any effect on the corresponding bit in the register.
DmacIntErrClrTr	0x010	W	7-0	When writing to this register, each data bit that is HIGH that causes the corresponding bit in the DmacIntErrStat register to be cleared. Data bit those are high does not have any effect on the corresponding bit in the register.
DmacSoftBReqTr	0x020	W	15-0	This register allows DMAC Burst Requests to

				be generated by software.
DmacSoftSReqTr	0x024	W	15-0	This register allows DMAC Single Requests to be generated by software.
DmacSoftLBReqTr	0x028	W	15-0	This register allows DMAC Last Burst Requests to be generated by software.
DmacSoftLSReqTr	0x02C	W	15-0	This register allows DMAC Last Single Requests to be generated by software.
DmacConfigTr	0x030	W	2-0	This register is used to configure the DMA controller.
DmacSyncTr	0x034	W	31-0	This register is used to enable the synchronisers for a particular group of DMAC requests.
DmacC0SrcAddrTr	0x100	W	31-0	DMAC channel 0 source address.
DmacC0DstAddrTr	0x104	W	31-0	DMAC channel 0 destination address.
DmacC0LLITr	0x108	W	31-4, 0	DMAC channel 0 linked list address.
DmacC0CtrlTr	0x10C	W	31-0	DMAC channel 0 control.
DmacC0ConfigTr	0x110	W	15-0	DMAC channel 0 configuration register.
DmacC1SrcAddrTr	0x120	W	31-0	DMAC channel 1 source address.
DmacC1DstAddrTr	0x124	W	31-0	DMAC channel 1 destination address.
DmacC1LLITr	0x128	W	31-4, 0	DMAC channel 1 linked list address.
DmacC1CtrlTr	0x12C	W	31-0	DMAC channel 1 control.
DmacC1ConfigTr	0x130	W	17-0	DMAC channel 1 configuration register.
DmacC2SrcAddrTr	0x140	W	31-0	DMAC channel 2 source address.
DmacC2DstAddrTr	0x144	W	31-0	DMAC channel 2 destination address.
DmacC2LLITr	0x148	W	31-4, 0	DMAC channel 2 linked list address.
DmacC2CtrlTr	0x14C	W	31-0	DMAC channel 2 control.
DmacC2ConfigTr	0x150	W	17-0	DMAC channel 2 configuration register.
DmacC3SrcAddrTr	0x160	W	31-0	DMAC channel 3 source address.
DmacC3DstAddrTr	0x164	W	31-0	DMAC channel 3 destination address.
DmacC3LLITr	0x168	W	31-4, 0	DMAC channel 3 linked list address.
DmacC3CtrlTr	0x16C	W	31-0	DMAC channel 3 control.
DmacC3ConfigTr	0x170	W	15-0	DMAC channel 3 configuration register.
DmacC4SrcAddrTr	0x180	W	31-0	DMAC channel 4 source address.

DmacC4DstAddrTr	0x184	W	31-0	DMAC channel 4 destination address.
DmacC4LLITr	0x188	W	31-4, 0	DMAC channel 4 linked list address.
DmacC4CtrlTr	0x18C	W	31-0	DMAC channel 4 control.
DmacC4ConfigTr	0x190	W	17-0	DMAC channel 4 configuration register.
DmacC5SrcAddrTr	0x1A0	W	31-0	DMAC channel 5 source address.
DmacC5DstAddrTr	0x1A4	W	31-0	DMAC channel 5 destination address.
DmacC5LLITr	0x1A8	W	31-4, 0	DMAC channel 5 linked list address.
DmacC5CtrlTr	0x1AC	W	31-0	DMAC channel 5 control.
DmacC5ConfigTr	0x1B0	W	17-0	DMAC channel 5 configuration register.
DmacC6SrcAddrTr	0x1C0	W	31-0	DMAC channel 6 source address.
DmacC6DstAddrTr	0x1C4	W	31-0	DMAC channel 6 destination address.
DmacC6LLITr	0x1C8	W	31-4, 0	DMAC channel 6 linked list address.
DmacC6CtrlTr	0x1CC	W	31-0	DMAC channel 6 control.
DmacC6ConfigTr	0x1D0	W	17-0	DMAC channel 6 configuration register.
DmacC7SrcAddrTr	0x1E0	W	31-0	DMAC channel 7 source address.
DmacC7DstAddrTr	0x1E4	W	31-0	DMAC channel 7 destination address.
DmacC7LLITr	0x1E8	W	31-4, 0	DMAC channel 7 linked list address.
DmacC7CtrlTr	0x1EC	W	31-0	DMAC channel 7 control.
DmacC7ConfigTr	0x1F0	W	17-0	DMAC channel 7 configuration register.
DmacTCRTr	0x500	W	9,8,0	This register is used to select the various test modes. This register is cleared upon reset.

There are several trickbox specific registers. When trickbox tries to access this register HSELT is used.

Register	Address (Trickbox Base + N)	W	Size	Description
ReqConfig	0x00C	W	17-0	This register helps in programming peripheral and memory modules on different busses.
GrantCount0	0x004	W	31-0	This register gives the provision of programming HGRANT0 generation.
GrantCount1	0x008	W	31-0	This register gives the provision of

				programming HGRANT1 generation.
DmacTrEn	0x00C	W	1	This register is responsible for enabling the trickbox module.

AHBMaster Interface (DmacTrAhbLite)

This block is responsible for bringing out necessary bus transfers based on the Address and control information given by the DMA Core logic block. The bus signals that got generated in this module are not put on the bus but are compared in the Checker module with respect to the signals generated from the UUT. This master behaves similar to UUT Master block.

Diagram of this module is shown in the **Fig 4.1.9** below.

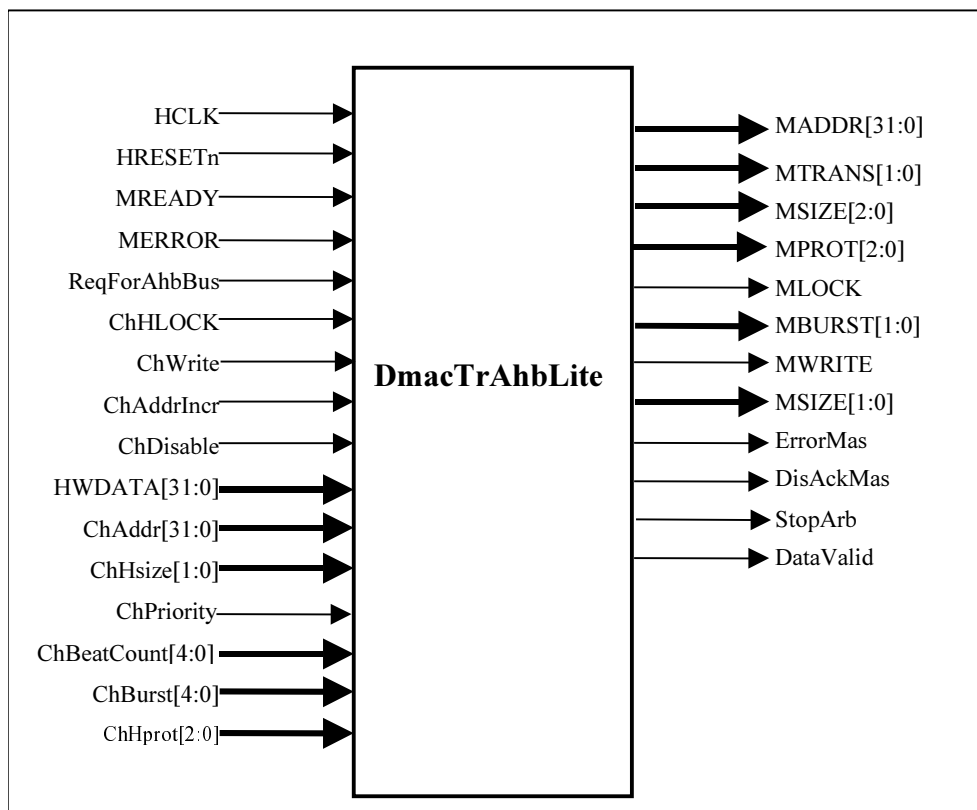


Figure 4.1-9 AHB Lite Master Interface diagram

This block has been implemented as a state machine.

The state transition diagram is as shown in **Figure 4.1-10**

The logic has the following states:

ST_AHBM_INIT: The ReqForAhbBus is sampled in this state. The transfer starts when the MREADY is sampled asserted. Here check is also done for feasibility of doing fixed increment transfers depending upon

the Address crossover of 1KB boundary. The State Machine moves to Address Transfer State (ST_AHBM_ADDRXFR) and the MTRANS are put as NSEQ.

ST_AHBM_ADDRXFR: This state is used for pipelining the address for next cycle. The MTRANS put while in this state always indicate a data transfer (i.e. MTRANS are always NSEQ or SEQ for this state). Hence the Next State after this one is always the ST_AHBM_ACTIVE State. Quite a bit of logic resource is common to this and ST_AHBM_ACTIVE State. And the state machine may alternate between this and the ST_AHBM_ACTIVE State during a committed amount of transfer. A crossover of 1KB range is also checked in this state. The next access is pipelined so that either a new burst is pipelined or the next sequential transfer of same burst is initiated.

The detection of new burst is as follows:

- AHB 32 bit Address Output bits crosses 1KB boundary. For example if HSIZE = BYTE and burst size is 256. Then if AHB 32 bit Address Output bits 9:0 are all 1's, indicates 1KB boundary cross over.
- Beat Count for current burst becomes = 1 indicating that current transfer is last being pipelined for the current ongoing burst.

ST_AHBM_ACTIVE: In this state the valid data transfer takes place. After every MREADY received with an OKAY response, the data valid for that particular transaction is routed. The transition to this state is always from ST_AHBM_ADDRXFR State.

In this state the MTRANS are changed for new burst on following conditions

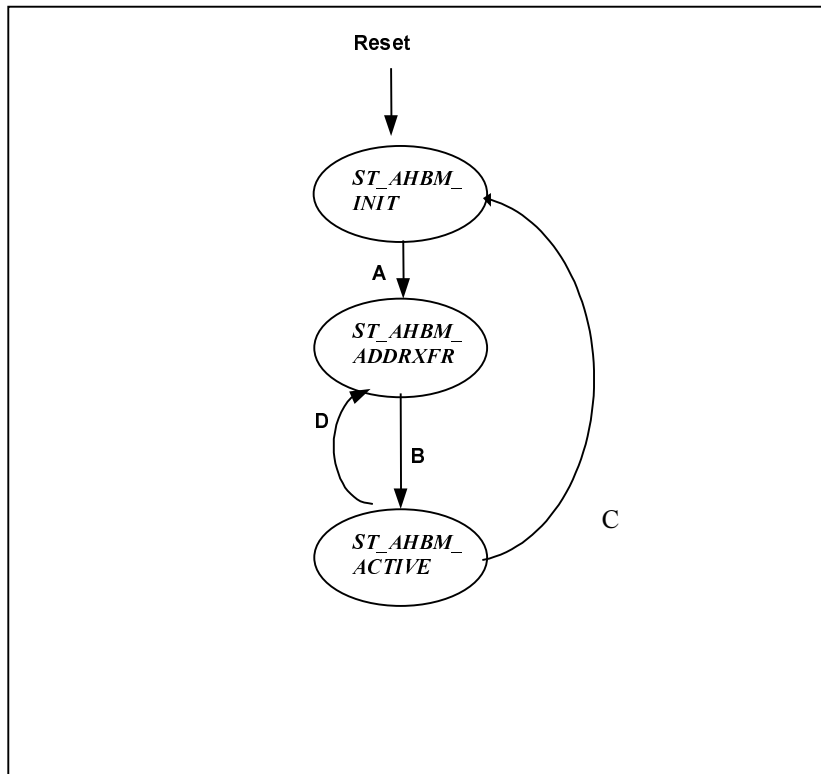
- AHB 32 bit Address Output bits indicating 1KB boundary cross over.
- Beat Count for current burst becomes = 1 indicating that current transfer is last being pipelined for the current ongoing burst, but given that the Number of beats committed for transfer are greater than 1.
- The MTRAN put for the previous cycle was IDLE, this may occur because of no further request from channel.

When in this state the BeatCount Committed for previous Request become equal to 1 and the MTRANS are put for a valid data transfer value (NSEQ or SEQ), indicating that the last of committed Number of transfers is being piped onto the AHB bus, the State machine again samples the Request and it may reinitiate the new committed transfer if channel continues to request for more data at this instant of sampling.

While in this state if the response is received as ERROR then the state machine moves to ST_AHBM_INIT and generates the "AHB Master Bus Error" signal.

While in this state if the committed number of transfer becomes one then, SM remains in this state by putting MTRANS as IDLE.

While in this state if the data for the last beat of the committed transfer is issued, and IDLE is sampled then the state machine will move to ST_AHBM_ADDRXFR State

**CONDITIONS :**

- A. MREADY; The request is asserted when ReqForAhbBus is sampled one in this state. MTRANS are put NSEQ before transitioning to ST_AHBM_ADDRXFR state. Depending upon BeatCount a Counter is maintained, this is called as Committed Number Of transfers or just ReqCount.
- B. MREADY is sampled asserted.
- C. At the end of transferring all the committed count if the ReqForAhbBus is sampled zero the state machine goes to ST_AHBM_INIT state. Or when Error response is sampled.
- D. At the end of active transfer no further request is there from channel immediately after Address phase but request comes when state machine is in ST_AHBM_ACTIVE state.

Figure 4.1-10 AHB Master Interface State Machine**6.3.3 Grant Generation Block (DmacTrGntGen)**

This block is responsible for generating Bus Grant (HGRANTx) signals for UUT and Trickbox. The different schemes used to generate grant are,

- Default granted method.
- Toggle method.
- Request based method without count dependent.
- Request based method with count dependent.
- Break in the middle of burst for fixed number of clock.
- Break in the middle of burst for 1 HCLK.

6.3.4 Checker Block

This block is responsible for doing the entire clock by clock comparisons of the UUT output with the expected value, which is getting generated in the Trickbox (All AHB Master1 and Master2 signals, and Non-AMBA signals).

A block diagram of the checker block is in **figure 4.1-11**. The list of signals, which gets compared with the UUT, is given below in table.

Signal Type	Signal Name
AHB Signal	HADDRM[31:0]
AHB Signal	HTRANSM[1:0]
AHB Signal	HBURSTM[1:0]
AHB Signal	HLOCKM
AHB Signal	HBUSREQM
AHB Signal	HSIZEM[2:0]
AHB Signal	HWDATAM[31:0]
AHB Signal	HPROTM[2:0]
AHB Signal	HWRITEM
DMAC Signal	DMACINTERR
DMAC Signal	DMACINTTC
DMAC Signal	DMACINTR
DMAC Signal	DMACCLR[15:0]
DMAC Signal	DMACTC[15:0]

Table 4.1-1 List of signals compared in the checker block

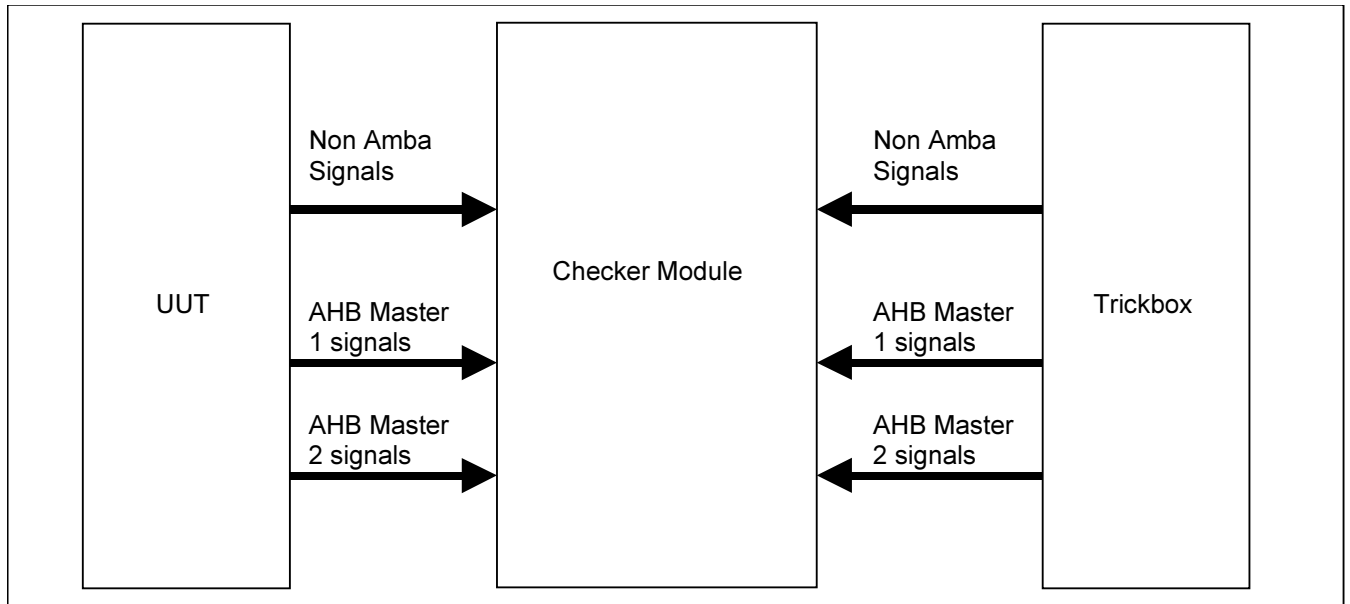


Figure 4.1-11 Block diagram of the checker module

For every check listed below, the message that may appear on the simulation log is also listed out.

1. Error Messages:

Error messages indicate that the functionality of the DMAC is not as expected or it is not configured correctly.

No	Message Displayed	Note
1	"DMAC initiated a WRAPx access on the AHB Bus"	Check whether DMAC master will ever initiate a wrap access on the bus.
2	"HPROT[0] is driven to '0' "	HPROT[0] should always be '1'.
3	"HLOCK should be asserted first along with an NSEQ access"	Check that HLOCK is asserted along with HBUSREQ.
4	"Slave was accessed with Non-Word transactions"	Check that the AHB accesses to the register through the DMAC slave interface are only a word access. If it is anything other than the word access, issue an error message.
5	"DMAC started on MasterX instead of MasterY"	Check that the channel is started by the expected master interface.
6	"DMACINTERR is NOT asserted" "DMACINTTC is NOT asserted" "DMACINTCOMBINE is NOT asserted" "DMACINTERR is asserted when masked" "DMACINTTC is asserted when masked" "DMACINTCOMBINE is asserted when masked"	Check for the assertion of DMACINTERR, DMACINTTC and DMACINTCOMBINE signals.
7	"DMACCLR is NOT asserted to Peripheral X" "DMACTC is NOT asserted to Peripheral X"	Check for DMACCLR and DMACTC of every peripheral.
8	"Peripheral ID should not be more than 15"	Check that the peripheral id in the CxConfiguration register is not programmed to be greater than 15.
9	"HSIZE should not be more than WORD"	Check that HSIZE is not more than a word transfer.
10	"Channel X is enabled when DMAC is disabled"	Check that DMAC is enabled before a channel is setup and enabled.
11	"DMAC disabled when ChannelX is active"	If there is atleast a channel active and DMAC is disabled, issue an error message.
12	"ChannelX enabled with DMAC as flow controller and Transfer size programmed as 0"	Issue an error message if DMAC is the flow controller and Transfer size is set to all zeroes while enabling the channel.
13	"SREQ / BREQ asserted along with LBREQ / LSREQ"	Issue an error message when SREQ / BREQ are asserted along with LSREQ / LBREQ of a peripheral.
14	"ChannelX enabled when Halt bit is active"	Issue an error message when the halt bit of a channel is not cleared when it is configured for a fresh data transfer.
15	"Writing to Channel Registers when ChannelX is Active"	If there is a write access to any of the channel registers when the channel is active, issue an error message.

16	"Endian bit of M1 / M2 changed when ChannelX is active"	If the endianness bits of the DMAC are changed when a channel is active, issue an error message.
17	"Write to SoftReq(B / S / LB / LS) register with 1 when it is not cleared"	Issue an error message when any soft request bit is set before it is reset by the DMAC.
18	"Source address for ChannelX not aligned to SrcWidth" "Destination address for ChannelX not aligned to DstWidth"	Check whether the programmed source/destination address is aligned to the source/destination width.
19	"Source width of channelX is an invalid value" "Destination width of channelX is an invalid value"	Issue an error message when the source/destination width is greater than 32.
20	"LSREQ and LBREQ of PeripheralX are raised together"	Check that LSREQ and LBREQ are not raised together
21	"ChannelX : All data taken from the source are NOT transferred to the Destination"	Check whether all the data fetched from the source are transferred to the destination.
22	"Illegal programming Width to Transfer ratio not correct"	Check whether the relation between the source and destination widths and the transfer size is maintained while enabling the channel. Conditions are : if srcwidth = destwidth -> Transfersize = n; (where n is an integer). if srcwidth = destwidth/2 -> Transfersize = 2n; (where n is an integer). if srcwidth = destwidth/4 -> Transfersize = 4n; (where n is an integer).
23	"Source and Destination peripheral values are programmed to be same for ChannelX"	Check if Source and Destination number are same for a channel.

2. Warning messages :

Warning messages indicate a performance issue even if the functionality is correct. It may also indicate issues that should not take place even though it does not affect the functionality of the DMAC.

No	Message Displayed	Note
1	"INTTC bit is active when ChannelX is enabled" "INTERR bit is active when ChannelX is enabled"	Issue a warning message if any of the interrupts are set and a channel is enabled.
2	"Channel X is disabled while the DMA is not yet over"	Issue a warning message if a channel is aborted due to an error response or due to the disabling of the channel by the software.
3	"LSREQ / LBREQ raised when DMAC being the flow controller."	Issue a warning message if the LSREQ and LBREQ are asserted when DMAC is the flow controller. Other conditions : P2M (DMAC) Only SREQ and BREQ are valid. P2M (SP) All requests are valid. M2P (DMAC) Only BREQ is valid. M2P (DP) All requests are valid. P2P (DMAC) Only BREQ and SREQ of Source and BREQ of destination peripheral are valid. P2P (SP) All requests from Source are valid. Only BREQ from destination is valid. P2P (DP) Only SREQ and BREQ from source are

		valid. All requests from destination peripheral are valid.
4	"ChannelX is halted and further requests are ignored till this bit is cleared"	Issue a warning message for any request being ignored by the DMAC because the channel halt bit is set.
5	"Writing All zeroes to Soft SREQ / BREQ / LSREQ / LBREQ register has no effect"	Check if HWDATA has atleast one '1' while writing to Soft request registers.

3. Notes :

The notes provide information during simulation.

No	Message Displayed	Note
1	"DMAC is enabled now" "DMAC is disabled now"	If the DMAC enable bit is set/reset, issue a message.
2	"ChannelX is Halted now"	Issue a message if a channel is halted.
3	"LLI Loading happening from AddressX for ChannelY"	Issue the message whenever the LLI loading takes place by the DMAC. The message should indicate the master information, the address from which the next LLI is being fetched.
4	"Both Soft and Hard request are active at same time for peripheralX"	Issue a message when both soft and hard requests are active.

Note : All the messages / warnings / errors related to SREQ / BREQ / LSREQ / LBREQ should be considered for the respective soft request from the register also.

Memory Block

The memory module will mimic the function of the memory on the buses where the DMAC acts as the master. This module will be a memory with AHB slave interface. There will be 2 AHB slave interfaces to the memory module. One will be used to program the control registers in the memory and the other will be accessed by the DMAC as a master. Therefore the registers can be accessed through HSELREG and the memory will be accessed through HSELMEM. This block also responsible for LLI (Linked List Items) loading. The LLI block is used to support scatter/gather data transfer. The LLI block consists of four words, the source address, destination address, pointer to next LLI block, and a control word. In the last LLI, next LLI pointer is set to NULL. The block diagram of the memory is shown in **Figure 4.1-12** below.

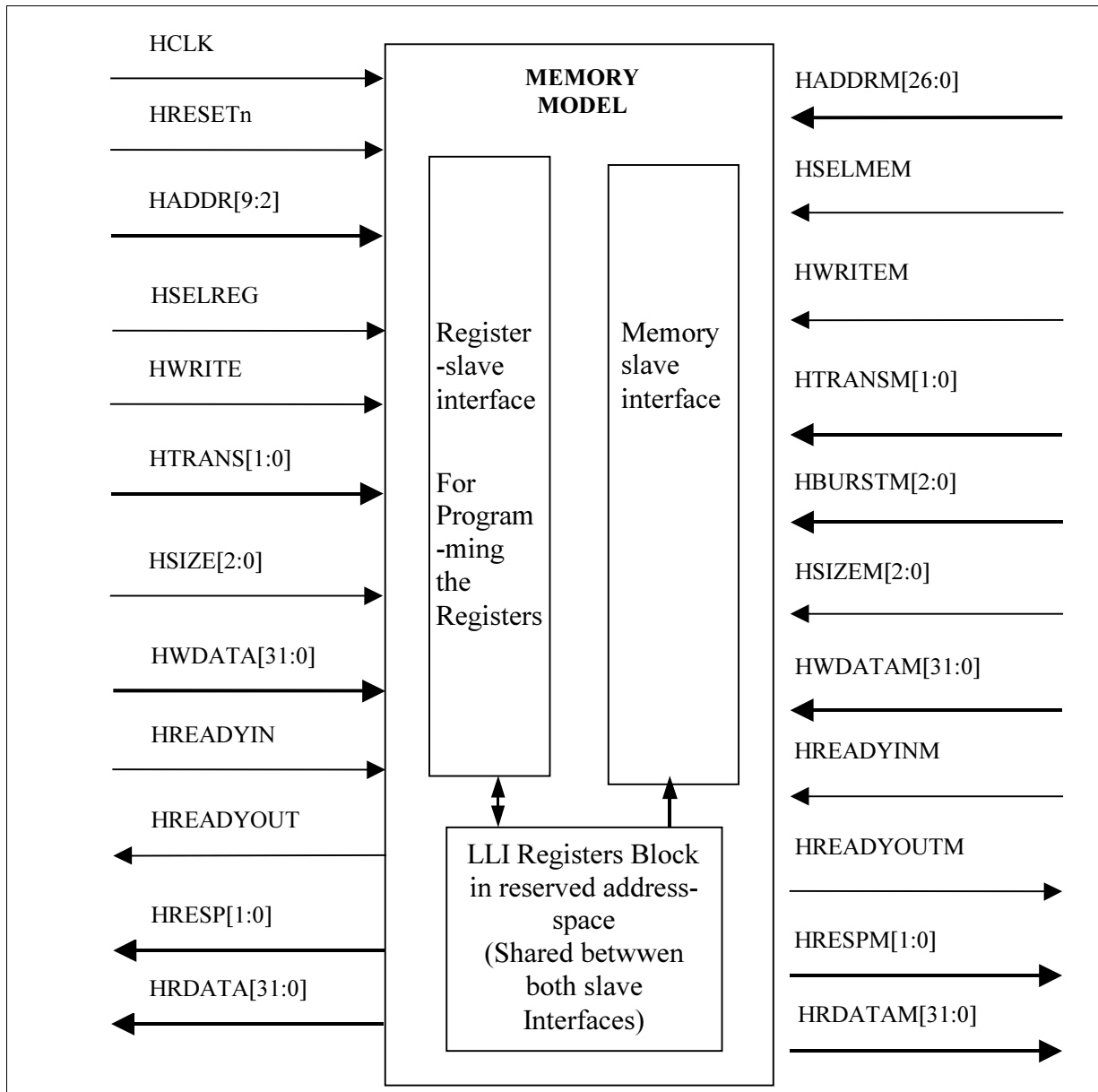


Figure 4.1-12 Block diagram of the memory block

Registers in Memory Module

Register : DMACTrMemEn

Address : Base Address

The memory will contain a register with the base address using HSELREG. This register will define memory enable bit and the default response that should be returned to the master whenever it is accessed through HSELMEM.

This will contain the memory enable, memory reset, endianness, data generation method, number of wait cycles and response bits. There are other control registers (**DMACTrMemCntlx**) and data register. The control registers indicate the number of wait cycles and the response for a specific data / address. The wait cycles and response types indicated in this register will be used for those that are not specified in any of the control registers. If the user requires

the memory to return a default response for all data transfers, then the memory needs only this register to be present. The data register contains the seed, offset and sub data generation method.

Bit	Name	Reset Value	Description
31	Memory Enable bit	0	This bit is used to enable the memory model.
30	Memory Reset	0	This is used to reset all the control registers in the memory.
29 - 11	Reserved	18'b0	Reserved bits
10	Endianess	1'b0	This bit indicates endianization. 0 – Little Endian mode 1 – Big Endian mode
9 – 8	Data generation method	2'b0	Indicates data generation methods. 00 – Random Data 01 – Gray code 10 – Address based 11 – Databased In address based method current address is used for generation for the Data depending on the submethod. In all other methods starting seed is used for generating first data. Then after that the previous data is used as a seed to generate the next data.
7 – 6	Retry / Split Count	2'b00	These 2 bits indicate the number of retry / split responses to be returned followed by an OKAY response. A maximum of 4 retry / split responses can be returned for any data transfer.
5 - 2	Wait cycles	0x0	These 4 bits indicate the number of wait cycles to be inserted for every data transfer. A maximum of 16 wait states can be inserted for any data transfer.
1 – 0	Response	2'b00	Indicates the response to be returned for a data transfer. 00 – OKAY 01 – ERROR 10 – RETRY 11 – SPLIT

Register : DMACTrMemData

Address : Base Address + 0x4

This register will contain the offset, seed and sub data generation method. This register will be used to generate data patterns depending on data generation method. The data generation method will be defined in **DMACTrMemEn** register. The sub data methods are Increment, Decrement, 1's complement and 2's complement. The seed will be

used for generation of Random, Gray and Databased data patterns. The sub methods are only applicable for Address and Databased data generation method. In Address based method, all sub methods are applicable. In Data based method, only first two sub methods are applicable i.e. Increment and Decrement method.

Bit	Name	Reset Value	Description
31-24	Seed	8'b0	Seed. These bits are used for generation of Random, Gray code and Databased data patterns.
23 – 8	Reserved	16'b0	Reserved bits
7 – 6	Sub Data method	2'b0	The two bits indicate sub data method. 00 – Increment Data = Address + Offset (In case of Address based method) Data = Seed + Offset (In case of Data based method) 01 – Decrement Data = Address - Offset (In case of Address based method) Data = Seed – Offset (In case of Data based method) 10 – 1's complement Data = ~Address (In case of Address based method) Data = ~Seed (In case of Data based method) 11 – 2's complement Data = 2's compliment of Address (Address Based) Data = 2's compliment of Seed (Data Based)
5 – 4	Reserved	16'b0	Reserved bits in case more offset expansion
3 – 0	Offset	2'b00	The offset is used for addressbased and databased data generation method. The offset is only applicable for increment and decrement sub methods.

Register : DMACTrMemCntlx (x = 3 to 63)

Address : Base Address +(4 * x)

This is a control register for programming the response to be returned to any specific data transfer. This register will contain register enable, address or data control bit, LSB of the address, number of valid bits of address / data, number of wait cycles, response type and address or data pattern.

This register is used to return a specific response to a specific data transfer as desired. The response may be based on the address or a specific data (for e.g., 2nd data of a burst). If the response is to be returned based on the address, then this register will be programmed to return the response for a particular address. Otherwise it can be set to return the response for a particular data in a burst.

This register can take upto 16 bits of address / data. Moreover the LSB of the address indicates whether it is a byte, half word or word transfer. Therefore a maximum of 2¹⁶ word addresses or 2¹⁶ data transfers can be individually controlled. However it may also be noted that the address and data can be aliased.

For example, if the user wants to return the same response for all the even addresses, then it can mentioned that only 1 bit (out of 16 bits) of the address is valid. It indicates that only bit 0 of this register is a valid address bit and the other bits (1 to 15) should be ignored. If bit 0 of this register is set to zero, then all transactions that belong to an even address will be returned with the wait cycle and response programmed in this register.

The following table provides a description of each of these bits.

Bit	Name	Reset Value	Description
31	Register enable	0	This bit indicates whether this register is a valid control register. The other bits are valid only if this bit is set to one.
30	Address / Data control	0	This bit indicates whether the response is controlled by the address or the data. 0 – Indicates that the response is controlled by the address. 1 – Indicates that the response is controlled by the data.
29 - 28	Transfer size	2'b00	This bit indicates the transfer size. 00 – Byte 01 – Half word 10 – Word If the transfer size is byte, then HADDR[0] is compared against bit 0 of this register, HADDR[1] is compared against bit 1 of this register and so on. If the transfer size is half word, then HADDR[1] is compared against bit 0 of this register, HADDR[2] is compared against bit 1 of this register and so on. If the transfer size is word, then HADDR[2] is compared against bit 0 of this register, HADDR[3] is compared against bit 1 of this register and so on.
27 - 24	Valid LSBs	0x0	These 4 bits indicate the number of valid bits from bit 0 of this register. There are 16 bits provided for indicating the address / data-count value.
23 - 22	Retry / Split Count	2'b00	These 2 bits indicate the number of retry / split responses to be returned followed by an OKAY response. A maximum of 4 retry / split responses can be returned for any data transfer.
21 - 18	Wait cycles	0x0	These 4 bits indicate the number of wait cycles to be inserted for the specified data transfer. A maximum of 16 wait states can be inserted for any data transfer.
17 – 16	Response	2'b00	Indicates the response to be returned for the specified data transfer. 00 – OKAY 01 – ERROR 10 – RETRY 11 – SPLIT

15 - 0	Address / Data-Count	0x0000	These 16 bits indicate the address or data-count for which the response defined in this register is to be returned. This register may contain address or data-count, based on bit 30 of this register. Out of the 16 bits, the number of bits valid from bit 0 is indicated by Valid LSBs (bits 27 – 24 of this register).
--------	----------------------	--------	--

There can be a maximum of 62 DMACTrMemCntl registers, from address offset 0x01 to 0xFF, in a memory block. The memory module will also be responsible for LLI loading. The LLI block will be programmed through its tbench slave port. The memory block will be responsible for the storage of 4 LLI blocks. The DMAC master will load the LLI for scatter/gather data transfer. The LLI block slave address range is 0x200 to 0x23F and master address range is 0x000000 to 0x000003F.

The algorithm to decide the response to be given to the master :

It starts checking the contents of the control registers one by one in the Memory module. If the data in the control register is valid then it is to be given. See the details in the above table for bit fields of this register. It checks whether it has to give the response programmed in that register to this transfer. (This is done by evaluating the contents of that register with the information about the current transfer. If this matches then this is the transfer for which the response programmed in the response field is to be given. See the details in the above table for bit fields of this register). If yes then it asserts that programmed response to the current transfer. Else it goes for checking the contents of the next register. In case it does not find any of the programmed response for this transfer after checking all the registers it asserts the default response to the current transfer.

Peripheral Block

This block is almost identical to the memory block. However, it is also expected to generate requests and receive DMACTC and DMACCLR signals. There will be 2 AHB slave interfaces to the peripheral module. One will be used to program the control registers in the peripheral and the other will be accessed by the DMAC as a master. Therefore the registers can be accessed through HSELREG and the peripheral will be accessed through HSELPERIPH. The block diagram of the peripheral is shown in **Figure 4.1-13** below.

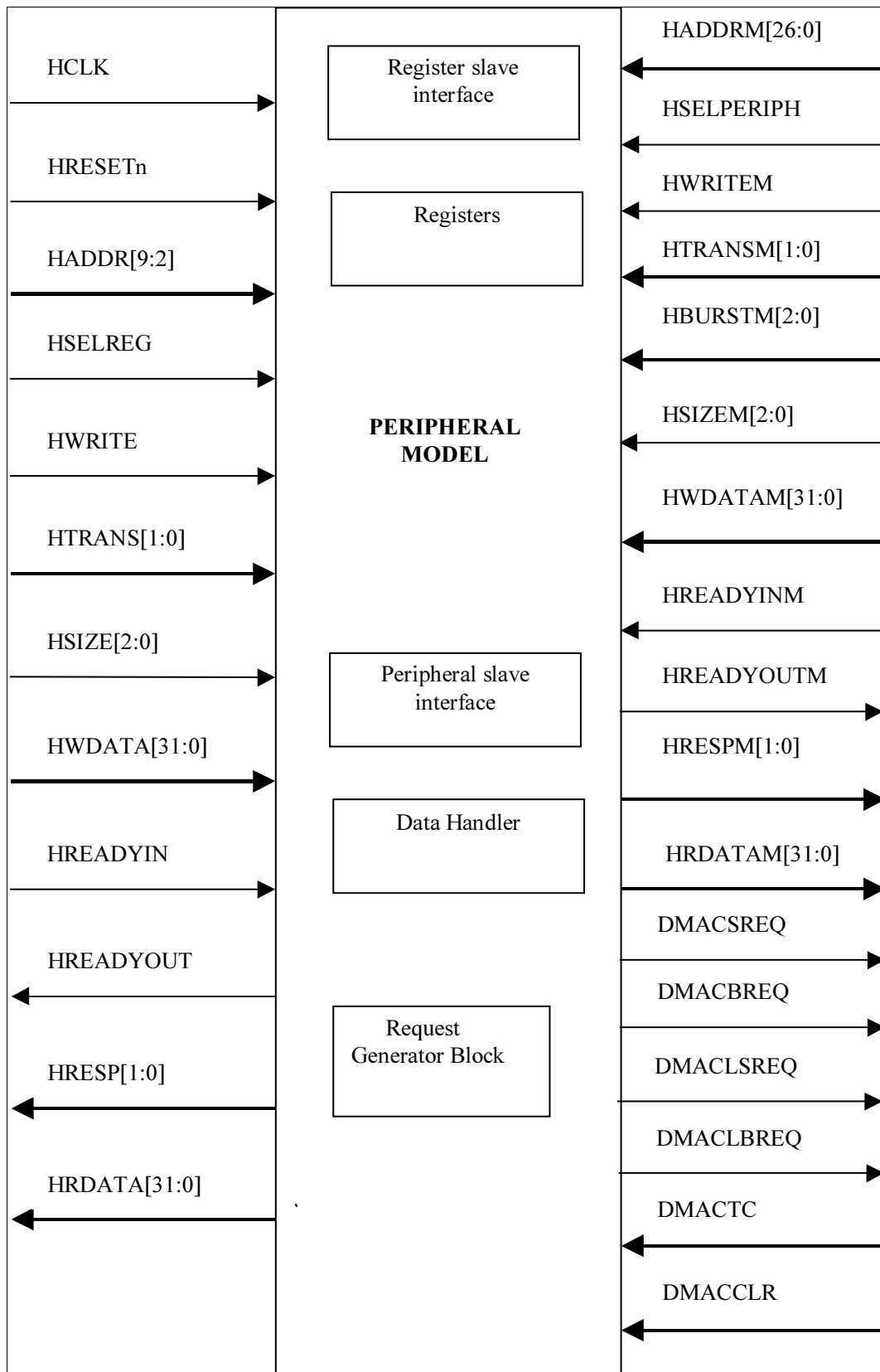


Figure 4.1-13 Block diagram of the peripheral block

Registers in Peripheral Module

Register : DMACTrPeriphEn

Address : Base Address

The peripheral will contain a register with the base address using HSELREG. This register will define peripheral enable bit and the default response that should be returned to the master whenever it is accessed through HSELPERIPH.

This will contain the peripheral enable, peripheral reset, number of wait cycles and response bits. There are other control registers (**DMACTrPeriphCntlx**) that indicate the number of wait cycles and the response for a specific data / address. The wait cycles and response types indicated in this register will be used for those that are not specified in any of the control registers. If the user requires the peripheral to return a default response for all data transfers, then the peripheral needs only this register to be present.

Bit	Name	Reset Value	Description
31	Peripheral Enable bit	0	This bit is used to enable the peripheral model.
30	Peripheral Reset	0	This is used to reset all the control registers in the peripheral.
29 - 8	Reserved	0x0000000	Reserved bits
10	Endianess	1'b0	This bit is indicates endianization. 0 – Little Endian mode 1 – Big Endian mode
9 – 8	Data generation method	2'b0	Indicates data generation methods. 00 – Random Data 01 – Gray code 10 – Address based 11 – Databased
7 - 6	Retry / Split Count	2'b00	These 2 bits indicate the number of retry / split responses to be returned followed by an OKAY response. A maximum of 4 retry / split responses can be returned for any data transfer.
5 - 2	Wait cycles	0x0	These 4 bits indicate the number of wait cycles to be inserted for every data transfer. A maximum of 16 wait states can be inserted for any data transfer.
1 – 0	Response	2'b00	Indicates the response to be returned for a data transfer. 00 – OKAY 01 – ERROR 10 – RETRY 11 – SPLIT

Register : DMACTrPeriphData

Address : Base Address + 4

This register will contain the offset, seed and sub data generation method. This register will be used to generate data patterns depending on data generation method. The data generation method will be defined in **DMACTrPeriphEn** register. The sub data methods are Increment, Decrement, 1's complement and 2's complement. The seed will be used for generation of Random, Gray and Databased data patterns. The sub methods are only applicable for Address and Databased data generation method. In Address based method, all sub methods are applicable. In Data based method, only first two sub methods are applicable i.e. Increment and Decrement method.

Bit	Name	Reset Value	Description
31-24	Seed	8'b0	Seed. These bits are used for generation of Random, Gray code and Databased data patterns.
23 – 8	Reserved	16'b0	Reserved bits
7 – 6	Sub Data method	2'b0	The two bits indicate sub data method. 00 – Increment 01 – Decrement 10 – 1's complement 11 – 2's complement
5 – 4	Reserved	16'b0	Reserved bits
3 – 0	Offset	2'b00	The offset is used for addressbased and databased data generation method. The offset is only applicable for increment and decrement sub methods.

Register : DMACTrReqReg**Address : Base Address + 8**

This register is used to specify number of HCLKs till DMA requests goes active. The DMA request will remain asserted until DMACCLR is received. The bit description is as follow.

Bit	Name	Reset Value	Description
31– 23	DMACLBReq	8'b00	This value is used to specify the number of HCLKs till DMACLBReq goes active. DMACLBReq will stay active until DMACCLR and DMACCTC are received. DMACLBReq will not go active if 0 is programmed.
23 –16	DMACLSReq	8'b00	This value is used to specify the number of HCLKs till DMACLSReq goes active. DMACLSReq will stay active until DMACCLR and DMACTC are received. DMACLSReq will not go active if 0 is programmed.
15 – 8	DMACBReq	8'b00	This value is used to specify the number of HCLKs till DMACBReq goes active. DMACBReq will stay active until DMACCLR is received. DMACBReq will not go active if 0 is programmed.
7 – 0	DMACSReq	8'b00	This value is used to specify the number of HCLKs till DMACSReq goes active. DMACSReq will stay active until DMACCLR is received. DMACSReq will not go active if 0 is

			programmed.
--	--	--	-------------

The registers for DMA clear and DMACTC are given in following table.

Register	Addr	Size	Reset	Description
DMACTrCLRReg	Base Address + 0x0C	31-0	0x00	This register indicates the duration (number of clocks) in which DMACCLR signal is expected from the DMAC. If the register is configured to zero, then DMACCLR is not expected from DMAC.
DMACTrTCReg	Base Address + 0x10	31-0	0x00	This register indicates the duration (number of clocks) in which DMACTC signal is expected from the DMAC. If the register is configured to zero, then DMACTC is not expected from DMAC.

Register : DMACTrPeriphCntlx (x = 5 to 63)

Address : Base Address +(4 * x)

This is a control register for programming the response to be returned to any specific data transfer. This register will contain register enable, address or data control bit, LSB of the address, number of valid bits of address / data, number of wait cycles, response type and address or data pattern.

This register is used to return a specific response to a specific data transfer as desired. The response may be based on the address or a specific data (for e.g., 2nd data of a burst). If the response is to be returned based on the address, then this register will be programmed to return the response for a particular address. Otherwise it can be set to return the response for a particular data in a burst.

This register can take upto 16 bits of address / data. Moreover the LSB of the address indicates whether it is a byte, half word or word transfer. Therefore a maximum of 2¹⁶ word addresses or 2¹⁶ data transfers can be individually controlled. However it may also be noted that the address and data can be aliased.

For example, if the user wants to return the same response for all the even addresses, then it can mentioned that only 1 bit (out of 16 bits) of the address is valid. It indicates that only bit 0 of this register is a valid address bit and the other bits (1 to 15) should be ignored. If bit 0 of this register is set to zero, then all transactions that belong to an even address will be returned with the wait cycle and response programmed in this register.

The following table provides a description of each of these bits.

Bit	Name	Reset Value	Description
31	Register enable	0	This bit indicates whether this register is a valid control register. The other bits are valid only if this bit is set to one.
30	Address / Data control	0	This bit indicates whether the response is controlled by the address or the data. 0 – Indicates that the response is controlled by the address. 1 – Indicates that the response is controlled by the data.
29 - 28	Transfer size	2'b00	This bit indicates the transfer size. 00 – Byte

			01 – Half word 10 – Word If the transfer size is byte, then HADDR[0] is compared against bit 0 of this register, HADDR[1] is compared against bit 1 of this register and so on. If the transfer size is half word, then HADDR[1] is compared against bit 0 of this register, HADDR[2] is compared against bit 1 of this register and so on. If the transfer size is word, then HADDR[2] is compared against bit 0 of this register, HADDR[3] is compared against bit 1 of this register and so on.
27 - 24	Valid LSBs	0x0	These 4 bits indicate the number of valid bits from bit 0 of this register. There are 16 bits provided for indicating the address / data-count value.
23 - 22	Retry / Split Count	2'b00	These 2 bits indicate the number of retry / split responses to be returned followed by an OKAY response. A maximum of 4 retry / split responses can be returned for any data transfer.
21 - 18	Wait cycles	0x0	These 4 bits indicate the number of wait cycles to be inserted for the specified data transfer. A maximum of 16 wait states can be inserted for any data transfer.
17 – 16	Response	2'b00	Indicates the response to be returned for the specified data transfer. 00 – OKAY 01 – ERROR 10 – RETRY 11 – SPLIT
15 - 0	Address / Data-Count	0x0000	These 16 bits indicate the address or data-count for which the response defined in this register is to be returned. This register may contain address or data-count, based on bit 30 of this register. Out of the 16 bits, the number of bits valid from bit 0 is indicated by Valid LSBs (bits 27 – 24 of this register).

There can be a maximum of 61 DMACTrPeriphCntl registers, in a peripheral block.

The algorithm to decide the response to be given to the master :

It starts checking the contents of the control registers one by one in the Memory module. If the data in the control register is valid then it checks whether it has to give the response programmed in that register to this transfer. If yes then it asserts that programmed response to the current transfer. Else it goes for checking the contents of the next register. In case it does not find any of the programmed response for this transfer after checking all the registers it asserts the default response to the current transfer.

Request Generator Block: -

This block consists four 8 bit and two 32 bit down counter. The counters are:

The requests are generated after the registers DmacTrReqReg(In effect the counter) is programmed with some count value. The request goes active after the counter expires. The de-activation of the request is done when the DMACCLR

is received. The counter gets reloaded with every DMACCLR signal and thus generation of request signals goes on continuously. But when DMACCLR and DMACTC come simultaneously counter does not get loaded after that. So from now on request would not be generated. If the generation of the request has to be started then again we have to programme the DmacTrReqReg.

DMACTrSCounter:-

This is eight bit down counter. This counter is used to generate DMA single request (DMACSREQ). The counter is loaded with the value as indicated in DMACSReq bit fields of DMACTrReqReg register. When the counter reaches to zero, it generates terminal count signal, which is used to generate DMACSREQ signal.

DMACTrBCounter:-

This is eight bit down counter. This counter is used to generate DMA burst request (DMACBREQ). The counter is loaded with the value as indicated in DMACBReq bit fields of DMACTrReqReg register. When the counter reaches to zero, it generates terminal count signal, which is used to generate DMACBREQ signal.

DMACTrLSCounter:-

This is eight bit down counter. This counter is used to generate DMA last single request (DMACLSREQ). The counter is loaded with the value as indicated in DMACLSReq bit fields of DMACTrReqReg. When the counter reaches to zero, it generates terminal signal, which is used to generate DMACLSREQ signal.

DMACTrLBCounter:-

This is eight bit down counter. This counter is used to generate DMA last burst request (DMACLBREQ). The counter is loaded with the value as indicated in DMACLBReq bit fields of DMACTrReqReg register. When the counter reaches to zero, it generates terminal count signal, which is used to generate DMACLBREQ signal.

DMACTrCLRCCounter:-

This is 32 bit down counter. It indicates the duration (i.e. number of clock cycle) in which the DMA clear signal is expected from the DMAC. The counter is loaded with the value as indicated in DMACTrCLRReg register. When this counter reaches to zero and if DMA clear signal is not asserted by DMAC, the peripheral flags the error message.

DMACTrDMACTCCnt:-

This is 32 bit down counter. It indicates the duration (i.e. number of clock cycle) in which the DMACTC is expected from the DMAC. The counter is loaded with the value as indicated in DMACTrTCReg register. When this counter reaches to zero and if the DMACTC is not asserted by the DMAC, the peripheral flags the error message.

Memory and Peripheral block address range :

The address range of memory and peripheral block is as follow.

Sr No.	Memory/Peipheral Module Number	Master Address Range
1.	Memory Module 0	0x08000000 - 0x0FFFFFFF
2.	Memory Module 1	0x10000000 - 0x17FFFFFFF
3.	Peripheral Module 0	0x18000000 - 0x1FFFFFFF
4.	Peripheral Module 1	0x20000000 - 0x27FFFFFFF
5.	Peripheral Module 2	0x28000000 - 0x2FFFFFFF
6.	Peripheral Module 3	0x30000000 - 0x37FFFFFFF
7.	Peripheral Module 4	0x38000000 - 0x3FFFFFFF

8.	Peripheral Module 5	0x40000000 - 0x47FFFFFF
9.	Peripheral Module 6	0x48000000 - 0x4FFFFFFF
10.	Peripheral Module 7	0x50000000 - 0x57FFFFFF
11.	Peripheral Module 8	0x58000000 - 0x5FFFFFFF
12.	Peripheral Module 9	0x60000000 - 0x67FFFFFF
13.	Peripheral Module 10	0x68000000 - 0x6FFFFFFF
14.	Peripheral Module 11	0x70000000 - 0x77FFFFFF
15.	Peripheral Module 12	0x78000000 - 0x7FFFFFFF
16.	Peripheral Module 13	0x80000000 - 0x87FFFFFF
17.	Peripheral Module 14	0x88000000 - 0x8FFFFFFF
18.	Peripheral Module 15	0x90000000 - 0xA7FFFFFF

Sr No.	Memory/Peripheral Module Number	Slave Address Range
1.	Memory Module 0	0xA001000 - 0xA001FFFF
2.	Memory Module 1	0xA002000 - 0xA002FFFF
3.	Peripheral Module 0	0xA003000 - 0xA003FFFF
4.	Peripheral Module 1	0xA004000 - 0xA004FFFF
5.	Peripheral Module 2	0xA005000 - 0xA005FFFF
6.	Peripheral Module 3	0xA006000 - 0xA006FFFF
7.	Peripheral Module 4	0xA007000 - 0xA007FFFF
8.	Peripheral Module 5	0xA008000 - 0xA008FFFF
9.	Peripheral Module 6	0xA009000 - 0xA009FFFF
10.	Peripheral Module 7	0xA00A000 - 0xA00AFFFF
11.	Peripheral Module 8	0xA00B000 - 0xA00BFFFF
12.	Peripheral Module 9	0xA00C000 - 0xA00CFFFF
13.	Peripheral Module 10	0xA00D000 - 0xA00DFFFF
14.	Peripheral Module 11	0xA00E000 - 0xA00EFFFF
15.	Peripheral Module 12	0xA00F000 - 0xA00FFFFF
16.	Peripheral Module 13	0xA010000 - 0xA010FFFF
17.	Peripheral Module 14	0xA011000 - 0xA011FFFF
18.	Peripheral Module 15	0xA012000 - 0xA012FFFF

6.4 Changes which needs to be done for above Trickbox for DMAC variant

6.4.1 Module level changes to be done

1. DmacTrAhbSlaveIf : No Change
2. DmacTrAhbMaster : No Change
3. DmacTrAhbLite : No Change
4. DmacTrIntArb : No Change
5. DmacTrMasWrap : No Change
6. DmacTrPackage : No Change
7. DmacTrGntGen : No Change
8. DmacTrRouter : The priority level of channel 1 is made LOW. Changes required are in the process p_Bus1RouteComb Line571(VHDL) should be changed to CombPriBus1 <= '0';.
9. DmacTrProChkr : All the bus2 signals to this module from the behavioral module removed and the comparison of only AHB Master1 signals is kept in place.
10. DmacTrBehaviour : No Change
11. DmacTrick : There will be no UUT AHB Master Bus2 signals coming as input to this module, and there will be no AHB Master 2 Bus signals going as output from this module. While port mapping and instantiating DmacTrProChkr module all bus2 references are removed. While portmapping DmacTrGntGen module for bus2 references 0's should be port mapped. The input signals HRESPM2Beh and HRDATAM2Beh are kept as they are.
12. DmacTrChLogic : The Master selects only select the master 1 only as the second AHB master is not present in the design. From RTL implementation these change details are as below.
 Line 1337 and 1338(VHDL) are changed to
 SrcAhbMasSel <= '0';
 DstAhbMasSel <= '0';
 Line 1358 are changed to
 NextLLIAhbMasSel <= '0';

6.4.2 Changes which needs to be done in Tbench for Behaviour Trickbox

In tbench (top level file) port map the for the Dmac and the trickbox is changed accordingly to remove the any reference to the AHB Master 2 related signals.

6.5 Functional Tests

The following sections describe the test cases to validate the DMAC. These tests assume the following configuration of DMAC.

- 32 bit AHB bus
- 16 Requestors
- 2 Channels
- Single Master

Key to the tables in this section :

M2M : Indicates Memory to Memory Dma transfer.

M2P : Indicates Memory to Peripheral Dma transfer.

P2M : Indicates Peripheral to Memory Dma transfer.

P2P : Indicates Peripheral to Peripheral Dma transfer.

Src Inc (Source Increment) : Indicates whether the source address will be incremented or not. I – Incrementing, NI – Non Incrementing.

Dest Inc (Destination Increment) : Indicates whether the destination address will be incremented or not. I – Incrementing, NI – Non Incrementing.

Src Width (Source Width) : 8, 16, 32. The width, in bits, of the source transfers.

Dest Width (Destination Width) : 8, 16, 32. The width, in bits, of the destination transfers.

Src Burst (Source Burst) : 1, 4, 8, 16, 32, 64, 128, 256. The size of the source burst transfer.

Dest Burst (Destination Burst) : 1, 4, 8, 16, 32, 64, 128, 256. The size of the destination burst transfer.

Transfer Size : 0 – 4096, NA – Not Applicable.

Src Endian (Source Endianness) : 0 – Little Endian, 1 – Big Endian. The Endianness of the source bus.

Dest Endian (Destination Endianness) : 0 – Little Endian, 1 – Big Endian. The Endianness of the destination bus.

Src Response (Source Response) : O – Okay, R – Retry, S – Split, E – Error, W – Wait. The AHB slave responses that will be returned by the source of the DMA transfers.

Dest Response (Destination Response) : O – Okay, R – Retry, S – Split, E – Error, W – Wait. The AHB slave responses that will be returned by the destination of the DMA transfers.

TC Mask (Terminal Count Interrupt Mask) : 1 – Mask the TC interrupt , 0 – Do not mask the TC interrupt.

Error Mask (Error Interrupt Mask) : 1 – Mask the error interrupt , 0 – Do not mask the error interrupt.

Note :

In the following test cases,

'Setup channel' means programming the channel's source address register, destination address register, control register and LLI register with the parameters provided in the table. The table only specifies the parameters that are relevant to the test. Other parameters are not expected to affect that particular test. Therefore the user can program any preferred value into those registers.

'Configure channel' means programming the channel's configuration register with the test parameters. The configuration parameters are the type of DMA, flow controller, source and destination peripheral numbers, lock and

protection bits etc. These values are not specified in the table. However they can be defined from the test category itself.

'Enable channel' means setting the DMA enable in the channel's configuration register without modifying any of the other bits.

6.5.1 Register Tests

Test Identification No : DMA_REG_RST

Reset Value Tests

1. Assert HRESETn for one HCLK.
2. Read (HSIZE = 10 only) all the registers in the DMA controller.
3. Check that there are only OKAY responses returned by the DMAC.
4. Check for the reset values as defined below. The actual address of the register will be the Base address of the DMAC + address offset.

DMAC Register Name	Address Offset	Reset Value
DMACIntStat	0x000	0x00000000
DMACIntTCStat	0x004	0x00000000
DMACIntTCClr	0x008	0x00000000
DMACIntErrStat	0x00C	0x00000000
DMACIntErrClr	0x010	0x00000000
DMACRawIntTC	0x014	0x00000000
DMACRawIntErr	0x018	0x00000000
DMACEnbldChns	0x01C	0x00000000
DMACSoftBReq	0x020	0x00000000
DMACSoftSReq	0x024	0x00000000
DMACSoftLBReq	0x028	0x00000000
DMACSoftLSReq	0x02C	0x00000000
DMACConfig	0x030	0x00000000
DMACSync	0x034	0x00000000
DMACC0SrcAddr	0x100	0x00000000
DMACC0DestAddr	0x104	0x00000000
DMACC0LLIReg	0x108	0x00000000
DMACC0Control	0x10C	0x00000000
DMACC0Config	0x110	0x00000000
DMACC1SrcAddr	0x120	0x00000000

DMACC1DestAddr	0x124	0x00000000
DMACC1LLIReg	0x128	0x00000000
DMACC1Control	0x12C	0x00000000
DMACC1Config	0x130	0x00000000
DMACTCR	0x500	0x00000000
DMACITOP1	0x504	0x00000000
DMACITOP2	0x508	0x00000000
DMACITOP3	0x50C	0x00000000
DMACPeriphId0	0xFE0	0x00000081
DMACPeriphId1	0xFE4	0x00000010
DMACPeriphId2	0xFE8	0x00000004
DMACPeriphId3	0xFEC	0x00000000
DMACPCellId0	0xFF0	0x0000000D
DMACPCellId1	0xFF4	0x000000F0
DMACPCellId2	0xFF8	0x00000005
DMACPCellId3	0xFFC	0x000000B1

Table 5.1-1 Reset values of DMAC registers**Test Identification No : DMA_REG_RW****Read – Write Tests**

This test targets the register bits individually. The objective of this tests are:

- Every bit that is a read only bit should return only the read-only value (zero / one) irrespective of the values of other bits / registers.
- Every bit that is a read write bit should accept the new value written into it, irrespective of the values of other bits / registers.

The DMAC contains three types of registers. They are read-only, write-only and read-write registers. The following table 5.1-2 lists the read-only registers of the DMAC. It also indicates the read-only value of these registers. During the register read-write tests, the register will be written with different write patterns as listed in the test cases below. However while reading the same register, the expected value should be same as it is in table 5.1-2.

Register	Address Offset	Read Value
DMACIntStat	0x000	0x00000000
DMACIntTCStat	0x004	0x00000000
DMACIntErrStat	0x00C	0x00000000
DMACRawIntTC	0x014	0x00000000

DMACRawIntErr	0x018	0x00000000
DMACEnbldChns	0x01C	0x00000000
DMACPeriphId0	0xFE0	0x00000081
DMACPeriphId1	0xFE4	0x00000010
DMACPeriphId2	0xFE8	0x00000004
DMACPeriphId3	0xFEC	0x00000000
DMACPCellId0	0xFF0	0x0000000D
DMACPCellId1	0xFF4	0x000000F0
DMACPCellId2	0xFF8	0x00000005
DMACPCellId3	0xFFC	0x000000B1

Table 5.1-2 Read value of read-only registers of the DMAC

The following table 5.1-3 lists the write-only registers of the DMAC. These register values are not defined during read, and hence they will not be tested with the register read-write tests.

Register	Address Offset
DMACIntTCClr	0x008
DMACIntErrClr	0x010

Table 5.1-3 Mask patterns for the register read access

The following table 5.1-4 lists the read-write registers of the DMAC and the valid bits. The valid bits are defined as the actual read-write bits in the register. The other bits may be read-only, write-only or reserved bits. It also provides a mask pattern for every register. In the tests described below, write the pattern as per the tests. While reading a register, the write pattern should be AND-ed with the mask pattern of the register and that forms the expected value to be read from the register.

Note 1: Please note that the Mask pattern for Dmac soft-request registers (SoftBReq, SoftSReq, SoftLBReq and SoftLSReq) are written as 0x000000 even though there are 16 bits that are writeable and corresponds to each of the 16 peripherals. As per the Dmac functionality, these can be written by an AHB master, but can be cleared only by the Dma controller internally. But during the register tests, the Dmac is disabled as enabling Dmac may initiate unwanted transactions on the AHB bus. Since the Dmac is disabled, the soft request registers cannot return the expected value during register read access, as writing zeroes into the registers does not have any effect on the register bits at all. Therefore the soft-request registers' read data are not compared with the expected data.

Note 2: The transfer size is loaded into the registers only when the Dmac channel is enabled, as per the implementation. Since the channel enable bits are not set, to avoid unpredictable behaviour of the Dmac, the transfer size is not compared with the expected data, while reading the channel control registers.

Register	Address Offset	Valid bits	Mask Pattern
DMACSoftBReq	0x020	15 – 0	0x0000FFFF
DMACSoftSReq	0x024	15 – 0	0x0000FFFF
DMACSoftLBReq	0x028	15 – 0	0x0000FFFF
DMACSoftLSReq	0x02C	15 – 0	0x0000FFFF

DMACConfig	0x030	1-0	0x00000003
DMACSync	0x034	15 – 0	0x0000FFFF
DMACC0SrcAddr	0x100	31 – 0	0xFFFFFFFF
DMACC0DestAddr	0x104	31 – 0	0xFFFFFFFF
DMACC0LLIReg	0x108	31-2	0xFFFFFFFFFC
DMACC0Control	0x10C	31 – 0	0xFFFFFFFF
DMACC0Config	0x110	17, 15 – 0	0X0002FFFF
DMACC1SrcAddr	0x120	31 – 0	0xFFFFFFFF
DMACC1DestAddr	0x124	31 – 0	0xFFFFFFFF
DMACC1LLIReg	0x128	31-2	0xFFFFFFFFFC
DMACC1Control	0x12C	31 – 0	0xFFFFFFFF
DMACC1Config	0x130	17, 15 – 0	0X0002FFFF
DMACTCR	0x500	1 – 0	0x00000003
DMACITOP1	0x504	15 – 0	0x0000FFFF
DMACITOP2	0x508	15 – 0	0x0000FFFF
DMACITOP3	0x50C	1 – 0	0x00000003

Table 5.1-4 Mask patterns for the read-write registers of the DMAC**Test No: DMA_REG_RW_1**

The following test ensures that every bit is independent of other bits in the same register.

- All the register accesses should be a word access (HSIZE = 10).

- 1) Write 0x00000000 to all the registers.
- 2) Read them back with the expected value (Write pattern AND Mask pattern).
- 3) Check that there are only OKAY responses returned by the DMAC.
- 4) Repeat steps 1) to 4) with the following write patterns:

0xFFFFFFFF, 0x55555555, 0xAAAAAAAA, 0x33333333, 0xCCCCCCCC, 0x0F0F0F0F, 0xF0F0F0F0, 0x00FF00FF, 0xFF00FF00, 0x0000FFFF, 0xFFFF0000.

Test No : DMA_REG_RW_2

The following test ensures that every register is independent of other registers in DMAC (There are 63 registers, each of 32 bit).

- All the register accesses should be a word access (HSIZE = 10).

- 1) Write 0x00000000 to all even registers (Reg0, Reg2, Reg4...) and 0xFFFFFFFF to all odd registers (Reg1, Reg3, Reg5...)
- 2) Read them back with the expected value (Write pattern AND Mask pattern)

- 3) Check that there are only OKAY responses returned by the DMAC.
- 4) Invert every bit of the write pattern and repeat steps 1) to 4)

Test No : DMA_REG_RW_3

- Group 2 successive registers (Reg0 and Reg1 forms the first group, Reg2 and Reg3 form the second group...)
 - All the register accesses should be a word access (HSIZE = 10).
- 1) Write 0x00000000 to all even group of registers (Reg0, Reg1, Reg4, Reg5...) and 0xFFFFFFFF to all odd group of registers (Reg2, Reg3, Reg6, Reg7...)
 - 2) Read them back with the expected value (Write pattern AND Mask pattern)
 - 3) Check that there are only OKAY responses returned by the DMAC.
 - 4) Invert every bit of the write pattern and repeat steps 1) to 4)

Test No : DMA_REG_RW_4 - DMA_REG_RW_7

- Repeat the above test with each group consisting of 4 / 8 / 16 / 32 registers.

Byte / Half word access**Test No : DMA_REG_BHW_1**

- All the register accesses should be a word access (HSIZE = 10).
- Write and read access to every register with a random data (ensure that DMAC is disabled).
- Write access to every register with HSIZE as Byte / Half word / Reserved and HTRANS as NSEQ / SEQ.
- Check that there are no wait states inserted by the DMAC.
- Check that there are only ERROR responses returned by the DMAC.
- Read access to every register with HSIZE as Word and check that the data is not modified.
- Read access to every register with HSIZE as Byte / Half word / Reserved and HTRANS as NSEQ / SEQ.
- Check that there are no wait states inserted by the DMAC.
- Check that there are only ERROR responses returned by the DMAC

Test No : DMA_REG_BHW_2

- Read / Write access with HSIZE as Byte / Half word / Reserved and HTRANS as IDLE / BUSY to DMAC.
- Check that there are no wait states inserted by the DMAC.
- Check that there are only OKAY responses returned by the DMAC.

6.5.2 Enabled Channel Test

This test will enable and then disable the one at a time and check that the relevant channel bit in the EnabledChannels register is set.

Note : In the following test cases, as the DMAC is disable.

Test No : DMA_CHNL_NABL_1

Objective : To test the “DMAEnabledChannel” register by enabling and disabling the channels individually.

- 1) Disable the DMAC through the DMAC configuration register (0x030).
- 2) With all channels disabled, read the EnabledChannel register (0x01C) and check that it returns zero for all the channels.
- 3) Enable Channel0, by writing into the DMAC0Configuration register.
- 4) Read the EnabledChannel register and check that the bit corresponding to Channel0 goes high. Also check that the other bits are still low.
- 5) Disable Channel0, by writing into the DMAC0Configuration register.
- 6) Read the EnabledChannel register and check that all the bits are low.
- 7) Repeat steps 3) to 6) for all channels separately.

Test No : DMA_CHNL_NABL_2

Objective : To test the “DMAEnabledChannel” register by enabling and disabling all the channels together.

- 1) Disable the DMAC through the DMAC configuration register (0x030).
- 2) With all channels disabled, read the EnabledChannel register (0x01C) and check that it returns zero for all the channels.
- 3) Enable Channel0, by writing into the DMAC0Configuration register.
- 4) Read the EnabledChannel register and check that the bit corresponding to Channel0 goes high. Also check that the other bits are still low.
- 5) Enable Channel1, by writing into the DMAC1Configuration register.
- 6) Read the EnabledChannel register and check that the bit corresponding to Channel1 and Channel0 are high and the other bits are low.
- 7) Disable Channel0, by writing into the DMAC0Configuration register.
- 8) Read the EnabledChannel register and check that the bit corresponding to Channel0 is low and that of other channels are high.
- 9) Disable Channel1, by writing into the DMAC1Configuration register.
- 10) Read the EnabledChannel register and check that the bits corresponding to Channel0 and Channel1 are low and that of other channels are high.

6.5.3 Interrupt Tests

The objective of this test is to create the TC (Terminal Count) and Error interrupts from the DMAC and to verify that they can be cleared by writing into the corresponding clear registers.

All the transactions in this section are only M2M transactions, as the main objective is to test that the registers can be cleared by writing into the clear register. The generation of DMACTC and DMACCLR signals for the requestor are tested in the Channel Test section.

Test No : DMA_CHNL_INTR_1

Objective : The following tests will target the generation and clearance of individual channel TC status bit.

- 1) Setup Channel0 with the parameters in table 5.1-5 (TC is masked).
- 2) Enable Channel0, by writing into the channel configuration register, for an M2M transaction.
- 3) Wait for the transaction to be over on the bus.
- 4) Check that bit 0 of DMACRawIntTCStatus register goes high. All other bits have to be low.
- 5) Check that all the bits of DMACIntTCStatus register are low.
- 6) Check that all the bits of DMACIntStatus register are low.
- 7) Write 0x00000001 to DMACIntTCClear register.
- 8) Check that all the bits of DMACRawIntTCStatus, DMACIntTCStatus and DMACIntStatus register are low.
- 9) Repeat steps 1) to 8) for channel 1.

Test No	Src Width	Dest Width	Src Inc	Dest Inc	Src Burst	Dest Burst	Transfer Size	TC Mask
1.	8	8	NI	NI	1	1	1	1

Table 5.1-5 DMAC parameters for interrupt tests

Test No : DMA_CHNL_INTR_2

Objective : The following tests will target the generation and clearance of individual channel TC status bit.

- 1) Setup Channel0 with the parameters in table 5.1-6 (TC is not masked).
- 2) Enable Channel0, by writing into the channel configuration register, for an M2M transaction.
- 3) Wait for the transaction to be over on the bus.
- 4) Check that bit 0 of DMACRawIntTCStatus register goes high. All other bits are low.
- 5) Check that bit 0 of DMACIntTCStatus register goes high. All other bits are low.
- 6) Check that bit 0 of DMACIntStatus register goes high. All other bits are low.
- 7) Write 0x00000001 to DMACIntTCClear register.
- 8) Check that all the bits of DMACRawIntTCStatus, DMACIntTCStatus and DMACIntStatus register are low.
- 9) Repeat steps 1) to 8) for channel 1.

Test No	Src Width	Dest Width	Src Inc	Dest Inc	Src Burst	Dest Burst	Transfer Size	TC Mask
1.	16	16	NI	NI	1	1	1	0

Table 5.1-6 DMAC parameters for interrupt tests**Test No : DMA_CHNL_INTR_3**

Objective : The following tests will target the generation and clearance of individual channel TC status bit.

- 1) Setup Channel0 with the parameters in table 5.1-7 (TC is not masked).
- 2) Enable Channel0, by writing into the channel configuration register, for an M2M transaction.
- 3) Wait for the transaction to be over on the bus.
- 4) Check that bit 0 of DMACRawIntTCStatus register goes high. All other bits are low.
- 5) Check that bit 0 of DMACIntTCStatus register goes high. All other bits are low.
- 6) Check that bit 0 of DMACIntStatus register goes high. All other bits are low.
- 7) Repeat steps 1) to 6) for channel 1.
- 8) Check that bits 7 – 0 of DMACRawIntTCStatus, DMACIntTCStatus and DMACIntStatus registers goes high. All other bits are low.
- 9) Clear the status bit of one channel at a time (Channel0 – Channel1) by writing '1' into the relevant bit of the DMACIntTCClear register. Check that the relevant bit is only reset to zero.

Test No	Src Width	Dest Width	Src Inc	Dest Inc	Src Burst	Dest Burst	Transfer Size	TC Mask
1.	32	32	I	I	1	1	1	0

Table 5.1-7 DMAC parameters for interrupt tests**Test No : DMA_CHNL_INTR_4**

Objective : The following tests will target the generation and clearance of individual channel Error status bit.

- 1) Setup Channel0 with the parameters in table 5.1-8 (Error is masked).
- 2) Enable Channel0, by writing into the channel configuration register, for an M2M transaction.

- 3) Wait for the transaction to be over on the bus. The slave should return an error response for the DMA transfer.
- 4) Check that bit 0 of DMACRawIntErrorStatus register goes high. All other bits have to be low.
- 5) Check that all the bits of DMACIntErrorStatus register are low.
- 6) Check that all the bits of DMACIntStatus register are low.
- 7) Write 0x00000001 to DMACIntErrorClear register.
- 8) Check that all the bits of DMACRawIntErrorStatus, DMACIntErrorStatus and DMACIntStatus register are low.
- 9) Repeat steps 1) to 8) for channel 1.

Test No	Src Width	Dest Width	Src Inc	Dest Inc	Src Burst	Dest Burst	Transfer Size	Error Mask
1.	8	8	NI	NI	1	1	1	1

Table 5.1-8 DMAC parameters for interrupt tests**Test No : DMA_CHNL_INTR_5**

Objective : The following tests will target the generation and clearance of individual channel Error status bit.

- 1) Setup Channel0 with the parameters in table 5.1-9 (Error is not masked).
- 2) Enable Channel0, by writing into the channel configuration register, for an M2M transaction.
- 3) Wait for the transaction to be over on the bus. The slave should return an error response for the DMA transfer.
- 4) Check that bit 0 of DMACRawIntErrorStatus register goes high. All other bits are low.
- 5) Check that bit 0 of DMACIntErrorStatus register goes high. All other bits are low.
- 6) Check that bit 0 of DMACIntStatus register goes high. All other bits are low.
- 7) Write 0x00000001 to DMACIntErrorClear register.
- 8) Check that all the bits of DMACRawIntErrorStatus, DMACIntErrorStatus and DMACIntStatus register are low.
- 9) Repeat steps 1) to 8) for channel 1.

Test No	Src Width	Dest Width	Src Inc	Dest Inc	Src Burst	Dest Burst	Transfer Size	Error Mask
1.	16	16	NI	NI	1	1	1	0

Table 5.1-9 DMAC parameters for interrupt tests**Test No : DMA_CHNL_INTR_6**

Objective : The following tests will target the generation and clearance of individual channel Error status bit.

- 1) Setup Channel0 with the parameters in table 5.1-10 (Error is not masked).
- 2) Enable Channel0, by writing into the channel configuration register, for an M2M transaction.
- 3) Wait for the transaction to be over on the bus. The slave should return an error response for the DMA transfer.
- 4) Check that bit 0 of DMACRawIntErrorStatus register goes high. All other bits are low.
- 5) Check that bit 0 of DMACIntErrorStatus register goes high. All other bits are low.
- 6) Check that bit 0 of DMACIntStatus register goes high. All other bits are low.
- 7) Repeat steps 1) to 6) for channel 1.
- 8) Check that bits 7 – 0 of DMACRawIntErrorStatus, DMACIntErrorStatus and DMACIntStatus registers goes high. All other bits are low.
- 9) Clear the status bit of one channel at a time (Channel0 – Channel1) by writing '1' into the relevant bit of the DMACIntErrorClear register. Check that the relevant bit is only reset to zero.

Test No	Src Width	Dest Width	Src Inc	Dest Inc	Src Burst	Dest Burst	Transfer Size	Error Mask
1.	32	32	I	I	1	1	1	0

Table 5.1-10 DMAC parameters for interrupt tests

6.5.4 Channel Test

The channel tests will target the DMA channel hardware. The channel hardware consists of the channel configuration registers, control logic - to monitor the requests and complete the DMA transfer - and the channel FIFOs.

The following tests will be performed to test the channel hardware for all possible configurations of the channel.

The major classification of the channel testing is based on the transfer type and the flow controller configurations. All these configurations will be tested individually with other variations in the channel parameters.

The channel tests will be run on all channels of the DMAC. The source and destination increments will be handled with all possible combinations, as follows in table 5.1-11. Similarly the source and destination masters will also be configured with all possible combinations, as follows in table 5.1-11. They are not listed in the table of test parameters.

S.No	Channel	Dma Transfer Type	Src Inc	Dest Inc
1	0	M2M	NI	NI

2	0	M2P	I	NI
3	0	P2M	I	I
4	0	P2P	NI	I
5	1	M2M	NI	NI
6	1	M2P	I	NI
7	1	P2M	I	I
8	1	P2P	NI	I

Table 5.1-11 Address incrementing and AHB master test combinations**Memory to Memory Transfer – Flow Controller DMAC:**

The following tests will target an M2M DMA transfer with DMAC as the flow controller. The relevant test parameters are listed in the following tables.

Test No : DMA_M2M_SDT_1**Objective : Single data transfer tests**

The objective of these tests is to check that a single data transfer takes place under the following configurations.

- 1) Setup the channel with the parameters in table 5.1-12.
- 2) Configure and enable the channel for an M2M transaction.
- 3) Check that DMAC transfers only one data of the source width.
- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-12

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	1	1	1
2.	16	8	1	1	1
3.	16	16	1	1	1
4.	32	8	1	1	1
5.	32	16	1	1	1
6.	32	32	1	1	1
7.	8	8	4	4	1
8.	16	8	4	4	1
9.	16	16	4	4	1
10.	32	8	4	4	1
11.	32	16	4	4	1

12.	32	32	4	4	1
13.	8	8	256	256	1
14.	16	8	256	256	1
15.	16	16	256	256	1
16.	32	8	256	256	1
17.	32	16	256	256	1
18.	32	32	256	256	1

Table 5.1-12 Parameters for M2M single data transfer**Test No : DMA_M2M_BDT_1****Objective : Source Burst Transfers**

The objective of these tests is to check that a burst data transfer takes place under the following configurations.

- These tests target the burst transfer between the source and destination.
- This will test that all the bytes in the FIFO are written.
- The total number of transfers in a burst is based on the number of transfers it takes to fill one word of FIFO or the complete FIFO.

- 1) Setup the channel with the parameters in table 5.1-13.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the total number of data transferred by the DMAC is same as transfer size.
- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-13

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	1	1	2
2.	8	8	1	1	3
3.	8	8	1	1	4
4.	8	8	4	1	1
5.	8	8	4	1	2
6.	8	8	4	1	3
7.	8	8	4	1	4
8.	8	8	4	1	5
9.	8	8	4	1	6
10.	8	8	4	1	7
11.	8	8	4	1	8

12.	8	8	4	1	16
13.	8	8	8	1	1
14.	8	8	8	1	2
15.	8	8	8	1	3
16.	8	8	8	1	4
17.	8	8	8	1	5
18.	8	8	8	1	6
19.	8	8	8	1	7
20.	8	8	8	1	8
21.	8	8	8	1	9
22.	8	8	8	1	10
23.	8	8	8	1	11
24.	8	8	8	1	12
25.	8	8	8	1	13
26.	8	8	8	1	14
27.	8	8	8	1	15
28.	8	8	8	1	16
29.	8	8	8	1	32
30.	8	8	16	1	1
31.	8	8	16	1	2
32.	8	8	16	1	3
33.	8	8	16	1	4
34.	8	8	16	1	5
35.	8	8	16	1	6
36.	8	8	16	1	7
37.	8	8	16	1	8
38.	8	8	16	1	9
39.	8	8	16	1	10
40.	8	8	16	1	11
41.	8	8	16	1	12
42.	8	8	16	1	13
43.	8	8	16	1	14

44.	8	8	16	1	15
45.	8	8	16	1	16
46.	8	8	16	1	17
47.	8	8	16	1	18
48.	8	8	16	1	19
49.	8	8	16	1	20
50.	8	8	16	1	21
51.	8	8	16	1	22
52.	8	8	16	1	23
53.	8	8	16	1	24
54.	8	8	16	1	25
55.	8	8	16	1	26
56.	8	8	16	1	27
57.	8	8	16	1	28
58.	8	8	16	1	29
59.	8	8	16	1	30
60.	8	8	16	1	31
61.	8	8	16	1	32
62.	8	8	16	1	64
63.	8	8	32	1	1
64.	8	8	32	1	4
65.	8	8	32	1	32
66.	8	8	32	1	128
67.	8	8	64	1	1
68.	8	8	64	1	8
69.	8	8	64	1	64
70.	8	8	64	1	256
71.	8	8	128	1	1
72.	8	8	128	1	16
73.	8	8	128	1	128
74.	8	8	128	1	512
75.	8	8	256	1	1

76.	8	8	256	1	32
77.	8	8	256	1	256
78.	8	8	256	1	1024

Table 5.1-13 Parameters for M2M source burst data transfer**Test No : DMA_M2M_BDT_2****Objective : Destination Burst Transfers**

- 1) Setup the channel with the parameters in table 5.1-14.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the total number of data transferred by the DMAC is same as transfer size.
- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-14

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	1	4	1
2.	8	8	1	4	2
3.	8	8	1	4	3
4.	8	8	1	4	4
5.	8	8	1	4	5
6.	8	8	1	4	6
7.	8	8	1	4	7
8.	8	8	1	4	8
9.	8	8	1	4	16
10.	8	8	1	8	1
11.	8	8	1	8	2
12.	8	8	1	8	3
13.	8	8	1	8	4
14.	8	8	1	8	5
15.	8	8	1	8	6
16.	8	8	1	8	7
17.	8	8	1	8	8
18.	8	8	1	8	9

19.	8	8	1	8	10
20.	8	8	1	8	11
21.	8	8	1	8	12
22.	8	8	1	8	13
23.	8	8	1	8	14
24.	8	8	1	8	15
25.	8	8	1	8	16
26.	8	8	1	8	32
27.	8	8	1	16	1
28.	8	8	1	16	2
29.	8	8	1	16	3
30.	8	8	1	16	4
31.	8	8	1	16	5
32.	8	8	1	16	6
33.	8	8	1	16	7
34.	8	8	1	16	8
35.	8	8	1	16	9
36.	8	8	1	16	10
37.	8	8	1	16	11
38.	8	8	1	16	12
39.	8	8	1	16	13
40.	8	8	1	16	14
41.	8	8	1	16	15
42.	8	8	1	16	16
43.	8	8	1	16	17
44.	8	8	1	16	18
45.	8	8	1	16	19
46.	8	8	1	16	20
47.	8	8	1	16	21
48.	8	8	1	16	22
49.	8	8	1	16	23
50.	8	8	1	16	24

51.	8	8	1	16	25
52.	8	8	1	16	26
53.	8	8	1	16	27
54.	8	8	1	16	28
55.	8	8	1	16	29
56.	8	8	1	16	30
57.	8	8	1	16	31
58.	8	8	1	16	32
59.	8	8	1	16	64
60.	8	8	1	32	1
61.	8	8	1	32	4
62.	8	8	1	32	32
63.	8	8	1	32	128
64.	8	8	1	64	1
65.	8	8	1	64	8
66.	8	8	1	64	64
67.	8	8	1	64	256
68.	8	8	1	128	1
69.	8	8	1	128	16
70.	8	8	1	128	128
71.	8	8	1	128	512
72.	8	8	1	256	1
73.	8	8	1	256	64
74.	8	8	1	256	256
75.	8	8	1	256	1024

Table 5.1-14 Parameters for M2M destination burst data transfer**Test No : DMA_M2M_BDT_3****Objective : Source and Destination burst combinations with Src Width = 8 and Dest Width = 8 are tested**

- 1) Setup the channel with the parameters in table 5.1-15.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the total number of data transferred by the DMAC is same as transfer size.

- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-15

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	4	4	1
2.	8	8	4	4	3
3.	8	8	4	4	4
4.	8	8	4	4	7
5.	8	8	8	4	1
6.	8	8	8	4	5
7.	8	8	8	4	8
8.	8	8	8	8	3
9.	8	8	8	8	7
10.	8	8	8	8	8
11.	8	8	8	8	15
12.	8	8	16	4	2
13.	8	8	16	4	12
14.	8	8	16	4	23
15.	8	8	16	8	5
16.	8	8	16	8	8
17.	8	8	16	8	15
18.	8	8	16	16	10
19.	8	8	16	16	16
20.	8	8	16	16	100
21.	8	8	16	16	255

Table 5.1-15 Burst data transfers with source width = 8 and destination width = 8

Test No : DMA_M2M_BDT_4

Objective : Source and Destination burst combinations with Src Width = 8 and Dest Width = 16 are tested

- 1) Setup the channel with the parameters in table 5.1-16.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the total number of data transferred by the DMAC is same as transfer size.

- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-16

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	16	4	4	2
2.	8	16	4	4	4
3.	8	16	4	4	6
4.	8	16	8	4	2
5.	8	16	8	4	6
6.	8	16	8	4	8
7.	8	16	8	8	4
8.	8	16	8	8	8
9.	8	16	8	8	18
10.	8	16	16	4	2
11.	8	16	16	4	10
12.	8	16	16	4	50
13.	8	16	16	8	6
14.	8	16	16	8	20
15.	8	16	16	8	100
16.	8	16	16	16	14
17.	8	16	16	16	16
18.	8	16	16	16	200
19.	8	16	16	16	3000

Table 5.1-16 Burst data transfers with source width = 8 and destination width = 16

Test No : DMA_M2M_BDT_5

Objective : Source and Destination burst combinations with Src Width = 8 and Dest Width = 32 are tested

- 1) Setup the channel with the parameters in table 5.1-17.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the total number of data transferred by the DMAC is same as transfer size.

- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-17

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	32	4	4	4
2.	8	32	4	4	8
3.	8	32	8	4	4
4.	8	32	8	4	8
5.	8	32	8	4	12
6.	8	32	8	8	4
7.	8	32	8	8	16
8.	8	32	8	8	28
9.	8	32	16	4	4
10.	8	32	16	4	12
11.	8	32	16	4	16
12.	8	32	16	8	8
13.	8	32	16	8	12
14.	8	32	16	8	16
15.	8	32	16	16	4
16.	8	32	16	16	16
17.	8	32	16	16	1024

Table 5.1-17 Burst data transfers with source width = 8 and destination width = 32

Test No : DMA_M2M_BDT_6

Objective : Source and Destination burst combinations with Src Width = 16 and Dest Width = 8 are tested

- 1) Setup the channel with the parameters in table 5.1-18.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the total number of data transferred by the DMAC is same as transfer size.
- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-18

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	16	8	4	4	1

2.	16	8	4	4	2
3.	16	8	4	4	3
4.	16	8	4	4	4
5.	16	8	4	4	15
6.	16	8	8	4	1
7.	16	8	8	4	4
8.	16	8	8	4	6
9.	16	8	8	4	8
10.	16	8	8	4	9
11.	16	8	8	8	1
12.	16	8	8	8	2
13.	16	8	8	8	8
14.	16	8	8	8	11
15.	16	8	8	8	256
16.	16	8	16	4	1
17.	16	8	16	4	4
18.	16	8	16	4	8
19.	16	8	16	4	12
20.	16	8	16	8	3
21.	16	8	16	8	6
22.	16	8	16	8	8
23.	16	8	16	8	16
24.	16	8	16	8	500
25.	16	8	16	16	5
26.	16	8	16	16	16
27.	16	8	16	16	1200

Table 5.1-18 Burst data transfers with source width = 16 and destination width = 8**Test No : DMA_M2M_BDT_7****Objective : Source and Destination burst combinations with Src Width = 16 and Dest Width = 16 are tested**

- 1) Setup the channel with the parameters in table 5.1-19.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the total number of data transferred by the DMAC is same as transfer size.

- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-19

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	16	16	4	4	1
2.	16	16	4	4	2
3.	16	16	4	4	3
4.	16	16	4	4	4
5.	16	16	4	4	7
6.	16	16	4	4	255
7.	16	16	8	4	1
8.	16	16	8	4	5
9.	16	16	8	4	8
10.	16	16	8	4	15
11.	16	16	8	8	1
12.	16	16	8	8	4
13.	16	16	8	8	8
14.	16	16	8	8	45
15.	16	16	16	4	3
16.	16	16	16	4	6
17.	16	16	16	4	8
18.	16	16	16	4	100
19.	16	16	16	8	5
20.	16	16	16	8	16
21.	16	16	16	8	250
22.	16	16	16	16	8
23.	16	16	16	16	16
24.	16	16	16	16	800

Table 5.1-19 Burst data transfers with source width = 16 and destination width = 16

Test No : DMA_M2M_BDT_8

Objective : Source and Destination burst combinations with Src Width = 16 and Dest Width = 32 are tested

- 1) Setup the channel with the parameters in table 5.1-20.
- 2) Configure and enable channel for an M2M transaction.

- 3) Check that the total number of data transferred by the DMAC is same as transfer size.
- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-20

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	16	32	4	4	2
2.	16	32	4	4	4
3.	16	32	4	4	40
4.	16	32	8	4	2
5.	16	32	8	4	4
6.	16	32	8	4	8
7.	16	32	8	8	2
8.	16	32	8	8	8
9.	16	32	8	8	56
10.	16	32	16	4	2
11.	16	32	16	4	4
12.	16	32	16	4	8
13.	16	32	16	8	6
14.	16	32	16	8	8
15.	16	32	16	8	12
16.	16	32	16	8	2048

Table 5.1-20 Burst data transfers with source width = 16 and destination width = 32

Test No : DMA_M2M_BDT_9

Objective : Source and Destination burst combinations with Src Width = 32 and Dest Width = 8 are tested

- 1) Setup the channel with the parameters in table 5.1-21.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the total number of data transferred by the DMAC is same as transfer size.
- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-21

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	32	8	4	4	1
2.	32	8	4	4	2

3.	32	8	4	4	3
4.	32	8	4	4	4
5.	32	8	4	4	5
6.	32	8	4	4	6
7.	32	8	4	4	7
8.	32	8	4	4	8
9.	32	8	8	4	1
10.	32	8	8	4	4
11.	32	8	8	4	25
12.	32	8	8	8	1
13.	32	8	8	8	5
14.	32	8	8	8	8
15.	32	8	8	8	1024

Table 5.1-21 Burst data transfers with source width = 32 and destination width = 8

Test No : DMA_M2M_BDT_10

Objective : Source and Destination burst combinations with Src Width = 32 and Dest Width = 16 are tested

- 1) Setup the channel with the parameters in table 5.1-22.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the total number of data transferred by the DMAC is same as transfer size.
- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-22

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	32	16	4	4	1
2.	32	16	4	4	2
3.	32	16	4	4	3
4.	32	16	4	4	4
5.	32	16	4	4	5
6.	32	16	4	4	6
7.	32	16	4	4	7
8.	32	16	4	4	8

9.	32	16	8	4	1
10.	32	16	8	4	3
11.	32	16	8	4	100
12.	32	16	8	8	1
13.	32	16	8	8	8
14.	32	16	8	8	125
15.	32	16	8	8	512

Table 5.1-22 Burst data transfers with source width = 32 and destination width = 16

Test No : DMA_M2M_BDT_11

Objective : Source and Destination burst combinations with Src Width = 32 and Dest Width = 32 are tested

- 1) Setup the channel with the parameters in table 5.1-22a.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the total number of data transferred by the DMAC is same as transfer size.
- 4) Check that DMACIntTCStatus is set at the end of the transfer.
- 5) Repeat steps 1) to 5) for all the combinations listed in table 5.1-22a.

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	32	32	4	4	1
2.	32	32	4	4	2
3.	32	32	4	4	3
4.	32	32	4	4	4
5.	32	32	4	4	5
6.	32	32	4	4	6
7.	32	32	4	4	7
8.	32	32	4	4	8
9.	32	32	8	4	1
10.	32	32	8	4	6
11.	32	32	8	4	38
12.	32	32	8	8	1
13.	32	32	8	8	7
14.	32	32	8	8	8

15.	32	32	8	8	225
16.	32	32	8	8	4096

Table 5.1-22a Burst data transfers with source width = 32 and destination width = 32**Lock and Protection signals test****Test No : DMA_M2M_LP_1****Objective : The functionality of Lock and Protection signals are tested here.**

- 1) Setup the channel with the parameters in table 5.1-22b.
- 2) Configure and enable channel for an M2M transaction.
- 3) Check that the lock and protection signals are driven to the programmed values.
- 4) Check that the total number of data transferred by the DMAC is same as transfer size.
- 5) Check that DMACIntTCStatus is set at the end of the transfer.
- 6) Repeat steps 1) to 5) for all the combinations listed in table 5.1-22b.

Test No.	Src Width	Dest Width	Src Inc	Dest Inc	Src Burst	Dest Burst	Transfer Size	LLI	Protection	Lock
1	32	32	NI	NI	1	1	4	0	pbc	NL
2	32	32	NI	I	1	1	4	0	pbc	L
3	32	32	I	NI	1	1	4	0	pbC	NL
4	32	32	I	I	1	1	4	0	pBc	NL
5	32	32	NI	I	1	1	4	0	Pbc	NL
6	32	32	I	I	1	1	4	0	PBC	L

Table 5.1-22b Parameters for protection and lock testing.

Peripheral to Memory Transfer – Flow Controller DMAC:

The following tests will target a P2M DMA transfer with DMAC as the flow controller.

- If the testing requires only single data request (DMACSREQ), then the following table 5.1-23 defines the parameters of testing. The tests are repeated for each set of parameters defined in the table.

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	16	1	9
2.	8	16	16	16	10
3.	8	32	32	32	20
4.	16	8	8	8	11
5.	16	16	8	1	8
6.	16	32	8	8	16
7.	32	8	4	4	3
8.	32	16	8	8	12
9.	32	32	4	1	9

Table 5.1-23 Parameters for DMACSREQ testing

- If the testing requires burst data request (DMACBREQ), then the following table 5.1-24 defines the parameters of testing. The tests are repeated for each set of parameters defined in the table.

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	16	1	49
2.	8	16	16	16	16
3.	8	32	32	32	20
4.	16	8	8	8	3
5.	16	16	8	1	29
6.	16	32	8	8	6
7.	32	8	4	4	1
8.	32	16	8	8	2
9.	32	32	4	1	19

Table 5.1-24 Parameters for DMACBREQ testing

- The sequence of peripheral's request generation described in the following tests only indicates the timing of the request signal generation for the first time. For subsequent transfers, the request is initiated on sampling the assertion of DMACCLR for the previous request. This request generation continues till the terminal count interrupt (DMACTC) is sampled asserted for a burst transfer.
- Check that DMACCLR is asserted after transferring one data for a DMACSREQ assertion.
- Check that DMACCLR is asserted after a burst data transfer for a DMACBREQ assertion.

Test No : DMA_P2M_DMAC_1**Objective : To test DMACSREQ request**

The objective of this test is to check that DMACSREQ of the source peripheral can be asserted at any point of time with respect to configuring a channel.

The sequence of events is listed in the table from left to right in a row in table 5.1-25 below.

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-23	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACSREQ signal
2.	Setup the channel with a set of parameters in table 5.1-23	Configure the channel for a P2M transfer	Assert source DMACSREQ signal	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-23	Assert source DMACSREQ signal.	Configure the channel for a P2M transfer	Enable the channel
4.	Assert source DMACSREQ signal.	Setup the channel with a set of parameters in table 5.1-23	Configure the channel for a P2M transfer	Enable the channel

Table 5.1-25 Sequence of DMACSREQ test

Test No : DMA_P2M_DMAC_2**Objective : To test DMACBREQ request**

The objective of this test is to check that DMACBREQ of the source peripheral can be asserted at any point of time with respect to configuring a channel.

The sequence of events is listed in the table from left to right in a row in table 5.1-26 below.

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-24	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-24	Configure the channel for a P2M transfer	Assert source DMACBREQ signal	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-24	Assert source DMACBREQ signal	Configure the channel for a P2M transfer	Enable the channel
4.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-24	Configure the channel for a P2M transfer	Enable the channel

Table 5.1-26 Sequence of DMACBREQ test

Test No : DMA_P2M_DMAC_3**Objective : Assertion of DMACSREQ followed by DMACBREQ of the Source Peripheral.**

The sequence of events are listed in the table from left to right in a row in table 5.1-27

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1.24	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACSREQ signal	Assert source DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1.24	Configure the channel for a P2M transfer	Assert source DMACSREQ signal	Enable the channel	Assert source DMACBREQ signal
3.	Setup the channel with a set of parameters in table 5.1.24	Configure the channel for a P2M transfer	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1.24	Assert source DMACSREQ signal	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACBREQ signal
5.	Setup the channel with a set of parameters in table 5.1.24	Assert source DMACSREQ signal	Configure the channel for a P2M transfer	Assert source DMACBREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1.24	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2M transfer	Enable the channel
7.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1.24	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACBREQ signal
8.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1.24	Configure the channel for a P2M transfer	Assert source DMACBREQ signal	Enable the channel
9.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1.24	Assert source DMACBREQ signal	Configure the channel for a P2M transfer	Enable the channel
10.	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1.24	Configure the channel for a P2M transfer	Enable the channel

Table 5.1-27 Sequence of DMACSREQ and DMACBREQ test

Test No : DMA_P2M_DMAC_4**Objective : Assertion of a combination of Source DMACSREQ and Source DMACBREQ**

- 1) Setup the channel with a set of parameters in table 5.1-28
- 2) Configure and enable the channel for a P2M transaction.
- 3) Generate a burst request using Source DMACBREQ.
- 4) Check that DMACCLR, of the source peripheral is asserted after the burst transfer.
- 5) Wait for few clocks, and assert a combination of DMACSREQ and DMACSREQ signals of the Source Peripheral, till all the data transfers are completed.
- 6) Repeat steps 4) and 5) till the DMA transfer is completed.
- 7) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	16	16	79
2.	8	16	16	16	14
3.	8	32	32	32	24
4.	16	8	8	8	7
5.	16	16	8	8	87
6.	16	32	8	8	6
7.	32	8	4	4	4
8.	32	16	8	8	8
9.	32	32	4	4	31

Table 5.1-28 Combination of DMACSREQ and DMACBREQ test**Test No : DMA_P2M_DMAC_5****Objective : Assertion of a combination of Source DMACSREQ and Source DMACBREQ.**

- 1) Setup the Channel0 with the following parameters as in table 5.1-29 below.
- 2) Configure and enable Channel0 for a P2M transaction.
- 3) Generate a single data request using Source DMACSREQ.
- 4) Check that DMACCLR, of the source peripheral is asserted after a single data transfer.
- 5) Wait for few clocks, and assert DMACBREQ if the previous request was DMACSREQ and vice versa.
- 6) Repeat steps 5) and 6) till the DMA transfer is completed.
- 7) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	16	16	79
2.	8	16	16	16	14
3.	8	32	32	32	24
4.	16	8	8	8	7
5.	16	16	8	8	87
6.	16	32	8	8	6
7.	32	8	4	4	4
8.	32	16	8	8	8
9.	32	32	4	4	31

Table 5.1-29 Combination of DMACSREQ and DMACBREQ test**Test No : DMA_P2M_DMAL_6****Objective : Random sequence of Source DMACSREQ and Source DMACBREQ**

Generate a random sequence Source DMACSREQ and Source DMACBREQ till DMACTC is sampled asserted. The parameters are same as mentioned in table 5.1-29 above.

Memory to Peripheral Transfer – Flow Controller DMAC:

The following tests will target an M2P DMA transfer with DMAC as the flow controller.

Notes:

- The following table 5.1-30 defines the parameters of testing. The tests are repeated for each set of parameters defined in the table.

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	16	1	49
2.	8	16	16	16	30
3.	8	32	32	32	4
4.	16	8	8	8	11
5.	16	16	8	1	66
6.	16	32	8	8	2
7.	32	8	4	4	15
8.	32	16	8	8	1
9.	32	32	4	1	99

Table 5.1-30 Parameters for DMACBREQ testing

- The sequence of peripheral's request generation described in the following tests only indicates the timing of the request signal generation for the first time. For subsequent transfers, the request is initiated on sampling the assertion of DMACCLR for the previous request. This request generation continues till the terminal count interrupt (DMACTC) is sampled asserted for a burst transfer.
- Check that DMACCLR is asserted after a burst data transfer for a DMACBREQ assertion.

Test No : DMA_M2P_DMAC_1**Objective : Assertion of DMACBREQ request**

The sequence of events are listed in the table from left to right in a row in table 5.1-31

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-30	Configure the channel for an M2P transfer	Enable the channel	Wait for the memory transfer to start on the source bus	Assert destination DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-30	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACBREQ signal	NA
3.	Setup the channel with a set of parameters in table 5.1-30	Configure the channel for an M2P transfer	Assert destination DMACBREQ signal	Enable the channel	NA

4.	Setup the channel with a set of parameters in table 5.1-30	Assert destination DMACBREQ signal	Configure the channel for an M2P transfer	Enable the channel	NA
5.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-30	Configure the channel for an M2P transfer	Enable the channel	NA

Table 5.1-31 Sequence of Destination DMACBREQ test**Test No : DMA_M2P_DMAC_2****Objective : Random sequence of Destination DMACSREQ and Destination DMACBREQ**

Generate a random sequence Source DMACSREQ and Source DMACBREQ till DMACTC is sampled asserted. The parameters are same as mentioned in table 5.1-30 above.

Peripheral to Peripheral Transfer – Flow Controller DMAC:

The following tests will target a P2P DMA transfer with DMAC as the flow controller.

Notes:

- The configuration of the DMAC is such that the source and destination widths will be same (8/8, 16/16 and 32/32) and the burst sizes will be any one of the valid values upto a maximum that is required to fill the complete FIFO.
- The following table 5.1-32 defines the parameters of testing.

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	16	16	49
2.	16	16	8	4	49
3.	32	32	4	1	49

Table 5.1-32 Parameters for DMACBREQ testing

- The sequence of request generation described in the following tests only indicates the timing of the request signal generation for the first time. For subsequent transfers, the request is initiated on sampling the assertion of DMACCLR for the previous request. This request generation continues till the terminal count interrupt (DMACTC) is sampled asserted for a burst transfer.
- Check that DMACCLR is asserted after a single data transfer for a DMACSREQ assertion.
- Check that DMACCLR is asserted after a burst data transfer for a DMACBREQ assertion.

Test No : DMA_P2P_DMAC_1**Objective : Source DMACSREQ followed by Destination DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-33

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal

5.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal
8.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel
9.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-33 Sequence of Source DMACSREQ and Destination DMACBREQ**Test No : DMA_P2P_DMAC_1A****Objective : Simultaneous assertion of Source DMACSREQ and Destination DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-34

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ and destination DMACBREQ signals simultaneously.
2.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACSREQ and destination DMACBREQ signals simultaneously.	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ and destination DMACBREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel

4.	Assert source DMACSREQ and destination DMACBREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel
----	---	--	--	--------------------

Table 5.1-34 Sequence of Source DMACSREQ and Destination DMACBREQ**Test No : DMA_P2P_DMAC_2****Objective : Destination DMACBREQ followed by Source DMACSREQ**

The sequence of events are listed in the table from left to right in a row as in table 5.1-35

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACSREQ signal
2.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACSREQ signal
3.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal
5.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel

9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-35 Sequence of Destination DMACBREQ followed by Source DMACSREQ testing**Test No : DMA_P2P_DMAC_3****Objective : Source DMACBREQ followed by Destination DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-36

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal
5.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal

8.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel
9.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-36 Source DMACBREQ followed by Destination DMACBREQ testing**Test No : DMA_P2P_DMAC_4****Objective : Simultaneous assertion of Source DMACBREQ and Destination DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-37

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.
2.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-32	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel
4.	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-37 Simultaneous assertion of Source DMACBREQ and Destination DMACBREQ testing**Test No : DMA_P2P_DMAC_5****Objective : Destination DMACBREQ followed by Source DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-38

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
----------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal
5.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-38 Destination DMACBREQ followed by Source DMACBREQ testing**Test No : DMA_P2P_DMAC_6****Objective : Assertion of Source DMACSREQ followed by Source DMACBREQ followed by Destination DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-39

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Assert destination DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACBREQ signal
4.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Enable the channel
5.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACBREQ signal
6.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACBREQ signal
7.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Enable the channel
8.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel
9.	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel

10.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACBREQ signal
11.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal
12.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel
13.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
14.	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
15.	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-39 Sequence of Source DMACSREQ followed by Source DMACBREQ followed by Destination DMACBREQ testing

Test No : DMA_P2P_DMAC_7

Objective : Simultaneous assertion of Source DMACSREQ, Source DMACBREQ and Destination DMACBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-40

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert Source DMACSREQ, Source DMACBREQ and Destination DMACBREQ signals simultaneously

2.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert Source DMACSREQ, Source DMACBREQ and Destination DMACBREQ signals simultaneously	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-32	Assert Source DMACSREQ, Source DMACBREQ and Destination DMACBREQ signals simultaneously	Configure the channel for a P2P transfer	Enable the channel
4.	Assert Source DMACSREQ, Source DMACBREQ and Destination DMACBREQ signals simultaneously	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-40 Simultaneous assertion of Source DMACSREQ, Source DMACBREQ and Destination DMACBREQ testing

Test No : DMA_P2P_DMAC_8

Objective : Assertion of Destination DMACBREQ followed by Source DMACSREQ followed by Source DMACBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-41

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Assert source DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACSREQ signal	Assert source DMACBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Enable the channel	Assert source DMACBREQ signal
4.	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Enable the channel

5.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert source DMACBREQ signal
6.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert source DMACBREQ signal
7.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Enable the channel
8.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel
9.	Setup the channel with a set of parameters in table 5.1-32	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert source DMACBREQ signal
11.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert source DMACBREQ signal
12.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Enable the channel
13.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel

14.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
15.	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-32	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
16.	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-32	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-41 Assertion of Destination DMACBREQ followed by Source DMACSREQ followed by Source DMACBREQ testing

Test No : DMA_P2P_DMAC_9

Objective : Assertion of a combination of Source DMACSREQ, Source DMACBREQ and Destination DMACBREQ

- 1) Setup the channel with a set of parameters in table 5.1-32.
- 2) Configure and Enable the channel for a P2P transaction.
- 3) Generate a burst request using Source DMACBREQ signal.
- 4) Generate a burst request using Destination DMACBREQ signal.
- 5) Check that DMACCLR, of the source peripheral is asserted after the data transfer.
- 6) Wait for few clocks, and assert DMACSREQ/DMACBREQ of the Source Peripheral again.
- 7) Repeat steps 5) and 6) till the DMA transfer is completed.
- 8) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.
- 9) Check that DMACCLR, of the Destination Peripheral is asserted after the burst transfer to the destination peripheral.
- 10) Wait for few clocks, and assert DMACBREQ of the Destination Peripheral.
- 11) Repeat steps 9) and 10) till the DMA transfer is completed.
- 12) Check that DMACCLR and DMACTC of the Destination Peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_DMAC_10

Objective : Assertion of a combination of Source DMACSREQ, Source DMACBREQ and Destination DMACBREQ.

- 1) Setup the channel with a set of parameters in table 5.1-32.
- 2) Configure and enable the channel for a P2P transaction.

- 3) Generate a single data request using Source DMACSREQ signal.
- 4) Generate a burst request using Destination DMACBREQ signal.
- 5) Check that DMACCLR, of the source peripheral is asserted after the burst transfer.
- 6) Wait for few clocks, and assert DMACBREQ if the previous request was DMACSREQ and vice versa.
- 7) Repeat steps 5) and 6) till the DMA transfer is completed.
- 8) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.
- 9) Check that DMACCLR, of the Destination Peripheral is asserted after the burst transfer to the destination peripheral.
- 10) Wait for few clocks, and assert DMACBREQ of the Destination Peripheral.
- 11) Repeat steps 9) and 10) till the DMA transfer is completed.
- 12) Check that DMACCLR and DMACTC of the Destination Peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_DMAC_11

Objective : Random sequence of Source DMACSREQ, Source DMACBREQ, Destination DMACSREQ and Destination DMACBREQ assertion.

Generate Source DMACSREQ, Source DMACBREQ, Destination DMACSREQ and Destination DMACBREQ randomly till their respective DMACTC are sampled asserted. The parameters can also be set to one of the valid values, from table 5.1-32.

Peripheral to Memory Transfer – Flow Controller Peripheral:

The following tests will target a P2M DMA transfer with the respective peripheral as the flow controller.

Notes:

- The configuration of the DMAC is such that the source and destination widths will be same (8/8, 16/16 and 32/32) and the burst sizes will be any one of the valid values upto a maximum that is required to fill the complete FIFO.
- The following table 5.1-42 defines the parameters of testing.

Test No	Src Width	Dest Width	Src Burst	Dest Burst
1.	8	8	16	1
2.	16	16	8	1
3.	32	32	4	1

Table 5.1-42 Parameters for P2M testing

- The sequence of request generation described in the following tests only indicates the timing of the request signal generation for the first time. For subsequent transfers, the request is initiated on sampling the assertion of DMACCLR for the previous request. This request generation continues till the terminal count interrupt (DMACTC) is sampled asserted for a burst transfer.
- Check that DMACCLR is asserted after a single data transfer for a DMACSREQ / DMACLSREQ assertion.
- Check that DMACCLR is asserted after a burst data transfer for a DMACBREQ / DMACLBREQ assertion.

Test No : DMA_P2M_SP_1**Objective : Assertion of DMACLSREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-43

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACLSREQ signal
2.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Assert source DMACLSREQ signal	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-42	Assert source DMACLSREQ signal	Configure the channel for a P2M transfer	Enable the channel
4.	Assert source DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel

Table 5.1-43 Sequence of DMACLSREQ testing**Test No : DMA_P2M_SP_2****Objective : Assertion of DMACLBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-44

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACLBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Assert source DMACLBREQ signal	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-42	Assert source DMACLBREQ signal	Configure the channel for a P2M transfer	Enable the channel
4.	Assert source DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel

Table 5.1-44 Sequence of DMACLBREQ testing**Test No : DMA_P2M_SP_3****Objective : Assertion of DMACSREQ followed by DMACLSREQ**

The sequence of events are listed in the table from left to right in a row table 5.1-45

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACSREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Assert source DMACSREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-42	Assert source DMACSREQ signal	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.

4.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.
----	-------------------------------	--	--	--------------------	---

Table 5.1-45 DMACSREQ followed by DMACLSREQ testing**Test No : DMA_P2M_SP_4****Objective : Assertion of DMACSREQ followed by DMACLBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-46

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACSREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Assert source DMACSREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-42	Assert source DMACSREQ signal	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for DMACSREQ request.
4.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for DMACSREQ request.

Table 5.1-46 DMACSREQ followed by DMACLBREQ testing

Test No : DMA_P2M_SP_5**Objective : Assertion of DMACBREQ followed by DMACLSREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-47

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Assert source DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-42	Assert Source DMACBREQ	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.
4.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.

Table 5.1.47 DMACBREQ followed by DMACLSREQ testing

Test No : DMA_P2M_SP_6**Objective : Assertion of DMACBREQ followed by DMACLBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-48

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
----------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Assert source DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-42	Assert source DMACBREQ signal	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.
4.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.

Table 5.1-48 DMACBREQ followed by DMACLBREQ testing**Test No : DMA_P2M_SP_7****Test : Assertion of 2 DMACSREQ followed by DMACLSREQ.**

The objective of this test is to check that a DMACSREQ followed by DMACSREQ can be handled by the DMAC.

The sequence of events are listed in the table from left to right in a row in table 5.1-49

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACSREQ signal	Assert Source DMACSREQ on sampling DMACCLR asserted for DMACSREQ	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.

2.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Assert source DMACSREQ signal	Enable the channel	Assert Source DMACSREQ on sampling DMACCLR asserted for DMACSREQ	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-42	Assert source DMACSREQ signal	Configure the channel for a P2M transfer	Enable the channel	Assert Source DMACSREQ on sampling DMACCLR asserted for DMACSREQ	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.
4.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert Source DMACSREQ on sampling DMACCLR asserted for DMACSREQ	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.

Table 5.1-49 Sequence of 2 DMACSREQ followed by DMACLSREQ testing**Test No : DMA_P2M_SP_8****Test : Assertion of 2 DMACBREQ followed by DMACLBREQ.**

The objective of this test is to check that a DMACBREQ followed by DMACLBREQ can be handled by the DMAC.

The sequence of events are listed in the table from left to right in a row in table 5.1-50

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert source DMACBREQ signal	Assert Source DMACBREQ on sampling DMACCLR asserted for DMACBREQ request.	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Assert source DMACBREQ signal	Enable the channel	Assert Source DMACBREQ on sampling DMACCLR asserted for DMACBREQ request.	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.

3.	Setup the channel with a set of parameters in table 5.1-42	Assert source DMACBREQ signal	Configure the channel for a P2M transfer	Enable the channel	Assert Source DMACBREQ on sampling DMACCLR asserted for DMACBREQ request.	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.
4.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-42	Configure the channel for a P2M transfer	Enable the channel	Assert Source DMACBREQ on sampling DMACCLR asserted for DMACBREQ request.	Assert source DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.

Table 5.1-50 Sequence of 2 DMACBREQ followed by DMACLBREQ testing**Test No : DMA_P2M_SP_9****Objective : Assertion of multiple DMACBREQ and DMACLSREQ to complete the DMA transfer**

- 1) Setup the channel with the parameters in table 5.1-42.
- 2) Configure and enable the channel for a P2M transaction.
- 3) Generate a single data transfer request, using Source DMACBREQ.
- 4) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ of the Source Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a last single data transfer request, using Source DMACLSREQ.
- 8) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.

Test No : DMA_P2M_SP_10**Objective : Assertion of multiple DMACBREQ and DMACLBREQ to complete the DMA transfer**

- 1) Setup the channel with the parameters in table 5.1-42
- 2) Configure and enable the channel for a P2M transaction.
- 3) Generate a single data transfer request, using Source DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the Source Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a last burst data transfer request, using Source DMACLBREQ signal.
- 8) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.

Test No : DMA_P2M_SP_11**Objective : Assertion of multiple DMACBREQ and DMACLSREQ to complete the DMA transfer**

- 1) Setup the channel with the parameters in table 5.1-42
- 2) Configure and enable the channel for a P2M transaction.
- 3) Generate a burst data transfer request, using Source DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the Source Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a last single data transfer request, using Source DMACLSREQ signal.
- 8) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.

Test No : DMA_P2M_SP_12**Objective : Assertion of multiple DMACBREQ and DMACLBREQ to complete the DMA transfer**

- 1) Setup the channel with the parameters in table 5.1-42
- 2) Configure and enable the channel for a P2M transaction.
- 3) Generate a burst data transfer request, using Source DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the Source Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a last burst data transfer request, using Source DMACLBREQ signal.
- 8) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.

Test No : DMA_P2M_SP_13**Objective : Assertion of multiple DMACBREQ / DMACSREQ and DMACLSREQ to complete the DMA transfer**

- 1) Setup the channel with the parameters in table 5.1-42
- 2) Configure and enable the channel for a P2M transaction.
- 3) Generate a burst data transfer request, using Source DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the Source Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a single data transfer request, using Source DMACSREQ signal.
- 8) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.

- 9) Wait for few clocks, and assert DMACSREQ signal of the Source Peripheral.
- 10) Repeat steps 8) and 9) 10 times.
- 11) Generate a last single data transfer request, using Source DMACLSREQ signal.
- 12) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.

Test No : DMA_P2M_SP_14

Objective : Assertion of multiple DMACSREQ / DMACBREQ and DMACLBREQ to complete the DMA transfer

- 1) Setup the Channel0 with the parameters in table 5.1-42
- 2) Configure and enable Channel0 for a P2M transaction.
- 3) Generate a single data transfer request, using Source DMACSREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a single data transfer of the source width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the Source Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a burst data transfer request, using Source DMACBREQ signal.
- 8) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 9) Wait for few clocks, and assert DMACBREQ signal of the Source Peripheral.
- 10) Repeat steps 8) and 9) 10 times.
- 11) Generate a last burst data transfer request, using Source DMACLBREQ signal.
- 12) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.

Test No : DMA_P2M_SP_15

Objective : Assertion of alternate DMACSREQ / DMACBREQ and DMACLBREQ to complete the DMA transfer

- 1) Setup the channel with the parameters in table 5.1-42
- 2) Configure and enable the channel for a P2M transaction.
- 3) Generate a single data transfer request, using Source DMACSREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a single data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the Source Peripheral.
- 6) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 7) Wait for few clocks, and assert DMACSREQ signal of the Source Peripheral.
- 8) Repeat steps 4) and 7) 10 times.
- 9) Generate a last burst data transfer request, using Source DMACLBREQ signal.
- 10) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.

Test No : DMA_P2M_SP_16**Objective : Assertion of alternate DMACBREQ / DMACSREQ and DMACLSREQ to complete the DMA transfer**

- 1) Setup the channel with the parameters in table 5.1-42
- 2) Configure and enable the channel for a P2M transaction.
- 3) Generate a burst data transfer request, using Source DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the Source Peripheral.
- 6) Check that DMACCLR, of the source peripheral is asserted after a single data transfer of the source width.
- 7) Wait for few clocks, and assert DMACBREQ signal of the Source Peripheral.
- 8) Repeat steps 4) and 7) 10 times.
- 9) Generate a last single data transfer request, using Source DMACLSREQ signal.
- 10) Check that DMACCLR and DMACTC of the Source Peripheral are asserted at the end of the transfer.

Memory to Peripheral Transfer – Flow Controller Peripheral:

The following tests will target an M2P DMA transfer with the respective peripheral as the flow controller.

Notes:

- The following table 5.1-51 defines the parameters of testing.

Test No	Src Width	Dest Width	Src Burst	Dest Burst	Transfer Size
1.	8	8	16	1	49
2.	8	16	16	16	30
3.	8	32	32	32	4
4.	16	8	8	8	11
5.	16	16	8	1	66
6.	16	32	8	8	2
7.	32	8	4	4	15
8.	32	16	8	8	1
9.	32	32	4	1	99

Table 5.1-51 Parameters for M2P testing

- The sequence of request generation described in the following tests only indicates the timing of the request signal generation for the first time. For subsequent transfers, the request is initiated on sampling the assertion of DMACCLR for the previous request. This request generation continues till the terminal count interrupt (DMACTC) is sampled asserted for a burst transfer.
- Check that DMACCLR is asserted after a single data transfer for a DMACSREQ / DMACLSREQ assertion.
- Check that DMACCLR is asserted after a burst data transfer for a DMACBREQ / DMACLBREQ assertion.

Test No : DMA_M2P_DP_1**Objective : Assertion of DMACLSREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-52

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACLSREQ signal
2.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Assert destination DMACLSREQ signal	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-51	Assert destination DMACLSREQ signal	Configure the channel for an M2P transfer	Enable the channel

4.	Assert destination DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel
----	-------------------------------------	--	---	--------------------

Table 5.1-52 Sequence of DMACLSREQ testing**Test No : DMA_M2P_DP_2****Objective : Assertion of DMACLBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-53

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACLBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Assert destination DMACLBREQ signal	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-51	Assert destination DMACLBREQ signal	Configure the channel for an M2P transfer	Enable the channel
4.	Assert destination DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel

Table 5.1-53 Sequence of DMACLBREQ testing**Test No : DMA_M2P_DP_3****Objective : Assertion of DMACSREQ followed by DMACLSREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-54

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.

3.	Setup the channel with a set of parameters in table 5.1-51	Assert destination DMACSREQ signal	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.
4.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.

Table 5.1-54 Sequence of DMACSREQ followed by DMACLSREQ testing**Test No : DMA_M2P_DP_3A****Objective : Assertion of DMACSREQ followed by DMACLBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-55

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-51	Assert destination DMACSREQ signal	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACSREQ request.

4.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACSREQ request.
----	------------------------------------	--	---	--------------------	--

Table 5.1-55 Sequence of DMACSREQ followed by DMACLBREQ testing**Test No : DMA_M2P_DP_4****Objective : Assertion of DMACBREQ followed by DMACLSREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-56

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-51	Assert Source DMACBREQ	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.
4.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACBREQ request.

Table 5.1-56 Sequence of DMACBREQ followed by DMACLSREQ testing**Test No : DMA_M2P_DP_5**

Objective : Assertion of DMACBREQ followed by DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-57

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-51	Assert destination DMACBREQ signal	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.
4.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.

Table 5.1-57 Sequence of DMACBREQ followed by DMACLBREQ testing

Test No : DMA_M2P_DP_6

Test : Assertion of 2 DMACSREQ followed by DMACLSREQ.

The objective of this test is to check that a DMACSREQ followed by DMACSREQ can be handled by the DMAC.

The sequence of events are listed in the table from left to right in a row in table 5.1-58

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
----------	--------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert Source DMACSREQ on sampling DMACCLR asserted for DMACSREQ request.	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert Source DMACSREQ on sampling DMACCLR asserted for DMACSREQ request.	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-51	Assert destination DMACSREQ signal	Configure the channel for an M2P transfer	Enable the channel	Assert Source DMACSREQ on sampling DMACCLR asserted for DMACSREQ request.	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.
4.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert Source DMACSREQ on sampling DMACCLR asserted for DMACSREQ request.	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for DMACSREQ request.

Table 5.1-58 Sequence of 2 DMACSREQ followed by DMACLSREQ testing**Test No : DMA_M2P_DP_7****Test : Assertion of 2 DMACBREQ followed by DMACLBREQ.**

The objective of this test is to check that a DMACBREQ followed by DMACBREQ can be handled by the DMAC.

The sequence of events are listed in the table from left to right in a row in table 5.1-59

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert Source DMACBREQ on sampling DMACCLR asserted for DMACBREQ request.	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert Source DMACBREQ on sampling DMACCLR asserted for DMACBREQ request.	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-51	Assert destination DMACBREQ signal	Configure the channel for an M2P transfer	Enable the channel	Assert Source DMACBREQ on sampling DMACCLR asserted for DMACBREQ request.	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.
4.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-51	Configure the channel for an M2P transfer	Enable the channel	Assert Source DMACBREQ on sampling DMACCLR asserted for DMACBREQ request.	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for DMACBREQ request.

Table 5.1-59 Sequence of 2 DMACBREQ followed by DMACLBREQ testing**Test No : DMA_M2P_DP_8****Objective : Assertion of multiple DMACSREQ and DMACLSREQ to complete the DMA transfer**

- 1) Setup the channel with a set of parameters in table 5.1-51
- 2) Configure and enable the channel for an M2P transaction.
- 3) Generate a single data transfer request, using destination DMACSREQ signal.
- 4) Check that DMACCLR, of the destination peripheral is asserted after one data transfer of the destination width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the Destination Peripheral.
- 6) Repeat steps 4) and 5) 10 times.

- 7) Generate a last single data transfer request, using Destination DMACLSREQ.
- 8) Check that DMACCLR and DMACTC of the Destination Peripheral are asserted at the end of the transfer.

Test No : DMA_M2P_DP_9**Objective : Assertion of multiple DMACSREQ and DMACLBREQ to complete the DMA transfer**

- 1) Setup the channel with a set of parameters in table 5.1-51
- 2) Configure and enable the channel for an M2P transaction.
- 3) Generate a single data transfer request, using Destination DMACSREQ signal.
- 4) Check that DMACCLR, of the destination peripheral is asserted after one data transfer of the destination width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the Destination Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a last burst data transfer request, using Destination DMACLBREQ signal.
- 8) Check that DMACCLR and DMACTC of the Destination Peripheral are asserted at the end of the transfer.

Test No : DMA_M2P_DP_10**Objective : Assertion of multiple DMACBREQ and DMACLSREQ to complete the DMA transfer**

- 1) Setup the channel with a set of parameters in table 5.1-51
- 2) Configure and enable the channel for an M2P transaction.
- 3) Generate a burst data transfer request, using Destination DMACBREQ signal.
- 4) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer of the destination width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the Destination Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a last single data transfer request, using Destination DMACLSREQ signal.
- 8) Check that DMACCLR and DMACTC of the Destination Peripheral are asserted at the end of the transfer.

Test No : DMA_M2P_DP_11**Objective : Assertion of multiple DMACBREQ and DMACLBREQ to complete the DMA transfer**

- 1) Setup the channel with a set of parameters in table 5.1-51
- 2) Configure and enable the channel for an M2P transaction.
- 3) Generate a burst data transfer request, using Destination DMACBREQ signal.
- 4) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer of the destination width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the Destination Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a last burst data transfer request, using Destination DMACLBREQ signal.
- 8) Check that DMACCLR and DMACTC of the Destination Peripheral are asserted at the end of the transfer.

Test No : DMA_M2P_DP_12**Objective : Assertion of multiple DMACBREQ / DMACSREQ and DMACLSREQ to complete the DMA transfer**

- 1) Setup the channel with a set of parameters in table 5.1-51
- 2) Configure and enable the channel for an M2P transaction.
- 3) Generate a burst data transfer request, using Destination DMACBREQ signal.
- 4) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer of the destination width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the Destination Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a single data transfer request, using Destination DMACSREQ signal.
- 8) Check that DMACCLR, of the destination peripheral is asserted after one data transfer of the destination width.
- 9) Wait for few clocks, and assert DMACSREQ signal of the Destination Peripheral.
- 10) Repeat steps 8) and 9) 10 times.
- 11) Generate a last single data transfer request, using Destination DMACLSREQ signal.
- 12) Check that DMACCLR and DMACTC of the Destination Peripheral are asserted at the end of the transfer.

Test No : DMA_M2P_DP_13**Objective: Assertion of multiple DMACSREQ / DMACBREQ and DMACLBREQ to complete the DMA transfer**

- 1) Setup the channel with a set of parameters in table 5.1-51
- 2) Configure and enable the channel for an M2P transaction.
- 3) Generate a single data transfer request, using Destination DMACSREQ signal.
- 4) Check that DMACCLR, of the destination peripheral is asserted after a single data transfer of the destination width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the Destination Peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a burst data transfer request, using Destination DMACBREQ signal.
- 8) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer of the destination width.
- 9) Wait for few clocks, and assert DMACBREQ signal of the Destination Peripheral.
- 10) Repeat steps 8) and 9) 10 times.
- 11) Generate a last burst data transfer request, using Destination DMACLBREQ signal.
- 12) Check that DMACCLR and DMACTC of the Destination Peripheral are asserted at the end of the transfer.

Test No : DMA_M2P_DP_14**Objective : Assertion of alternate DMACSREQ / DMACBREQ and DMACLBREQ to complete the DMA transfer**

- 1) Setup the channel with a set of parameters in table 5.1-51
- 2) Configure and enable the channel for an M2P transaction.
- 3) Generate a single data transfer request, using Destination DMACSREQ signal.
- 4) Check that DMACCLR, of the destination peripheral is asserted after a single data transfer of the destination width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the Destination Peripheral.
- 6) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer of the destination width.
- 7) Wait for few clocks, and assert DMACSREQ signal of the Destination Peripheral.
- 8) Repeat steps 4) and 7) 10 times.
- 9) Generate a last burst data transfer request, using Destination DMACLBREQ signal.
- 10) Check that DMACCLR and DMACTC of the Destination Peripheral are asserted at the end of the transfer.

Test No : DMA_M2P_DP_15

Objective : Assertion of alternate DMACBREQ / DMACSREQ and DMACLSREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-51
- 2) Configure and enable the channel for an M2P transaction.
- 3) Generate a burst data transfer request, using Destination DMACBREQ signal.
- 4) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer of the destination width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the Destination Peripheral.
- 6) Check that DMACCLR, of the destination peripheral is asserted after a single data transfer of the destination width.
- 7) Wait for few clocks, and assert DMACBREQ signal of the Destination Peripheral.
- 8) Repeat steps 4) and 7) 10 times.
- 9) Generate a last single data transfer request, using Destination DMACLSREQ signal.
- 10) Check that DMACCLR and DMACTC of the Destination Peripheral are asserted at the end of the transfer.

Peripheral to Peripheral Transfer – Flow Controller Source Peripheral:

The following tests will target a P2P DMA transfer with the respective source peripheral as the flow controller.

Notes:

- The configuration of the DMAC is such that the source and destination widths will be same (8/8, 16/16 and 32/32) and the burst sizes will be as defined in table 5.1-60.

Test No	Src Width	Dest Width	Src Burst	Dest Burst
1.	8	8	16	1
2.	16	16	8	1
3.	32	32	4	1

Table 5.1-60 Parameters for P2P testing

- The sequence of request generation described in the following tests only indicates the timing of the request signal generation for the first time. For subsequent transfers, the request is initiated on sampling the assertion of DMACCLR for the previous request. This request generation continues till the terminal count interrupt (DMACTC) is sampled asserted for a burst transfer.
- Check that DMACCLR is asserted after a single data transfer for a DMACSREQ / DMACLSREQ assertion.
- Check that DMACCLR is asserted after a burst data transfer for a DMACBREQ / DMACLBREQ assertion.

Test No : DMA_P2P_SP_1**Objective : Assertion of Source DMACLSREQ followed by Destination DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-61

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal	Assert destination DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACLSREQ signal	Enable the channel	Assert destination DMACBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACLSREQ signal	Assert destination DMACBREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACLSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal
5.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACLSREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel

6.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACLSREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert source DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal
8.	Assert source DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel
9.	Assert source DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert source DMACLSREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-61 Sequence of Source DMACLSREQ followed by Destination DMACBREQ testing

Test No : DMA_P2P_SP_2

Objective : Assertion of Simultaneous Source DMACLSREQ and Destination DMACBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-62

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert Source DMACLSREQ and Destination DMACBREQ signals simultaneously
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert Source DMACLSREQ and Destination DMACBREQ signals simultaneously	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-60	Assert Source DMACLSREQ and Destination DMACBREQ signals simultaneously	Configure the channel for a P2P transfer	Enable the channel
4.	Assert Source DMACLSREQ and Destination DMACBREQ signals simultaneously	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-62 Sequence of Simultaneous Source DMACLSREQ and Destination DMACBREQ testing**Test No : DMA_P2P_SP_3****Objective : Assertion of Destination DMACBREQ followed by Source DMACLSREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-62a

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal
5.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACLSREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACLSREQ signal	Enable the channel
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACLSREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-62a Sequence of Destination DMACBREQ followed by Source DMACLSREQ testing**Test No : DMA_P2P_SP_4****Objective : Assertion of Source DMACLBREQ followed by Destination DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-63

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal	Assert destination DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACLBREQ signal	Enable the channel	Assert destination DMACBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACLBREQ signal	Assert destination DMACBREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACLBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal
5.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACLBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACLBREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert source DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal
8.	Assert source DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel
9.	Assert source DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert source DMACLBREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-63 Sequence of Source DMACLBREQ followed by Destination DMACBREQ testing**Test No : DMA_P2P_SP_5****Objective : Assertion of Simultaneous Source DMACLBREQ and Destination DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-64

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert Source DMACLBREQ and Destination DMACBREQ signals simultaneously
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert Source DMACLBREQ and Destination DMACBREQ signals simultaneously	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-60	Assert Source DMACLBREQ and Destination DMACBREQ signals simultaneously	Configure the channel for a P2P transfer	Enable the channel
4.	Assert Source DMACLBREQ and Destination DMACBREQ signals simultaneously	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-64 Sequence of Simultaneous Source DMACLBREQ and Destination DMACBREQ testing**Test No : DMA_P2P_SP_6****Objective : Assertion of Destination DMACBREQ followed by Source DMACLBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-65

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal	Enable the channel

4.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal
5.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACLBREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACLBREQ signal	Enable the channel
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACLBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-65 Sequence of Destination DMACBREQ followed by Source DMACLBREQ testing**Test No : DMA_P2P_SP_7****Objective : Assertion of Source DMACSREQ followed by Destination DMACBREQ and completing with Source DMACLSREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-66

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
6.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

7.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
8.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
9.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
10.	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

Table 5.1-66 Sequence of Source DMACSREQ followed by Destination DMACBREQ and completing with Source DMACLSREQ testing

Test No : DMA_P2P_SP_8

Objective : Assertion Simultaneous assertion of Source DMACSREQ and Destination DMACBREQ, and completing with Source DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-67

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
----------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert Source DMACSREQ and Destination DMACBREQ signals simultaneously.	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert Source DMACSREQ and Destination DMACBREQ signals simultaneously.	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Assert Source DMACSREQ and Destination DMACBREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
4.	Assert Source DMACSREQ and Destination DMACBREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

Table 5.1-67 Sequence of Simultaneous assertion of Source DMACSREQ and Destination DMACBREQ, and completing with Source DMACLSREQ testing

Test No : DMA_P2P_SP_9

Objective : Assertion of Destination DMACBREQ followed by Source DMACSREQ and completing with Source DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1.68

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
----------	--------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACSREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

6.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
10.	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

Table 5.1-68 Sequence of Destination DMACBREQ followed by Source DMACSREQ and completing with Source DMACLSREQ testing

Test No : DMA_P2P_SP_10

Objective : Assertion of Source DMACBREQ followed by Destination DMACBREQ and completing with Source DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-69

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.

6.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
7.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
8.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
9.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
10.	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.

Table 4.6-69 Sequence of Source DMACBREQ followed by Destination DMACBREQ and completing with Source DMACLSREQ testing

Test No : DMA_P2P_SP_11

Objective : Simultaneous assertion of Source DMACBREQ and Destination DMACBREQ, and completing with Source DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-70

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
4.	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.

Table 5.1-70 Sequence of Source DMACBREQ and Destination DMACBREQ, and completing with Source DMACLSREQ testing

Test No : DMA_P2P_SP_12

Objective : Assertion of Destination DMACBREQ followed by Source DMACBREQ and completing with Source DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-71

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
----------	--------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.

6.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
10.	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLSREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.

Table 5.1-71 Sequence of Destination DMACBREQ followed by Source DMACBREQ and completing with Source DMACLSREQ testing

Test No : DMA_P2P_SP_13

Objective : Assertion of Source DMACSREQ followed by Destination DMACBREQ and completing with Source DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-72

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

6.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
7.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
8.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
9.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
10.	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

Table 5.1-72 Sequence of Source DMACSREQ followed by Destination DMACBREQ and completing with Source DMACLBREQ testing

Test No : DMA_P2P_SP_14

Objective : Assertion of Simultaneous assertion of Source DMACSREQ and Destination DMACBREQ, and completing with Source DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-73

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert Source DMACSREQ and Destination DMACBREQ signals simultaneously.	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert Source DMACSREQ and Destination DMACBREQ signals simultaneously.	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Assert Source DMACSREQ and Destination DMACBREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
4.	Assert Source DMACSREQ and Destination DMACBREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

Table 5.1-73 Sequence of Simultaneous assertion of Source DMACSREQ and Destination DMACBREQ, and completing with Source DMACLBREQ testing

Test No : DMA_P2P_SP_15

Objective : Assertion of Destination DMACBREQ followed by Source DMACSREQ and completing with Source DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-74

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
----------	--------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACSREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

6.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.
10.	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACSREQ request.

Table 5.1-74 Sequence of Destination DMACBREQ followed by Source DMACSREQ and completing with Source DMACLBREQ testing

Test No : DMA_P2P_SP_16

Objective : Assertion of Source DMACBREQ followed by Destination DMACBREQ and completing with Source DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-75

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.

6.	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
7.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
8.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
9.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
10.	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.

Table 5.1-75 Sequence of Source DMACBREQ followed by Destination DMACBREQ and completing with Source DMACLBREQ testing

Test No : DMA_P2P_SP_17

Objective : Assertion of Simultaneous assertion of Source DMACBREQ and Destination DMACBREQ, and completing with Source DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-76

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
4.	Assert Source DMACBREQ and Destination DMACBREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.

Table 5.1-76 Sequence of Simultaneous assertion of Source DMACBREQ and Destination DMACBREQ, and completing with Source DMACLBREQ testing

Test No : DMA_P2P_SP_18

Objective : Assertion of Destination DMACBREQ followed by Source DMACBREQ and completing with Source DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-77

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
----------	--------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.

6.	Setup the channel with a set of parameters in table 5.1-60	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.
10.	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-60	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACLBREQ signal on sampling DMACCLR asserted for Source DMACBREQ request.

Table 5.1-77 Sequence of Destination DMACBREQ followed by Source DMACBREQ and completing with Source DMACLBREQ testing

Test No : DMA_P2P_SP_19

Objective : Assertion of multiple Source DMACSREQ and Destination DMACBREQ and Source DMACLSREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-60
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a single data transfer request, using source peripheral's DMACSREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a burst data transfer request, using destination peripheral's DMACBREQ signal.
- 8) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 9) Wait for a few clocks, and assert DMACBREQ signal of the destination peripheral.
- 10) Repeat steps 8) and 9) till the data transfer is completed by source peripheral.
- 11) Generate a last single data transfer request, using source peripheral's DMACLSREQ signal.
- 12) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 13) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_SP_20

Objective : Assertion of multiple Source DMACSREQ and Destination DMACBREQ and Source DMACLBREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-60
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a single data transfer request, using source DMACSREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a burst data transfer request, using destination peripheral's DMACBREQ signal.
- 8) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 9) Wait for a few clocks, and assert DMACBREQ signal of the destination peripheral.
- 10) Repeat steps 8) and 9) till the data transfer is completed by source peripheral.
- 11) Generate a last burst data transfer request, using source peripheral's DMACLBREQ signal.
- 12) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 13) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_SP_21

Objective : Assertion of multiple Source and Destination DMACBREQ and Source DMACLSREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-60
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a burst data transfer request, using source peripheral's DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a burst data transfer request, using destination peripheral's DMACBREQ signal.
- 8) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 9) Wait for a few clocks, and assert DMACBREQ of the destination peripheral.
- 10) Repeat steps 8) and 9) till the data transfer is completed by source peripheral.
- 11) Generate a last single data transfer request, using source DMACLSREQ signal.
- 12) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 13) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_SP_22

Objective : Assertion of multiple Source DMACBREQ and Destination DMACBREQ and Source DMACLBREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-60
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a burst data transfer request, using source peripheral's DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a burst data transfer request, using destination peripheral's DMACBREQ signal.
- 8) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 9) Wait for a few clocks, and assert DMACBREQ signal of the destination peripheral.
- 10) Repeat steps 8) and 9) till the data transfer is completed by source peripheral.
- 11) Generate a last burst data transfer request, using source peripheral's DMACLBREQ signal.
- 12) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 13) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_SP_23

Objective : Assertion of multiple Source DMACBREQ, Source DMACSREQ and Destination DMACBREQ and Source DMACLSREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-60
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a burst data transfer request, using source peripheral's DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a single data transfer request, using source peripheral's DMACSREQ signal.
- 8) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.
- 9) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 10) Repeat steps 8) and 9) 10 times.
- 11) Generate a burst data transfer request, using destination peripheral's DMACBREQ signal.
- 12) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 13) Wait for a few clocks, and assert DMACBREQ signal of the destination peripheral.
- 14) Repeat steps 12) and 13) till the data transfer is completed by source peripheral.
- 15) Generate a last single data transfer request, using source peripheral's DMACLSREQ signal.
- 16) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 17) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_SP_24

Objective : Assertion of multiple Source DMACBREQ, Source DMACSREQ and Destination DMACBREQ and Source DMACLBREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-60
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a burst data transfer request, using source peripheral's DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a single data transfer request, using source peripheral's DMACSREQ signal.
- 8) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.
- 9) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 10) Repeat steps 8) and 9) 10 times.
- 11) Generate a burst data transfer request, using destination peripheral's DMACBREQ signal.
- 12) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 13) Wait for a few clocks, and assert DMACBREQ signal of the destination peripheral.

- 14) Repeat steps 12) and 13) till the data transfer is completed by source peripheral.
- 15) Generate a last single data transfer request, using source peripheral's DMACLBREQ signal.
- 16) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 17) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_SP_25

Objective : Assertion of alternate Source DMACSREQ / DMACBREQ and multiple Destination DMACBREQ and Source DMACLBREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-60
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a single data transfer request, using source peripheral's DMACSREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a single data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.
- 6) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 7) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 8) Repeat steps 4) and 7) 10 times.
- 9) Generate a burst data transfer request, using destination peripheral's DMACBREQ signal.
- 10) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 11) Wait for a few clocks, and assert DMACBREQ signal of the destination peripheral.
- 12) Repeat steps 12) and 13) till the data transfer is completed by source peripheral.
- 13) Generate a last burst data transfer request, using source peripheral's DMACLBREQ signal.
- 14) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 15) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_SP_26

Objective : Assertion of alternate Source DMACSREQ / DMACBREQ and multiple Destination DMACBREQ and Source DMACLBREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-60
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a single data transfer request, using source peripheral's DMACSREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a single data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.
- 6) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 7) Wait for few clocks, and assert DMACSREQ of the source peripheral.

- 8) Repeat steps 4) and 7) 10 times.
- 9) Generate a burst data transfer request, using destination peripheral's DMACBREQ signal.
- 10) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 11) Wait for a few clocks, and assert DMACBREQ signal of the destination peripheral.
- 12) Repeat steps 12) and 13) till the data transfer is completed by source peripheral.
- 13) Generate a last burst data transfer request, using source peripheral's DMACLBREQ signal.
- 14) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 15) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Peripheral to Peripheral Transfer – Flow Controller Destination Peripheral:

The following tests will target a P2P DMA transfer with the respective destination peripheral as the flow controller.

Notes:

- The following table 5.1-78 defines the parameters of testing.

Test No	Src Width	Dest Width	Src Burst	Dest Burst
1.	8	8	16	1
2.	16	16	8	1
3.	32	32	4	1

Table 5.1-78 Parameters for P2P testing

- The sequence of request generation described in the following tests only indicates the timing of the request signal generation for the first time. For subsequent transfers, the request is initiated on sampling the assertion of DMACCLR for the previous request. This request generation continues till the terminal count interrupt (DMACTC) is sampled asserted for a burst transfer.
- Check that DMACCLR is asserted after a single data transfer for a DMACSREQ / DMACLSREQ assertion.
- Check that DMACCLR is asserted after a burst data transfer for a DMACBREQ / DMACLBREQ assertion

Test No : DMA_P2P_DP_1**Objective : Assertion of Destination DMACLSREQ followed by Source DMACSREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-79

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal	Assert source DMACSREQ signal
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLSREQ signal	Enable the channel	Assert source DMACSREQ signal
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLSREQ signal	Assert source DMACSREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACLSREQ	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal
5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLSREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel

6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLSREQ signal	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert destination DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal
8.	Assert destination DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel
9.	Assert destination DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert destination DMACLSREQ signal	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-79 Sequence of Source DMACLSREQ followed by Destination DMACBREQ testing

Test No : DMA_P2P_DP_2

Objective : Assertion of Simultaneous Destination DMACLSREQ and Source DMACSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-80

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACLSREQ and Source DMACSREQ signals simultaneously
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACLSREQ and Source DMACSREQ signals simultaneously	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACLSREQ and Source DMACSREQ signals simultaneously	Configure the channel for a P2P transfer	Enable the channel
4.	Assert Destination DMACLSREQ and Source DMACSREQ signals simultaneously	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-80 Sequence of Simultaneous Destination DMACLSREQ and Source DMACSREQ testing

Test No : DMA_P2P_DP_3

Objective : Assertion of Source DMACSREQ followed by Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-81

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal
5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert destination DMACLSREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal
8.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLSREQ signal	Enable the channel
9.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLSREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-81 Sequence of Source DMACSREQ followed by Destination DMACLSREQ testing

Test No : DMA_P2P_DP_4

Objective : Assertion of Destination DMACLBREQ followed by Source DMACSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-82

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal	Assert source DMACSREQ signal
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLBREQ signal	Enable the channel	Assert source DMACSREQ signal
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLBREQ signal	Assert source DMACSREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal
5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLBREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLBREQ signal	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert destination DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal
8.	Assert destination DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel
9.	Assert destination DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert destination DMACLBREQ signal	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-82 Sequence of Destination DMACLBREQ followed by Source DMACSREQ testing

Test No : DMA_P2P_DP_5

Objective : Assertion of Simultaneous Destination DMACLBREQ and Source DMACSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-83

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACLBREQ and Source DMACSREQ signals simultaneously
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACLBREQ and Source DMACSREQ signals simultaneously	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACLBREQ and Source DMACSREQ signals simultaneously	Configure the channel for a P2P transfer	Enable the channel
4.	Assert Destination DMACLBREQ and Source DMACSREQ signals simultaneously	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-83 Sequence of Simultaneous Destination DMACLBREQ and Source DMACSREQ testing

Test No : DMA_P2P_DP_6

Objective : Assertion of Source DMACSREQ followed by Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-85

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal

5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert destination DMACLBREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal
8.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLBREQ signal	Enable the channel
9.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-85 Sequence of Source DMACSREQ followed by Destination DMACLBREQ testing**Test No : DMA_P2P_DP_7****Objective : Assertion of Destination DMACSREQ followed by Source DMACSREQ and completing with Destination DMACLSREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-86

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert source DMACSREQ signal	Assert Destination DMACLSREQ on sampling DMACCLR asserted for Destination DMACSREQ request.

2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

7.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
8.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
9.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
10.	Assert destination DMACSREQ signal	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

Table 5.1-86 Sequence of Destination DMACSREQ followed by Source DMACSREQ and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_8

Objective : Assertion of Simultaneous assertion of Destination DMACSREQ and Source DMACSREQ, and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-87

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
----------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACSREQ and Source DMACSREQ signals simultaneously.	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACSREQ and Source DMACSREQ signals simultaneously.	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACSREQ and Source DMACSREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
4.	Assert Destination DMACSREQ and Source DMACSREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

Table 5.1-87 Sequence of Simultaneous assertion of Destination DMACSREQ and Source DMACSREQ, and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_9

Objective : Assertion of Source DMACSREQ followed by Destination DMACSREQ and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-88

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
----------	--------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
7.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
8.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
9.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
10.	Assert source DMACSREQ signal	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

Table 5.1-88 Sequence of Source DMACSREQ followed by Destination DMACSREQ and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_10

Objective : Assertion of Destination DMACBREQ followed by Source DMACSREQ and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-89

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

10.	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
-----	------------------------------------	-------------------------------	--	--	--------------------	--

Table 5.1-89 Sequence of Destination DMACBREQ followed by Source DMACSREQ and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_11

Objective : Assertion of Simultaneous assertion of Destination DMACBREQ and Source DMACSREQ, and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-90

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACBREQ and Source DMACSREQ signals simultaneously.	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACBREQ and Source DMACSREQ signals simultaneously.	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACBREQ and Source DMACSREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

4.	Assert Destination DMACBREQ and Source DMACSREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
----	---	--	--	--------------------	--

Table 5.1-90 Sequence of Simultaneous assertion of Destination DMACBREQ and Source DMACSREQ, and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_12

Objective : Assertion of Source DMACSREQ followed by Destination DMACBREQ and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-91

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
7.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
8.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

9.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
10.	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

Table 5.1-91 Sequence of Source DMACSREQ followed by Destination DMACBREQ and completing with Destination DMACLSREQ

Test No : DMA_P2P_DP_13

Objective : Assertion of Destination DMACSREQ followed by Source DMACSREQ and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-92

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
7.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

8.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
9.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
10.	Assert destination DMACSREQ signal	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

Table 5.1-92 Sequence of Destination DMACSREQ followed by Source DMACSREQ and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_14

Objective : Assertion of Simultaneous assertion of Destination DMACSREQ and Source DMACSREQ, and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-93

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACSREQ and Source DMACSREQ signals simultaneously.	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACSREQ and Source DMACSREQ signals simultaneously.	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACSREQ and Source DMACSREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
4.	Assert Destination DMACSREQ and Source DMACSREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

Table 5.1-93 Sequence of Simultaneous assertion of Destination DMACSREQ and Source DMACSREQ, and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_15

Objective : Assertion of Source DMACSREQ followed by Destination DMACSREQ and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-94

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

7.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
8.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
9.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
10.	Assert source DMACSREQ signal	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

Table 5.1-94 Sequence of Source DMACSREQ followed by Destination DMACSREQ and completing with Destination DMACLBREQ

Test No : DMA_P2P_DP_16

Objective : Assertion of Destination DMACLBREQ followed by Source DMACSREQ and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-95

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
----------	--------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
10.	Assert destination DMACBREQ signal	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

Table 5.1-95 Sequence of Destination DMACBREQ followed by Source DMACSREQ and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_17

Objective : Assertion of Simultaneous assertion of Destination DMACBREQ and Source DMACSREQ, and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-96

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACBREQ and Source DMACSREQ signals simultaneously.	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACBREQ and Source DMACSREQ signals simultaneously.	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACBREQ and Source DMACSREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
4.	Assert Destination DMACBREQ and Source DMACSREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

Table 5.1-96 Sequence of Simultaneous assertion of Destination DMACBREQ and Source DMACSREQ, and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_18

Objective : Assertion of Source DMACSREQ followed by Destination DMACBREQ and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-97

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
7.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
8.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
9.	Assert source DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
10.	Assert source DMACSREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

Table 5.1-97 Sequence of Source DMACSREQ followed by Destination DMACBREQ and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_19**Objective : Assertion of Destination DMACLSREQ followed by Source DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-98

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal	Assert source DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLSREQ signal	Enable the channel	Assert source DMACBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLSREQ signal	Assert source DMACBREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal
5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLSREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLSREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert destination DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal
8.	Assert destination DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel
9.	Assert destination DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert destination DMACLSREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-98 Sequence of Destination DMACLSREQ followed by Source DMACBREQ testing

Test No : DMA_P2P_DP_20**Objective : Assertion of Simultaneous Destination DMACLSREQ and Source DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-99

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACLSREQ and Source DMACBREQ signals simultaneously
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACLSREQ and Source DMACBREQ signals simultaneously	Enable the channel
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACLSREQ and Source DMACBREQ signals simultaneously	Configure the channel for a P2P transfer	Enable the channel
4.	Assert Destination DMACLSREQ and Source DMACBREQ signals simultaneously	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-99 Sequence of Simultaneous Destination DMACLSREQ and Source DMACBREQ testing

Test No : DMA_P2P_DP_21**Objective : Assertion of Source DMACBREQ followed by Destination DMACLSREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-100

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal

5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACLSREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal
8.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLSREQ signal	Enable the channel
9.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLSREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-100 Sequence of Source DMACBREQ followed by Destination DMACLSREQ testing

Test No : DMA_P2P_DP_22

Objective : Assertion of Destination DMACLBREQ followed by Source DMACBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-101

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal	Assert source DMACBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLBREQ signal	Enable the channel	Assert source DMACBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLBREQ signal	Assert source DMACBREQ signal	Enable the channel

4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal
5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLBREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLBREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert destination DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal
8.	Assert destination DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel
9.	Assert destination DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert destination DMACLBREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-101 Sequence of Destination DMACLBREQ followed by Source DMACBREQ testing**Test No : DMA_P2P_DP_23****Objective : Assertion of Simultaneous Destination DMACLBREQ and Source DMACBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-102

Test No.	Step 1	Step 2	Step 3	Step 4
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACLBREQ and Source DMACBREQ signals simultaneously
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACLBREQ and Source DMACBREQ signals simultaneously	Enable the channel

3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACLBREQ and Source DMACBREQ signals simultaneously	Configure the channel for a P2P transfer	Enable the channel
4.	Assert Destination DMACLBREQ and Source DMACBREQ signals simultaneously	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-102 Sequence of Simultaneous Destination DMACLBREQ and Source DMACBREQ**Test No : DMA_P2P_DP_24****Objective : Assertion of Source DMACBREQ followed by Destination DMACLBREQ**

The sequence of events are listed in the table from left to right in a row in table 5.1-103

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal	Enable the channel
4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal
5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACLBREQ signal	Enable the channel
6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal	Configure the channel for a P2P transfer	Enable the channel
7.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal

8.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACLBREQ signal	Enable the channel
9.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACLBREQ signal	Configure the channel for a P2P transfer	Enable the channel
10.	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel

Table 5.1-103 Sequence of Source DMACBREQ followed by Destination DMACLBREQ testing

Test No : DMA_P2P_DP_25

Objective : Assertion of Destination DMACSREQ followed by Source DMACBREQ and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-104

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
7.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

8.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
9.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
10.	Assert destination DMACSREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

Table 5.1-104 Sequence of Destination DMACSREQ followed by Source DMACBREQ and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_26

Objective : Assertion of Simultaneous assertion of Destination DMACSREQ and Source DMACBREQ, and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-105

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACSREQ and Source DMACBREQ signals simultaneously.	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACSREQ and Source DMACBREQ signals simultaneously.	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACSREQ and Source DMACBREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
4.	Assert Destination DMACSREQ and Source DMACBREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

Table 5.1-105 Sequence of Simultaneous assertion of Destination DMACSREQ and Source DMACBREQ, and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_27

Objective : Assertion of Source DMACBREQ followed by Destination DMACSREQ and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-106

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

7.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
8.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
9.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
10.	Assert source DMACBREQ signal	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

Table 5.1-106 Sequence of Source DMACBREQ followed by Destination DMACSREQ and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_28

Objective : Assertion of Destination DMACBREQ followed by Source DMACBREQ and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-107

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
----------	--------	--------	--------	--------	--------	--------

1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
10.	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

Table 5.1-107 Sequence of Destination DMACBREQ followed by Source DMACBREQ and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_29

Objective : Assertion of Simultaneous assertion of Destination DMACBREQ and Source DMACBREQ, and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-108

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACBREQ and Source DMACBREQ signals simultaneously.	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACBREQ and Source DMACBREQ signals simultaneously.	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACBREQ and Source DMACBREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
4.	Assert Destination DMACBREQ and Source DMACBREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

Table 5.1-108 Sequence of Simultaneous assertion of Destination DMACBREQ and Source DMACBREQ, and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_30

Objective : Assertion of Source DMACBREQ followed by Destination DMACBREQ and completing with Destination DMACLSREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-109

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
7.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
8.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
9.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
10.	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLSREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

Table 5.1-109 Sequence of Source DMACBREQ followed by Destination DMACBREQ and completing with Destination DMACLSREQ testing

Test No : DMA_P2P_DP_31

Objective : Assertion of Destination DMACSREQ followed by Source DMACBREQ and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-110

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
7.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
8.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
9.	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

10.	Assert destination DMACSREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
-----	------------------------------------	-------------------------------	--	--	--------------------	--

Table 5.1-110 Sequence of Destination DMACSREQ followed by Source DMACBREQ and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_32

Objective : Assertion of Simultaneous assertion of Destination DMACSREQ and Source DMACBREQ, and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-111

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACSREQ and Source DMACBREQ signals simultaneously.	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACSREQ and Source DMACBREQ signals simultaneously.	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACSREQ and Source DMACBREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

4.	Assert Destination DMACSREQ and Source DMACBREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
----	---	--	--	--------------------	--

Table 5.1-111 Sequence of Simultaneous assertion of Destination DMACSREQ and Source DMACBREQ, and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_33

Objective : Assertion of Source DMACBREQ followed by Destination DMACSREQ and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-112

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
7.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACSREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
8.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACSREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

9.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACSREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.
10.	Assert source DMACBREQ signal	Assert destination DMACSREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACSREQ request.

Table 5.1-112 Sequence of Source DMACBREQ followed by Destination DMACSREQ and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_34

Objective : Assertion of Destination DMACBREQ followed by Source DMACBREQ and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-113

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
7.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

8.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
9.	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
10.	Assert destination DMACBREQ signal	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

Table 5.1-113 Sequence of Destination DMACBREQ followed by Source DMACBREQ and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_35

Objective : Assertion of Simultaneous assertion of Destination DMACBREQ and Source DMACBREQ, and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-114

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert Destination DMACBREQ and Source DMACBREQ signals simultaneously.	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert Destination DMACBREQ and Source DMACBREQ signals simultaneously.	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Assert Destination DMACBREQ and Source DMACBREQ signals simultaneously.	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
4.	Assert Destination DMACBREQ and Source DMACBREQ signals simultaneously.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

Table 5.1-114 Sequence of Simultaneous assertion of Destination DMACBREQ and Source DMACBREQ, and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_36

Objective : Assertion of Source DMACBREQ followed by Destination DMACBREQ and completing with Destination DMACLBREQ

The sequence of events are listed in the table from left to right in a row in table 5.1-115

Test No.	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6
1.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

2.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
3.	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
4.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
5.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
6.	Setup the channel with a set of parameters in table 5.1-78	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

7.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACBREQ signal	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
8.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Assert destination DMACBREQ signal	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
9.	Assert source DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Assert destination DMACBREQ signal	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.
10.	Assert source DMACBREQ signal	Assert destination DMACBREQ signal	Setup the channel with a set of parameters in table 5.1-78	Configure the channel for a P2P transfer	Enable the channel	Assert destination DMACLBREQ signal on sampling DMACCLR asserted for Destination DMACBREQ request.

Table 5.1-115 Sequence of Source DMACBREQ followed by Destination DMACBREQ and completing with Destination DMACLBREQ testing

Test No : DMA_P2P_DP_37

Objective : Assertion of multiple Source DMACSREQ and Destination DMACSREQ and Destination DMACLSREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-78
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a single data transfer request, using source peripheral's DMACSREQ signal.

- 4) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a single data transfer request, using destination peripheral's DMACSREQ signal.
- 8) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 9) Wait for a few clocks, and assert DMACSREQ signal of the destination peripheral.
- 10) Repeat steps 8) and 9) till the data transfer is completed by source peripheral.
- 11) Generate a last single data transfer request, using destination peripheral's DMACSREQ signal.
- 12) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 13) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_DP_38

Objective : Assertion of multiple Source DMACSREQ and Destination DMACSREQ and Destination DMACLBREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-78
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a single data transfer request, using source peripheral's DMACSREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a single data transfer request, using destination peripheral's DMACSREQ signal.
- 8) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 9) Wait for a few clocks, and assert DMACSREQ signal of the destination peripheral.
- 10) Repeat steps 8) and 9) till the data transfer is completed by source peripheral.
- 11) Generate a last burst data transfer request, using destination peripheral's DMACLBREQ signal.
- 12) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 13) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_DP_39

Objective : Assertion of multiple Source and Destination DMACBREQ and Destination DMACLSREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-78
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a burst data transfer request, using source peripheral's DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.

- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a burst data transfer request, using destination peripheral's DMACBREQ signal.
- 8) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 9) Wait for a few clocks, and assert DMACBREQ signal of the destination peripheral.
- 10) Repeat steps 8) and 9) till the data transfer is completed by source peripheral.
- 11) Generate a last single data transfer request, using destination peripheral's DMACLSREQ signal.
- 12) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 13) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_DP_40

Objective : Assertion of multiple Source DMACBREQ and Destination DMACBREQ and Destination DMACLBREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-78
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a burst data transfer request, using source peripheral's DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a burst data transfer request, using destination peripheral's DMACBREQ signal.
- 8) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer.
- 9) Wait for a few clocks, and assert DMACBREQ signal of the destination peripheral.
- 10) Repeat steps 8) and 9) till the data transfer is completed by source peripheral.
- 11) Generate a last burst data transfer request, using destination peripheral's DMACLBREQ signal.
- 12) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 13) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_DP_41

Objective : Assertion of multiple Source DMACBREQ, Source DMACSREQ and Destination DMACSREQ, Destination DMACBREQ and Destination DMACLSREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-78
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a burst data transfer request, using source peripheral's DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a single data transfer request, using source peripheral's DMACSREQ signal.

- 8) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.
- 9) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 10) Repeat steps 8) and 9) 10 times.
- 11) Generate a burst data transfer request, using destination peripheral's DMACSREQ signal.
- 12) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer of the source width.
- 13) Wait for few clocks, and assert DMACSREQ signal of the destination peripheral.
- 14) Repeat steps 4) and 5) 10 times.
- 15) Generate a single data transfer request, using destination peripheral's DMACBREQ signal.
- 16) Check that DMACCLR, of the destination peripheral is asserted after one data transfer of the source width.
- 17) Wait for few clocks, and assert DMACBREQ signal of the destination peripheral.
- 18) Repeat steps 8) and 9) 10 times.
- 19) Generate a last single data transfer request, using destination peripheral's DMACLSREQ signal.
- 20) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 21) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_DP_42

Objective : Assertion of multiple Source DMACBREQ, Source DMACSREQ and Destination DMACSREQ, Destination DMACBREQ and Destination DMACLBREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-78
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a burst data transfer request, using source peripheral's DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.
- 6) Repeat steps 4) and 5) 10 times.
- 7) Generate a single data transfer request, using source peripheral's DMACSREQ signal.
- 8) Check that DMACCLR, of the source peripheral is asserted after one data transfer of the source width.
- 9) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 10) Repeat steps 8) and 9) 10 times.
- 11) Generate a burst data transfer request, using destination peripheral's DMACSREQ signal.
- 12) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer of the source width.
- 13) Wait for few clocks, and assert DMACSREQ signal of the destination peripheral.
- 14) Repeat steps 4) and 5) 10 times.
- 15) Generate a single data transfer request, using destination peripheral's DMACBREQ signal.
- 16) Check that DMACCLR, of the destination peripheral is asserted after one data transfer of the source width.
- 17) Wait for few clocks, and assert DMACBREQ signal of the destination peripheral.

- 18) Repeat steps 8) and 9) 10 times.
- 19) Generate a last single data transfer request, using destination peripheral's DMACLBREQ signal.
- 20) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 21) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_DP_43

Objective : Assertion of alternate Source DMACSREQ / DMACBREQ, Destination DMACSREQ / DMACBREQ and Destination DMACLSREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-78
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a single data transfer request, using source peripheral's DMACSREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a single data transfer of the source width.
- 5) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.
- 6) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 7) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 8) Repeat steps 4) and 7) 10 times.
- 9) Generate a single data transfer request, using destination peripheral's DMACSREQ signal.
- 10) Check that DMACCLR, of the destination peripheral is asserted after a single data transfer of the source width.
- 11) Wait for few clocks, and assert DMACBREQ signal of the destination peripheral.
- 12) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer of the source width.
- 13) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 14) Repeat steps 10) and 13) 10 times.
- 15) Generate a last burst data transfer request, using destination peripheral's DMACLSREQ signal.
- 16) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 17) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

Test No : DMA_P2P_DP_44

Objective : Assertion of alternate Source DMACBREQ / DMACSREQ, Destination DMACBREQ / DMACSREQ and Destination DMACLBREQ to complete the DMA transfer

- 1) Setup the channel with a set of parameters in table 5.1-78
- 2) Configure and enable the channel for a P2P transaction.
- 3) Generate a burst data transfer request, using source peripheral's DMACBREQ signal.
- 4) Check that DMACCLR, of the source peripheral is asserted after a burst data transfer of the source width.
- 5) Wait for few clocks, and assert DMACSREQ signal of the source peripheral.
- 6) Check that DMACCLR, of the source peripheral is asserted after a single data transfer of the source width.
- 7) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.

- 8) Repeat steps 4) and 7) 10 times.
- 9) Generate a single data transfer request, using destination peripheral's DMACBREQ signal.
- 10) Check that DMACCLR, of the destination peripheral is asserted after a burst data transfer of the source width.
- 11) Wait for few clocks, and assert DMACSREQ signal of the destination peripheral.
- 12) Check that DMACCLR, of the destination peripheral is asserted after a single data transfer of the source width.
- 13) Wait for few clocks, and assert DMACBREQ signal of the source peripheral.
- 14) Repeat steps 10) and 13) 10 times.
- 15) Generate a last burst data transfer request, using destination peripheral's DMACLBREQ signal.
- 16) Check that DMACCLR and DMACTC of the source peripheral are asserted at the end of the transfer.
- 17) Check that DMACCLR and DMACTC of the destination peripheral are asserted at the end of the transfer.

6.5.5 DMAC DISABLE Test

Test No : DMAC_DISABLE_1 - DMAC_DISABLE_39

- 1) Setup all the channels of the DMAC as in table 5.1-116 below
- 2) Check that the DMAC is active on both the AHB buses.
- 3) Disable the DMAC by writing zero into bit 0 of the DMAConfiguration register.
- 4) Check that DMAC completes the current AHB transfer as per AHB protocol.
- 5) Check that all the channel enable bits are disabled in the respective DMACxConfiguration registers.
- 6) Check that there are no further AHB accesses initiated by the DMAC and it is insensitive to any peripheral's request.

Test No	Channel No	Transfer Type	Flow Controller	Src Width	Dest Width	Src Inc	Dest Inc	Src Burst	Dest Burst	Transfer Size
1.	0	M2M	D	8	8	NI	NI	16	1	100
2.	1	P2M	D	16	16	NI	I	8	1	75
9.	0	M2M	D	8	8	NI	NI	16	16	100
10.	1	M2M	D	8	8	NI	NI	16	1	100
17.	0	M2M	D	8	8	NI	NI	16	1	100
18.	1	M2M	D	8	8	NI	NI	16	1	100
25.	1	M2M	D	8	8	NI	NI	16	16	100

32	0	M2M	D	8	8	NI	NI	16	1	100
33.	1	M2M	D	8	8	NI	NI	16	1	100

Table 5.1-116 DMAC Halt test parameters.

6.5.6 DMAC HALT Test

Test No : DMAC_HALT_1 - DMAC_HALT_8

1. Setup all the channels of the DMAC as in table 5.1-117 below
2. Check that the DMAC is active on both the AHB buses.
3. HALT the DMAC by writing one into Halt bit of the DMAConfiguration register.
4. Check that DMAC completes the current AHB transfer as per AHB protocol.
5. Clear the halt bit by writing zero into halt bit of the DMAConfiguration register.
6. Check that DMAC completes the remaining AHB transfer.

Test No	Channel No	Transfer Type	Flow Controller	Src Width	Dest Width	Src Inc	Dest Inc	Src Burst	Dest Burst	Transfer Size
1.	0	M2M	D	8	8	NI	NI	16	1	100
2.	1	P2M	D	16	16	NI	I	8	1	75

Table 5.1-117 DMAC Halt test parameters.

6.5.7 LLI Tests

The parameters for the LLI tests are indicated in table 5.1-117a. These parameters will be programmed into a memory module and will be repeatedly used by the DMAC for LLI transactions.

- The transfer size, protection bits, lock bit and the next LLI value will be generated as a random number during testing and will be programmed into the memory.

Test No : DMAC_LLI_1 - DMAC_LLI_64

Objective : M2M transactions with LLI.

- 1) Setup channel0 for an M2M transaction, with the parameters as specified in the first row of table 5.1-117a and enable Channel0.
- 2) Check that the LLIs are loaded after the every DMA transfer.
- 3) Wait till the LLIs are completed and the channel is disabled.
- 4) Enable the DMACxLLI register with the address of the next LLI location and enable the channels.
- 5) Repeat steps 2) to 4) till the given set of parameters are completed.

6) Repeat steps 1) to 5) for channel 1.

Test No	Src Width	Dest Width	Src Inc	Dest Inc	Src Burst	Dest Burst
1.	8	8	NI	NI	1	1
5.	16	32	I	NI	1	32
10.	32	32	I	I	4	4
23.	16	16	I	NI	8	128
39.	16	16	I	NI	32	128
49.	8	8	NI	NI	128	1
53.	16	32	I	NI	128	32
58.	32	32	I	I	256	4

Table 5.1-117a DMAC LLI Test parameters

6.5.8 MultiChannel Tests

The main objective of multi-channel tests is to check the implementation of channel priority of DMA Controller. Following tests are targeted to test the implementation of channel priority of DMA controller. The main parameters considered are AHB response, number of LLIs and assertion of HGRANTx. AHB master is programmed for both little or big endian mode. Source and destination peripherals may be in incrementing or non incrementing mode.

The following steps which needs to be followed to test the implementation of channel priority.

Two Channel Test Cases :-

Test No: - DMA_2Ch_1

1. Set up the channel 0 and 1 as given in table no 5.1-118.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Generate a source burst transfer request of channel 0.
5. Generate a destination burst transfer request of channel 0 and 1.
6. Check that channel 1 performs source data transfer.
7. Check that channel 0 performs source data transfer.
8. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
9. Check that channel 1 performs destination data transfer.

10. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
11. Check that channel 0 performs destination data transfer.
12. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	DMAC	32	32	4	4	1	0	3	9	OK
CH1	M2P	DMAC	32	32	4	4	1	0	NA	1	OK

Table 5.1-118 Test parameters for DMA_2Ch_1

Test No :- DMA_2Ch_2

1. Set up the channel 0 and 1 as given in table no 5.1-119.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Generate a source last burst transfer request of channel 0.
5. Generate a destination burst transfer request of channel 0.
6. Check that channel 1 performs source data transfer.
7. Check that channel 0 performs source data transfer.
8. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
9. Check that Channel 0 performs destination data transfer.
10. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
11. Check that Channel 4 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	Src. Periph	32	32	4	4	NA	0	2	8	OK
CH1	M2M	DMAC	32	32	4	4	4	0	NA	NA	OK

Table 5.1-119 Test parameters for DMA_2Ch_2

Test No:- DMA_2Ch_3

1. Set up the channel 0 and 1 as given in table no 5.1-120.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure source peripheral of channel 0 such that it should give two split responses for second data transfer.
4. Enable channel 0 first and then channel 1.
5. Generate a destination last burst transfer request of channel 1.
6. Generate a source last burst transfer request of channel 0.
7. Check that channel 1 performs source data transfer.
8. Check that channel 0 performs source data transfer.
9. Check that for second data transfer, source peripheral of channel 0 gives two split responses and also verify that lower priority channel should not start data transfer.
10. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
11. Check that channel 0 performs destination data transfer.
12. Check that channel 1 performs destination data transfer.
13. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2M	Src. Periph	16	16	4	4	NA	0	2	NA	Split response for 2 nd data transfer
CH1	M2P	Dest. Periph	32	32	4	4	NA	0	NA	8	OK

Table 5.1-120 Test parameters for DMA_2Ch_3

Test No:- DMA_2Ch_5

1. Set up the channel 0 and 1 as given in table no 5.1-121.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure source peripheral of channel 0 such that it should give four split responses for second data transfer.
4. Configure destination peripheral of channel 1 such that it should give two retry responses for second data transfer.
5. Enable channel 0 first and then channel 1.
6. Generate a source burst and destination last burst transfer request of channel 1.
7. Generate a source last burst and destination burst transfer request of channel 0.
8. Check that channel 1 should start source data transfer.

9. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
10. Check that channel 0 performs source data transfer.
11. Check that for second data transfer, source peripheral of channel 0 gives four split responses.
12. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
13. Check that channel 0 performs destination data transfer.
14. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
15. Check that channel 1 performs destination data transfer.
16. Check that for second data transfer, destination peripheral of channel 1 gives two retry responses.
17. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	Src. Periph	32	32	4	4	NA	0	2	8	Split response for 2 nd data transfer
CH1	P2P	Dest. Periph	32	32	4	4	NA	0	10	15	Retry response for 2 nd data transfer

Table 5.1-121 Test parameters for DMA_2Ch_5**Test No :- DMA_2Ch_6**

1. Set up the channel 0 and 1 as given in table no 5.1-122.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Generate a source burst transfer request of channel 0.
5. Generate a destination burst transfer request of channel 0 and 1.
6. Check that channel 1 performs source data transfer.
7. Check that channel 0 performs source data transfer.
8. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
9. Check that channel 1 performs destination data transfer.
10. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer
11. Check that channel 0 performs destination data transfer.

12. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	DMAC	16	16	4	4	1	0	4	5	OK
CH1	M2P	DMAC	8	8	4	4	1	0	NA	6	OK

Table 5.1-122 Test parameters for DMA_2Ch_6

Test No:- DMA_2Ch_9

- Set up the channel 0 and 1 as given in table no 5.1-123.
- Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
- Configure source peripheral of channel 0 such that it should give two split responses for second data transfer.
- Enable channel 0 first and then channel 1.
- Generate a destination last burst transfer request of channel 1.
- Generate a source last burst transfer request of channel 0.
- Check that channel 1 performs source data transfer.
- Check that channel 0 performs source data transfer.
- Check that for second data transfer, source peripheral of channel 0 gives two split responses and also verify that lower priority channel should not start data transfer.
- Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
- Check that Channel 0 performs destination data transfer.
- Check that Channel 1 performs destination data transfer.
- Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2M	Src. Periph	16	16	4	4	NA	0	2	NA	Split response for

											2 nd data transfer
CH1	M2P	Dest. Periph	32	32	4	4	NA	0	NA	7	OK

Table 5.1-123 Test parameters for DMA_2Ch_9**Test No :- DMA_2Ch_11**

1. Set up the channel 0 and 1 as given in table no 5.1-124.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Generate a destination burst transfer request of channel 1.
5. Generate a source burst and destination last burst transfer request of channel 0.
6. Check that channel 1 performs source data transfer.
7. Check that channel 0 performs source data transfer.
8. Check that DMACCLR is asserted for source peripheral of channel 0 after the end of burst transfer.
9. Check that channel 0 performs destination data transfer.
10. Generate a source burst transfer request of channel 0.
11. Check that DMACCLR is asserted for destination peripheral of channel 0 after the end of burst transfer.
12. Check that channel 1 performs destination data transfer.
13. Generate a destination last burst transfer request of channel 0.
14. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
15. Check that channel 0 performs source data transfer.
16. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of burst transfer.
17. Check that channel 0 performs destination data transfer.
18. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of burst transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	Dest. Periph	32	32	4	4	NA	0	13	5	OK
CH1	M2P	DMAC	32	32	4	4	4	0	NA	1	OK

Table 5.1-124 Test parameters for DMA_2Ch_11**Test No :- DMA_2Ch_14**

1. Set up the channel 0 and 1 as given in table no 5.1-125.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure source memory of channel 0 such that it should give 2 retry responses when last two bits of source address are 00.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 should start source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that when last two bits of source address are 00, source memory of channel 0 gives two retry responses.
8. Check that channel 0 performs destination data transfer.
9. Check that channel 1 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	32	32	4	4	4	0	NA	NA	Retry response when last 2 bits of address are 00.
CH1	M2M	DMAC	32	32	4	4	4	0	NA	NA	OK

Table 5.1-125 Test parameters for DMA_2Ch_14

Test No :- DMA_2Ch_16

1. Set up the channel 0 and 1 as given in table no 5.1-126.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 1 first and then channel 0.
4. Generate a destination last burst transfer request of channel 1.
5. Generate a source last burst and destination burst transfer request of channel 0.
6. Check that channel 1 performs source data transfer.
7. Check that channel 0 performs source data transfer.
8. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
9. Check that channel 0 performs destination data transfer.
10. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
11. Check that channel 1 performs destination data transfer.
12. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
13. Check that channel 0 LLI gets loaded.

14. Generate a source last burst and destination burst transfer request of channel 0.
15. Check that channel 1 LLI gets loaded.
16. Check that channel 0 performs source data transfer.
17. Generate a destination last burst transfer request of channel 1.
18. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
19. Check that Channel 0 performs destination data transfer.
20. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
21. Check that channel 1 performs source data transfer.
22. Check that channel 1 performs destination data transfer.
23. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	Src. Periph	32	32	4	4	NA	1	4	8	OK
CH1	M2P	Dest. Periph	32	32	4	4	NA	1	NA	12	OK

Table 5.1-126 Test parameters for DMA_2Ch_16

Test No :- DMA_2Ch_17

1. Set up the channel 0 and 1 as given in table no 5.1-127.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 1 first and then channel 0.
4. Generate a source last burst transfer request of channel 1.
5. Generate a source last burst and destination burst transfer request of channel 0.
6. Check that channel 1 performs source data transfer.
7. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
8. Check that channel 0 performs source data transfer.
9. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
10. Check that channel 0 performs destination data transfer.
11. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
12. Check that channel 1 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	Src. Periph	32	32	8	8	NA	0	4	8	OK
CH1	P2M	Src. Periph	32	32	8	8	NA	0	12	NA	OK

Table 5.1-127 Test parameters for DMA_2Ch_17**Test No :- DMA_2Ch_19**

1. Set up the channel 0 and 1 as given in table no 5.1-128.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure source peripheral of channel 0 such that it gives error response for second data transfer.
4. Enable channel 1 first and then channel 0.
5. Generate a source last burst and destination burst transfer request of channel 1.
6. Generate a source and destination burst transfer request of channel 0.
7. Check that channel 1 should start source data transfer.
8. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of burst transfer.
9. Check that channel 0 performs source data transfer.
10. Check that for 2 data transfer, source peripheral of channel 0 gives error response.
11. Check that channel 0 get disabled and whole transfer is aborted.
12. Check that channel 1 performs destination data transfer.
13. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	DMAC	32	32	4	4	4	0	12	8	Error response for 2 nd data transfer.
CH1	P2P	Src. Periph	32	32	4	4	NA	0	1	2	OK

Table 5.1-128 Test parameters for DMA_2Ch_19**Test No :- DMA_2Ch_20**

1. Set up the channel 0 and 1 as given in table no 5.1-129.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Generate a source last burst transfer request of channel 0.
5. Generate a destination burst transfer request of channel 0.
6. Check that channel 1 performs source data transfer.
7. Check that channel 0 performs source data transfer.
8. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
9. Check that channel 0 performs destination data transfer.
10. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
11. Check that channel 1 performs destination data transfer.
12. Generate a source last burst transfer request of channel 0.
13. Generate a destination burst transfer request channel 0.
14. Check that channel 0 LLI gets loaded.
15. Check that Channel 1 LLI gets loaded.
16. Check that channel 0 performs source data transfer.
17. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
18. Check that channel 0 performs destination data transfer.
19. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
20. Check that channel 1 performs source data transfer.
21. Check that channel 1 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	DMAC	32	32	4	4	8	0	12	8	OK
CH1	M2M	DMAC	32	32	4	4	4	0	NA	NA	OK

Table 5.1-129 Test parameters for DMA_2Ch_20

Test No :- DMA_2Ch_22

1. Set up the channel 0 and 1 as given in table no 5.1-130.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 1 first and then channel 0.
4. Generate a source last burst transfer request of channel 1.
5. Generate a source burst transfer request of channel 0.
6. Check that channel 1 performs source data transfer.
7. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
8. Check that channel 0 performs source data transfer.
9. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
10. Check that channel 0 performs destination data transfer.
11. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
12. Check that channel 1 performs destination data transfer.
13. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
14. Generate source burst transfer request of channel 0.
15. Generate source last burst transfer request of channel 1.
16. Check that channel 0 LLI gets loaded.
17. Check that Channel 1 LLI gets loaded.
18. Check that channel 0 performs source data transfer.
19. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
20. Check that channel 0 performs destination data transfer.
21. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
22. Check that channel 1 performs source data transfer.
23. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
24. Check that channel 1 performs destination data transfer.
25. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
---------	---------------	-----------------	-----------	------------	-----------	------------	---------------	-----	-------------	--------------	--------------

CH0	P2M	DMAC	32	32	4	4	4	1	1	NA	OK
CH1	P2M	Src. Periph	32	32	4	4	NA	1	3	NA	OK

Table 5.1-130 Test parameters for DMA_2Ch_22**Test No :- DMA_2Ch_24**

1. Set up the channel 0 and 1 as given in table no 5.1-131.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives error response for 4th data transfer.
4. Enable channel 1 first and then channel 0.
5. Generate a source last burst transfer request of channel 1.
6. Generate a source burst and destination last burst transfer request of channel 0.
7. Check that channel 1 performs source data transfer.
8. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
9. Check that channel 0 performs source data transfer.
10. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
11. Check that channel 0 performs destination data transfer.
12. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
13. Check that channel 1 performs destination data transfer.
14. Check that for fourth data transfer, destination memory gives error response.
15. Generate a destination burst transfer request of channel 0.
16. Check that channel 0 LLI gets loaded.
17. Check that channel 0 performs source data transfer.
18. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
19. Check that channel 0 performs destination data transfer.
20. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	Dest. Periph	32	32	4	4	NA	1	1	2	OK
CH1	P2M	Src. Periph	32	32	4	4	NA	1	3	NA	Error Response for 4 th data transfer

Table 5.1-131 Test parameters for DMA_2Ch_24**Test No :- DMA_2Ch_26**

1. Set up the channel 0 and 1 as given in table no 5.1-132.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives waited error response when the last 16 bits of destination address is 0x3800.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that when the last 16 bits of destination address are 0x3800.for fourth data transfer, destination memory gives waited error response.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 1 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	8	8	8	1	NA	NA	OK
CH1	M2M	DMAC	8	8	1	1	1	0	NA	NA	Waited Error response when last 16 bits of address are 0x3800.

Table 5.1-132 Test parameters for DMA_2Ch_26**Test No :- DMA_2Ch_27**

1. Set up the channel 0 and 1 as given in table no 5.1-133.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Check that channel 1 performs source data transfer.
5. Check that channel 0 performs source data transfer.
6. Check that channel 0 aborts DMA transfer.
7. Disable channel 0 by writing zero in channel enable bit of channel 0.
8. Check that channel 1 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	16	16	32	0	NA	NA	OK
CH1	M2M	DMAC	8	8	16	16	32	0	NA	NA	OK

Table 5.1-133 Test parameters for DMA_2Ch_27**Test No :- DMA_2Ch_28**

1. Set up the channel 0 and 1 as given in table no 5.1-134.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure channel 1 source and destination memory such that it asserts three wait cycles for each data transfer.
4. Enable channel 0 first and then channel 1.
5. Check that channel 1 performs source data transfer.
6. Check that channel 1 source memory asserts three wait cycles for each data transfer.
7. Check that channel 0 performs source data transfer.
8. Disable channel 0 by writing zero in channel enable bit of channel 0.
9. Check that channel 0 aborts DMA transfer.
10. Check that channel 1 performs destination data transfer.
11. Check that channel 1 destination memory asserts three wait cycles for each data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	16	16	32	0	NA	NA	OK
CH1	M2M	DMAC	8	8	16	16	32	0	NA	NA	OK

Table 5.1-134 Test parameters for DMA_2Ch_28**Test No :- DMA_2Ch_29**

1. Set up the channel 0 and 1 as given in table no 5.1-135.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure channel 0 and channel 1 source and destination memory such that it asserts one wait cycle for each data transfer.
4. Enable channel 0 first and then channel 1.

5. Check that channel 1 performs source data transfer.
6. Check that channel 1 source memory asserts one wait cycle for each data transfer.
7. Check that channel 0 performs source data transfer.
8. Check that channel 0 source memory asserts one wait cycles for each data transfer.
9. Disable channel 0 by writing zero in channel enable bit of channel 0.
10. Check that channel 0 aborts DMA transfer.
11. Check that channel 1 performs destination data transfer.
12. Check that channel 1 destination memory asserts one wait cycle for each data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	16	16	32	0	NA	NA	OK
CH1	M2M	DMAC	8	8	16	16	32	0	NA	NA	OK

Table 5.1-135 Test parameters for DMA_2Ch_29

Test No :- DMA_2Ch_30

1. Set up the channel 1 and 0 as given in table no 5.1-136.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Generate a destination last burst transfer request of channel 0.
5. Generate a source last burst and destination burst transfer request of channel 1.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
9. Generate a destination last burst transfer request of channel 0.
10. Check that channel 1 performs source data transfer.
11. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
12. Check that channel 0 LLI gets loaded.
13. Check that channel 1 performs destination data transfer.
14. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
15. Generate a source last burst and destination burst transfer request of channel 1.

16. Check that channel 0 performs source data transfer.
17. Check that channel 0 performs destination data transfer.
18. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
19. Check that channel 1 LLI gets loaded.
20. Check that channel 1 performs source data transfer.
21. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
22. Check that Channel 1 performs destination data transfer.
23. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH1	P2P	Src. Periph	32	32	4	4	NA	1	4	8	OK
CH0	M2P	Dest. Periph	32	32	4	4	NA	1	NA	12	OK

Table 5.1-136 Test parameters for DMA_2Ch_30

Test No :- DMA_2Ch_31

1. Set up the channel 1 and 0 as given in table no 5.1-137.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Generate a destination last burst transfer request of channel 0.
5. Generate a source last burst and destination burst transfer request of channel 1.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
9. Generate a destination last burst transfer request of channel 0.
10. Check that channel 1 performs source data transfer.
11. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
12. Check that channel 0 LLI gets loaded.
13. Check that channel 1 performs destination data transfer.
14. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

15. Generate a source last burst and destination burst transfer request of channel 1.
16. Check that channel 0 performs source data transfer.
17. Check that channel 0 performs destination data transfer.
18. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
19. Check that channel 1 LLI gets loaded.
20. Check that channel 1 performs source data transfer.
21. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
22. Check that Channel 1 performs destination data transfer.
23. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH1	P2P	Src. Periph	32	32	4	4	NA	1	4	8	OK
CH0	M2P	Dest. Periph	32	32	4	4	NA	1	NA	12	OK

Table 5.1-137 Test parameters for DMA_2Ch_31

Test No :- DMA_2Ch_32

1. Set up the channel 1 and 0 as given in table no 5.1-138.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Generate a destination last burst transfer request of channel 0.
5. Generate a source last burst and destination burst transfer request of channel 1.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
9. Generate a destination last burst transfer request of channel 0.
10. Check that channel 1 performs source data transfer.
11. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
12. Check that channel 0 LLI gets loaded.

13. Check that channel 1 performs destination data transfer.
14. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
15. Generate a source last burst and destination burst transfer request of channel 1.
16. Check that channel 0 performs source data transfer.
17. Check that channel 0 performs destination data transfer.
18. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
19. Check that channel 1 LLI gets loaded.
20. Check that channel 1 performs source data transfer.
21. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
22. Check that Channel 1 performs destination data transfer.
23. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH1	P2P	Src. Periph	32	32	4	4	NA	1	4	8	OK
CH0	M2P	Dest. Periph	32	32	4	4	NA	1	NA	12	OK

Table 5.1-138 Test parameters for DMA_2Ch_32

Test No :- DMA_2Ch_33

1. Set up the channel 1 and 0 as given in table no 5.1-139.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Generate a destination last burst transfer request of channel 0.
5. Generate a source last burst and destination burst transfer request of channel 1.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
9. Generate a destination last burst transfer request of channel 0.
10. Check that channel 1 performs source data transfer.

11. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
12. Check that channel 0 LLI gets loaded.
13. Check that channel 1 performs destination data transfer.
14. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
15. Generate a source last burst and destination burst transfer request of channel 1.
16. Check that channel 0 performs source data transfer.
17. Check that channel 0 performs destination data transfer.
18. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
19. Check that channel 1 LLI gets loaded.
20. Check that channel 1 performs source data transfer.
21. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
22. Check that Channel 1 performs destination data transfer.
23. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH1	P2P	Src. Periph	32	32	4	4	NA	1	4	8	OK
CH0	M2P	Dest. Periph	32	32	4	4	NA	1	NA	12	OK

Table 5.1-139 Test parameters for DMA_2Ch_33

Test No :- DMA_2Ch_34

1. Set up the channel 1 and 0 as given in table no 5.1-140.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 0 first and then channel 1.
4. Generate a destination last burst transfer request of channel 0 .
5. Generate a source last burst and destination burst transfer request of channel 1.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
9. Generate a destination last burst transfer request of channel 0.

10. Check that channel 1 performs source data transfer.
11. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
12. Check that channel 0 LLI gets loaded.
13. Check that channel 1 performs destination data transfer.
14. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
15. Generate a source last burst and destination burst transfer request of channel 1.
16. Check that channel 0 performs source data transfer.
17. Check that channel 0 performs destination data transfer.
18. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
19. Check that channel 1 LLI gets loaded.
20. Check that channel 1 performs source data transfer.
21. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 1 after the end of DMA transfer.
22. Check that Channel 1 performs destination data transfer.
23. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH1	P2P	Src. Periph	32	32	4	4	NA	1	4	8	OK
CH0	M2P	Dest. Periph	32	32	4	4	NA	1	NA	12	OK

Table 5.1-140 Test parameters for DMA_2Ch_34

Test No :- DMA_2Ch_35

1. Set up the channel 0 and 1 as given in table no 5.1-141.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 1 first and then channel 0.
4. Generate a destination last burst transfer request of channel 1.
5. Generate a source last burst and destination burst transfer request of channel 0.
6. Check that channel 1 performs source data transfer.
7. Check that channel 4 performs source data transfer.
8. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.

9. Check that channel 0 performs destination data transfer.
10. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
11. Generate a source last burst and destination burst transfer request of channel 0.
12. Check that channel 1 performs destination data transfer.
13. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
14. Generate a destination last burst transfer request of channel 1.
15. Check that channel 0 LLI gets loaded.
16. Check that channel 1 LLI gets loaded.
17. Check that channel 0 performs source data transfer.
18. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
19. Check that Channel 0 performs destination data transfer.
20. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
21. Check that channel 1 performs source data transfer.
22. Check that channel 1 performs destination data transfer.
23. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	Src. Periph	32	32	4	4	NA	1	4	8	OK
CH1	M2P	Dest. Periph	32	32	4	4	NA	1	NA	12	OK

Table 5.1-141 Test parameters for DMA_2Ch_35

Test No :- DMA_2Ch_36

1. Set up the channel 0 and 1 as given in table no 5.1-142.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 1 first and then channel 0.
4. Generate a destination last burst transfer request of channel 1.
5. Generate a source last burst and destination burst transfer request of channel 0.
6. Check that channel 1 performs source data transfer.
7. Check that channel 0 performs source data transfer.

8. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
9. Check that channel 0 performs destination data transfer.
10. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
11. Generate a source last burst and destination burst transfer request of channel 0.
12. Check that channel 1 performs destination data transfer.
13. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
14. Generate a destination last burst transfer request of channel 1.
15. Check that channel 0 LLI gets loaded.
16. Check that channel 1 LLI gets loaded.
17. Check that channel 0 performs source data transfer.
18. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
19. Check that Channel 0 performs destination data transfer.
20. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
21. Check that channel 1 performs source data transfer.
22. Check that channel 1 performs destination data transfer.
23. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	Src. Periph	32	32	4	4	NA	1	4	8	OK
CH1	M2P	Dest. Periph	32	32	4	4	NA	1	NA	12	OK

Table 5.1-142 Test parameters for DMA_2Ch_36

Test No :- DMA_2Ch_37

1. Set up the channel 0 and 1 as given in table no 5.1-143.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Enable channel 1 first and then channel 0.
4. Generate a destination last burst transfer request of channel 1.
5. Generate a source last burst and destination burst transfer request of channel 0.
6. Check that channel 1 performs source data transfer.

7. Check that channel 0 performs source data transfer.
8. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
9. Check that channel 0 performs destination data transfer.
10. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
11. Generate a source last burst and destination burst transfer request of channel 0.
12. Check that channel 1 performs destination data transfer.
13. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.
14. Generate a destination last burst transfer request of channel 1.
15. Check that channel 0 LLI gets loaded.
16. Check that channel 1 LLI gets loaded.
17. Check that channel 0 performs source data transfer.
18. Check that DMACCLR and DMACTC are asserted for source peripheral of channel 0 after the end of DMA transfer.
19. Check that Channel 0 performs destination data transfer.
20. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 0 after the end of DMA transfer.
21. Check that channel 1 performs source data transfer.
22. Check that channel 1 performs destination data transfer.
23. Check that DMACCLR and DMACTC are asserted for destination peripheral of channel 1 after the end of DMA transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	P2P	Src. Periph	32	32	4	4	NA	1	4	8	OK
CH1	M2P	Dest. Periph	32	32	4	4	NA	1	NA	12	OK

Table 5.1-143 Test parameters for DMA_2Ch_37

Test No :- DMA_2Ch_38

1. Set up the channel 0 and 1 as given in table no 5.1-144.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.

3. Configure destination memory of channel 1 such that it gives error response when last 16 bits of destination address are 0x440F.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory of channel 1 give error response when last 16 bits of destination address are 0x440F.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	16	16	16	1	NA	NA	OK
CH1	M2M	DMAC	8	8	16	16	16	0	NA	NA	Error Response when last 16 bits of address are 0x440F

Table 5.1-144 Test parameters for DMA_2Ch_38

Test No :- DMA_2Ch_39

1. Set up the channel 0 and 1 as given in table no 5.1-145.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 2 such that it gives waited error response when last 16 bits of destination address are 0xA700.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory of channel 1 give error response when last 16 bits of destination address are 0xA700.
10. Check that channel 0 LLI gets loaded.

11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	8	8	8	1	NA	NA	OK
CH1	M2M	DMAC	8	8	1	1	1	0	NA	NA	Error Response when last 16 bits of address are 0xA700

Table 5.1-145 Test parameters for DMA_2Ch_39

Test No :- DMA_2Ch_40

1. Set up the channel 0 and 1 as given in table no 5.1-146.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives error response when last 16 bits of destination address are 0xC60F.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory gives of channel 1 error response when last 16 bits of destination address are 0xC60F.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	16	16	16	1	NA	NA	OK
CH1	M2M	DMAC	8	8	16	16	16	0	NA	NA	Error Response when last 16 bits of address are

											0xC60F
--	--	--	--	--	--	--	--	--	--	--	--------

Table 5.1-146 Test parameters for DMA_2Ch_40**Test No :- DMA_2Ch_41**

1. Set up the channel 0 and 1 as given in table no 5.1-147.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives waited error response when last 16 bits of destination address are 0x5900.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory of channel 1 give error response when last 16 bits of destination address are 0x5900.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	8	8	8	1	NA	NA	OK
CH1	M2M	DMAC	8	8	1	1	1	0	NA	NA	Error Response when last 16 bits of address are 0x5900

Table 5.1-147 Test parameters for DMA_2Ch_41**Test No :- DMA_2Ch_42**

1. Set up the channel 0 and 1 as given in table no 5.1-148.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives error response when last 16 bits of destination address are 0xC40F.
4. Enable channel 1 first and then channel 0.

5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory gives of channel 1 error response when last 16 bits of destination address are 0xC40F.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	16	16	16	1	NA	NA	OK
CH1	M2M	DMAC	8	8	16	16	16	0	NA	NA	Error Response when last 16 bits of address are 0xC40F

Table 5.1-148 Test parameters for DMA_2Ch_42

Test No :- DMA_2Ch_43

1. Set up the channel 0 and 1 as given in table no 5.1-149.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives waited error response when last 16 bits of destination address are 0x4E00.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory of channel 1 give error response when last 16 bits of destination address are 0x4E00.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	8	8	8	1	NA	NA	OK
CH1	M2M	DMAC	8	8	1	1	1	0	NA	NA	Error Response when last 16 bits of address are 0x4E00

Table 5.1-149 Test parameters for DMA_2Ch_43

Test No :- DMA_2Ch_44

1. Set up the channel 0 and 1 as given in table no 5.1-150.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives error response when last 16 bits of destination address are 0x300F.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory gives of channel 1 error response when last 16 bits of destination address are 0x300F.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	16	16	16	1	NA	NA	OK
CH1	M2M	DMAC	8	8	16	16	16	0	NA	NA	Error Response when last 16 bits of address are 0x300F

Table 5.1-150 Test parameters for DMA_2Ch_44

Test No :- DMA_2Ch_45

1. Set up the channel 0 and 1 as given in table no 5.1-151.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives waited error response when last 16 bits of destination address are 0x9000.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory of channel 1 give error response when last 16 bits of destination address are 0x9000.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	8	8	8	1	NA	NA	OK
CH1	M2M	DMAC	8	8	1	1	1	0	NA	NA	Error Response when last 16 bits of address are 0x9000

Table 5.1-151 Test parameters for DMA_2Ch_45

Test No :- DMA_2Ch_46

1. Set up the channel 0 and 1 as given in table no 5.1-152.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 6 such that it gives error response when last 16 bits of destination address are 0x590F.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.

9. Check that destination memory gives of channel 1 error response when last 16 bits of destination address are 0x590F.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	16	16	16	1	NA	NA	OK
CH1	M2M	DMAC	8	8	16	16	16	0	NA	NA	Error Response when last 16 bits of address are 0x590F

Table 5.1-152 Test parameters for DMA_2Ch_46

Test No :- DMA_2Ch_47

1. Set up the channel 0 and 1 as given in table no 5.1-153.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives waited error response when last 16 bits of destination address are 0xE000.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory of channel 1 give error response when last 16 bits of destination address are 0xE000.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
---------	---------------	-----------------	-----------	------------	-----------	------------	---------------	-----	-------------	--------------	--------------

CH0	M2M	DMAC	8	8	8	8	8	1	NA	NA	OK
CH1	M2M	DMAC	8	8	1	1	1	0	NA	NA	Error Response when last 16 bits of address are 0xE000

Table 5.1-153 Test parameters for DMA_2Ch_47**Test No :- DMA_2Ch_48**

1. Set up the channel 0 and 1 as given in table no 5.1-154.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives error response when last 16 bits of destination address are 0x910F.
4. Enable channel 1 first and then channel 0.
5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory gives of channel 1 error response when last 16 bits of destination address are 0x910F.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	16	16	16	1	NA	NA	OK
CH1	M2M	DMAC	8	8	16	16	16	0	NA	NA	Error Response when last 16 bits of address are 0x910F

Table 5.1-154 Test parameters for DMA_2Ch_48**Test No :- DMA_2Ch_49**

1. Set up the channel 0 and 1 as given in table no 5.1-155.
2. Configure Trickbox such that HGRANTx should be delayed by 7 clocks after assertion of HBUSREQx by DMAC.
3. Configure destination memory of channel 1 such that it gives waited error response when last 16 bits of destination address are 0x4D00.
4. Enable channel 1 first and then channel 0.

5. Check that channel 1 performs source data transfer.
6. Check that channel 0 performs source data transfer.
7. Check that channel 0 performs destination data transfer.
8. Check that channel 1 performs destination data transfer.
9. Check that destination memory of channel 1 give error response when last 16 bits of destination address are 0x4D00.
10. Check that channel 0 LLI gets loaded.
11. Check that channel 0 performs source data transfer.
12. Check that channel 0 performs destination data transfer.

Channel	Transfer type	Flow controller	Src Width	Dest Width	Src Burst	Dest Burst	Transfer size	LLI	Src Periph.	Dest Periph.	AHB Response
CH0	M2M	DMAC	8	8	8	8	8	1	NA	NA	OK
CH1	M2M	DMAC	8	8	1	1	1	0	NA	NA	Error Response when last 16 bits of address are 0x4D00

Table 5.1-155 Test parameters for DMA_2Ch_49

6.5.9 AHB Master Test Cases

The general steps, which need to be followed to carry out the following test cases, is given below.

DMAC_MSTR_1 (Back to Back Transfers) :

1. Set-up the Channel0 as shown in **Table 5.1-180**.
2. Configure and enable Channel0 for a M2M transaction.
3. Configure Source Address to 0x08000040 and Destination address to 0x10000040.
4. Program the Channel registers so that all possible combinations are tried out.
5. Enable Channel0.
6. Wait for the transactions to get over by polling the Channel flag to go low.

Channel	Transfer type	Flow Controller	Src Width	Dest Width	Src Burst	Dest Burst	Tx Size	Src Inc	Dest Inc	Src Periph	Dst Periph	Note
CH0	M2M	DMAC	32	32	4	4	32	I	I	0	1	1

CH0	M2M	DMAC	16	16	8	8	32	I	I	0	1	2
CH0	M2M	DMAC	8	8	16	16	32	I	I	0	1	3
CH0	M2M	DMAC	16	8	8	8	32	I	I	0	1	4
CH0	M2M	DMAC	32	16	16	16	32	I	I	0	1	4
CH0	M2M	DMAC	32	32	8	8	32	NI	NI	0	1	5
CH0	M2M	DMAC	16	8	16	16	32	NI	NI	0	1	5
CH0	M2M	DMAC	16	8	8	8	32	I	I	0	1	6
CH0	M2M	DMAC	32	16	16	16	32	I	I	0	1	6
CH0	M2M	DMAC	32	32	8	8	32	NI	NI	0	1	6
CH0	M2M	DMAC	16	8	16	16	32	NI	NI	0	1	6
CH0	M2M	DMAC	32	32	4	1	8	I	I	0	1	7
CH0	M2M	DMAC	16	8	4	1	8	I	I	0	1	8
CH0	M2M	DMAC	16	8	4	1	8	I	I	0	1	9

Table 5.1-180: DMAC being flow controller**Note:**

1. This test will test INCR4 followed by INCR4 back to back.
2. This test will test INCR8 followed by INCR8 back to back.
3. This test will test INCR16 followed by INCR16 back to back.
4. These tests ensure that different HSIZE values are programmed. Otherwise it is similar to case1and case3.
5. These test checks UINCR transfers on different HSIZE.
6. The above tests (Case4 and Case5) are repeated with Lock set to 1.
7. This test will test INCR4 followed by Single back to back. This will also test single followed by burst.
8. These tests ensure that different HSIZE values are programmed. Otherwise it is similar to case7
9. Case7 and Case8 are repeated with Lock set to 1.

The corner test cases are given in **Table 5.1-181**.

Src width	Dst width	Src Inc	Dst Inc	Src Burst	Dst Burst	Tx Size	Source Starting Addr [31:0]	Destn Starting Addr[31:0]	Hlock
32	32	I	I	4	4	1	0x08000040	0x10000040	0
32	32	I	I	4	4	8	0x08000FF8	0x10000048	0

32	16	I	I	4	8	8	0x08000FFC	0x1000004A	0
----	----	---	---	---	---	---	------------	------------	---

Table 5.1-181: Corner test cases

For each of the test the values to be loaded into the registers are given below.

- Master is of little endian configuration.
- HPROT lines are driven to user access mode HPROT[0] = 0.
- LLI register is loaded with null.
- The transaction is M2M.
- Flow Controller is DMAC.

DMAC_MSTR_2 (Default Master Test Case) :

The purpose of this test is to check the behavior of DMAC when it is default master.

Make DMAC the default master. So the master should come out with valid transactions on the next clock edge(i.e. the edge after DMAC is enabled) before external arbiter sees the bus request. i.e. NSEQ comes with assertion of HBUSREQ.

- Poll for Channel0 Enabled bit to go low.
- Program DMAC for M2M transfers with DMAC being flow controller as shown in 1st row of Table 5.1-181.
- Enable Channel0.
- Wait for the transactions to get over by polling the Channel Enable flag to go low.
- Re-program the channel for other combination.

DMAC_MSTR_3 (Split/Retry Master Test Case) :

This test tests the Master with Split/Retry response. Here the Split/Retry is given for few times and then OK response is given so that the same address gets retried repeatedly. This test is carried out with HGRANT toggling. Since the grant lines are toggled this test will also check that the burst is rebuild appropriately. The steps needed to carry out this test are given below

- Poll for Channel0 Enabled bit to go low.
- Program the Memory module to give 3 consecutive Retry's followed by OK response.
- Program the Source and Destination register near 1KB boundhary.
- Program the Grant block to toggle the grant for every 2 HCLK.
- Program DMAC for M2M transfers with DMAC being flow controller as shown in 2nd row of Table 5.1-181.
- Enable Channel0.
- Wait for the transactions to get over by polling the Channel Enable flag to go low.
- Re-program the channel for other combination.

DMAC_MSTR_4 (Testing for Error Response) :

This test tests the Master with Error response. This test is carried out with HGRANT toggling. The Grant is toggled such that it is not there when Error happens. The steps needed to carry out this test is given below

- Poll for Channel0 Enabled bit to go low.

- Program the Memory module to give Error response.
- Program the Source and Destination register near 1KB boundary.
- Program the Grant block to toggle the grant for every HCLK.
- Program DMAC for M2M transfers with DMAC being flow controller as shown in 2nd row of Table 5.1-181.
- Enable Channel0.
- Wait for the transactions to get over by polling the Channel Enable flag to go low.
- Re-program the channel for other combination.

DMAC_MSTR_5 (Testing for 1KB Boundary without Grant Toggling) :

This test ensures that pipelining is maintained on AHB and BURST is split properly near 1KB, also

HBUSREQ does not get deasserted as there are back to back transfers. The steps needed to carry out this test is given below

- Poll for Channel0 Enabled bit to go low.
- Program the Memory module to give OK response.
- Program the Source and Destination register near 1KB boundary.
- Program the Grant block so that Master is always granted.
- Program DMAC for M2M transfers with DMAC being flow controller as shown in 3rd row of Table 5.1-181.
- Enable Channel0.
- Wait for the transactions to get over by polling the Channel Enable flag to go low.
- Re-program the channel for other combination.

DMAC_MSTR_6 (Testing for 1KB Boundary with Grant Toggling) :

This test ensures that pipelining is maintained on AHB and BURST is split properly near 1KB. This test will also check that the rebuilding of burst happens appropriately. The steps needed to carry out this test is given below

- Poll for Channel0 Enabled bit to go low.
- Program the Memory module to give OK response.
- Program the Source and Destination register near 1KB boundary.
- Program the Grant block so that grant is toggled on every HCLK.
- Program DMAC for M2M transfers with DMAC being flow controller as shown in 3rd row of Table 5.1-181.
- Enable Channel0.
- Wait for the transactions to get over by polling the Channel Enable flag to go low.
- Re-program the channel for other combination.

6.5.10 Corner cases

The corner cases identified after the implementation details are available. The corner test cases are identified from the uncovered portion of the vhdl/verilog rtl of the UUT.

- The busgrant signal is toggled while the memory is returning a wait response.
- The addresses are programmed to be all 1's and the incrementing the address for further data transfers.

- Test to cover big endian mode operation.
- A 2 channel operation, when the first channel gets an error response preceeded by a wait response and the second channel is expected to continue its operation.
- Peripheral requests initiated purely from the softreq registers and the dma transfer is aborted with an error response.
- The different address patterns for the source and destination address in incrementing mode of transfer are given so to cover the address increment logic.
- The disabling of the channel happens in the middle of the burst started by the channel and next channel comes with the new burst of transfers with numbers of transfers = 1 in one case and > 1 in another. These cases are repeated with the waited and also for the non-waited transfer responses from the AHB slave to the master interface.
- The above disable cases repeated with the incrementing and non-incrementing mode of transfers and again for low priority channel followed by HIGH priority or vice-versa.
- Disabling of the channel happens when INCR burst is going on AHB at 1KB boundary and the channel is disabled.

Verification Test Vector Generation

A **make sourcefilename** (without the .c extension) in the **verification/bustests** directory will create .bif formatted and .sim formatted vectors in the **verification/bustest/invec** directory. The makefile in **verification/bustest** directory can be referred to for the make options.

The **make all** option can be used to create a test vector file including all the tests.

7 DMAC INTEGRATION TEST DETAILS

The DMAC integration test vector suite allows the integrator to verify that the DMAC has been connected into the target system correctly. These vectors test 2 classes of signals:

- AMBA signals (DMAC signals connected directly to the AHB)
- Intra-Chip signals (DMAC signals connected to the Interrupt Controllers and other DMA supporting peripherals)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 7].

The DMAC Integration testbench is based on an AHB TICTalk testbench. The VHDL and Verilog TICTalk testbenches are used to verify that the DMAC has been connected into the target system correctly. The VHDL and Verilog TICTalk testbench files reside in the **integration/vhdl** and **integration/verilog** directories respectively. The test vectors that are applied to the TICTalk testbenches are in the **integration/bustest** directory. The testvectors for integration testing are written in BusTalk and then converted into .tif/.sim format to be applied to the TICTalk integration testbench.

7.1 DMAC VHDL Integration testbench

The AMBA TICTalk VHDL testbench used for integration testing of the DMAC is located in the **integration/vhdl/tbench** directory. The Integration test vectors are located in the **integration/bustest** directory.

The test vectors used with this testbench can be applied to the DMAC when it is part of an AMBA system design.

Figure 7.1-1 illustrates the VHDL testbench to apply TICTalk test vectors.

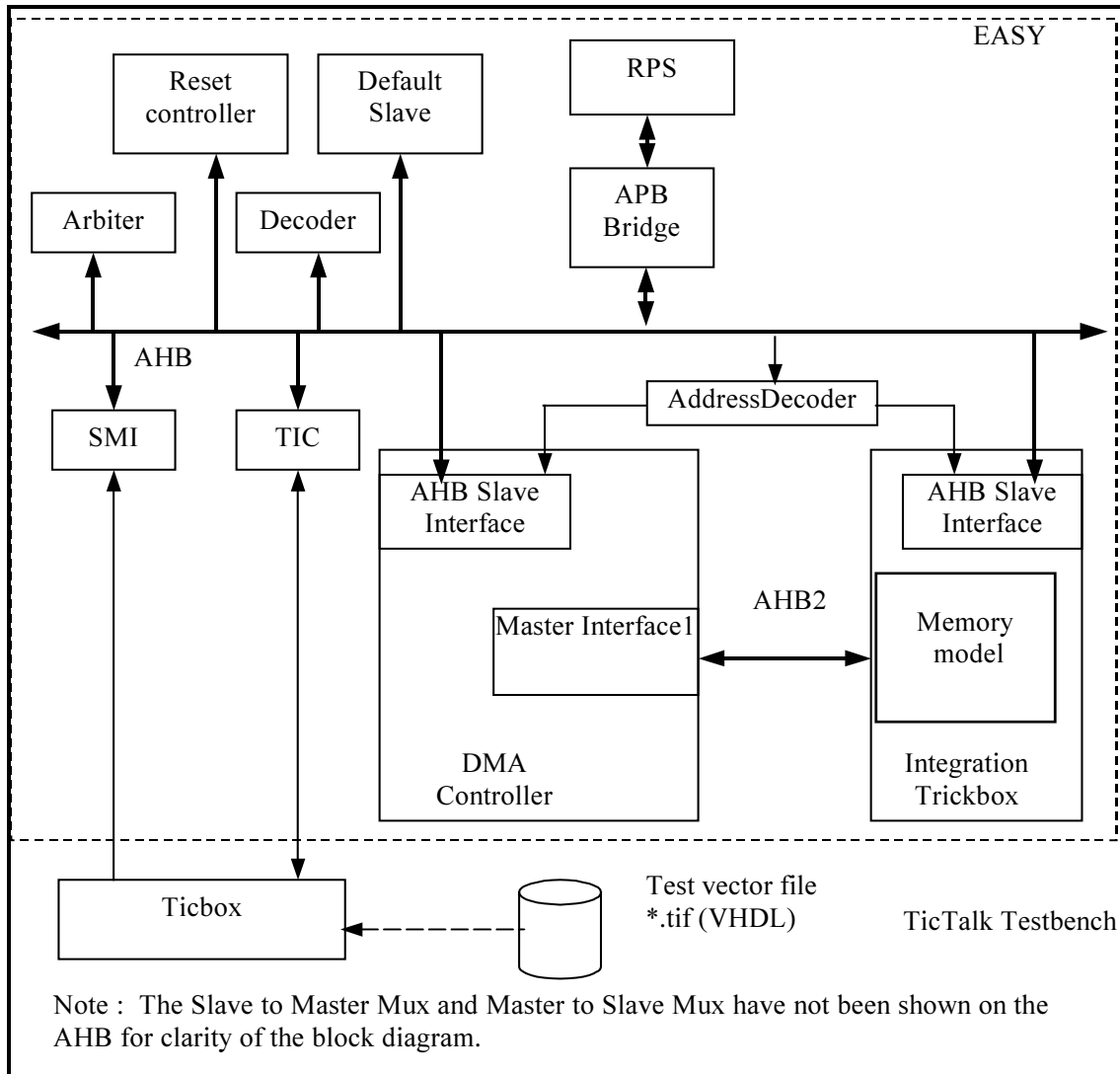


Figure 7.1-1 VHDL Testbench for simulating TICTalk test vectors

The DMAC VHDL Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide [Ref. 6] which is part of ARM Micropack documentation.

Figure 7.1-2 illustrates the hierarchy for the TICTalk VHDL testbench.

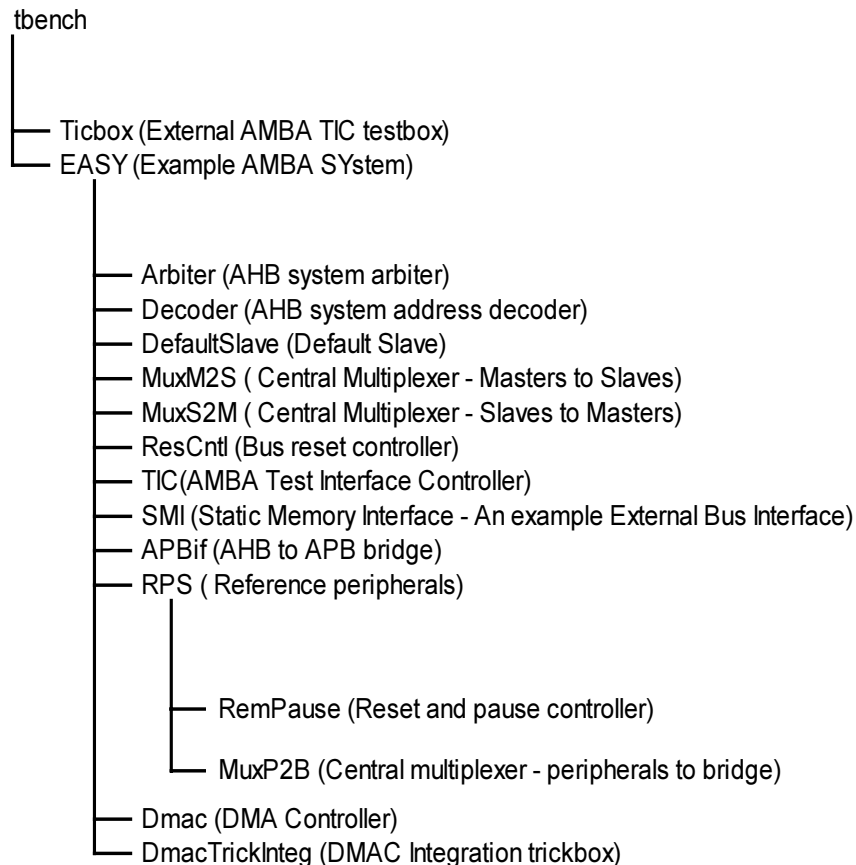


Figure 0-2 VHDL Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.vhd**, the top-level of the DMAC Integration testbench. This module instantiates the model of the EASY microcontroller and the Ticbox. The DMAC is instantiated in the EASY.vhd. All the Scan input pins are tied low to keep them from interfering with the test process.

The integration trickbox is used to implement memory models, which mimic an AHB Slave on the AHB for the AHB Master interfaces on the DMAC. For further details on the memory model in the integration trickbox, refer to **Section 6.3.5 Memory Block**.

7.1.1 Unit Under Test

The Unit Under Test may be one of the following

DMAC VHDL RTL

DMAC scan inserted VHDL netlist generated from VHDL RTL Source

One out of these two versions may be referenced by creating a link [named **uut**] in the **integration/vhdl** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

7.1.2 Test Vectors

The test vectors for the integration testing of the DMAC are coded into separate C files for testing accesses to all the DMAC registers and for Integration testing. Make options are provided to run the tests together or individually.

7.1.3 Running Simulations

The **README** file in the *integration/vhdl/tbench* directory gives step by step instructions for running simulations using the DMAC Integration test vectors on the VHDL source code.

Toggle coverage numbers (for block-level I/Os) when the integration test vectors are run on the VHDL RTL and netlist forms of the DMAC are available in the *integration/vhdl/Dmac_rtl* and *integration/vhdl/Dmac_scaninsert_vhdl_net_max* directories.

Dmac_rtl_modelsim.untog

Dmac_scaninsert_vhdl_vhdl_net_max_modelsim.untog

A sample toggle report (*.untog) file is shown below:

Toggle Report	Node	-> 0	->1
	Signal 1	x	y
	Signal 2	a	b
Total Node Count	= C		
Toggled Node Count	= T		
Untoggled Node Count	C-T = U		
Toggle Coverage	T/C = P %		

The column "Node" in the .untog file lists the individual nodes that are not toggled when the test vectors are run on the VHDL in RTL or netlist forms. The column "->0" indicates the number of times that particular node has toggled to 0 and the column "->1" indicates the number of times it has toggled to 1. In the above example, the Node "Signal 1" toggles to 0 "x" times and it toggles to 1 "y" times. The "Total Node Count", indicates the total number of I/O nodes in the block and the "Toggled Node Count" indicates the number of these nodes that are toggled when the test vectors are run. The number "Untoggled Node Count" indicates the number of I/O nodes in the block that are not toggled when the test vectors are run and the "Toggle Coverage" gives the percentage of the total number of nodes in the block that are toggled when the test vectors are run.

Run time logs for all the combinations are available in the following files in the *integration/vhdl/Dmac_rtl* and *integration/vhdl/Dmac_scaninsert_vhdl_net_max* directories.

Dmac_rtl_modelsim.log

Dmac_scaninsert_vhdl_vhdl_net_max_modelsim.log

7.2 DMAC Verilog Integration testbench

The AMBA TICTalk Verilog testbench used for integration testing of the DMAC is located in the *integration/verilog/tbench* directory. The Integration test vectors are located in the *integration/bustest/invec* directory.

The test vectors used with this testbench can be applied to the DMAC when it is part of an AMBA system design.

Figure 7.2-1 illustrates the Verilog testbench to apply TICTalk test vectors.

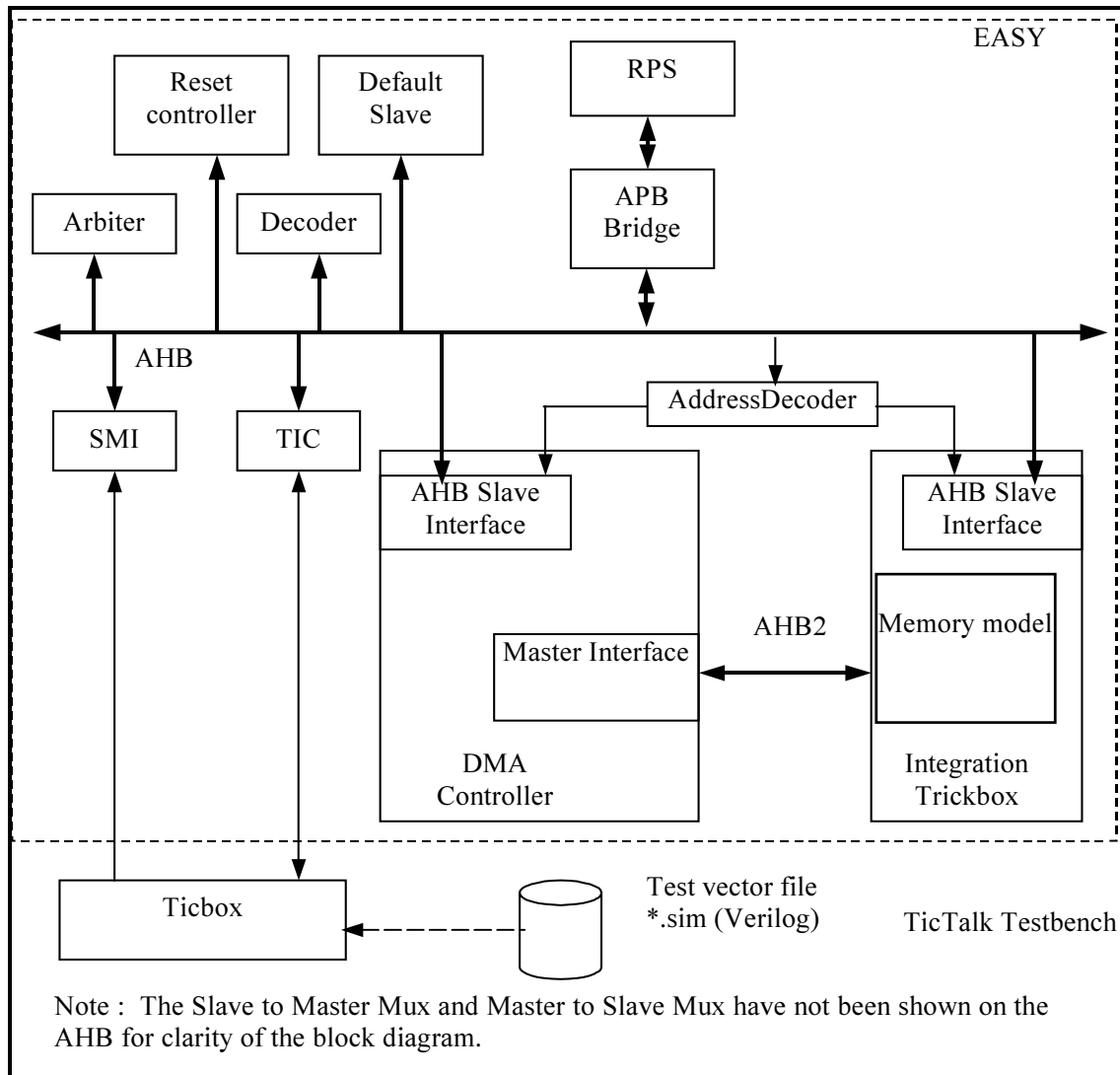


Figure 7.2-1 Verilog Testbench for simulating TICTalk test vectors

The DMAC Verilog Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 7.2-2 illustrates the hierarchy for the TICTalk Verilog testbench.

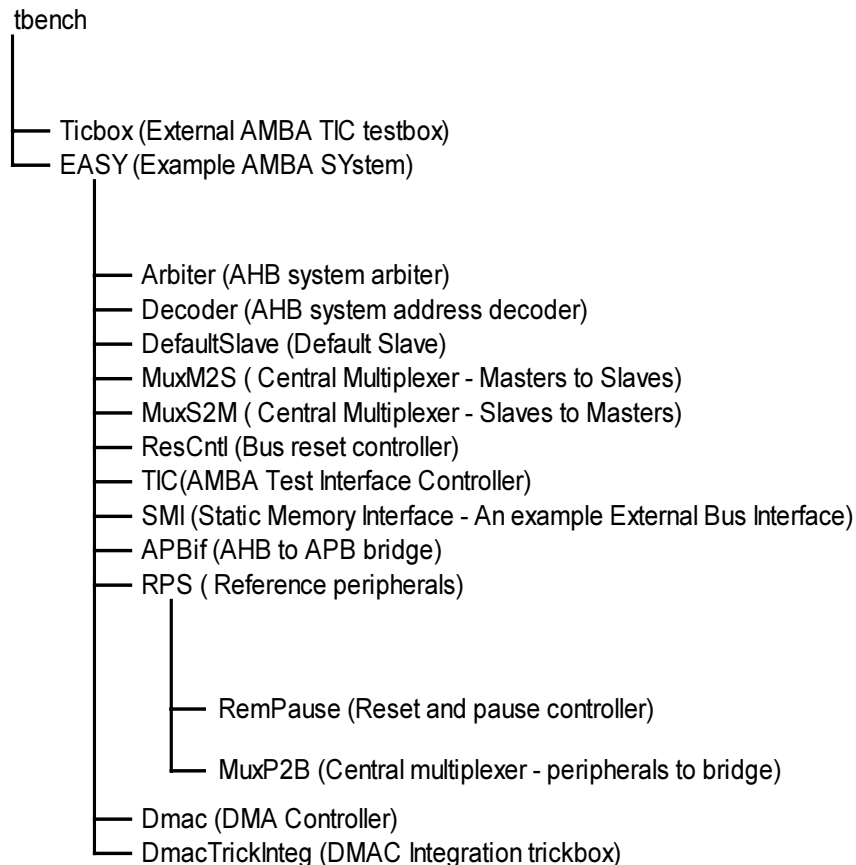


Figure 7.2-2 Verilog Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.vhd**, the top-level of the DMAC Integration testbench. This module instantiates the model of the EASY microcontroller and the Ticbox. The DMAC is instantiated in the EASY.vhd. All the Scan input pins are tied low to keep them from interfering with the test process.

The integration trickbox is used to implement memory models, which mimic an AHB Slave on the AHB for the AHB Master interfaces on the DMAC. For further details on the memory model in the integration trickbox, refer to **Section 6.3.5 Memory Block**.

7.2.1 Unit Under Test

The Unit Under Test may be one of the following

DMAC Verilog RTL

DMAC scanready Verilog netlist generated from Verilog RTL Source

DMAC non-scan Verilog netlist generated from VHDL RTL Source

One out of these two versions may be referenced by creating a link [named **uut**] in the **integration/verilog** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

7.2.2 Test Vectors

The test vectors for the integration testing of the DMAC are coded into separate C files for testing accesses to all the DMAC registers and for Integration testing. Make options are provided to run the tests together or individually.

7.2.3 Running Simulations

The **README** file in the *integration/verilog/tbench* directory gives step by step instructions for running simulations using the DMAC Integration test vectors on the VERILOG source code.

Toggle coverage numbers (for block-level I/Os) when the integration test vectors are run on the Verilog RTL and netlist forms of the DMAC are available in the *integration/verilog/Dmac_rtl* and *integration/verilog/Dmac_noscan_vhdl_net_min* directories.

Dmac_rtl_modelsim.untog

Dmac_noscan_vhdl_verilog_net_min_modelsim.untog

A sample toggle report (*.untog) file is shown below:

Toggle Report	Node	-> 0	->1
	Signal 1	x	y
	Signal 2	a	b
Total Node Count	= C		
Toggled Node Count	= T		
Untoggled Node Count	C-T = U		
Toggle Coverage	T/C = P %		

The column "Node" in the .untog file lists the individual nodes that are not toggled when the test vectors are run on the Verilog in RTL or netlist forms. The column "->0" indicates the number of times that particular node has toggled to 0 and the column "->1" indicates the number of times it has toggled to 1. In the above example, the Node "Signal 1" toggles to 0 "x" times and it toggles to 1 "y" times. The "Total Node Count", indicates the total number of I/O nodes in the block and the "Toggled Node Count" indicates the number of these nodes that are toggled when the test vectors are run. The number "Untoggled Node Count" indicates the number of I/O nodes in the block that are not toggled when the test vectors are run and the "Toggle Coverage" gives the percentage of the total number of nodes in the block that are toggled when the test vectors are run.

Run time logs for all the combinations are available in the following files in the integration/verilog/Dmac_rtl, integration/verilog/Dmac_noscan_vhdl_net_min and integration/verilog/Dmac_scanready_verilog_net_typ directories.

Dmac_rtl_modelsim.log

Dmac_rtl_LDV.log

Dmac_noscan_vhdl_verilog_net_min_modelsim.log

Dmac_scanready_verilog_verilog_net_typ_LDV.log

7.3 Integration Trickbox

The Integration trickbox implements 2 memory models, one for each AHB Master port of the DMAC. These memory modules act as AHB Slaves, responding to accesses initiated by the DMAC through the AHB master interface. For further details of the memory models refer to **Section 6.3.5 Memory Block**. These memory modules have the same implementation as in the verification trickbox.

7.4 Integration Tests

When the DMAC is used in the AMBA system, it must be ensured that all the I/O pins are correctly connected. Integration vectors allow the user to verify that the DMAC has been wired into the system correctly.

This test is performed with `DmacIntM2Mtests()`, `RegisterTests()`, `ResetTests()` and `DmacIntegTests()` function calls. The `RegisterTests()` and `ResetTests()` function calls are used to increase the toggle coverage of the AHB Slave ports. The `DmacIntM2Mtests()` function call is used for the integration testing of the two AHB Master ports. The `DmacIntegTests()` function call is used for integration testing the Non-AMBA ports of the DMAC.

7.4.1 Integration Testing of non-AMBA intra-chip outputs

Signals : DMACINTR, DMACINTTC, DMACINTERR, DMACCLR, DMACTC.

When Integration tests are run with the DMAC in a stand-alone test setup :

- Write a '1' to the ITEN bit in the Control register. This selects the test path from the corresponding bit the DMACITOP register bits to the intra-chip output signals.
- Write a '1' and then a '0' to the corresponding bit in the DMACITOP register and read the same register bits to verify that the value written is read out.

When Integration tests are run in an integrated system :

- Write a '1' to the ITEN bit in the Control register. This selects the test path from the corresponding bit the DMACITOP register bits to the intra-chip output signals.
- Write a '1' and then a '0' to the corresponding bit in the DMACITOP register and read the ITIP register (Integration Test Input register) bits to verify that the value written to the DMAC is read out of the target peripheral.

In the `DmacIntegTests()` function, the user needs to replace the read commands from the DMACITOP register by the suitable command to read from the ITIP registers of the destination peripherals.

7.4.2 Integration Testing of non-AMBA intra-chip inputs

For the Non-AMBA inputs (i.e. all DMA Request lines), Integration testing is done through the DMACSoftReq registers.

Signals : DMACSREQ, DMACBREQ, DMACLSREQ, DMACLBREQ.

When Integration tests are run with the DMAC in a stand-alone test setup:

- Write a '1' to the ITEN bit in the Control register. This selects the test path from the corresponding bit in the DMACSoftReq (one of DMACSoftSreq, DMACSoftBreq, DMACSoftLBreq or DMACSoftLSreq) register to the internal DMA request signal.
- Write a '1' and then a '0' to the corresponding bit in the DMACSoftReq register and read the same register bit to ensure that the value written is read out.

When Integration tests are run in an integrated system, the input signals to the DMAC are toggled by writing '1' and '0' into internal registers (ITOP registers) of the Source peripheral of the signal and the connection is verified by reading out from the corresponding bit in the DMACSoftReq register. The user should ensure that the ITEN bit in the

7.4.3 Integration Testing of AMBA ports

The AMBA ports can be classified into 2 categories

1. AHB Slave Interface ports
2. AHB Master Interface ports

The AHB Slave Interface ports are tested with normal read-write of registers in the DMAC through the DMAC Slave Interface. The addresses and the data patterns are so defined to get the maximum possible toggle coverage of the AHB Slave ports. When these tests are run in the integrated environment these tests may be used as is without any modification.

AHB Master Interface ports are tested by programming the DMAC to perform Memory-to-Memory transfers from a memory model connected to one AHB Master port of the DMAC to another memory model connected to the other AHB Master port of the DMAC. The two memory models are located in the Integration trickbox. The memory models respond to a wide range of addresses and support transfers of byte, half-word and word widths. These tests are only intended to help in defining the actual tests for use in an integrated system, because the tests are very much dependent on the address map which, in turn, is system-specific. In addition, the tests may need modifications for compatibility with the transfer widths supported by the AHB slaves in the integrated system.

7.5 Integration Test Vector Generation

A **make sourcefilename** (without the .c extension) in the **integration/bustests** directory will create .tif formatted and .sim formatted vectors in the **integration/bustest/invec** directory. The makefile in **integration/bustest** directory can be referred for the make options.

The **make all** option can be used to create a test vector file including all the tests.